

Dokumentacja Techniczna Projektu

PLIK: celery_app.py

```
"""celery_app.py
```

```
Konfiguracja Celery i taski generowania PDF.
```

```
WZORZEC: ContextTask – aplikacja Flask inicjowana RAZ przy starcie workera.
```

```
Każde zadanie jest automatycznie owijane w istniejący app_context,
```

```
zamiast tworzyć go od nowa (co niszczyło pulę połączeń do bazy).
```

```
"""
```

```
import os
```

```
import logging
```

```
from celery import Celery
```

```
logger = logging.getLogger(__name__)
```

```
BROKER_URL = os.environ.get('CELERY_BROKER_URL', 'redis://localhost:6379/0')
```

```
RESULT_URL = os.environ.get('CELERY_RESULT_BACKEND', 'redis://localhost:6379/0')
```

```
# — 1. Inicjalizacja Celery —————
```

```
celery = Celery(
```

```
    'ans_praktyki',
```

```
    broker=BROKER_URL,
```

```
    backend=RESULT_URL,
```

```
    include=['celery_app'],
```

```
)
```

```
celery.conf.update(
```

```
    task_serializer='json',
```

```
    result_serializer='json',
```

```
    accept_content=['json'],
```

```
    timezone='Europe/Warsaw',
```

```

    enable_utc=True,

    task_track_started=True,

    broker_connection_retry_on_startup=True,

    result_expires=3600, # wyniki trzymamy 1h
)

# — 2. Flask app – jedno wywołanie create_app() dla całego workera —————

# Celery-worker budowany jest na bazie app_admin/Dockerfile,

# więc importujemy fabrykę stamtąd. Baza danych jest współdzielona

# i modele (ZapisPraktyki, WpisDziennika) są identyczne w obu aplikacjach.

from app_admin import create_app as _create_flask_app

flask_app = _create_flask_app()

logger.info("Flask app initialized once for Celery worker (ContextTask pattern)")

# — 3. ContextTask – automatyczne opakowanie każdego taska w app_context ———

class ContextTask(celery.Task):

    """Nadpisana klasa bazowa Celery Task.

    Każde zadanie jest wykonywane wewnątrz istniejącego app.app_context(),
    eliminując potrzebę tworzenia nowego kontekstu (i nowej puli połączeń DB)
    przy każdym wywołaniu taska.

    """

    abstract = True

    def __call__(self, *args, **kwargs):

        with flask_app.app_context():

            return self.run(*args, **kwargs)

celery.Task = ContextTask

```

```
@celery.task(bind=True, name='generate_pdf_dziennik', base=ContextTask)
```

```
def generate_pdf_dziennik(self, enrollment_id: str) -> dict:
```

```
    import uuid
```

```
    from pathlib import Path
```

```
    try:
```

```
        self.update_state(state='STARTED', meta={'progress': 0})
```

```
        # Importy modeli – baza jest wspólna, importujemy z app_admin
```

```
        from core.extensions import db
```

```
        from core.modele import ZapisPraktyki, WpisDziennika
```

```
        zapis = db.session.get(ZapisPraktyki, uuid.UUID(enrollment_id))
```

```
        if not zapis:
```

```
            raise ValueError(f'Zapis {enrollment_id} nie istnieje')
```

```
        wpisy = (
```

```
            db.session.query(WpisDziennika)
```

```
                .filter_by(zapis_id=zapis.id)
```

```
                .order_by(WpisDziennika.data_wpisu)
```

```
                .all()
```

```
        )
```

```
        self.update_state(state='STARTED', meta={'progress': 30})
```

```
        import httpx
```

```
        context = {
```

```
            'student': {
```

```
                'first_name': zapis.student.first_name,
```

```
                'last_name': zapis.student.last_name,
```

```
                'album_number': zapis.student.album_number,
```

```
            },
```

```

'zapis': {

    'total_hours_logged': zapis.total_hours_logged,

    'praktyka': {

        'rok_uczelnianny': zapis.praktyka.rok_uczelnianny,

        'semestr': zapis.praktyka.semestr,

    },

},

'wpisy': [

    {

        'entry_date': w.entry_date.isoformat(),

        'duration_hours': w.duration_hours,

        'description': w.description,

        'efekt': {'kod': f"{w.learning_outcome_id:02d}"},

    }

    for w in sorted(wpisy, key=lambda x: x.entry_date)

]

}

```

```

tex_service_url = flask_app.config.get('TEX_SERVICE_URL', 'http://tex-service:5002')

```

```

resp = httpx.post(

    f"{tex_service_url}/generuj",

    json={'template': 'zal6_dziennik.tex.j2', 'context': context},

    timeout=60.0

)

```

```

if resp.status_code != 200:

    raise Exception(f"Tex service error: {resp.text}")

```

```

pdf_bytes = resp.content

```

```

self.update_state(state='STARTED', meta={'progress': 80})

```

```

output_dir = Path('/app/pdf_output')

```

```

output_dir.mkdir(exist_ok=True)

```

```

filename = f"dziennik_{enrollment_id}.pdf"

```

```

output_path = output_dir / filename

```

```

output_path.write_bytes(pdf_bytes)

self.update_state(state='STARTED', meta={'progress': 100})

return {

    'status': 'success',

    'path': str(output_path),

    'filename': f"dziennik_{zapis.student.last_name}_{zapis.student.first_name}.pdf",

}

except Exception as exc:

    logger.error("Task generate_pdf_dziennik failed: %s", str(exc), exc_info=True)

    return {'status': 'error', 'message': str(exc)}

@celery.task(bind=True, name='compile_raw_tex_task', base=ContextTask)

def compile_raw_tex_task(self, tex_source: str, filename_prefix: str) -> dict:

    import uuid

    from pathlib import Path

    try:

        self.update_state(state='STARTED', meta={'progress': 10})

        from tex_service.compiler import compile_raw_tex

        pdf_bytes = compile_raw_tex(tex_source)

        self.update_state(state='STARTED', meta={'progress': 80})

        output_dir = Path('/app/pdf_output')

        output_dir.mkdir(exist_ok=True)

        filename = f"{filename_prefix}_{uuid.uuid4().hex[:8]}.pdf"

        output_path = output_dir / filename

        output_path.write_bytes(pdf_bytes)

        self.update_state(state='STARTED', meta={'progress': 100})

        return {

```

```

        'status': 'success',

        'path': str(output_path),

        'filename': filename,

    }

```

except Exception as exc:

```

    logger.error("Task compile_raw_tex_task failed: %s", str(exc), exc_info=True)

    return {'status': 'error', 'message': str(exc)}

```

```

@celery.task(bind=True, name='generate_zip_task', base=ContextTask,

             max_retries=3, default_retry_delay=5)

```

```

def generate_zip_task(self, enrollment_id: str) -> dict:

```

```

    import uuid

```

```

    import zipfile

```

```

    from pathlib import Path

```

```

    try:

```

```

        from core.extensions import db

```

```

        from core.modele import ZapisPraktyki

```

```

        from tex_service.compiler import render_tex, compile_raw_tex

```

```

        from tex_service.pdf_service import pdf_service

```

```

        from app_admin.routes.dokumenty import DOCUMENTS

```

```

        self.update_state(state='STARTED', meta={'progress': 5})

```

```

        zapis = db.session.get(ZapisPraktyki, uuid.UUID(enrollment_id))

```

```

        if not zapis:

```

```

            raise ValueError('Zapis nie istnieje')

```

```

        output_dir = Path('/app/pdf_output')

```

```

        output_dir.mkdir(exist_ok=True)

```

```

        import tempfile

```

```

        with tempfile.TemporaryDirectory() as tmpdir:

```

```

tmp_path = Path(tmpdir)

total_docs = len(DOCUMENTS)

doc_errors = [] # śledzenie błędów poszczególnych dokumentów

docs_ok = 0

for idx, (doc_type, (template_name, title)) in enumerate(DOCUMENTS.items()):

    if doc_type == 'ZAL_6':

        context = pdf_service._build_context_dziennik(zapis)

    elif doc_type == 'ZAL_4':

        context = pdf_service._build_context_efekty(zapis)

    elif doc_type == 'ZAL_7':

        tresc = {

            'charakterystyka_miejsca': zapis.sprawozdanie.charakterystyka_miejsca if zapis.sprawozdanie else
'',

            'opis_prac': zapis.sprawozdanie.opis_i_analiza if zapis.sprawozdanie else '',

            'efekty_opisy': ['' for _ in range(13)]

        }

        context = pdf_service._build_context_sprawozdanie(zapis, tresc)

    else:

        context = {'student': zapis.student, 'firma': zapis.dane_miejsca, 'zapis': zapis}

    try:

        raw_tex = render_tex(template_name, context)

        pdf_bytes = compile_raw_tex(raw_tex)

        pdf_file_path = tmp_path / f"{doc_type}.pdf"

        pdf_file_path.write_bytes(pdf_bytes)

        docs_ok += 1

    except Exception as e:

        error_msg = f"{doc_type}: {str(e)}"

        logger.warning("ZIP task: błąd generowania %s: %s", doc_type, e)

        doc_errors.append(error_msg)

        err_file = tmp_path / f"{doc_type}_ERROR.txt"

        err_file.write_text(f"Błąd kompilacji {doc_type}: {str(e)}")

self.update_state(state='STARTED', meta={

```

```

        'progress': 10 + int(80 * (idx + 1) / total_docs),

        'docs_ok': docs_ok,

        'docs_failed': len(doc_errors),

    })

zip_filename = f"komplet_{zapis.student.last_name}_{zapis.student.first_name}_{uuid.uuid4().hex[:6]}.zip"
zip_path = output_dir / zip_filename

with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zipf:

    for f in tmp_path.iterdir():

        zipf.write(f, arcname=f.name)

self.update_state(state='STARTED', meta={'progress': 100})

# Raportowanie częściowego sukcesu zamiast cichego "SUCCESS"
if doc_errors:

    return {

        'status': 'partial_success',

        'path': str(zip_path),

        'filename': zip_filename,

        'docs_ok': docs_ok,

        'docs_failed': len(doc_errors),

        'errors': doc_errors,

        'message': f'Wygenerowano {docs_ok}/{total_docs} dokumentów. '

                    f'Błędy: {"; ".join(doc_errors[:3])}'

                    f'{"..." if len(doc_errors) > 3 else ""}',

    }

return {

    'status': 'success',

    'path': str(zip_path),

    'filename': zip_filename,

}

```



```
except (ConnectionError, OSError) as exc:

    # Błędy przejściowe (sieć, I/O) – ponów zadanie

    logger.warning("ZIP task transient error, retry %d/%d: %s",

                   self.request.retries, self.max_retries, exc)

    raise self.retry(exc=exc, countdown=5 * (self.request.retries + 1))

except Exception as exc:

    logger.error("Task generate_zip_task failed: %s", str(exc), exc_info=True)

    return {'status': 'error', 'message': str(exc)}
```

PLIK: docker-compose.patch.yml

```
version: '3.8'
```

```
services:
```

```
  tex-service:
```

```
    build:
```

```
      context: ./tex_service
```

```
      dockerfile: Dockerfile
```

```
    container_name: sop_tex_service
```

```
    restart: unless-stopped
```

```
    environment:
```

```
      - TZ=Europe/Warsaw
```

```
    volumes:
```

```
      - ./tex_engine/templates:/app/templates:ro
```

```
    healthcheck:
```

```
      test: ["CMD", "curl", "-f", "http://localhost:5001/health"]
```

```
      interval: 10s
```

```
      timeout: 5s
```

```
      retries: 3
```

```
    expose:
```

```
      - "5001"
```

PLIK: docker-compose.yml

services:

db:

image: postgres:16-alpine

container_name: sop_db

restart: unless-stopped

environment:

POSTGRES_USER: \${POSTGRES_USER:-ans_admin}

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD in .env}

POSTGRES_DB: \${POSTGRES_DB:-ans_praktyki}

ports:

- "5432:5432"

networks:

- internal

volumes:

- ./database/init.sql:/docker-entrypoint-initdb.d/init.sql

- pgdata:/var/lib/postgresql/data

deploy:

resources:

limits:

cpus: '1.0'

memory: 512M

redis:

image: redis:7-alpine

container_name: sop_redis

restart: unless-stopped

ports:

- "6379:6379"

networks:

- internal

volumes:

- redis_data:/data

deploy:

resources:

limits:

cpus: '0.25'

memory: 128M

admin:

build:

context: .

dockerfile: app_admin/Dockerfile

container_name: sop_admin

restart: unless-stopped

environment:

- FLASK_ENV=\${FLASK_ENV:-production}

- DATABASE_URL=postgresql+psycopg2://\${POSTGRES_USER:-ans_admin}:\${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD in .env}@db:5432/\${POSTGRES_DB:-ans_praktyki}

- SECRET_KEY=\${SECRET_KEY:?Set SECRET_KEY in .env}

- TEX_SERVICE_URL=http://tex-service:5002

- CELERY_BROKER_URL=redis://redis:6379/0

- CELERY_RESULT_BACKEND=redis://redis:6379/0

ports:

- "5000:5000"

networks:

- internal

- public

depends_on:

tex-service:

condition: service_healthy

db:

condition: service_started

redis:

condition: service_started

volumes:

- pdf_data:/app/pdf_output

- uploads_data:/app/uploads

- ./app_admin:/app/app_admin:ro

```
- ./core:/app/core:ro

read_only: true

tmpfs:

- /tmp

- /app/app_admin/logs

security_opt:

- no-new-privileges:true

cap_drop:

- ALL

deploy:

resources:

limits:

cpus: '0.5'

memory: 256M

student:

build:

context: .

dockerfile: app_student/Dockerfile

container_name: sop_student

restart: unless-stopped

environment:

- FLASK_ENV=${FLASK_ENV:-production}

- DATABASE_URL=postgresql+psycopg2://${POSTGRES_USER:-ans_admin}:${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD in .env}@db:5432/${POSTGRES_DB:-ans_praktyki}

- SECRET_KEY=${SECRET_KEY:?Set SECRET_KEY in .env}

- TEX_SERVICE_URL=http://tex-service:5002

- CELERY_BROKER_URL=redis://redis:6379/0

- CELERY_RESULT_BACKEND=redis://redis:6379/0

ports:

- "5001:5001"

networks:

- internal

- public

depends_on:
```

```
tex-service:

  condition: service_healthy

db:

  condition: service_started

redis:

  condition: service_started

volumes:

  - pdf_data:/app/pdf_output

  - uploads_data:/app/uploads

  - ./app_student:/app/app_student:ro

  - ./core:/app/core:ro

read_only: true

tmpfs:

  - /tmp

security_opt:

  - no-new-privileges:true

cap_drop:

  - ALL

deploy:

  resources:

    limits:

      cpus: '0.5'

      memory: 256M

celery-worker:

  build:

    context: .

    dockerfile: app_admin/Dockerfile

  container_name: sop_celery_worker

  restart: unless-stopped

  environment:

    - FLASK_ENV=${FLASK_ENV:-production}

    - DATABASE_URL=postgresql+psycopg2://${POSTGRES_USER:-ans_admin}:${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD in .env}@db:5432/${POSTGRES_DB:-ans_praktyki}

    - SECRET_KEY=${SECRET_KEY:?Set SECRET_KEY in .env}
```

- TEX_SERVICE_URL=http://tex-service:5002
- CELERY_BROKER_URL=redis://redis:6379/0
- CELERY_RESULT_BACKEND=redis://redis:6379/0

command: celery -A celery_app worker --loglevel=info --concurrency=2

networks:

- internal

depends_on:

redis:

condition: service_started

db:

condition: service_started

tex-service:

condition: service_healthy

volumes:

- pdf_data:/app/pdf_output
- uploads_data:/app/uploads
- ./app_admin:/app/app_admin:ro
- ./core:/app/core:ro

read_only: true

tmpfs:

- /tmp
- /app/app_admin/logs

security_opt:

- no-new-privileges:true

cap_drop:

- ALL

deploy:

resources:

limits:

cpus: '1.0'

memory: 512M

tex-service:

build:

```
context: ./tex_service

dockerfile: Dockerfile

container_name: sop_tex_service

restart: unless-stopped

environment:

  - TZ=Europe/Warsaw

volumes:

  - ./tex_service/templates:/app/templates:ro

expose:

  - "5002"

networks:

  - internal

healthcheck:

  test: ["CMD", "curl", "-f", "http://localhost:5002/health"]

  interval: 10s

  timeout: 5s

  retries: 5

  start_period: 30s

read_only: true

tmpfs:

  - /tmp:size=256m,mode=1777

security_opt:

  - no-new-privileges:true

cap_drop:

  - ALL

deploy:

  resources:

    limits:

      cpus: '1.0'

      memory: 512M

    reservations:

      memory: 128M
```

```
networks:
```


Sieć wewnętrzna – bez dostępu do internetu.

Wszystkie serwisy backendowe komunikują się wyłącznie przez tę sieć.

internal:

internal: true

Sieć publiczna – wyłącznie dla admin i student (porty 5000/5001).

public:

volumes:

pgdata:

pdf_data:

uploads_data:

redis_data:

PLIK: x.py

```
import os

import unicodedata

from fpdf import FPDF, XPos, YPos


# Funkcja ratunkowa: usunie polskie znaki, jeśli czcionki systemowe zawiodą,
# żeby skrypt się nie wywalił na literce "ż" lub "ł".

def usun_polskie_znaki(tekst):

    return unicodedata.normalize('NFKD', tekst).encode('ASCII', 'ignore').decode('ASCII')


class CodeToPDF(FPDF):

    def header(self):

        if self.page_no() == 1:

            self.set_font(self.font_family_name, 'B', 16)

            # Zamiast ln=True używamy nowych standardów fpdf2

            self.cell(0, 10, 'Dokumentacja Techniczna Projektu', new_x=XPos.LMARGIN, new_y=YPos.NEXT, align='C')

            self.ln(10)


def create_pdf_from_project(source_dir, output_filename):

    if not os.path.exists(source_dir):

        print(f" BŁĄD: Folder '{source_dir}' nie istnieje!")

        return

    print(f" Rozpoczynam skanowanie folderu: {os.path.abspath(source_dir)}")

    pdf = CodeToPDF()

    pdf.set_auto_page_break(auto=True, margin=15)

    # --- SZUKANIE CZCIONKI ---

    font_name = 'Courier'

    ma_polskie_znaki = False

    # Lista popularnych czcionek (Consolas jest najlepsza do kodu)

    czcionki_do_sprawdzenia = [
```

```

('Consolas', r'C:\Windows\Fonts\consola.ttf', r'C:\Windows\Fonts\consolab.ttf'),

('Arial', r'C:\Windows\Fonts\arial.ttf', r'C:\Windows\Fonts\arialbd.ttf'),

('CourierNew', r'C:\Windows\Fonts\cour.ttf', r'C:\Windows\Fonts\courbd.ttf')

]

for nazwa, plik_zwykly, plik_pogrubiony in czcionki_do_sprawdzenia:

    if os.path.exists(plik_zwykly) and os.path.exists(plik_pogrubiony):

        try:

            pdf.add_font(nazwa, '', plik_zwykly)

            pdf.add_font(nazwa, 'B', plik_pogrubiony)

            font_name = nazwa

            ma_polskie_znaki = True

            print(f" Znaleziono czcionkę obsługującą polskie znaki: {nazwa}")

            break

        except Exception:

            continue

if not ma_polskie_znaki:

    print(" UWAGA: Nie mogę załadować systemowej czcionki TTF.")

    print(" Uruchamiam protokół ratunkowy: konwersja na ASCII (ż -> z, ł -> l).")

pdf.font_family_name = font_name

ext_list = ('.py', '.html', '.css', '.yaml', '.yml', 'Dockerfile')

znalezione_pliki = 0

for root, dirs, files in os.walk(source_dir):

    # Ignorujemy niepotrzebne foldery środowiskowe

    dirs[:] = [d for d in dirs if d not in ['.git', 'venv', '.venv', '__pycache__', 'node_modules']]

    for file in files:

        if file.endswith(ext_list) or file == 'Dockerfile':

            znalezione_pliki += 1

            file_path = os.path.join(root, file)

```

```

rel_path = os.path.relpath(file_path, source_dir)

print(f" Przetwarzam: {rel_path}")

pdf.add_page()

# --- NAGŁÓWEK PLIKU ---

pdf.set_fill_color(240, 240, 240)

pdf.set_text_color(0, 51, 102)

pdf.set_font(font_name, 'B', 11)

tytul_pliku = f" PLIK: {rel_path}"

if not ma_polskie_znaki: tytul_pliku = usun_polskie_znaki(tytul_pliku)

# Nowe wywołania metody cell

pdf.cell(0, 8, tytul_pliku, new_x=XPos.LMARGIN, new_y=YPos.NEXT, fill=True)

pdf.ln(4)

# --- TREŚĆ KODU ---

pdf.set_text_color(30, 30, 30)

pdf.set_font(font_name, '', 8)

try:

    with open(file_path, 'r', encoding='utf-8', errors='replace') as f:

        for linia in f:

            format_linii = linia.replace('\t', '    ')

            if not ma_polskie_znaki:

                format_linii = usun_polskie_znaki(format_linii)

            pdf.multi_cell(0, 4, format_linii)

except Exception as e:

    pdf.set_text_color(200, 0, 0)

    pdf.set_font(font_name, 'B', 10)

    blad = f"Bład odczytu: {e}"

    if not ma_polskie_znaki: blad = usun_polskie_znaki(blad)

```

```
pdf.cell(0, 10, blad, new_x=XPos.LMARGIN, new_y=YPos.NEXT)
```

```
if znalezione_pliki == 0:
```

```
    print(f" Nie znaleziono plików.")
```

```
    return
```

```
pdf.output(output_filename)
```

```
print(f" SUKCES! Gotowe. Złożono {znalezione_pliki} plików w dokumencie: '{output_filename}'.")
```

```
# --- ODPALAMY ---
```

```
folder_zrodlowy = '.'
```

```
nazwa_pdf = 'dokumentacja_kodu.pdf'
```

```
create_pdf_from_project(folder_zrodlowy, nazwa_pdf)
```

PLIK: y.py

```
import os

import unicodedata

from fpdf import FPDF, XPos, YPos


def usun_polskie_znaki(tekst):

    """Awaryjne usuwanie ogonków, gdyby system zablokował czcionki."""

    return unicodedata.normalize('NFKD', tekst).encode('ASCII', 'ignore').decode('ASCII')


class TreePDF(FPDF):

    def header(self):

        if self.page_no() == 1:

            self.set_font(self.font_family_name, 'B', 16)

            self.cell(0, 10, 'Struktura Drzewa Projektu', new_x=XPos.LMARGIN, new_y=YPos.NEXT, align='C')

            self.ln(5)


def generate_tree(dir_path, prefix="", ignore_dirs=None):

    """Rekurencyjnie generuje linie tekstu reprezentujące drzewo folderów."""

    if ignore_dirs is None:

        # Lista folderów, które nie będą zaśmiecać drzewa

        ignore_dirs = {'.git', 'venv', '.venv', '__pycache__', 'node_modules', '.idea', '.vscode'}

    try:

        items = sorted(os.listdir(dir_path))

    except PermissionError:

        return [f"{prefix}[Brak dostępu]"]

    # Filtrujemy ignorowane foldery

    filtered_items = []

    for item in items:

        full_path = os.path.join(dir_path, item)

        if os.path.isdir(full_path) and item in ignore_dirs:

            continue

        filtered_items.append(item)
```

```

lines = []

for i, item in enumerate(filtered_items):

    is_last = (i == len(filtered_items) - 1)

    # Używamy bezpiecznych znaków ASCII do rysowania gałęzi

    connector = "\-- " if is_last else "+-- "

    lines.append(f"{prefix}{connector}{item}")

    full_path = os.path.join(dir_path, item)

    if os.path.isdir(full_path):

        extension = "    " if is_last else "|    "

        lines.extend(generate_tree(full_path, prefix=prefix + extension, ignore_dirs=ignore_dirs))

return lines

def create_tree_pdf(source_dir, output_filename):

    if not os.path.exists(source_dir):

        print(f" BŁĄD: Folder '{source_dir}' nie istnieje!")

        return

    print(f" Skanuję strukturę folderu: {os.path.abspath(source_dir)}")

    pdf = TreePDF()

    pdf.set_auto_page_break(auto=True, margin=15)

    # --- SZUKANIE BEZPIECZNEJ CZCIONKI MONOSPACE ---

    font_name = 'Courier'

    ma_polskie_znaki = False

    czcionki_do_sprawdzenia = [

        ('Consolas', r'C:\Windows\Fonts\consola.ttf', r'C:\Windows\Fonts\consolab.ttf'),

        ('CourierNew', r'C:\Windows\Fonts\cour.ttf', r'C:\Windows\Fonts\courbd.ttf')

    ]

```

```

for nazwa, plik_zwykly, plik_pogrubiony in czcionki_do_sprawdzenia:

    if os.path.exists(plik_zwykly) and os.path.exists(plik_pogrubiony):

        try:

            pdf.add_font(nazwa, '', plik_zwykly)

            pdf.add_font(nazwa, 'B', plik_pogrubiony)

            font_name = nazwa

            ma_polskie_znaki = True

            break

        except Exception:

            continue

pdf.font_family_name = font_name

pdf.add_page()

# --- RYSOWANIE DRZEWA ---

pdf.set_text_color(30, 30, 30)

pdf.set_font(font_name, 'B', 10)

# Nazwa głównego folderu

root_name = os.path.basename(os.path.abspath(source_dir))

if not root_name: root_name = "Projekt"

if not ma_polskie_znaki: root_name = usun_polskie_znaki(root_name)

pdf.cell(0, 6, f"Katalog: {root_name}", new_x=XPos.LMARGIN, new_y=YPos.NEXT)

pdf.set_font(font_name, '', 10)

# Pobieranie linii drzewa z funkcji

drzewo_linie = generate_tree(source_dir)

# Zapisywanie każdej linii do PDF

for linia in drzewo_linie:

    tekst_linii = linia

    if not ma_polskie_znaki:

```



```
tekst_linii = usun_polskie_znaki(tekst_linii)

pdf.cell(0, 5, tekst_linii, new_x=XPos.LMARGIN, new_y=YPos.NEXT)

pdf.output(output_filename)

print(f" SUKCES! Struktura projektu zapisana jako: '{output_filename}'.")

# --- KONFIGURACJA ---

folder_zrodlowy = '.' # '.' oznacza obecny folder

nazwa_pdf = 'struktura_projektu.pdf'

create_tree_pdf(folder_zrodlowy, nazwa_pdf)
```

PLIK: app_admin\config.py

```
import os

class Config:

    # Osobne nazwy ciasteczek dla admina i studenta, żeby nie "nadpisywały się" nazzajem na localhost

    SESSION_COOKIE_NAME = 'admin_session'

    # Podstawowe zabezpieczenie sesji i formularzy WTF (zmień na produkcji!)

    SECRET_KEY = os.environ.get('SECRET_KEY', 'super-tajny-klucz-tylko-na-dev-xd')

    # Konfiguracja połączenia z bazą danych – sterownik psycopg2 (C, wydajny)

    SQLALCHEMY_DATABASE_URI = os.environ.get(

        'DATABASE_URL',

        'postgresql+psycopg2://ans_admin:secure_password_123@localhost:5432/ans_praktyki'

    )

    # Wyłączamy system zdarzeń SQLAlchemy, którego nie używamy (oszczędza pamięć)

    SQLALCHEMY_TRACK_MODIFICATIONS = False


class DevelopmentConfig(Config):

    DEBUG = True

    ENV = 'development'


class ProductionConfig(Config):

    DEBUG = False

    ENV = 'production'

    # Na produkcji wymusimy silniejsze zabezpieczenia ciasteczek (tylko HTTPS)

    SESSION_COOKIE_SECURE = True


# Słownik ułatwiający Fabryce Aplikacji wybór odpowiedniej klasy

config_dict = {
```

```
'development': DevelopmentConfig,  
  
'production': ProductionConfig,  
  
'default': DevelopmentConfig  
  
}
```

PLIK: app_admin\Dockerfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \

    PYTHONUNBUFFERED=1 \

    PYTHONPATH=/app

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \

    curl \

    && rm -rf /var/lib/apt/lists/*

COPY app_admin/requirements.txt requirements.txt

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Bezpieczeństwo: dedykowany nieuprzywilejowany użytkownik (nie root)

RUN addgroup --system appgroup \

    && adduser --system --ingroup appgroup --no-create-home appuser \

    && chown -R appuser:appgroup /app

USER appuser

EXPOSE 5000

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app_admin:create_app()"]
```

PLIK: app_admin__init__.py

```
import os

from flask import Flask

from jinja2 import select_autoescape

from core.extensions import db, login_manager

from core.error_handlers import register_error_handlers

import logging

from logging.handlers import RotatingFileHandler

from pathlib import Path


def create_app():

    from pathlib import Path as _Path

    from flask import Blueprint as _Blueprint

    app = Flask(__name__)

    # — Core static assets (CSS design system + templates) —————

    core_bp = _Blueprint(

        'core_static', __name__,

        static_folder=str(_Path(__file__).parent.parent / 'core' / 'static'),

        static_url_path='/core/static',

        template_folder=str(_Path(__file__).parent.parent / 'core' / 'templates'),

    )

    app.register_blueprint(core_bp)

    app.jinja_options = app.jinja_options.copy()

    app.jinja_options.update(dict(

        autoescape=select_autoescape(['html', 'xml'])

    ))

    # Configure file-based logging so exceptions are captured when console is silent

    logs_dir = Path(app.root_path) / 'logs'

    try:

        logs_dir.mkdir(exist_ok=True)
```

```

file_handler = RotatingFileHandler(logs_dir / 'app.log', maxBytes=1000000, backupCount=5)

file_handler.setLevel(logging.INFO)

formatter = logging.Formatter('%(asctime)s %(levelname)s: %(message)s [in %(pathname)s:%(lineno)d]')

file_handler.setFormatter(formatter)

app.logger.addHandler(file_handler)

app.logger.setLevel(logging.INFO)

except Exception:

    pass


from app_admin.config import config_dict

env = os.environ.get('FLASK_ENV', 'development')

app.config.from_object(config_dict.get(env, config_dict['default']))


db.init_app(app)

login_manager.init_app(app)

register_error_handlers(app)


login_manager.login_view = 'auth.logowanie'

# Polish translation for Flask-Login unauthorized message

login_manager.login_message = 'Zaloguj się, aby uzyskać dostęp do tej strony.'

login_manager.login_message_category = 'warning'

with app.app_context():

    from core.autoryzacja import stworz_blueprint_auth, wymaga_rol

    from core.pliki import stworz_blueprint_pliki

    from core.modele import RolaUzytkownika

    from app_admin.routes.pulpit import dashboard_bp

    from app_admin.routes.zarzadzanie import zarzadzanie_bp

    from app_admin.routes.ocenianie import evaluation_bp

    from app_admin.routes.dziennik import journal_bp

    from app_admin.routes.dokumenty import documents_bp


auth_bp = stworz_blueprint_auth(

    dozwolone_role=[RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ],

    template_logowania='auth/logowanie.html',

```

```

        template_zmiany_hasla='auth/zmien_haslo.html',

    )

    uploads_bp = stworz_blueprint_pliki()

    app.register_blueprint(auth_bp)

    app.register_blueprint(dashboard_bp, url_prefix='/panel')

    app.register_blueprint(zarzadzanie_bp, url_prefix='/zarzadzanie')

    app.register_blueprint(evaluation_bp, url_prefix='/oceny')

    app.register_blueprint(journal_bp, url_prefix='/dzienniki')

    app.register_blueprint(documents_bp, url_prefix='/dokumenty')

    app.register_blueprint(uploads_bp, url_prefix='/uploads')


@app.context_processor
def inject_nav_counts():

    counts = {}

    try:

        from flask_login import current_user

        if current_user.is_authenticated:

            from core.modele import ZapisPraktyki, StatusZapisu

            counts['nav_oczekujace'] = db.session.query(ZapisPraktyki).filter(

                ZapisPraktyki.status == StatusZapisu.AWAITING_APPROVAL

            ).count()

            counts['nav_komisja'] = db.session.query(ZapisPraktyki).filter(

                ZapisPraktyki.status == StatusZapisu.COMMISSION_REVIEW

            ).count()

            counts['nav_dziekan'] = db.session.query(ZapisPraktyki).filter(

                ZapisPraktyki.status == StatusZapisu.DEAN_APPROVAL

            ).count()

        except Exception:

            pass

    return counts


@app.context_processor
def inject_tlumacz():

```

```
return dict(tlumacz_status=lambda val: {

    'PENDING': 'Oczekuje',

    'IN_PROGRESS': 'W trakcie',

    'COMPLETED': 'Zakończona',

    'ACTIVE': 'Aktywna',

    'INACTIVE': 'Nieaktywna',

    'AWAITING_APPROVAL': 'Oczekuje na akceptację',

    'COMMISSION_REVIEW': 'Weryfikacja komisji',

    'DEAN_APPROVAL': 'Oczekuje na dziekana',

    'REJECTED': 'Odrzucony'

}).get(val, val))
```

```
@app.before_request
```

```
def sprawdz_zmiane_hasla():
```

```
    from flask import request, redirect, url_for
```

```
    from flask_login import current_user
```

```
    if (current_user.is_authenticated
```

```
        and getattr(current_user, 'wymagana_zmiana_hasla', False)
```

```
        and request.endpoint not in ('auth.zmien_haslo', 'auth.wylogowanie', 'static')
```

```
        and not (request.endpoint and request.endpoint.endswith('.static'))):
```

```
        return redirect(url_for('auth.zmien_haslo'))
```

```
return app
```

```
@login_manager.user_loader
```

```
def wczytaj_uzytkownika(user_id):
```

```
    from core.modele import Uzytkownik
```

```
    return db.session.get(Uzytkownik, user_id)
```


PLIK: app_admin\routes\dokumenty.py

```
import uuid

import json

import time

from flask import Blueprint, render_template, redirect, url_for, flash, request, jsonify, abort, send_file, current_app, Response

from flask_login import login_required, current_user

from pathlib import Path

from core.modele import ZapisPraktyki, DocumentAuditLog, RolaUzytkownika

from core.extensions import db

from core.autoryzacja import wymaga_rola

import logging

logger = logging.getLogger(__name__)

documents_bp = Blueprint('documents', __name__)

DOCUMENTS = {

    'ZAL_1': ('zal1_porozumienie.tex.j2', 'Załącznik 1 (Porozumienie)'),

    'ZAL_2a': ('zal2a_harmonogram.tex.j2', 'Załącznik 2a (Harmonogram Uczelni)'),

    'ZAL_3': ('zal3_skierowanie.tex.j2', 'Załącznik 3 (Skierowanie i ocena ZOPZ)'),

    'ZAL_4': ('zal4_efekty.tex.j2', 'Załącznik 4 (Efekty Ucznia)'),

    'ZAL_4a': ('zal4a_komisja.tex.j2', 'Załącznik 4a (Opinia Komisji do weryfikacji)'),

    'ZAL_4b': ('zal4b_wniosek.tex.j2', 'Załącznik 4b (Wniosek o działalność gospodarczą)'),

    'ZAL_6': ('zal6_dziennik.tex.j2', 'Załącznik 6 (Dziennik Praktyki)'),

    'ZAL_7': ('zal7_sprawozdanie.tex.j2', 'Załącznik 7 (Sprawozdanie Instytucji)'),

    'ZAL_7a': ('zal7a_wniosek.tex.j2', 'Załącznik 7a (Wniosek o pracę zawodową)'),

    'ZAL_8': ('zal8_protokol.tex.j2', 'Załącznik 8 (Protokół Egzaminacyjny)'),

    'ZAL_9': ('zal9_zobowiazanie.tex.j2', 'Załącznik 9 (Zobowiązanie poufności)'),

}

def log_audit(zapis_id, doc_type, action, details=""):

    db.session.add(DocumentAuditLog(
```

```

        id=uuid.uuid4(),

        user_id=current_user.id,

        enrollment_id=zapis_id,

        document_type=doc_type,

        action=action,

        details=details

    ))

    db.session.commit()

@documents_bp.route('/zapis/<uuid:id>')

@wymaga_rol(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def panel(id):

    zapis = db.session.get(ZapisPraktyki, id) or abort(404)

    logs = db.session.query(DocumentAuditLog).filter_by(zapis_id=id).order_by(DocumentAuditLog.wykonano_o.desc()).limit(20).all()

    return render_template('documents/panel.html', zapis=zapis, docs=DOCUMENTS, logs=logs)

@documents_bp.route('/zapis/<uuid:id>/edytuj/<doc_type>', methods=['GET'])

@wymaga_rol(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def edytuj(id, doc_type):

    zapis = db.session.get(ZapisPraktyki, id) or abort(404)

    if doc_type not in DOCUMENTS: abort(404)

    template_name, doc_title = DOCUMENTS[doc_type]

    from tex_service.compiler import render_tex

    from tex_service.pdf_service import pdf_service

    context = {}

    if doc_type == 'ZAL_6':

        context = pdf_service.build_context_dziennik(zapis)

    elif doc_type == 'ZAL_4':

        context = pdf_service.build_context_efekty(zapis)

    elif doc_type == 'ZAL_7':

        tresc = {'charakterystyka_miejsca': zapis.sprawozdanie.charakterystyka_miejsca if zapis.sprawozdanie else '',

```

```

        'opis_prac': zapis.sprawozdanie.opis_i_analiza if zapis.sprawozdanie else '',

        'efekty_opisy': ['' for _ in range(13)]}

    context = pdf_service.build_context_sprawozdanie(zapis, tresc)

else:

    context = {'student': zapis.student, 'firma': zapis.dane_miejsca, 'zapis': zapis}

try:

    raw_tex = render_tex(template_name, context)

except Exception as e:

    raw_tex = f"% BŁĄD RENDEROWANIA SZABLONU: {str(e)}\n\n% Edytuj dokument od zera jeśli wymagane.\n"

log_audit(zapis.id, doc_type, 'VIEWED_MANUAL', 'Otwarto edytor ręczny LaTeX')

return render_template('documents/edytor.html', zapis=zapis, raw_tex=raw_tex, doc_type=doc_type, doc_title=doc_title)

@documents_bp.route('/zapis/<uuid:id>/generuj_auto/<doc_type>', methods=['POST'])

@wymaga_rol(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def generuj_auto(id, doc_type):

    from tex_service.compiler import render_tex

    from tex_service.pdf_service import pdf_service

    from celery_app import compile_raw_tex_task

    zapis = db.session.get(ZapisPraktyki, id) or abort(404)

    if doc_type not in DOCUMENTS: abort(404)

    template_name, doc_title = DOCUMENTS[doc_type]

    context = {}

    if doc_type == 'ZAL_6':

        context = pdf_service.build_context_dziennik(zapis)

    elif doc_type == 'ZAL_4':

        context = pdf_service.build_context_efekty(zapis)

    elif doc_type == 'ZAL_7':

        tresc = {'charakterystyka_miejsca': zapis.sprawozdanie.charakterystyka_miejsca if zapis.sprawozdanie else '',

                'opis_prac': zapis.sprawozdanie.opis_i_analiza if zapis.sprawozdanie else '',

                'efekty_opisy': ['' for _ in range(13)]}

```

```

        context = pdf_service.build_context_sprawozdanie(zapis, tresc)

    else:

        context = {'student': zapis.student, 'firma': zapis.dane_miejsca, 'zapis': zapis}

    try:

        raw_tex = render_tex(template_name, context)

        task = compile_raw_tex_task.delay(raw_tex, doc_type.lower())

        log_audit(zapis.id, doc_type, 'GENERATED_AUTO', f'Wygenerowano {doc_type}')

        return jsonify({'task_id': task.id, 'status': 'PENDING'})

    except Exception as e:

        return jsonify({'error': str(e)}), 500


@documents_bp.route('/zapis/<uuid:id>/generuj_zip', methods=['POST'])

@wymaga_rol(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def generuj_zip(id):

    zapis = db.session.get(ZapisPraktyki, id) or abort(404)

    try:

        from celery_app import generate_zip_task

        task = generate_zip_task.delay(str(zapis.id))

        log_audit(zapis.id, 'PAKIET_ZIP', 'GENERATED_AUTO', 'Wygenerowano komplet dokumentów (ZIP)')

        return jsonify({'task_id': task.id, 'status': 'PENDING'})

    except Exception as e:

        return jsonify({'error': str(e)}), 500


@documents_bp.route('/zapis/<uuid:id>/kompiluj/<doc_type>', methods=['POST'])

@wymaga_rol(RolaUzytkownika.ADMIN)

def kompiluj_recznie(id, doc_type):

    tex_source = request.form.get('tex_source')

    if not tex_source: abort(400)

    log_audit(id, doc_type, 'COMPILED_MANUAL', f'Rozpoczęto kompilację ręczną {doc_type}')

    try:

        from celery_app import compile_raw_tex_task

        task = compile_raw_tex_task.delay(tex_source, doc_type.lower())

        return jsonify({'task_id': task.id, 'status': 'PENDING'})

```

```
except Exception as e:
```

```
    return jsonify({'error': str(e)}), 500
```

```
@documents_bp.route('/status/<task_id>')
```

```
@wymaga_rola(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)
```

```
def status_pdf(task_id):
```

```
    """Legacy polling endpoint – zachowany dla kompatybilności wstecznej."""
```

```
    try:
```

```
        from celery_app import celery
```

```
        task = celery.AsyncResult(task_id)
```

```
        if task.state == 'SUCCESS':
```

```
            res = task.result
```

```
            if isinstance(res, dict) and res.get('status') == 'error':
```

```
                return jsonify({'status': 'FAILURE', 'error': res.get('message')})
```

```
                return jsonify({'status': 'SUCCESS', 'download_url': url_for('documents.pobierz_pdf', task_id=task_id)})
```

```
        elif task.state == 'FAILURE':
```

```
            return jsonify({'status': 'FAILURE', 'error': str(task.info)})
```

```
        else:
```

```
            progress = task.info.get('progress', 0) if task.info else 0
```

```
            return jsonify({'status': task.state, 'progress': progress})
```

```
    except Exception as e:
```

```
        return jsonify({'error': str(e)}), 500
```

```
@documents_bp.route('/stream/<task_id>')
```

```
@wymaga_rola(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)
```

```
def stream_status(task_id):
```

```
    """Server-Sent Events endpoint."""
```

```
    from celery_app import celery
```

```
def generate():
```

```
    while True:
```

```
        try:
```

```
            task = celery.AsyncResult(task_id)
```

```
            if task.state == 'SUCCESS':
```

```

        res = task.result

        if isinstance(res, dict) and res.get('status') == 'error':

            data = {'status': 'FAILURE', 'error': res.get('message', 'Nieznany błąd')}

        else:

            data = {'status': 'SUCCESS', 'download_url': url_for('documents.pobierz_pdf', task_id=task_id)}

        yield f"data: {json.dumps(data)}\n\n"

        return

    elif task.state == 'FAILURE':

        data = {'status': 'FAILURE', 'error': str(task.info)}

        yield f"data: {json.dumps(data)}\n\n"

        return

    else:

        progress = task.info.get('progress', 0) if task.info else 0

        data = {'status': task.state, 'progress': progress}

        yield f"data: {json.dumps(data)}\n\n"

except GeneratorExit:

    return

except Exception as exc:

    logger.error("SSE stream error for task %s: %s", task_id, exc)

    yield f"data: {json.dumps({'status': 'FAILURE', 'error': str(exc)})}\n\n"

    return

time.sleep(1)

return Response(generate(), mimetype='text/event-stream',

                headers={'Cache-Control': 'no-cache', 'X-Accel-Buffering': 'no'})

@documents_bp.route('/pobierz/<task_id>')

@wymaga_rola(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def pobierz_pdf(task_id):

    try:

        from celery_app import celery

        task = celery.AsyncResult(task_id)

        if task.state != 'SUCCESS':

            abort(404)

```

```
result = task.result

if isinstance(result, dict) and result.get('status') == 'error':

    abort(500)

pdf_path = Path(result['path'])

if not pdf_path.exists():

    abort(404)

return send_file(pdf_path, mimetype='application/pdf', as_attachment=True, download_name=result['filename'])

except Exception:

    abort(500)
```

PLIK: app_admin\routes\dziennik.py

```
"""

app_admin/routes/dziennik.py

Monitor dzienników praktyk – podgląd wpisów, wykrywanie przerw, generowanie PDF.

Przemianowano z journal.py.

"""

import io

from datetime import date, timedelta

from flask import Blueprint, render_template, request, abort, send_file

from flask_login import login_required, current_user

from sqlalchemy import func

from core.modele import (ZapisPraktyki, WpisDziennika, Uzytkownik,
                          RolaUzytkownika, StatusZapisu)

from core.extensions import db

journal_bp = Blueprint('journal', __name__)

def _wykryj_przerwy(wpisy, prog_dni=3):

    """Wykrywa przerwy dłuższe niż prog_dni roboczych między wpisami."""

    if not wpisy:

        return []

    daty = sorted(w.entry_date for w in wpisy)

    przerwy = []

    for i in range(1, len(daty)):

        delta = 0

        d = daty[i - 1]

        while d < daty[i]:

            d += timedelta(days=1)

            if d.weekday() < 5:

                delta += 1

        if delta > prog_dni:

            przerwy.append({'od': daty[i - 1], 'do': daty[i], 'dni_roboczych': delta})
```



```
return przerwy
```

```
@journal_bp.route('/')
```

```
@login_required
```

```
def lista_dziennikow():
```

```
    szukaj = request.args.get('szukaj', '').strip()
```

```
    strona = request.args.get('strona', 1, type=int)
```

```
    q = db.session.query(ZapisPraktyki).filter(ZapisPraktyki.status == StatusZapisu.IN_PROGRESS)
```

```
    if current_user.role == RolaUzytkownika.UOPZ:
```

```
        q = q.filter_by(uopz_id=current_user.id)
```

```
    if szukaj:
```

```
        wzorzec = f'{szukaj}%'
```

```
        q = q.join(Uzytkownik, ZapisPraktyki.student_id == Uzytkownik.id).filter(
```

```
            db.or_(Uzytkownik.first_name.ilike(wzorzec), Uzytkownik.last_name.ilike(wzorzec))
```

```
        )
```

```
    zapisy = q.order_by(ZapisPraktyki.enrolled_at.desc()).paginate(page=strona, per_page=20, error_out=False)
```

```
    dane = []
```

```
    for z in zapisy.items:
```

```
        ostatni = db.session.query(func.max(WpisDziennika.data_wpisu)).filter_by(zapis_id=z.id).scalar()
```

```
        liczba_wpisow = db.session.query(func.count(WpisDziennika.id)).filter_by(zapis_id=z.id).scalar()
```

```
        alert = False
```

```
        if ostatni:
```

```
            delta = 0
```

```
            d = ostatni
```

```
            dzisiaj = date.today()
```

```
            while d < dzisiaj:
```

```
                d += timedelta(days=1)
```

```
        if d.weekday() < 5:
```

```
            delta += 1
```

```
        alert = delta > 3
```

```
        dane.append({'zapis': z, 'liczba_wpisow': liczba_wpisow, 'ostatni_wpis': ostatni, 'alert': alert})
```

```
    return render_template('journal/lista.html', dane=dane, zapisy=zapisy)
```

```
@journal_bp.route('/zapis/<uuid:id>')
```

```
@login_required
```

```
def dziennik_zapisu(id):
```

```
    zapis = db.session.get(ZapisPraktyki, id) or abort(404)
```

```
    wpisy = db.session.query(WpisDziennika)\
```

```
        .filter_by(zapis_id=id)\
```

```
        .order_by(WpisDziennika.data_wpisu.desc())\
```

```
        .all()
```

```
    przerwy = _wykryj_przerwy(wpisy)
```

```
    suma_godzin = sum(w.duration_hours for w in wpisy)
```

```
    return render_template('journal/dziennik.html', zapis=zapis, wpisy=wpisy,
```

```
        przerwy=przerwy, suma_godzin=suma_godzin)
```

```
@journal_bp.route('/zapis/<uuid:id>/pdf')
```

```
@login_required
```

```
def pdf_dziennik(id):
```

```
    zapis = db.session.get(ZapisPraktyki, id) or abort(404)
```

```
    wpisy = db.session.query(WpisDziennika)\
```

```
        .filter_by(zapis_id=id)\
```

```
        .order_by(WpisDziennika.data_wpisu)\
```

```
        .all()
```

```
    try:
```

```
        from tex_service.pdf_service import pdf_service
```

```

class _FakePraktyka:

    def __init__(self, z, w):

        self.id = z.id

        self.student = z.student

        self.zaklad = None

        self.start_date = z.enrolled_at.date() if z.enrolled_at else date.today()

        self.end_date = date.today()

        self.path_type = None

        self.total_hours_logged = z.total_hours_logged

        self.wpisy_dziennika = w

fake = _FakePraktyka(zapis, wpisy)

pdf_bytes = pdf_service.get_dziennik(fake)

nazwa = f"dziennik_{zapis.student.last_name}_{zapis.student.first_name}.pdf"

return send_file(io.BytesIO(pdf_bytes), mimetype='application/pdf',

                  as_attachment=True, download_name=nazwa)

except EnvironmentError:

    abort(503)

except Exception:

    abort(500)

```

PLIK: app_admin\routes\ocenie.py

```
"""

app_admin/routes/ocenie.py

Oceny efektów uczenia się – operuje na ZapisPraktyki (enrollment).

Przemianowano z evaluation.py.

"""

import uuid

from flask import Blueprint, render_template, redirect, url_for, flash, request, abort

from flask_login import login_required, current_user

from core.modele import (ZapisPraktyki, OcenaPraktyki, EfektUczenia,
                          RolaUzytkownika, StatusZapisu, WynikOceny)

from core.extensions import db

from core.autoryzacja import wymaga_rol

evaluation_bp = Blueprint('evaluation', __name__)

def _parse_grade(val: str):
    """Konwertuje ocenę z formularza na float, obsługując przecinki i kropki."""
    if not val or not val.strip():
        return None

    try:
        return float(val.strip().replace(',', '.'))
    except (ValueError, AttributeError):
        return None

from core.uslugi import SerwisOceniania

def get_pilne_oceny(uopz_id=None):
    return SerwisOceniania.get_pilne_oceny(uopz_id)
```

```

@evaluation_bp.route('/')

@login_required

def lista_ocen():

    SerwisOceniania.auto_complete_internships()

    q = db.session.query(ZapisPraktyki).filter(

        ZapisPraktyki.status.in_([StatusZapisu.IN_PROGRESS, StatusZapisu.COMPLETED])

    )

    if current_user.role == RolaUzytkownika.UOPZ:

        q = q.filter_by(uopz_id=current_user.id)

    zapisy = q.join(ZapisPraktyki.student).order_by(

        ZapisPraktyki.status.desc(),

        ZapisPraktyki.enrolled_at.desc()

    ).all()

    from datetime import date, timedelta

    zapisy_z_deadlinami = []

    for zapis in zapisy:

        deadline = None

        dni_do_deadline = None

        przekroczony = False

        if zapis.termin_do and zapis.status == StatusZapisu.COMPLETED:

            deadline = zapis.termin_do + timedelta(days=7)

            dni_do_deadline = (deadline - date.today()).days

            przekroczony = dni_do_deadline < 0

    zapisy_z_deadlinami.append({

        'zapis': zapis,

        'deadline': deadline,

```

```

'dni_do_deadline': dni_do_deadline,

'przekroczony': przekroczony,

'w_trakcie': zapis.status == StatusZapisu.IN_PROGRESS,

'zakonczone': zapis.status == StatusZapisu.COMPLETED,

})

```

```

w_trakcie = [z for z in zapisy_z_deadlinami if z['w_trakcie']]

zakonczone = [z for z in zapisy_z_deadlinami if z['zakonczone']]

```

```

return render_template('evaluation/lista_ocen.html',

                        zapisy_z_deadlinami=zapisy_z_deadlinami,

                        w_trakcie=w_trakcie,

                        zakonczone=zakonczone)

```

```

@evaluation_bp.route('/zapis/<uuid:id>/karta_ocen', methods=['GET', 'POST'])

```

```

@wymaga_rol(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

```

```

def ocen_praktyke(id):

```

```

    zapis = db.session.get(ZapisPraktyki, id) or abort(404)

```

```

    if request.method == 'POST':

```

```

        from core.modele import OcenyKoncowe, Sprawdzian as SprawdzianModel

```

```

        ok = zapis.oceny_koncowe or OcenyKoncowe(zapis_id=zapis.id)

```

```

        if ok not in db.session:

```

```

            db.session.add(ok)

```

```

        ok.ocena_opisowa_uopz = request.form.get('ocena_opisowa_uopz')

```

```

        ok.ocena_opisowa_zopz = request.form.get('ocena_opisowa_zopz')

```

```

        ok.ocena_sprawozdania = _parse_grade(request.form.get('ocena_sprawozdania', ''))

```

```

        ok.ocena_uopz = _parse_grade(request.form.get('ocena_uopz', ''))

```

```

        ok.ocena_zopz = _parse_grade(request.form.get('ocena_zopz', ''))

```

```

        sp = zapis.sprawdzian or SprawdzianModel(zapis_id=zapis.id)

```

```

        if sp not in db.session:

```

```

            db.session.add(sp)

```

```

        sp.pytanie_1 = request.form.get('sprawdzian_pytanie_1')

```

```

        sp.ocena_1 = _parse_grade(request.form.get('sprawdzian_ocena_1', ''))

```

```

sp.pytanie_2 = request.form.get('sprawdzian_pytanie_2')

sp.ocena_2 = _parse_grade(request.form.get('sprawdzian_ocena_2', ''))

sp.pytanie_3 = request.form.get('sprawdzian_pytanie_3')

sp.ocena_3 = _parse_grade(request.form.get('sprawdzian_ocena_3', ''))


if request.form.get('zakonczone'):

    zapis.status = StatusZapisu.COMPLETED

    db.session.commit()

    flash('Oceny zostały zapisane.', 'success')

    return redirect(url_for('evaluation.ocen_praktyke', id=zapis.id))


from flask_wtf import FlaskForm

csrf_form = FlaskForm()

return render_template('evaluation/karta_ocen.html', practically=zapis, zapis=zapis, csrf_form=csrf_form)


@evaluation_bp.route('/zapis/<uuid:id>/sprawozdanie')

@wymaga_rola(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def podglad_sprawozdania(id):

    zapis = db.session.get(ZapisPraktyki, id) or abort(404)

    return render_template('evaluation/podglad_sprawozdania.html', zapis=zapis)


@evaluation_bp.route('/zapis/<uuid:id>', methods=['GET', 'POST'])

@wymaga_rola(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def ocen_zapis(id):

    zapis = db.session.get(ZapisPraktyki, id) or abort(404)

    efekty = db.session.query(EfektUczenia).order_by(EfektUczenia.id).all()

    istniejace = {

        str(o.learning_outcome_id): o

        for o in db.session.query(OcenaPraktyki).filter_by(zapis_id=id).all()

    }

    if request.method == 'POST':

        for efekt in efekty:

```

```

wynik_str = request.form.get(f'wynik_{efekt.id}')

uwagi      = request.form.get(f'uwagi_{efekt.id}', '').strip()

if not wynik_str:

    continue

try:

    wynik = WynikOceny[wynik_str]

except KeyError:

    continue

ocena = istniejace.get(str(efekt.id))

if ocena:

    ocena.result          = wynik

    ocena.evaluator_notes = uwagi or None

else:

    db.session.add(OcenaPraktyki(

        id                  = uuid.uuid4(),

        enrollment_id       = zapis.id,

        learning_outcome_id = efekt.id,

        result              = wynik,

        evaluator_notes     = uwagi or None,

    ))

db.session.commit()

flash('Oceny zostały zapisane.', 'success')

return redirect(url_for('evaluation.ocen_zapis', id=id))


return render_template('evaluation/formularz_ocen.html',

                       zapis=zapis, efekty=efekty, istniejace=istniejace)


@evaluation_bp.route('/zapis/<uuid:id>/zakonc', methods=['POST'])

@wymaga_rol(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def zakonc_zapis(id):

    zapis = db.session.get(ZapisPraktyki, id) or abort(404)

    zapis.status = StatusZapisu.COMPLETED

    db.session.commit()

    flash(f'Praktyka studenta {zapis.student.first_name} {zapis.student.last_name} została zakończona.', 'success')

```



```
return redirect(url_for('evaluation.lista_ocen'))
```

```
@evaluation_bp.route('/zapis/<uuid:id>/protokol')
```

```
@wymaga_rola(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)
```

```
def generuj_protokol(id):
```

```
    """Generuje Protokół egzaminu (zał.8) przez tex-service."""
```

```
    import httpx, unicodedata
```

```
    from flask import make_response, current_app
```

```
    from datetime import date
```

```
    zapis = db.session.get(ZapisPraktyki, id) or abort(404)
```

```
    ok = zapis.oceny_koncowe
```

```
    sp = zapis.sprawdzian
```

```
    if not (ok and ok.ocena_uopz):
```

```
        flash('Protokół dostępny dopiero po wystawieniu oceny UOPZ.', 'warning')
```

```
        return redirect(url_for('evaluation.ocen_praktyke', id=id))
```

```
    s = zapis.student
```

```
    tex_url = current_app.config.get('TEX_SERVICE_URL', 'http://tex-service:5002')
```

```
    dm = zapis.dane_miejsca
```

```
    firma_nazwa = (zapis.firma.nazwa if zapis.firma else (dm.firma_nazwa if dm else None)) or ''
```

```
def _f(v):
```

```
    return float(v) if v is not None else None
```

```
ctx = {
```

```
    'zapis': {
```

```
        'firma_nazwa': firma_nazwa,
```

```
        'termin_od': zapis.termin_od.strftime('%d.%m.%Y') if zapis.termin_od else '',
```

```
        'termin_do': zapis.termin_do.strftime('%d.%m.%Y') if zapis.termin_do else '',
```

```
        'ocena_sprawozdania': _f(ok.ocena_sprawozdania if ok else None),
```

```
        'ocena_uopz': _f(ok.ocena_uopz if ok else None),
```

```
        'ocena_zopz': _f(ok.ocena_zopz if ok else None),
```

```
        'sprawdzian_pytanie_1': sp.pytanie_1 if sp else None,
```

```

'sprawdzian_ocena_1': _f(sp.ocena_1 if sp else None),

'sprawdzian_pytanie_2': sp.pytanie_2 if sp else None,

'sprawdzian_ocena_2': _f(sp.ocena_2 if sp else None),

'sprawdzian_pytanie_3': sp.pytanie_3 if sp else None,

'sprawdzian_ocena_3': _f(sp.ocena_3 if sp else None),

'uopz': {'first_name': zapis.uopz.first_name, 'last_name': zapis.uopz.last_name} if zapis.uopz else None,
},

'student': {

    'imie': s.first_name, 'nazwisko': s.last_name,

    'first_name': s.first_name, 'last_name': s.last_name,

    'nr_albumu': s.album_number or '', 'album_number': s.album_number or '',

    'kierunek': getattr(s, 'kierunek', 'Informatyka') or 'Informatyka',
},

'specjalnosc': getattr(s, 'specjalnosc', '') or getattr(zapis, 'specjalnosc', '') or '',

'praktyka': {

    'rok_uczelnianny': zapis.praktyka.rok_uczelnianny if zapis.praktyka else '',

    'semestr': zapis.praktyka.semestr if zapis.praktyka else '',

    'wymiar_godzin': zapis.praktyka.wymiar_godzin if zapis.praktyka else 960,
},

'data_egzaminu': date.today().strftime('%d.%m.%Y'),
}

try:

    r = httpx.post(f'{tex_url}/generuj',

                  json={'template': 'zal8_protokol.tex.j2', 'context': ctx, 'filename': 'zal8_protokol.pdf'},

                  timeout=60)

    if r.status_code == 200:

        safe = unicodedata.normalize('NFKD', s.last_name).encode('ascii', 'ignore').decode('ascii') or 'student'

        resp = make_response(r.content)

        resp.headers['Content-Type'] = 'application/pdf'

        resp.headers['Content-Disposition'] = f'attachment; filename="zal8_protokol_{safe}.pdf"'

        return resp

    flash(f'Błąd generowania protokołu: {r.text[:200]}', 'danger')

except Exception as e:

    flash(f'Błąd połączenia z tex-service: {str(e)}', 'danger')

```

```
return redirect(url_for('evaluation.ocen_praktyke', id=id))
```

PLIK: app_admin\routes\pdf.py

```
"""
```

```
app_admin/routes/pdf.py
```

Blueprint Flask do pobierania wygenerowanych PDF-ów.

Rejestracja w app_admin/__init__.py:

```
from routes.pdf import pdf_bp

app.register_blueprint(pdf_bp)
```

```
"""
```

```
from flask import Blueprint, send_file, abort, current_app
```

```
from flask_login import login_required, current_user
```

```
from tex_service.pdf_service import pdf_service
```

```
from tex_service.compiler import TexCompilationError
```

```
from core.modele import Praktyka, RolaUzytkownika
```

```
pdf_bp = Blueprint("pdf", __name__, url_prefix="/pdf")
```

```
def _get_praktyka_or_403(praktyka_id: str):
```

```
    """
```

```
    Pobiera praktykę z bazy i sprawdza prawa dostępu.
```

```
    Student widzi tylko własne praktyki, UOPZ/ADMIN widzą wszystkie.
```

```
    """
```

```
    praktyka = Praktyka.query.get_or_404(praktyka_id)
```

```
    if current_user.role == RolaUzytkownika.STUDENT:
```

```
        if str(praktyka.student_id) != str(current_user.id):
```

```
            abort(403)
```

```
    elif current_user.role == RolaUzytkownika.UOPZ:
```

```
        if str(praktyka.uopz_id) != str(current_user.id):
```

```

        abort(403)

# ADMIN widzi wszystko


return praktyka


@pdf_bp.route("/dziennik/<uuid:praktyka_id>")

@login_required

def download_dziennik(praktyka_id):

    """GET /pdf/dziennik/<uuid> → Załącznik 6 - Dziennik praktyki zawodowej."""

    praktyka = _get_praktyka_or_403(str(praktyka_id))

    try:

        pdf_bytes = pdf_service.get_dziennik(praktyka)

    except TexCompilationError as e:

        current_app.logger.error("Błąd kompilacji PDF (dziennik): %s\nLog:\n%s", e, e.log)

        abort(500, description="Błąd generowania dziennika. Skontaktuj się z administratorem.")

    return send_file(

        io.BytesIO(pdf_bytes),

        mimetype="application/pdf",

        as_attachment=True,

        download_name=f"dziennik_{praktyka.student.album_number}.pdf",

    )


@pdf_bp.route("/efekty/<uuid:praktyka_id>")

@login_required

def download_efekty(praktyka_id):

    """GET /pdf/efekty/<uuid> → Załącznik 4 - Potwierdzenie efektów uczenia."""

    praktyka = _get_praktyka_or_403(str(praktyka_id))

    try:

        pdf_bytes = pdf_service.get_efekty(praktyka)

    except TexCompilationError as e:

        current_app.logger.error("Błąd kompilacji PDF (efekty): %s\nLog:\n%s", e, e.log)

```

```

        abort(500, description="Błąd generowania dokumentu efektów.")

    return send_file(

        io.BytesIO(pdf_bytes),

        mimetype="application/pdf",

        as_attachment=True,

        download_name=f"efekty_{praktyka.student.album_number}.pdf",

    )

@pdf_bp.route("/sprawozdanie/<uuid:praktyka_id>")
@login_required
def download_sprawozdanie(praktyka_id):
    """
    GET /pdf/sprawozdanie/<uuid> → Załącznik 7 – Sprawozdanie studenta.

    Pobiera treść sprawozdania z JSONB kolumny tasks_scope lub

    z request.json jeśli wysłane POST-em.
    """

    praktyka = _get_praktyka_or_403(str(praktyka_id))

    # Treść sprawozdania trzymana w kolumnie JSONB tasks_scope

    tresc = praktyka.tasks_scope or {}

    try:

        pdf_bytes = pdf_service.get_sprawozdanie(praktyka, tresc)

    except TexCompilationError as e:

        current_app.logger.error("Błąd kompilacji PDF (sprawozdanie): %s\nLog:\n%s", e, e.log)

        abort(500, description="Błąd generowania sprawozdania.")

    return send_file(

        io.BytesIO(pdf_bytes),

        mimetype="application/pdf",

        as_attachment=True,

        download_name=f"sprawozdanie_{praktyka.student.album_number}.pdf",

```


PLIK: app_admin\routes\pliki.py

```
"""

app_admin/routes/pliki.py

Obsługa uploadu i pobierania plików dokumentów.

Przemianowano z uploads.py.

"""

import os

import uuid

from pathlib import Path

from werkzeug.utils import secure_filename

from flask import Blueprint, request, jsonify, send_from_directory, abort, flash, redirect, url_for

from flask_login import login_required, current_user


from core.modele import UploadedDocument, ZapisPraktyki, RolaUzytkownika

from core.extensions import db

from core.autoryzacja import wymaga_rol

uploads_bp = Blueprint('uploads', __name__)


UPLOAD_FOLDER = Path(__file__).parent.parent.parent / "uploads"

UPLOAD_FOLDER.mkdir(exist_ok=True)

MAX_FILE_SIZE = 10 * 1024 * 1024 # 10 MB

ALLOWED_EXTENSIONS = {'.pdf', '.doc', '.docx', '.jpg', '.jpeg', '.png', '.zip', '.rar'}

ALLOWED_MIME_TYPES = {

    'application/pdf',

    'application/msword',

    'application/vnd.openxmlformats-officedocument.wordprocessingml.document',

    'image/jpeg',

    'image/png',

    'application/zip',

    'application/x-rar-compressed',

    'application/vnd.rar',

}
```



```

def allowed_file(filename, content_type):

    if not filename:

        return False

    ext = Path(filename).suffix.lower()

    return ext in ALLOWED_EXTENSIONS and content_type in ALLOWED_MIME_TYPES


@uploads_bp.route('/enrollment/<uuid:enrollment_id>/upload', methods=['POST'])
@login_required
def upload_document(enrollment_id):

    zapis = db.session.get(ZapisPraktyki, enrollment_id)

    if not zapis:

        abort(404)

    if current_user.role != RolaUzytkownika.ADMIN:

        if current_user.role == RolaUzytkownika.UOPZ and zapis.uopz_id != current_user.id:

            abort(403)

    if 'file' not in request.files:

        return jsonify({'error': 'Brak pliku'}), 400

    file = request.files['file']

    document_type = request.form.get('document_type', '').strip()

    if not file.filename or not document_type:

        return jsonify({'error': 'Brak pliku lub typu dokumentu'}), 400

    file.seek(0, os.SEEK_END)

    file_size = file.tell()

    file.seek(0)

    if file_size > MAX_FILE_SIZE:

        return jsonify({'error': f'Plik zbyt duży. Maksymalny rozmiar: {MAX_FILE_SIZE // (1024*1024)}MB'}), 400

    content_type = file.content_type

    if not allowed_file(file.filename, content_type):

        return jsonify({'error': 'Niedozwolony typ pliku'}), 400

```

```

try:

    original_filename = secure_filename(file.filename)

    file_ext = Path(original_filename).suffix.lower()

    stored_filename = f"{uuid.uuid4().hex}{file_ext}"

    file_path = UPLOAD_FOLDER / stored_filename

    file.save(file_path)

    doc = UploadedDocument(

        enrollment_id      = enrollment_id,

        document_type      = document_type,

        original_filename  = original_filename,

        stored_filename    = stored_filename,

        file_path          = str(file_path),

        file_size          = file_size,

        mime_type          = content_type,

        uploaded_by_id    = current_user.id,

    )

    db.session.add(doc)

    db.session.commit()

    return jsonify({'success': True, 'document_id': str(doc.id),

                    'filename': original_filename, 'size': file_size})

except Exception as e:

    if 'file_path' in locals() and file_path.exists():

        file_path.unlink()

    db.session.rollback()

    return jsonify({'error': f'Błąd podczas zapisywania: {str(e)}'}), 500

```

```

@uploads_bp.route('/document/<uuid:document_id>/download')

```

```

@login_required

```

```

def download_document(document_id):

```

```

    doc = db.session.get(UploadedDocument, document_id)

```

```

if not doc:

    abort(404)

zapis = doc.enrollment

if current_user.role != RolaUzytkownika.ADMIN:

    if current_user.role == RolaUzytkownika.UOPZ and zapis.uopz_id != current_user.id:

        abort(403)

file_path = Path(doc.file_path)

if not file_path.exists():

    abort(404, "Plik nie został znaleziony na dysku")

return send_from_directory(file_path.parent, file_path.name,

                           as_attachment=True, download_name=doc.original_filename,

                           mimetype=doc.mime_type)


@uploads_bp.route('/document/<uuid:document_id>/delete', methods=['POST'])

@login_required

def delete_document(document_id):

    doc = db.session.get(UploadedDocument, document_id)

    if not doc:

        abort(404)

    zapis = doc.enrollment

    if current_user.rola != RolaUzytkownika.ADMIN:

        if current_user.rola == RolaUzytkownika.STUDENT and zapis.student_id != current_user.id:

            abort(403)

        elif current_user.rola == RolaUzytkownika.UOPZ:

            abort(403)

    try:

        file_path = Path(doc.file_path)

        if file_path.exists():

            file_path.unlink()

        db.session.delete(doc)

        db.session.commit()

        flash('Dokument został usunięty.', 'success')

    except Exception as e:

```

```

db.session.rollback()

flash(f'Błąd podczas usuwania dokumentu: {str(e)}', 'danger')

return redirect(request.referrer or url_for('dashboard.index'))


@uploads_bp.route('/enrollment/<uuid:enrollment_id>/documents')

@login_required

def list_documents(enrollment_id):

    zapis = db.session.get(ZapisPraktyki, enrollment_id)

    if not zapis:

        abort(404)

    if current_user.role != RolaUzytkownika.ADMIN:

        if current_user.role == RolaUzytkownika.UOPZ and zapis.uopz_id != current_user.id:

            abort(403)

    documents = db.session.query(UploadedDocument)\

        .filter_by(zapis_id=enrollment_id)\

        .order_by(UploadedDocument.przeslano_o.desc())\

        .all()

    return jsonify([

        'id':                str(doc.id),

        'document_type':     doc.document_type,

        'original_filename': doc.original_filename,

        'file_size':         doc.file_size,

        'mime_type':         doc.mime_type,

        'uploaded_at':       doc.uploaded_at.isoformat(),

        'uploaded_by':       f"{doc.uploaded_by.first_name} {doc.uploaded_by.last_name}" if doc.uploaded_by else None,

        'download_url':      url_for('uploads.download_document', document_id=doc.id),

        'delete_url':        url_for('uploads.delete_document', document_id=doc.id),

    ] for doc in documents])

```

PLIK: app_admin\routes\pulpit.py

```
from flask import Blueprint, render_template

from flask_login import login_required, current_user

from core.modele import Praktyka, ZapisPraktyki, Uzytkownik, RolaUzytkownika, StatusPraktyki, StatusZapisu

from core.extensions import db

from flask_wtf import FlaskForm


dashboard_bp = Blueprint('dashboard', __name__)


@dashboard_bp.route('/')
@login_required
def index():

    q = db.session.query(ZapisPraktyki)


    if current_user.role == RolaUzytkownika.UOPZ:

        q = q.filter(db.or_(ZapisPraktyki.uopz_id == current_user.id, ZapisPraktyki.status == StatusZapisu.PENDING))


    statystyki = {

        'praktyki_aktywne': q.filter(ZapisPraktyki.status == StatusZapisu.IN_PROGRESS).count(),

        'oczekujace_oceny': q.filter(ZapisPraktyki.status == StatusZapisu.COMPLETED).count(),

        'zakonczone':      db.session.query(Praktyka).filter_by(status=StatusPraktyki.NIEAKTYWNA).count(),

        'liczba_studentow': db.session.query(Uzytkownik).filter_by(

            rola=RolaUzytkownika.STUDENT, aktywny=True).count(),

    }


    ostatnie_zapisy = (q

        .order_by(ZapisPraktyki.enrolled_at.desc())

        .limit(8).all())


    # Dodaj ostrzeżenia o przekroczonych deadline'ach dla UOPZ

    pilne_oceny = []


    if current_user.role == RolaUzytkownika.UOPZ:

        from datetime import date, timedelta
```

```
completed_q = db.session.query(ZapisPraktyki).filter(  
  
    ZapisPraktyki.status == StatusZapisu.COMPLETED,  
  
    ZapisPraktyki.uopz_id == current_user.id  
  
)
```

```
for zapis in completed_q.all():  
  
    if zapis.termin_do:  
  
        deadline = zapis.termin_do + timedelta(days=7)  
  
        dni_do_deadline = (deadline - date.today()).days  
  
        if dni_do_deadline <= 2:  
  
            pilne_oceny.append({  
  
                'zapis': zapis,  
  
                'deadline': deadline,  
  
                'dni_do_deadline': dni_do_deadline,  
  
                'przekroczony': dni_do_deadline < 0  
  
            })
```

```
csrf_form = FlaskForm()
```

```
return render_template('dashboard/index.html',  
  
    statystyki=statystyki,  
  
    ostatnie_zapisy=ostatnie_zapisy,  
  
    pilne_oceny=pilne_oceny,  
  
    csrf_form=csrf_form)
```

PLIK: app_admin\routes\zarzadzanie\dziekan.py

```
import uuid

import csv

import io

import datetime

from flask import (Blueprint, render_template, redirect, url_for,
                   flash, request, abort)

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from flask_wtf.file import FileField, FileAllowed

from wtforms import StringField, SelectField

from wtforms.validators import DataRequired, Email, Length, Optional, ValidationError

from werkzeug.security import generate_password_hash


from core.modele import (Uzytkownik, Student, Praktyka, ZapisPraktyki, HarmonogramPraktyki, EfektUczenia,
                          RolaUzytkownika, StatusPraktyki, StatusZapisu, UploadedDocument, Firma)

from core.extensions import db

from core.uslugi import UsługaUzytkownikow as _UsługaUzytkownikow

_serwis_uzytkownikow = _UsługaUzytkownikow()

from core.autoryzacja import wymaga_rol

from . import zarzadzanie_bp

from .formularze import *


# — Dziekan —————

@zarzadzanie_bp.route('/dziekan')

@wymaga_rol(RolaUzytkownika.ADMIN)

def dziekan_lista():

    strona = request.args.get('page', 1, type=int)

    q = db.session.query(ZapisPraktyki)\

        .join(Uzytkownik, ZapisPraktyki.student_id == Uzytkownik.id)\

        .filter(ZapisPraktyki.status == StatusZapisu.DEAN_APPROVAL)\

        .filter(ZapisPraktyki.sieczka.in_(['EMPLOYMENT', 'OWN_BUSINESS']))
```

```
wnioski = q.order_by(ZapisPraktyki.enrolled_at.desc()).paginate(page=strona, per_page=25, error_out=False)

csrf_form = FlaskForm()

return render_template('zarzadzanie/dziekan/lista.html', wnioski=wnioski, csrf_form=csrf_form)
```

```
@zarzadzanie_bp.route('/dziekan/<uuid:id>/decyzja', methods=['GET', 'POST'])
```

```
@wymaga_rola(RolaUzytkownika.ADMIN)
```

```
def dziekan_decyzja(id):
```

```
    from flask_wtf import FlaskForm
```

```
    from wtforms import TextAreaField, SelectField, SubmitField
```

```
    from wtforms.validators import DataRequired, Optional
```

```
    zapis = db.session.get(ZapisPraktyki, id) or abort(404)
```

```
    if zapis.status != StatusZapisu.DEAN_APPROVAL:
```

```
        flash('Wniosek nie wymaga decyzji dziekana.', 'warning')
```

```
        return redirect(url_for('zarzadzanie.dziekan_lista'))
```

```
class FormularzDziekana(FlaskForm):
```

```
    decyzja = SelectField('Decyzja dziekana', choices=[
```

```
        ('APPROVED', 'Wyrażam zgodę na zaliczenie praktyki'),
```

```
        ('REJECTED', 'Nie wyrażam zgody na zaliczenie'),
```

```
    ], validators=[DataRequired()])
```

```
    komentarz = TextAreaField('Komentarz dziekana', validators=[Optional()])
```

```
    submit = SubmitField('Zapisz decyzję')
```

```
form = FormularzDziekana()
```

```
if form.validate_on_submit():
```

```
    from core.modele import ZdarzenieProces, TypZdarzenia
```

```
    from datetime import datetime
```

```
    if form.decyzja.data == 'APPROVED':
```

```
        zapis.status = StatusZapisu.IN_PROGRESS
```



```
flash('Wniosek zatwierdzony przez dziekana. Student może kontynuować praktykę.', 'success')

else:

    zapis.status = StatusZapisu.REJECTED

    db.session.add(ZdarzenieProces(

        zapis_id=zapis.id, typ=TypZdarzenia.UOPZ_KOMENTARZ,

        komentarz=f"Dziekan nie wyraził zgody: {form.komentarz.data}",

        wykonane_przez_id=current_user.id, wykonano_o=datetime.utcnow(),

    ))

    flash('Wniosek odrzucony przez dziekana.', 'warning')


db.session.add(ZdarzenieProces(

    zapis_id=zapis.id, typ=TypZdarzenia.DZIEKAN_DECYZJA,

    decyzja=form.decyzja.data, komentarz=form.komentarz.data,

    wykonane_przez_id=current_user.id, wykonano_o=datetime.utcnow(),

))

db.session.commit()

return redirect(url_for('zarzadzanie.dziekan_lista'))


return render_template('zarzadzanie/dziekan/decyzja.html', form=form, zapis=zapis)
```

PLIK: app_admin\routes\zarzadzanie\firmy.py

```
import uuid

import csv

import io

import datetime

from flask import (Blueprint, render_template, redirect, url_for,
                   flash, request, abort)

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from flask_wtf.file import FileField, FileAllowed

from wtforms import StringField, SelectField

from wtforms.validators import DataRequired, Email, Length, Optional, ValidationError

from werkzeug.security import generate_password_hash


from core.modele import (Uzytkownik, Student, Praktyka, ZapisPraktyki, HarmonogramPraktyki, EfektUczenia,
                          RolaUzytkownika, StatusPraktyki, StatusZapisu, UploadedDocument, Firma)

from core.extensions import db

from core.uslugi import UsługaUzytkownikow as _UsługaUzytkownikow

_serwis_uzytkownikow = _UsługaUzytkownikow()

from core.autoryzacja import wymaga_rol

from . import zarzadzanie_bp

from .formularze import *


# — Zakłady —————

class _EmptyPagination:

    def __init__(self):

        self.items = []

        self.page = 1

        self.pages = 1

    def iter_pages(self):

        return [1]
```

```
@zarzadzanie_bp.route('/zaklady')

@login_required

def lista_zakladow():

    zaklady = _EmptyPagination()

    csrf_form = FlaskForm()

    return render_template('zarzadzanie/zaklady.html', zaklady=zaklady, csrf_form=csrf_form)
```

```
@zarzadzanie_bp.route('/zaklady/nowy', methods=['GET', 'POST'])

@wymaga_rol(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def nowy_zaklad():

    flash('Funkcja dodawania zakładu jeszcze niezaimplementowana.', 'warning')

    return redirect(url_for('zarzadzanie.lista_zakladow'))
```

```
@zarzadzanie_bp.route('/zaklady/<uuid:id>/edytuj', methods=['GET', 'POST'])

@wymaga_rol(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def edytuj_zaklad(id):

    flash('Edycja zakładu jeszcze niezaimplementowana.', 'warning')

    return redirect(url_for('zarzadzanie.lista_zakladow'))
```

— Firmy —————

```
@zarzadzanie_bp.route('/firmy')

@wymaga_rol(RolaUzytkownika.ADMIN)

def lista_firm():

    strona = request.args.get('page', 1, type=int)

    szukaj = request.args.get('szukaj', '').strip()

    status = request.args.get('status', 'wszystkie')

    q = db.session.query(Firma)

    if status == 'aktywne':
```

```

        q = q.filter_by(aktywna=True)

elif status == 'nieaktywne':

    q = q.filter_by(aktywna=False)

if szukaj:

    q = q.filter(db.or_(

        Firma.nazwa.ilike(f'%{szukaj}%'),

        Firma.adres.ilike(f'%{szukaj}%'),

        Firma.miasto.ilike(f'%{szukaj}%'),

        Firma.nip_krs.ilike(f'%{szukaj}%'),

    ))

    firmy      = q.order_by(Firma.nazwa).paginate(page=strona, per_page=25, error_out=False)

    csrf_form = FlaskForm()

    return render_template('zarzadzanie/firmy/lista.html', firmy=firmy, csrf_form=csrf_form)


@zarzadzanie_bp.route('/firmy/dodaj', methods=['GET', 'POST'])

@wymaga_rola(RolaUzytkownika.ADMIN)

def dodaj_firme():

    form = FormularzFirmy()

    if form.validate_on_submit():

        istniejaca = db.session.query(Firma).filter_by(nazwa=form.nazwa.data.strip(), aktywna=True).first()

        if istniejaca:

            flash('Firma o tej nazwie już istnieje w systemie.', 'error')

            return render_template('zarzadzanie/firmy/formularz.html', form=form, tryb='dodaj')

        if form.nip_krs.data and form.nip_krs.data.strip():

            istniejaca_nip = db.session.query(Firma).filter_by(nip_krs=form.nip_krs.data.strip(), aktywna=True).first()

            if istniejaca_nip:

                flash(f'Firma z numerem NIP/KRS "{form.nip_krs.data.strip()}" już istnieje ({istniejaca_nip.nazwa}).',

                    'error')

                return render_template('zarzadzanie/firmy/formularz.html', form=form, tryb='dodaj')

        firma = Firma(

```

```

        id      = uuid.uuid4(),

        nazwa    = form.nazwa.data.strip(),

        adres    = form.adres.data.strip() if form.adres.data else None,

        miasto   = form.miasto.data.strip() if form.miasto.data else None,

        nip_krs  = form.nip_krs.data.strip() if form.nip_krs.data else None,

    )

    db.session.add(firma)

    db.session.commit()

    flash('Firma została dodana do systemu.', 'success')

    return redirect(url_for('zarzadzanie.lista_firm'))


return render_template('zarzadzanie/firmy/formularz.html', form=form, tryb='dodaj')


@zarzadzanie_bp.route('/firmy/<uuid:id>/edytuj', methods=['GET', 'POST'])
@wymaga_rola(RolaUzytkownika.ADMIN)
def edytuj_firme(id):

    firma = db.session.get(Firma, id) or abort(404)

    form = FormularzFirmy(obj=firma)

    if form.validate_on_submit():

        istniejaca = db.session.query(Firma)\

            .filter(Firma.nazwa == form.nazwa.data.strip(), Firma.id != firma.id, Firma.aktywna == True).first()

        if istniejaca:

            flash('Firma o tej nazwie już istnieje w systemie.', 'error')

            return render_template('zarzadzanie/firmy/formularz.html', form=form, tryb='edytuj', firma=firma)

        if form.nip_krs.data and form.nip_krs.data.strip():

            istniejaca_nip = db.session.query(Firma)\

                .filter(Firma.nip_krs == form.nip_krs.data.strip(), Firma.id != firma.id, Firma.aktywna == True).first()

            if istniejaca_nip:

                flash(f'Firma z NIP/KRS "{form.nip_krs.data.strip()}" już istnieje ({istniejaca_nip.nazwa}).', 'error')

                return render_template('zarzadzanie/firmy/formularz.html', form=form, tryb='edytuj', firma=firma)

```

```

        firma.nazwa = form.nazwa.data.strip()

        firma.adres = form.adres.data.strip() if form.adres.data else None

        firma.miasto = form.miasto.data.strip() if form.miasto.data else None

        firma.nip_krs = form.nip_krs.data.strip() if form.nip_krs.data else None

        db.session.commit()

        flash('Dane firmy zostały zaktualizowane.', 'success')

        return redirect(url_for('zarzadzanie.lista_firm'))

    return render_template('zarzadzanie/firmy/formularz.html', form=form, tryb='edytuj', firma=firma)

@zarzadzanie_bp.route('/firmy/<uuid:id>/usun', methods=['POST'])

@wymaga_rol(RolaUzytkownika.ADMIN)

def usun_firme(id):

    firma = db.session.get(Firma, id) or abort(404)

    wszystkie_praktyki = db.session.query(ZapisPraktyki).filter_by(firma_id=firma.id).count()

    if wszystkie_praktyki > 0:

        flash(f'Nie można usunąć firmy - ma {wszystkie_praktyki} praktyk w historii.', 'error')

        return redirect(url_for('zarzadzanie.lista_firm'))

    nazwa_firmy = firma.nazwa

    db.session.delete(firma)

    db.session.commit()

    flash(f'Firma "{nazwa_firmy}" została trwale usunięta z systemu.', 'success')

    return redirect(url_for('zarzadzanie.lista_firm'))

@zarzadzanie_bp.route('/firmy/<uuid:id>/przelacz-aktywnosc', methods=['POST'])

@wymaga_rol(RolaUzytkownika.ADMIN)

def przelacz_aktywnosc_firmy(id):

    firma = db.session.get(Firma, id) or abort(404)

    if firma.is_active:

        aktywne_praktyki = db.session.query(ZapisPraktyki)\

            .filter_by(firma_id=firma.id)\

```

```
.filter(ZapisPraktyki.status.in_(  
    StatusZapisu.AWAITING_APPROVAL, StatusZapisu.IN_PROGRESS,  
    StatusZapisu.COMMISSION_REVIEW, StatusZapisu.DEAN_APPROVAL,  
))).count()  
  
if aktywne_praktyki > 0:  
    flash(f'Nie można dezaktywować firmy - ma {aktywne_praktyki} aktywnych praktyk.', 'error')  
    return redirect(url_for('zarzadzanie.lista_firm'))  
  
firma.is_active = False  
  
flash('Firma została dezaktywowana.', 'success')  
  
else:  
    firma.is_active = True  
  
    flash('Firma została aktywowana.', 'success')  
  
  
db.session.commit()  
  
return redirect(url_for('zarzadzanie.lista_firm'))
```

PLIK: app_admin\routes\zarzadzanie\formularze.py

```
import uuid

import csv

import io

import datetime

from flask_wtf import FlaskForm

from flask_wtf.file import FileField, FileAllowed

from wtforms import StringField, SelectField

from wtforms.validators import DataRequired, Email, Length, Optional, ValidationError


from core.modele import Uzytkownik, Student

from core.extensions import db


# --- Formularze -----


class FormularzStudenta(FlaskForm):

    imie          = StringField('Imię',          validators=[DataRequired(), Length(max=100)])

    nazwisko      = StringField('Nazwisko',      validators=[DataRequired(), Length(max=100)])

    email         = StringField('E-mail',        validators=[DataRequired(), Email(), Length(max=255)])

    numer_albumu  = StringField('Nr albumu',     validators=[DataRequired(), Length(max=20)])

    plec          = SelectField('Płeć',          choices=[('', '--- Wybierz ---'), ('M', 'Mężczyzna'), ('K', 'Kobieta')],
    validators=[Optional()])

    kierunek      = StringField('Kierunek studiów', validators=[Optional(), Length(max=100)])

    specjalnosc   = StringField('Specjalność',   validators=[Optional(), Length(max=100)])

    tryb_studiow  = SelectField('Tryb studiów',  choices=[('', '--- Wybierz ---'), ('stacjonarne', 'Stacjonarne'),
    ('niestacjonarne', 'Niestacjonarne')], validators=[Optional()])

    uopz_id       = SelectField('Opiekun uczelniany (UOPZ)', choices=[], validators=[Optional()])

    def validate_email(self, pole):

        q = db.session.query(Uzytkownik).filter_by(email=pole.data.lower().strip()).first()

        if q:

            raise ValidationError('Konto z tym e-mailem już istnieje.')

    def validate_numer_albumu(self, pole):
```



```
q = db.session.query(Student).filter_by(numer_albumu=pole.data.strip()).first()
```

```
if q:
```

```
    raise ValidationError('Student z tym nr albumu już istnieje.')
```

```
class FormularzEdycjiStudenta(FormularzStudenta):
```

```
    def __init__(self, uzytkownik_id=None, *args, **kwargs):
```

```
        super().__init__(*args, **kwargs)
```

```
        self._uid = uzytkownik_id
```

```
    def validate_email(self, pole):
```

```
        q = db.session.query(Uzytkownik).filter_by(email=pole.data.lower().strip()).first()
```

```
        if q and str(q.id) != str(self._uid):
```

```
            raise ValidationError('Konto z tym e-mailem już istnieje.')
```

```
    def validate_numer_albumu(self, pole):
```

```
        q = db.session.query(Student).filter_by(numer_albumu=pole.data.strip()).first()
```

```
        if q and str(q.id) != str(self._uid):
```

```
            raise ValidationError('Student z tym nr albumu już istnieje.')
```

```
class FormularzPracownika(FlaskForm):
```

```
    imie      = StringField('Imię',      validators=[DataRequired(), Length(max=100)])
```

```
    nazwisko = StringField('Nazwisko', validators=[DataRequired(), Length(max=100)])
```

```
    email     = StringField('E-mail',    validators=[DataRequired(), Email(), Length(max=255)])
```

```
    rola      = SelectField('Rola', choices=[
```

```
        ('UOPZ', 'Opiekun uczelni (UOPZ)'),
```

```
        ('ADMIN', 'Administrator'),
```

```
    ], validators=[DataRequired()])
```

```
    def validate_email(self, pole):
```

```
        q = db.session.query(Uzytkownik).filter_by(email=pole.data.lower().strip()).first()
```

```
        if q:
```

```
            raise ValidationError('Konto z tym e-mailem już istnieje.')
```

```
class FormularzImportuCSV(FlaskForm):

    plik = FileField('Plik CSV', validators=[

        DataRequired(),

        FileAllowed(['csv'], 'Tylko pliki CSV.')

    ])
```

```
class FormularzFirmy(FlaskForm):

    nazwa = StringField('Nazwa firmy', validators=[DataRequired(), Length(max=255)])

    adres = StringField('Adres', validators=[Optional(), Length(max=255)])

    miasto = StringField('Miasto', validators=[Optional(), Length(max=100)])

    nip_krs = StringField('NIP/KRS', validators=[Optional(), Length(max=50)])
```

```
class FormularzPraktyki(FlaskForm):

    rok_uczelniany = StringField('Rok uczelniany', validators=[DataRequired(), Length(max=9)])

    semestr = SelectField('Semestr', choices=[

        ('zimowy', 'Zimowy'),

        ('letni', 'Letni'),

    ], validators=[DataRequired()])

    wymiar_godzin = StringField('Wymiar godzin (h)', validators=[DataRequired()])
```

PLIK: app_admin\routes\zarzadzanie\komisja.py

```
import uuid

import csv

import io

import datetime

from flask import (Blueprint, render_template, redirect, url_for,
                   flash, request, abort)

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from flask_wtf.file import FileField, FileAllowed

from wtforms import StringField, SelectField

from wtforms.validators import DataRequired, Email, Length, Optional, ValidationError

from werkzeug.security import generate_password_hash


from core.modele import (Uzytkownik, Student, Praktyka, ZapisPraktyki, HarmonogramPraktyki, EfektUczenia,
                          RolaUzytkownika, StatusPraktyki, StatusZapisu, UploadedDocument, Firma)

from core.extensions import db

from core.uslugi import UsługaUzytkownikow as _UsługaUzytkownikow

_serwis_uzytkownikow = _UsługaUzytkownikow()

from core.autoryzacja import wymaga_rol

from . import zarzadzanie_bp

from .formularze import *


# — Komisja weryfikująca —————

@zarzadzanie_bp.route('/komisja')

@wymaga_rol(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def komisja_lista():

    strona = request.args.get('page', 1, type=int)

    from sqlalchemy import or_

    from sqlalchemy import exists

    from core.modele import ZdarzenieProces, TypZdarzenia

    ma_komentarz_uopz = exists().where(
```

```

(ZdarzenieProces.zapis_id == ZapisPraktyki.id) &

(ZdarzenieProces.typ == TypZdarzenia.UOPZ_KOMENTARZ) &

ZdarzenieProces.komentarz.isnot(None)

)

q = db.session.query(ZapisPraktyki)\

    .join(Uzytkownik, ZapisPraktyki.student_id == Uzytkownik.id)\

    .filter(ZapisPraktyki.sieczka.in_(['EMPLOYMENT', 'OWN_BUSINESS']))\

    .filter(or_(

        ZapisPraktyki.status == StatusZapisu.COMMISSION_REVIEW,

        (ZapisPraktyki.status == StatusZapisu.AWAITING_APPROVAL) & ma_komentarz_uopz,

    ))

wnioski = q.order_by(ZapisPraktyki.enrolled_at.desc()).paginate(page=strona, per_page=25, error_out=False)

csrf_form = FlaskForm()

return render_template('zarzadzanie/komisja/lista.html', wnioski=wnioski, csrf_form=csrf_form)

```

```

@zarzadzanie_bp.route('/komisja/<uuid:id>/weryfikuj', methods=['GET', 'POST'])

```

```

@wymaga_rol(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

```

```

def komisja_weryfikuj(id):

```

```

    from flask_wtf import FlaskForm

```

```

    from wtforms import TextAreaField, SelectField, SubmitField

```

```

    from wtforms.validators import DataRequired, Optional

```

```

    zapis = db.session.get(ZapisPraktyki, id) or abort(404)

```

```

    if zapis.status != StatusZapisu.COMMISSION_REVIEW:

```

```

        flash('Wniosek nie wymaga weryfikacji komisji.', 'warning')

```

```

        return redirect(url_for('zarzadzanie.komisja_lista'))

```

```

class FormularzKomisji(FlaskForm):

```

```

    decyzja = SelectField('Decyzja komisji', choices=[

```

```

        ('APPROVED', 'Zatwierdzam - kieruję do dziekana'),

```

```

        ('PARTIALLY_APPROVED', 'Zatwierdzam częściowo - wymaga uzupełnień'),

```

```

        ('REJECTED', 'Odrzucam wniosek'),

```

```

], validators=[DataRequired()])

komentarz = TextAreaField('Komentarz komisji', validators=[Optional()])

submit      = SubmitField('Zapisz decyzję')


form = FormularzKomisji()


if form.validate_on_submit():

    from core.modele import ZdarzenieProces, TypZdarzenia

    from datetime import datetime


    if form.decyzja.data == 'APPROVED':

        decyzja_db = 'APPROVED'

        zapis.status = StatusZapisu.DEAN_APPROVAL

        flash('Wniosek zatwierdzony i przekazany do dziekana.', 'success')

    elif form.decyzja.data == 'PARTIALLY_APPROVED':

        decyzja_db = 'PARTIALLY_APPROVED'

        zapis.status = StatusZapisu.AWAITING_APPROVAL

        db.session.add(ZdarzenieProces(

            zapis_id=zapis.id, typ=TypZdarzenia.UOPZ_KOMENTARZ,

            komentarz=f"Komisja: {form.komentarz.data}",

            wykonane_przez_id=current_user.id, wykonano_o=datetime.utcnow(),

        ))

        flash('Wniosek wymaga uzupełnień - student zostanie powiadomiony.', 'info')

    else:

        decyzja_db = 'REJECTED'

        zapis.status = StatusZapisu.REJECTED

        db.session.add(ZdarzenieProces(

            zapis_id=zapis.id, typ=TypZdarzenia.UOPZ_KOMENTARZ,

            komentarz=f"Wniosek odrzucony przez komisję: {form.komentarz.data}",

            wykonane_przez_id=current_user.id, wykonano_o=datetime.utcnow(),

        ))

        flash('Wniosek został odrzucony.', 'warning')


db.session.add(ZdarzenieProces(

```

```
        zapis_id=zapis.id, typ=TypZdarzenia.KOMISJA_DECYZJA,

        decyzja=decyzja_db, komentarz=form.komentarz.data,

        wykonane_przez_id=current_user.id, wykonano_o=datetime.utcnow(),

    ))

    db.session.commit()

    return redirect(url_for('zarzadzanie.komisja_lista'))
```

```
dokumenty = db.session.query(UploadedDocument)\

    .filter_by(zapis_id=id)\

    .order_by(UploadedDocument.przeslano_o.desc())\

    .all()

return render_template('zarzadzanie/komisja/weryfikuj.html',

    form=form, zapis=zapis, dokumenty=dokumenty)
```

PLIK: app_admin\routes\zarzadzanie\praktyki.py

```
import uuid

import csv

import io

import datetime

from flask import (Blueprint, render_template, redirect, url_for,
                   flash, request, abort)

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from flask_wtf.file import FileField, FileAllowed

from wtforms import StringField, SelectField

from wtforms.validators import DataRequired, Email, Length, Optional, ValidationError

from werkzeug.security import generate_password_hash


from core.modele import (Uzytkownik, Student, Praktyka, ZapisPraktyki, HarmonogramPraktyki, EfektUczenia,
                          RolaUzytkownika, StatusPraktyki, StatusZapisu, UploadedDocument, Firma)

from core.extensions import db

from core.uslugi import UsługaUzytkownikow as _UsługaUzytkownikow

_serwis_uzytkownikow = _UsługaUzytkownikow()

from core.autoryzacja import wymaga_rol

from . import zarzadzanie_bp

from .formularze import *


# — Praktyki —————

@zarzadzanie_bp.route('/praktyki')

@login_required

def lista_praktyk():

    strona = request.args.get('strona', 1, type=int)

    praktyki = db.session.query(Praktyka)\

        .order_by(Praktyka.rok_ucelnianny.desc(), Praktyka.semestr)\

        .paginate(page=strona, per_page=25, error_out=False)

    csrf_form = FlaskForm()
```

```
return render_template('zarzadzanie/praktyki.html', praktyki=praktyki, csrf_form=csrf_form)
```

```
@zarzadzanie_bp.route('/praktyki/nowa', methods=['GET', 'POST'])
```

```
@wymaga_rola(RolaUzytkownika.ADMIN)
```

```
def nowa_praktyka():
```

```
    form = FormularzPraktyki()
```

```
    if form.validate_on_submit():
```

```
        rok = (form.rok_uczelnianny.data or '').strip()
```

```
        try:
```

```
            wymiar = int(form.wymiar_godzin.data)
```

```
        except Exception:
```

```
            flash('Wymiar godzin musi być liczbą całkowitą.', 'danger')
```

```
            return render_template('zarzadzanie/formularz_praktyki.html', form=form)
```

```
    p = Praktyka(
```

```
        id = uuid.uuid4(),
```

```
        rok_uczelnianny = rok,
```

```
        semestr = form.semestr.data,
```

```
        wymiar_godzin = wymiar,
```

```
        status = StatusPraktyki.INACTIVE,
```

```
    )
```

```
    db.session.add(p)
```

```
    db.session.commit()
```

```
    flash('Praktyka została utworzona.', 'success')
```

```
    return redirect(url_for('zarzadzanie.lista_praktyk'))
```

```
return render_template('zarzadzanie/formularz_praktyki.html', form=form)
```

```
@zarzadzanie_bp.route('/praktyki/<uuid:id>/aktywnosc', methods=['POST'])
```

```
@wymaga_rola(RolaUzytkownika.ADMIN)
```

```
def przelacz_aktywnosc_praktyki(id):
```

```
    p = db.session.get(Praktyka, id) or abort(404)
```

```
    p.status = StatusPraktyki.INACTIVE if p.status == StatusPraktyki.ACTIVE else StatusPraktyki.ACTIVE
```

```
    db.session.commit()
```



```
stan = 'aktywowana' if p.status == StatusPraktyki.ACTIVE else 'dezaktywowana'

flash(f'Praktyka {p.rok_uczelniany} ({p.semestr}) została {stan}.', 'success')

return redirect(url_for('zarzadzanie.lista_praktyk'))
```

— Zgłoszenia studentów —————

```
class FormularzPrzypiszUOPZ(FlaskForm):
```

```
    uopz_id = SelectField('Opiekun uczelniany (UOPZ)', choices=[], validators=[Optional()])
```

```
@zarzadzanie_bp.route('/zgloszenia')
```

```
@wymaga_rola(RolaUzytkownika.ADMIN)
```

```
def lista_zgloszen():
```

```
    strona          = request.args.get('page', 1, type=int)
```

```
    status_filter = request.args.get('status', '').strip()
```

```
q = db.session.query(ZapisPraktyki).join(Uzytkownik, ZapisPraktyki.student_id == Uzytkownik.id)
```

```
q = q.filter(ZapisPraktyki.track_type == 'STANDARD')
```

```
if status_filter:
```

```
    try:
```

```
        q = q.filter(ZapisPraktyki.status == StatusZapisu[status_filter])
```

```
    except KeyError:
```

```
        flash(f'Nieznany status: {status_filter}', 'warning')
```

```
zgloszenia = q.order_by(ZapisPraktyki.enrolled_at.desc()).paginate(page=strona, per_page=25, error_out=False)
```

```
csrf_form = FlaskForm()
```

```
return render_template('zarzadzanie/enrollments/list.html', zgloszenia=zgloszenia, csrf_form=csrf_form)
```

```
@zarzadzanie_bp.route('/zgloszenia/<uuid:id>/przypisz-uopz', methods=['GET', 'POST'])
```

```
@wymaga_rola(RolaUzytkownika.ADMIN)
```

```
def przypisz_uopz(id):
```

```

zapis      = db.session.get(ZapisPraktyki, id) or abort(404)

form       = FormularzPrzypiszUOPZ()

uopz_list = db.session.query(Uzytkownik).filter_by(rola=RolaUzytkownika.UOPZ, aktywny=True)\
                .order_by(Uzytkownik.nazwisko, Uzytkownik.imie).all()

form.uopz_id.choices = [('', '--- brak ---')] + [(str(u.id), f"{u.first_name} {u.last_name}") for u in uopz_list]

if form.validate_on_submit():

    if form.uopz_id.data:

        zapis.uopz_id = form.uopz_id.data

        zapis.status = StatusZapisu.AWAITING_APPROVAL

        db.session.commit()

        flash('Opiekun UOPZ przypisany, zgłoszenie przekazane do zatwierdzenia.', 'success')

    else:

        flash('Nie wybrano opiekuna.', 'warning')

    return redirect(url_for('zarzadzanie.lista_zgloszen'))

if request.method == 'GET':

    form.uopz_id.data = str(zapis.uopz_id) if zapis.uopz_id else ''

return render_template('zarzadzanie/enrollments/przypisz_uopz.html', form=form, zapis=zapis)

@zarzadzanie_bp.route('/zgloszenia/<uuid:id>/szczegoly', methods=['GET', 'POST'])

@wymaga_rol(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def szczegoly_zgloszenia(id):

    zapis = db.session.get(ZapisPraktyki, id) or abort(404)

    if current_user.role == RolaUzytkownika.UOPZ and zapis.uopz_id != current_user.id:

        abort(403)

    harmonogram      = db.session.query(HarmonogramPraktyki).filter_by(zapis_id=id).all()

    harmonogram_dict = {h.learning_outcome_id: h for h in harmonogram}

    efekty           = db.session.query(EfektUczenia).order_by(EfektUczenia.id).all()

```

```

from flask_wtf import FlaskForm

from wtforms import TextAreaField, SubmitField


class FormularzKomentarza(FlaskForm):

    komentarz = TextAreaField('Komentarz do studenta')

    zatwierdz = SubmitField('Zatwierdź zgłoszenie')

    odrzuc     = SubmitField('Wymagane poprawki')


form = FormularzKomentarza()


if form.validate_on_submit():

    from core.modele import ZdarzenieProces, TypZdarzenia

    from datetime import datetime


    if form.zatwierdz.data:

        if form.komentarz.data:

            db.session.add(ZdarzenieProces(

                zapis_id=zapis.id,

                typ=TypZdarzenia.ADMIN_KOMENTARZ if current_user.role == RolaUzytkownika.ADMIN else
TypZdarzenia.UOPZ_KOMENTARZ,

                komentarz=form.komentarz.data,

                wykonane_przez_id=current_user.id, wykonano_o=datetime.utcnow(),

            ))

            zapis.status = StatusZapisu.IN_PROGRESS

            flash('Zgłoszenie zostało zatwierdzone!', 'success')

        elif form.odrzuc.data:

            db.session.add(ZdarzenieProces(

                zapis_id=zapis.id,

                typ=TypZdarzenia.ADMIN_KOMENTARZ if current_user.role == RolaUzytkownika.ADMIN else
TypZdarzenia.UOPZ_KOMENTARZ,

                komentarz=form.komentarz.data,

                wykonane_przez_id=current_user.id, wykonano_o=datetime.utcnow(),

            ))

            db.session.add(ZdarzenieProces(

                zapis_id=zapis.id, typ=TypZdarzenia.POWIADOMIENIE_STUDENTA,

```

```

        wykonane_przez_id=current_user.id, wykonano_o=datetime.utcnow(),

    ))

    zapis.status = StatusZapisu.PENDING

    flash('Wysłano prośbę o poprawki do studenta.', 'info')

db.session.commit()

return redirect(url_for('zarzadzanie.lista_zgloszen') if current_user.role == RolaUzytkownika.ADMIN

                else url_for('zarzadzanie.moje_zgloszenia'))

uploaded_docs = db.session.query(UploadedDocument)\

    .filter_by(zapis_id=id)\

    .order_by(UploadedDocument.przeslano_o.desc())\

    .all()

return render_template('zarzadzanie/enrollments/szczegoly.html',

                        zapis=zapis, harmonogram_dict=harmonogram_dict,

                        efekty=efekty, form=form, uploaded_docs=uploaded_docs)

@zarzadzanie_bp.route('/zgloszenia/<uuid:id>/zatwierdz-zaklad', methods=['POST'])

@wymaga_rol(RolaUzytkownika.UOPZ, RolaUzytkownika.ADMIN)

def zatwierdz_zaklad(id):

    zapis = db.session.get(ZapisPraktyki, id) or abort(404)

    zapis.status = StatusZapisu.IN_PROGRESS

    db.session.commit()

    flash('Zakład zatwierdzony. Praktyka rozpoczęła się.', 'success')

    return redirect(url_for('zarzadzanie.lista_zgloszen'))

@zarzadzanie_bp.route('/zgloszenia/<uuid:id>/potwierdz', methods=['POST'])

@wymaga_rol(RolaUzytkownika.ADMIN, RolaUzytkownika.UOPZ)

def potwierdz_zapis(id):

    zapis = db.session.get(ZapisPraktyki, id) or abort(404)

    zapis.status = StatusZapisu.IN_PROGRESS

```

```

if current_user.role == RolaUzytkownika.UOPZ:

    zapis.uopz_id = current_user.id

db.session.commit()

flash('Zapis studenta na praktykę został potwierdzony. Zostałeś/aś przypisany/a jako opiekun.', 'success')

return redirect(request.referrer or url_for('dashboard.index'))

```

```
@zarzadzanie_bp.route('/moje-zgloszenia')
```

```
@wymaga_rol(RolaUzytkownika.UOPZ)
```

```
def moje_zgloszenia():
```

```
    """Lista zgłoszeń przypisanych do aktualnego UOPZ"""
```

```
    from app_admin.routes.ocenianie import get_pilne_oceny
```

```
    strona = request.args.get('page', 1, type=int)
```

```
    status_filter = request.args.get('status', '').strip()
```

```
    q = db.session.query(ZapisPraktyki).filter(ZapisPraktyki.uopz_id == current_user.id)
```

```
    if status_filter:
```

```
        try:
```

```
            q = q.filter(ZapisPraktyki.status == StatusZapisu[status_filter])
```

```
        except KeyError:
```

```
            flash(f'Nieznany status: {status_filter}', 'warning')
```

```
    base_query = db.session.query(ZapisPraktyki).filter(ZapisPraktyki.uopz_id == current_user.id)
```

```
    liczniki = {
```

```
        'wszystkie': base_query.count(),
```

```
        'oczekujace': base_query.filter(ZapisPraktyki.status == StatusZapisu.AWAITING_APPROVAL).count(),
```

```
        'zatwierdzone': base_query.filter(ZapisPraktyki.status == StatusZapisu.IN_PROGRESS).count(),
```

```
    }
```

```
    pilne_oceny = get_pilne_oceny(current_user.id)
```

```
    zgloszenia = q.order_by(ZapisPraktyki.enrolled_at.desc()).paginate(page=strona, per_page=25, error_out=False)
```

```
    csrf_form = FlaskForm()
```

```
return render_template('zarzadzanie/enrollments/moje_lista.html',  
                        zgloszenia=zgloszenia, liczniki=liczniki,  
                        pilne_oceny=pilne_oceny, csrf_form=csrf_form)
```

```
@zarzadzanie_bp.route('/praktyki/<uuid:id>/usun', methods=['POST'])
```

```
@wymaga_rola(RolaUzytkownika.ADMIN)
```

```
def usun_praktyke(id):
```

```
    p = db.session.get(Praktyka, id) or abort(404)
```

```
    opis = f'{p.rok_uczelniany} ({p.semestr})'
```

```
    from sqlalchemy import text as _text
```

```
    db.session.execute(_text("""
```

```
        DELETE FROM dokumenty_przeslane
```

```
        WHERE zapis_id IN (
```

```
            SELECT id FROM zapisy_praktyk WHERE praktyka_id = :pid
```

```
        )
```

```
    """), {'pid': str(id)})
```

```
    db.session.delete(p)
```

```
    db.session.commit()
```

```
    flash(f'Praktyka {opis} została usunięta.', 'success')
```

```
    return redirect(url_for('zarzadzanie.lista_praktyk'))
```

PLIK: app_admin\routes\zarzadzanie\uzytownicy.py

```
import uuid

import csv

import io

import datetime

from flask import (Blueprint, render_template, redirect, url_for,
                   flash, request, abort)

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from flask_wtf.file import FileField, FileAllowed

from wtforms import StringField, SelectField

from wtforms.validators import DataRequired, Email, Length, Optional, ValidationError

from werkzeug.security import generate_password_hash


from core.modele import (Uzytkownik, Student, Praktyka, ZapisPraktyki, HarmonogramPraktyki, EfektUczenia,
                          RolaUzytkownika, StatusPraktyki, StatusZapisu, UploadedDocument, Firma)

from core.extensions import db

from core.uslugi import UsługaUzytkownikow as _UsługaUzytkownikow

_serwis_uzytownikow = _UsługaUzytkownikow()

from core.autoryzacja import wymaga_rol

from . import zarzadzanie_bp

from .formularze import *


# — Użytkownicy —————

@zarzadzanie_bp.route('/uzytownicy')

@login_required

def lista_uzytownikow():

    strona      = request.args.get('strona', 1, type=int)

    szukaj      = request.args.get('szukaj', '').strip()

    filtr_rola  = request.args.get('rola', '').strip()

    q = db.session.query(Uzytkownik).outerjoin(Student, Uzytkownik.id == Student.id)
```

```
if szukaj:
```

```
    q = q.filter(db.or_(
        Uzytkownik.imie.ilike(f'%{szukaj}%'),
        Uzytkownik.nazwisko.ilike(f'%{szukaj}%'),
        Uzytkownik.email.ilike(f'%{szukaj}%'),
        Student.numer_albumu.ilike(f'%{szukaj}%'),
    ))
```

```
if filtr_rola:
```

```
    try:
        q = q.filter_by(rola=RolaUzytkownika[filtr_rola])
    except KeyError:
        pass
```

```
uzytkownicy = q.order_by(Uzytkownik.nazwisko, Uzytkownik.imie)\
    .paginate(page=strona, per_page=25, error_out=False)
```

```
csrf_form = FlaskForm()
```

```
return render_template('zarzadzanie/uzytkownicy.html',
                        uzytkownicy=uzytkownicy,
                        csrf_form=csrf_form)
```

```
@zarzadzanie_bp.route('/uzytkownicy/nowy-student', methods=['GET', 'POST'])
```

```
@wymaga_rola(RolaUzytkownika.ADMIN)
```

```
def nowy_student():
```

```
    form = FormularzStudenta()
```

```
    uopz_list = db.session.query(Uzytkownik).filter_by(rola=RolaUzytkownika.UOPZ, aktywny=True)\
        .order_by(Uzytkownik.nazwisko, Uzytkownik.imie).all()
```

```
    form.uopz_id.choices = [(str(u.id), f"{u.first_name} {u.last_name}") for u in uopz_list]
```

```
    if form.validate_on_submit():
```

```
        u = _serwis_uzytkownikow.utworz_studenta(
            email        = form.email.data,
            haslo        = form.numer_albumu.data,
            imie         = form.imie.data,
```



```

        nazwisko      = form.nazwisko.data,

        numer_albumu  = form.numer_albumu.data,

        plec          = form.plec.data          or None,

        kierunek       = form.kierunek.data      or None,

        specjalnosc    = form.specjalnosc.data   or None,

        tryb_studiow   = form.tryb_studiow.data  or None,

        wymagana_zmiana_hasla=True,

    )

    flash(

        f'Konto studenta {u.imie} {u.nazwisko} (nr alb. {u.numer_albumu}) '

        f'zostało utworzone. Hasło tymczasowe: {u.numer_albumu}',

        'success'

    )

    return redirect(url_for('zarzadzanie.lista_uzytkownikow'))

return render_template('zarzadzanie/formularz_studenta.html', form=form, uzytkownik=None)

```

```

@zarzadzanie_bp.route('/uzytkownicy/<uuid:id>/edytuj-student', methods=['GET', 'POST'])

```

```

@wymaga_rol(RolaUzytkownika.ADMIN)

```

```

def edytuj_studenta(id):

```

```

    u      = db.session.get(Uzytkownik, id) or abort(404)

```

```

    form = FormularzEdycjiStudenta(uzytkownik_id=id, obj=u)

```

```

    uopz_list = db.session.query(Uzytkownik).filter_by(rola=RolaUzytkownika.UOPZ, aktywny=True)\

```

```

        .order_by(Uzytkownik.nazwisko, Uzytkownik.imie).all()

```

```

    form.uopz_id.choices = [(str(x.id), f"{x.first_name} {x.last_name}") for x in uopz_list]

```

```

    if request.method == 'GET':

```

```

        form.imie.data      = u.first_name

```

```

        form.nazwisko.data   = u.last_name

```

```

        form.email.data      = u.email

```

```

        form.numer_albumu.data = u.album_number

```

```

    if form.validate_on_submit():

```

```

        u.first_name      = form.imie.data.strip()

```

```

u.last_name      = form.nazwisko.data.strip()

u.email          = form.email.data.lower().strip()

u.album_number  = form.numer_albumu.data.strip()

u.plec           = form.plec.data or None

u.kierunek       = form.kierunek.data or None

u.specjalnosc    = form.specjalnosc.data or None

u.tryb_studiow   = form.tryb_studiow.data or None

db.session.commit()

flash('Dane studenta zostały zaktualizowane.', 'success')

return redirect(url_for('zarzadzanie.lista_uzytkownikow'))

return render_template('zarzadzanie/formularz_studenta.html', form=form, uzytkownik=u)

```

```

@zarzadzanie_bp.route('/uzytkownicy/<uuid:id>/reset-hasla', methods=['POST'])

```

```

@wymaga_rola(RolaUzytkownika.ADMIN)

```

```

def reset_hasla(id):

    u = db.session.get(Uzytkownik, id) or abort(404)

    if u.role != RolaUzytkownika.STUDENT or not u.album_number:

        flash('Reset hasła dostępny tylko dla studentów z nr albumu.', 'danger')

        return redirect(url_for('zarzadzanie.lista_uzytkownikow'))

    u.password_hash      = generate_password_hash(u.album_number)

    u.wymagana_zmiana_hasla = True

    db.session.commit()

    flash(

        f'Hasło {u.first_name} {u.last_name} zresetowane do nr albumu ({u.album_number}).',

        'success'

    )

    return redirect(url_for('zarzadzanie.lista_uzytkownikow'))

```

```

@zarzadzanie_bp.route('/uzytkownicy/nowy-pracownik', methods=['GET', 'POST'])

```

```

@wymaga_rola(RolaUzytkownika.ADMIN)

```

```

def nowy_pracownik():

    form = FormularzPracownika()

```

```

if form.validate_on_submit():

    haslo_tymczasowe = form.email.data.lower().strip().split('@')[0]

    u = Uzytkownik(

        id                = uuid.uuid4(),

        first_name        = form.imie.data.strip(),

        last_name         = form.nazwisko.data.strip(),

        email             = form.email.data.lower().strip(),

        role              = RolaUzytkownika[form.rola.data],

        password_hash     = generate_password_hash(haslo_tymczasowe),

        wymagana_zmiana_hasla = True,

        is_active         = True,

    )

    db.session.add(u)

    db.session.commit()

    flash(

        f'Konto {u.first_name} {u.last_name} [{u.role.value}] utworzone. ',

        f'Hasło tymczasowe: {haslo_tymczasowe}',

        'success'

    )

    return redirect(url_for('zarzadzanie.lista_uzytkownikow'))

return render_template('zarzadzanie/formularz_pracownika.html', form=form, uzytkownik=None)

```

```

@zarzadzanie_bp.route('/uzytkownicy/import-csv', methods=['GET', 'POST'])

```

```

@wymaga_rol(RolaUzytkownika.ADMIN)

```

```

def import_csv():

```

```

    form        = FormularzImportuCSV()

    uopz_list = db.session.query(Uzytkownik).filter_by(rola=RolaUzytkownika.UOPZ, aktywny=True)\

                .order_by(Uzytkownik.nazwisko, Uzytkownik.imie).all()

    form.uopz_id      = type('X', (), {}]()

    form.uopz_id.choices = [(str(u.id), f"{u.first_name} {u.last_name}") for u in uopz_list]

    wyniki = None

```

```

if form.validate_on_submit():

```

```

plik      = form.plik.data

zawartosc = plik.read().decode('utf-8-sig')

czytnik   = csv.DictReader(io.StringIO(zawartosc))

utworzono, pominieto, bledy = 0, 0, []

wiersze_csv = []

for nr_wiersza, wiersz in enumerate(czytnik, start=2):

    imie      = (wiersz.get('imie')          or wiersz.get('Imię')          or '').strip()

    nazwisko   = (wiersz.get('nazwisko')      or wiersz.get('Nazwisko')    or '').strip()

    email      = (wiersz.get('email')         or wiersz.get('Email')       or '').strip().lower()

    nr_albumu  = (wiersz.get('numer_albumu')  or wiersz.get('Nr albumu')    or '').strip()

    plec       = (wiersz.get('plec')          or wiersz.get('Płeć')        or '').strip().upper() or None

    kierunek   = (wiersz.get('kierunek')      or wiersz.get('Kierunek')    or '').strip() or None

    specjalnosc = (wiersz.get('specjalnosc')  or wiersz.get('Specjalność') or '').strip() or None

    tryb_studiow = (wiersz.get('tryb_studiow') or wiersz.get('Tryb')        or '').strip().lower() or None

    if not all([imie, nazwisko, email, nr_albumu]):

        bledy.append(f'Wiersz {nr_wiersza}: brakujące dane (imie, nazwisko, email, numer_albumu)')

        pominieto += 1

        continue

    wiersze_csv.append({

        'nr': nr_wiersza, 'imie': imie, 'nazwisko': nazwisko,

        'email': email, 'nr_albumu': nr_albumu, 'plec': plec,

        'kierunek': kierunek, 'specjalnosc': specjalnosc,

        'tryb_studiow': tryb_studiow,

    })

if wiersze_csv:

    all_emails = [w['email'] for w in wiersze_csv]

    all_albums = [w['nr_albumu'] for w in wiersze_csv]

    existing = db.session.query(

```

```

        Uzytkownik.email, Student.numer_albumu

    ).outerjoin(Student, Uzytkownik.id == Student.id).filter(

        db.or_(

            Uzytkownik.email.in_(all_emails),

            Student.numer_albumu.in_(all_albums)

        )

    ).all()

existing_emails = {row.email for row in existing}

existing_albums = {row.numer_albumu for row in existing if row.numer_albumu}

for w in wiersze_csv:

    if w['email'] in existing_emails or w['nr_albumu'] in existing_albums:

        bledy.append(f"Wiersz {w['nr']}: {w['email']} lub nr {w['nr_albumu']} już istnieje")

        pominieto += 1

        continue

    try:

        _serwis_uzytkownikow.utworz_studenta(

            email          = w['email'],

            haslo          = w['nr_albumu'],

            imie           = w['imie'],

            nazwisko       = w['nazwisko'],

            numer_albumu   = w['nr_albumu'],

            plec           = w['plec'],

            kierunek       = w['kierunek'],

            specjalnosc    = w['specjalnosc'],

            tryb_studiow   = w['tryb_studiow'],

            wymagana_zmiana_hasla=True,

        )

        existing_emails.add(w['email'])

        existing_albums.add(w['nr_albumu'])

        utworzono += 1

    except Exception as e:

        bledy.append(f"Wiersz {w['nr']}: {str(e)}")

```

```
pominieto += 1
```

```
db.session.commit()
```

```
wyniki = {'utworzono': utworzono, 'pominieto': pominieto, 'bledy': bledy}
```

```
if utworzono:
```

```
    flash(f'Import zakończony: {utworzono} kont utworzonych.', 'success')
```

```
return render_template('zarzadzanie/import_csv.html', form=form, wyniki=wyniki)
```

```
@zarzadzanie_bp.route('/uzytkownicy/<uuid:id>/aktywnosc', methods=['POST'])
```

```
@wymaga_rol(RolaUzytkownika.ADMIN)
```

```
def przelacz_aktywnosc(id):
```

```
    u = db.session.get(Uzytkownik, id) or abort(404)
```

```
    if str(u.id) == str(current_user.id):
```

```
        flash('Nie możesz dezaktywować własnego konta.', 'danger')
```

```
        return redirect(url_for('zarzadzanie.lista_uzytkownikow'))
```

```
    u.is_active = not u.is_active
```

```
    db.session.commit()
```

```
    stan = 'aktywowane' if u.is_active else 'dezaktywowane'
```

```
    flash(f'Konto {u.first_name} {u.last_name} zostało {stan}.', 'success')
```

```
    return redirect(url_for('zarzadzanie.lista_uzytkownikow'))
```

```
@zarzadzanie_bp.route('/uzytkownicy/<uuid:id>/usun', methods=['POST'])
```

```
@wymaga_rol(RolaUzytkownika.ADMIN)
```

```
def usun_uzytkownika(id):
```

```
    u = db.session.get(Uzytkownik, id) or abort(404)
```

```
    if str(u.id) == str(current_user.id):
```

```
        flash('Nie możesz usunąć własnego konta.', 'danger')
```

```
        return redirect(url_for('zarzadzanie.lista_uzytkownikow'))
```

```
    imie_nazwisko = f'{u.first_name} {u.last_name}'
```

```
    db.session.delete(u)
```

```
db.session.commit()

flash(f'Konto {imie_nazwisko} zostało trwale usunięte.', 'success')

return redirect(url_for('zarzadzanie.lista_uzytkownikow'))
```

PLIK: app_admin\routes\zarzadzanie__init__.py

```
from flask import Blueprint
```

```
zarzadzanie_bp = Blueprint('zarzadzanie', __name__)
```

```
from . import uzytkownicy, praktyki, firmy, komisja, dziekan
```


PLIK: app_admin\services\serwis_oceniania.py

"""

app_admin/services/serwis_oceniania.py

Logika biznesowa oceniania praktyk.

Przemianowano z evaluation_service.py.

"""

from datetime import date, timedelta

from core.extensions import db

from core.modele import ZapisPraktyki, StatusZapisu

class SerwisOceniania:

@staticmethod

def get_pilne_oceny(uopz_id=None):

"""Zwraca listę praktyk z pilnymi ocenami dla danego UOPZ lub wszystkich."""

q = db.session.query(ZapisPraktyki).filter_by(status=StatusZapisu.COMPLETED)

if uopz_id:

q = q.filter_by(uopz_id=uopz_id)

pilne_oceny = []

for zapis in q.all():

if zapis.termin_do:

deadline = zapis.termin_do + timedelta(days=7)

dni_do_deadline = (deadline - date.today()).days

if dni_do_deadline <= 3:

pilne_oceny.append({

'zapis': zapis,

'deadline': deadline,

'dni_do_deadline': dni_do_deadline,

'przekroczony': dni_do_deadline < 0,

})

return sorted(pilne_oceny, key=lambda x: x['dni_do_deadline'])

```
@staticmethod
```

```
def auto_complete_internships():
```

```
    """Automatycznie przenosi praktyki z przekroczonym terminem do statusu COMPLETED."""
```

```
    do_zakonczenia = db.session.query(ZapisPraktyki).filter(
```

```
        ZapisPraktyki.status == StatusZapisu.IN_PROGRESS,
```

```
        ZapisPraktyki.termin_do < date.today()
```

```
    ).all()
```

```
    for praktyka in do_zakonczenia:
```

```
        praktyka.status = StatusZapisu.COMPLETED
```

```
    if do_zakonczenia:
```

```
        db.session.commit()
```

```
# Alias dla kompatybilności wstecznej
```

```
EvaluationService = SerwisOceniania
```

PLIK: app_admin\services\serwis_praktyk.py

```
"""

app_admin/services/serwis_praktyk.py

Logika biznesowa zarządzania praktykantami i zgłoszeniami.

Przemianowano z internship_service.py.

"""

import uuid

from werkzeug.security import generate_password_hash

from core.extensions import db

from core.modele import Uzytkownik, RolaUzytkownika, ZapisPraktyki, StatusZapisu


class SerwisPraktyk:

    @staticmethod

    def create_student(imie, nazwisko, email, numer_albumu,

                      plec=None, kierunek=None, specjalnosc=None, tryb_studiow=None):

        u = Uzytkownik(

            id                = uuid.uuid4(),

            first_name        = imie.strip(),

            last_name         = nazwisko.strip(),

            email             = email.lower().strip(),

            album_number      = numer_albumu.strip(),

            plec              = plec or None,

            kierunek          = kierunek or None,

            specjalnosc       = specjalnosc or None,

            tryb_studiow      = tryb_studiow or None,

            role               = RolaUzytkownika.STUDENT,

            password_hash     = generate_password_hash(numer_albumu.strip()),

            wymagana_zmiana_hasla = True,

            is_active         = True,

        )

        db.session.add(u)

        return u
```

```
@staticmethod
```

```
def approve_enrollment(enrollment_id, current_user):  
  
    """Zatwierdza zgłoszenie, opcjonalnie przypisując opiekuna UOPZ."""  
  
    zapis = db.session.get(ZapisPraktyki, enrollment_id)  
  
    if not zapis:  
  
        raise ValueError("Zapis nie istnieje.")  
  
    zapis.status = StatusZapisu.IN_PROGRESS  
  
    if current_user.role == RolaUzytkownika.UOPZ:  
  
        zapis.uopz_id = current_user.id  
  
    db.session.commit()  
  
    return zapis
```

```
@staticmethod
```

```
def assign_supervisor(enrollment_id, supervisor_id):  
  
    zapis = db.session.get(ZapisPraktyki, enrollment_id)  
  
    if not zapis:  
  
        raise ValueError("Zapis nie istnieje.")  
  
    zapis.uopz_id = supervisor_id  
  
    zapis.status = StatusZapisu.AWAITING_APPROVAL  
  
    db.session.commit()  
  
    return zapis
```

```
@staticmethod
```

```
def approve_company(enrollment_id):  
  
    zapis = db.session.get(ZapisPraktyki, enrollment_id)  
  
    if not zapis:  
  
        raise ValueError("Zapis nie istnieje.")  
  
    zapis.status = StatusZapisu.IN_PROGRESS  
  
    db.session.commit()  
  
    return zapis
```

```
# Alias dla kompatybilności wstecznej
```

```
InternshipService = SerwisPraktyk
```

```
/* =====  
  
admin.css – Punkt wejścia arkusza stylów panelu  
  
System Praktyk Zawodowych – ANS Elbląg  
  
Moduły ładowane w kolejności zależności:  
  
1. zmienne – tokeny CSS, reset, dostępność (WCAG 2.1 AA)  
2. komunikaty – flash messages i powiadomienia  
3. formularze – pola input, checkboxy  
4. logowanie – strona logowania i zmiana hasła  
5. układ – sidebar, panel-main, nagłówek  
6. karty – komponent .karta  
7. tabele – tabele danych  
8. przyciski – przyciski i grupy akcji  
9. statusy – odznaki statusów i ról  
10. komponenty – pasek postępu, paginacja, filtry  
11. dashboard – statystyki, strona błędu  
  
===== */  
  
@import url('/core/static/css/modul/zmienne.css');  
  
@import url('/core/static/css/modul/komunikaty.css');  
  
@import url('/core/static/css/modul/formularze.css');  
  
@import url('/core/static/css/modul/logowanie.css');  
  
@import url('/core/static/css/modul/uklad.css');  
  
@import url('/core/static/css/modul/karty.css');  
  
@import url('/core/static/css/modul/tabele.css');  
  
@import url('/core/static/css/modul/przyciski.css');  
  
@import url('/core/static/css/modul/statusy.css');  
  
@import url('/core/static/css/modul/komponenty.css');  
  
@import url('/core/static/css/modul/dashboard.css');
```

PLIK: app_admin\static\css\student.css

```
/* Copied student styles so Flask serves them from app_admin's static/ */

/* Minimal student panel styles (copied from app_student/static/css/student.css) */

:root{--kolor-glowny:#1769aa;--kolor-tekst-drugor:#6b7280;--kolor-tlo:#f4f6f9;--kolor-sukces:#16a34a}

body{font-family:Inter,system-ui,Arial, sans-serif;background:#fff;color:#111}

.przycisk{display:inline-block;padding:.5rem
.75rem;border-radius:6px;border:0;background:var(--kolor-glowny);color:#fff;text-decoration:none}

.przycisk--drugorzedy{background:#e5e7eb;color:#111}

.karta{background:#fff;border:1px solid #e6e9ef;padding:1rem;border-radius:8px}

/* add more as needed */
```

PLIK: app_admin\templates\auth\logowanie.html

```
{% extends "layouts/base_auth.html" %}

{% block tytul %}Logowanie{% endblock %}

{% block body_class %}strona-logowania{% endblock %}

{% block body %}

<div class="panel-logowania" role="main">

    <!-- Lewa kolumna – identyfikacja uczelni -->

    <aside class="panel-logowania__branding" aria-label="Informacje o instytucji">

        <div class="branding__tresc">

            <div class="branding__logo-kontener">

                

            </div>

            <div class="branding__separator" aria-hidden="true"></div>

            <p class="branding__nazwa-systemu">System zarządzania<br>praktykami zawodowymi</p>

            <p class="branding__podtytul">Kierunek Informatyka</p>

            <!-- Ozdobny element geometryczny – aria-hidden bo dekoracyjny -->

            <div class="branding__dekoracja" aria-hidden="true">

                <span></span><span></span><span></span></div>

        </div>

    </aside>
```

```

<!-- Prawa kolumna – formularz logowania -->

<main class="panel-logowania__formularz" id="main-content">

    <div class="formularz__kontener">

        <div class="formularz__naglowek">

            <!-- WCAG 2.4.6: nagłówki opisujące temat -->

            <h1 class="formularz__tytul">Panel administracyjny</h1>

            <p class="formularz__opis" id="opis-formularza">

                Zaloguj się, aby zarządzać praktykami, użytkownikami i dokumentacją.

            </p>

        </div>

        <!-- Komunikaty błędów – WCAG 1.3.3, 4.1.3 -->

        {% with wiadomosci = get_flashed_messages(with_categories=True) %}

        {% if wiadomosci %}

            <div role="alert" aria-live="assertive" class="komunikaty">

                {% for kategoria, wiadomosc in wiadomosci %}

                    <div class="komunikat komunikat--{{ kategoria }}">

                        <span class="komunikat__ikona" aria-hidden="true">

                            {% if kategoria == 'danger' %}{% elif kategoria == 'success' %}{% else %}{% endif %}

                        </span>

                        <span>{{ wiadomosc }}</span>

                    </div>

                {% endfor %}

            </div>

        {% endif %}

        {% endwith %}

        <form

            method="POST"

            action="{{ url_for('auth.logowanie') }}"

            novalidate

            aria-describedby="opis-formularza"

            class="formularz__forma"

```


>

```
<!-- CSRF token – zabezpieczenie przed atakami -->
```

```
{{ form.hidden_tag() }}
```

```
<!-- Pole: adres e-mail -->
```

```
<div class="pole-formularza {% if form.email.errors %}pole-formularza--blad{% endif %}">
```

```
<label
```

```
    for="email"
```

```
    class="pole-formularza__etykieta"
```

```
>
```

```
    Adres e-mail
```

```
    <span class="pole-formularza__wymagane" aria-label="(pole wymagane)">*</span>
```

```
</label>
```

```
<input
```

```
    type="email"
```

```
    id="email"
```

```
    name="email"
```

```
    class="pole-formularza__input"
```

```
    value="{{ form.email.data or '' }}"
```

```
    autocomplete="email"
```

```
    spellcheck="false"
```

```
    required
```

```
    aria-required="true"
```

```
    {% if form.email.errors %}
```

```
        aria-invalid="true"
```

```
        aria-describedby="email-blad"
```

```
    {% endif %}
```

```
    placeholder="np. 1234@ans-elblag.pl"
```

```
>
```

```
{% if form.email.errors %}
```

```
    <span
```

```
        id="email-blad"
```

```
        class="pole-formularza__blad"
```

```
        role="alert"
```

```

>

    {% for blad in form.email.errors %}{ { blad }}{% endfor %}

</span>

{% endif %}

</div>


<!-- Pole: hasło -->

<div class="pole-formularza {% if form.haslo.errors %}pole-formularza--blad{% endif %}">

    <div class="pole-formularza__naglowek-haslo">

        <label

            for="haslo"

            class="pole-formularza__etykieta"

        >

            Hasło

            <span class="pole-formularza__wymagane" aria-label="(pole wymagane)">*</span>

        </label>

    </div>

    <div class="pole-formularza__input-wrapper">

        <input

            type="password"

            id="haslo"

            name="haslo"

            class="pole-formularza__input"

            autocomplete="current-password"

            required

            aria-required="true"

            {% if form.haslo.errors %}

                aria-invalid="true"

                aria-describedby="haslo-blad"

            {% endif %}

        >

        <!-- Przycisk pokaż/ukryj hasło – WCAG 1.3.5 -->

        <button

            type="button"

```

```

        class="pole-formularza__toggle-haslo"

        aria-label="Pokaż hasło"

        aria-pressed="false"

        onclick="toggleHaslo(this)"
    >

    <span class="ikona-ok" aria-hidden="true"></span>

</button>

</div>

{% if form.haslo.errors %}

    <span

        id="haslo-blad"

        class="pole-formularza__blad"

        role="alert"

    >

        {% for blad in form.haslo.errors %}{{ blad }}{% endfor %}

    </span>

{% endif %}

</div>

<!-- Zapamiętaj mnie -->

<div class="pole-formularza pole-formularza--checkbox">

    <label class="pole-formularza__etykieta-checkbox">

        <input

            type="checkbox"

            name="zapamietaj"

            id="zapamietaj"

            class="pole-formularza__checkbox"

            value="1"

        >

        <span class="pole-formularza__checkbox-label">Zapamiętaj mnie na tym urządzeniu</span>

    </label>

</div>

<!-- Przycisk submit -->

```

```
<button
    type="submit"
    class="przycisk-logowania"
    aria-label="Zaloguj się do panelu administracyjnego"
>
    Zaloguj się
</button>
```

```
</form>
```

```
<!-- Stopka formularza -->
```

```
<footer class="formularz__stopka">
```

```
<p class="formularz__stopka-tekst">
```

```
    Dostęp wyłącznie dla upoważnionych pracowników Instytutu.
```

```
</p>
```

```
<p class="formularz__stopka-tekst">
```

```
    Problemy z logowaniem?
```

```
<a href="mailto:admin@ans.elblag.edu.pl" class="formularz__link">
```

```
    Skontaktuj się z administratorem
```

```
</a>
```

```
</p>
```

```
</footer>
```

```
</div>
```

```
</main>
```

```
</div>
```

```
{% endblock %}
```

```
{% block scripts_extra %}
```

```
<script>
```

```
function toggleHaslo(przycisk) {
```

```
    const input = document.getElementById('haslo');
```

```
    const ukryte = input.type === 'password';
```

```
input.type = ukryte ? 'text' : 'password';

przycisk.setAttribute('aria-pressed', ukryte ? 'true' : 'false');

przycisk.setAttribute('aria-label', ukryte ? 'Ukryj hasło' : 'Pokaż hasło');

}

</script>

{% endblock %}
```

PLIK: app_admin\templates\dashboard\index.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Dashboard{% endblock %}

{% block naglowek %}Przegląd systemu{% endblock %}


{% block naglowek_akcje %}

    {% if current_user.role.value == 'ADMIN' %}

        <div class="akcje-panelu" style="display: flex; gap: 0.5rem; align-items: center;">

            <a href="{{ url_for('zarzadzanie.nowy_student') }}" class="przycisk przycisk--glowny">

                <span aria-hidden="true">+</span> Dodaj studenta

            </a>

            <a href="{{ url_for('zarzadzanie.nowy_pracownik') }}" class="przycisk przycisk--glowny">

                <span aria-hidden="true">+</span> Dodaj pracownika

            </a>

        </div>

    {% endif %}

{% endblock %}


{% block tresc %}


{% if current_user.role.value == 'UOPZ' and pilne_oceny %}

<div style="margin-bottom: 2rem;">

    {% for item in pilne_oceny %}

        <div class="komunikat komunikat--{% if item.przekroczony %}danger{% else %}warning{% endif %}" style="margin-bottom: 1rem;">

            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round" style="flex-shrink: 0; margin-top: 2px;">

                <path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"/>

                <line x1="12" y1="9" x2="12" y2="13"/>

                <line x1="12" y1="17" x2="12.01" y2="17"/>

            </svg>

            <div>

                <div style="font-weight: 600;">

                    {% if item.przekroczony %}
```

```

        Przekroczony deadline oceny: {{ item.zapis.student.last_name }} {{ item.zapis.student.first_name }}

    {% elif item.dni_do_deadline == 0 %}

        Dziś ostatni dzień oceny: {{ item.zapis.student.last_name }} {{ item.zapis.student.first_name }}

    {% else %}

        Pilna ocena za {{ item.dni_do_deadline }} dni: {{ item.zapis.student.last_name }} {{
item.zapis.student.first_name }}

    {% endif %}

</div>

<div style="font-size: 0.9rem; margin-top: 0.5rem;">

    Deadline: {{ item.deadline.strftime('%d.%m.%Y') }}

    <a href="{{ url_for('evaluation.ocen_praktyke', id=item.zapis.id) }}"

        style="margin-left: 1rem; color: inherit; text-decoration: underline;">

        Wystaw ocenę →

    </a>

</div>

</div>

</div>

{% endfor %}

</div>

{% endif %}

```

```

<section class="siatka-statystyk" aria-label="Kluczowe wskaźniki">

```

```

<div class="karta-stat">

    <span class="karta-stat__ikona" aria-hidden="true"><svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12"
r="10"/><polyline points="12 6 12 12 16 14"/></svg></span>

    <span class="karta-stat__liczba">{{ statystyki.praktyki_aktywne }}</span>

    <span class="karta-stat__etykieta">W trakcie realizacji</span>

</div>

```

```

<div class="karta-stat">

    <span class="karta-stat__ikona" aria-hidden="true"><svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M9 11l3 3L22 4"/><path
d="M21 12v7a2 2 0 0 1-2 2H5a2 2 0 0 1-2 2V5a2 2 0 0 1 2-2h11"/></svg></span>

    <span class="karta-stat__liczba">{{ statystyki.oczekujace_oceny }}</span>

    <span class="karta-stat__etykieta">Czeka na ocenę</span>

```

</div>

<div class="karta-stat">

<svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M22 10v6M2 10 5-10 5z"/><path d="M6 12v5c0 2 4 4 6 4s6-2 6-4v-5"/></svg>

{{ statystyki.zakonczone }}

Zakończzone

</div>

<div class="karta-stat">

<svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2"/><circle cx="9" cy="7" r="4"/><path d="M23 21v-2a4 4 0 0 0-3-3.87"/><path d="M16 3.13a4 4 0 0 1 0 7.75"/></svg>

{{ statystyki.liczba_studentow }}

Aktywnych studentów

</div>

</section>

<div class="uklad-dashboardu" style="display: grid; grid-template-columns: 2fr 1fr; gap: 2rem;">

<section class="karta" aria-labelledby="tytul-ostatnie">

<header class="karta__naglowek">

<h2 id="tytul-ostatnie" class="karta__tytul">Ostatnie aktywności</h2>

Wszystkie

</header>

<div class="karta__tresc">

<div class="tabela-wrapper">

<table class="tabela">

<thead>

<tr>

<th scope="col">Student</th>

<th scope="col">Praktyka</th>

<th scope="col">Status</th>


```

        <th scope="col" style="text-align:right;">Akcje</th>

    </tr>

</thead>

<tbody>

    {% for z in ostatnie_zapisy %}

    <tr>

        <td>{{ z.student.first_name }} {{ z.student.last_name }}</td>

        <td>{{ z.praktyka.rok_uczelniiany }} ({{ z.praktyka.semestr }})</td>

        <td>

            <span class="status status--{{ z.status.value|lower|replace('_', '-') }}">

                {{ tlumacz_status(z.status.value) }}

            </span>

        </td>

        <td style="text-align:right;">

            {% if z.status.value == 'PENDING' %}

                <form method="POST" action="{{ url_for('zarzadzanie.potwierdz_zapis', id=z.id) }}"
style="display:inline;">

                    {{ csrf_form.hidden_tag() }}

                    <button type="submit" class="przycisk przycisk--drugorzeczny przycisk--maly" onclick="return
confirm('Potwierdzić ten zapis?')">Zatwierdź</button>

                </form>

            {% endif %}

        </td>

    </tr>

    {% else %}

    <tr>

        <td colspan="3" class="tabela--pusta">Brak aktywnych zapisów.</td>

    </tr>

    {% endfor %}

</tbody>

</table>

</div>

</div>

</section>

```

```
<section class="karta" aria-labelledby="tytul-oczekujace">

  <header class="karta__naglowek">

    <h2 id="tytul-oczekujace" class="karta__tytul">Aktywne praktyki</h2>

  </header>

  <div class="karta__tresc">

    <ul class="lista-prosta" role="list" style="list-style: none; padding: 0;">

      {% for z in ostatnie_zapisy %}

        <li style="padding: 1rem 0; border-bottom: 1px solid var(--kolor-ramka); display: flex; justify-content: space-between; align-items: center;">

          <div>

            <span style="display: block; font-weight: 600;">{{ z.student.first_name }} {{ z.student.last_name }}</span>

            <span style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">{{ z.praktyka.rok_uczelnianny }}</span>

          </div>

        </li>

        {% else %}

        <li style="padding: 2rem; text-align: center; color: var(--kolor-tekst-drugor);">Brak zapisów.</li>

        {% endfor %}

      </ul>

    </div>

  </section>

</div>

{% endblock %}
```

PLIK: app_admin\templates\documents\edytor.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Edytor Dokumentów (LaTeX){% endblock %}

{% block naglowek %}Tryb Ręczny: {{ doc_title }}{% endblock %}


{% block naglowek_akcje %}

<a href="{{ url_for('documents.panel', id=zapis.id) }}" class="przycisk przycisk--drugorzeczny">Powrót do panelu</a>

{% endblock %}


{% block tresc %}

<div style="max-width: 1200px; margin: 0 auto; padding-bottom: 4rem;">

    <div class="karta" style="margin-bottom: 2rem;">

        <div class="karta__naglowek" style="background: var(--kolor-tlo); display: flex; justify-content: space-between; align-items: center;">

            <h2 class="karta__tytul">Surowy kod LaTeX</h2>

            <span style="font-size: 0.85rem; color: var(--kolor-blad); font-weight: 600;">Uwaga: Jesteś w trybie "Manual Override"</span>

        </div>

        <div class="karta__tresc">

            <p style="font-size: 0.9rem; color: var(--kolor-tekst-drugor); margin-bottom: 1.5rem;">

                Edytujesz właśnie wyrenderowany szablon. Możesz wprowadzać dowolne zmiany zgodnie z syntaktyką LaTeX (LuaLaTeX). Zmieniony dokument nie zostanie nadpisany w systemie, lecz od razu skompilowany do PDF. Jeśli system automatyczny generuje błędne dane, możesz je tutaj poprawić i pobrać PDF ręcznie.

            </p>

            <form id="tex-form">

                <input type="hidden" name="csrf_token" value="{{ csrf_token() }}" />

                <div style="border: 1px solid var(--kolor-ramka); border-radius: 4px; overflow: hidden; margin-bottom: 1.5rem;">

                    <textarea name="tex_source" id="tex_source" style="width: 100%; height: 60vh; max-height: 800px; font-family: 'IBM Plex Mono', monospace; font-size: 0.85rem; padding: 1rem; border: none; background: #fbfbfb; resize: vertical; spellcheck="false">{{ raw_tex }}</textarea>

                </div>

            </div>

        </div>

    </div>

</div>
```

```
<div style="display: flex; justify-content: space-between; align-items: center;">

  <div id="status-box" style="font-size: 0.9rem; font-weight: 500; color: var(--kolor-tekst-drugor);">

    Gotowy do kompilacji

  </div>

  <button type="submit" id="btn-kompiluj" class="przycisk przycisk--glowny" style="padding: 0.75rem 2.5rem; font-size: 1.05rem;">

    Kompiluj do PDF

  </button>

</div>

</form>

</div>

</div>

</div>
```

```
{% block scripts_extra %}
```

```
<script>
```

```
document.addEventListener('DOMContentLoaded', () => {

  const form = document.getElementById('tex-form');

  const btn = document.getElementById('btn-kompiluj');

  const statusBox = document.getElementById('status-box');

  form.addEventListener('submit', async (e) => {

    e.preventDefault();

    btn.disabled = true;

    btn.innerHTML = 'Kompilowanie w tle...';

    statusBox.innerHTML = '<span style="color: var(--kolor-glowny);">Czekam na uruchomienie zadania...</span>';

    try {

      const formData = new FormData(form);

      const res = await fetch("{ url_for('documents.kompiluj_recznie', id=zapis.id, doc_type=doc_type) }", {

        method: 'POST',

        body: formData,
```

```

        headers: {

            'X-Requested-With': 'XMLHttpRequest'

        }
    });

    const data = await res.json();

    if(!res.ok) {

        throw new Error(data.error || 'Błąd serwera');

    }

    const taskId = data.task_id;

    watchTaskSSE(taskId);

} catch(err) {

    statusBox.innerHTML = `

```

```

    } else if (data.status === 'FAILURE') {

        es.close();

        statusBox.innerHTML = `

```

PLIK: app_admin\templates\documents\panel.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Dokumenty i Załączniki{% endblock %}

{% block naglowek %}Generowanie dokumentów: {{ zapis.student.imie }} {{ zapis.student.nazwisko }}{% endblock %}

{% block naglowek_akcje %}

<a href="{% url_for('zarzadzanie.szczegoly_praktyki', id=zapis.internship_id) %}" class="przycisk
przycisk--drugorzeczny">Powrót do szczegółów</a>

{% endblock %}

{% block tresc %}

<div class="uklad-szczegolow" style="display: grid; grid-template-columns: 2fr 1fr; gap: 2rem;">

    <div class="lewa-kolumna" style="display: flex; flex-direction: column; gap: 2rem;">

        <!-- Lista dokumentów -->

        <div class="karta">

            <div class="karta__naglowek" style="background: var(--kolor-tlo); display: flex; justify-content: space-between;
            align-items: center;">

                <h2 class="karta__tytul">Dostępne szablony (LaTeX)</h2>

                <button onclick="triggerAuto('ZIP')" id="btn-auto-ZIP" class="przycisk przycisk--glowny przycisk--maly"
                style="background: #27ae60; border-color: #27ae60;"> Generuj Komplet (ZIP)</button>

            </div>

            <div class="karta__tresc">

                <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.5rem;">

                    Wybierz dokument, który chcesz wygenerować ze stałych szablonów. Kliknij "Tryb ręczny" aby edytować surowy
                    szablon TeX przed kompilacją.

                </p>

                <div id="status-ZIP" style="text-align: right; margin-bottom: 1rem; font-size: 0.85rem; font-weight: 600;"></div>

                <div style="display: flex; flex-direction: column; gap: 1rem;">

                    {% for doc_key, doc_info in docs.items() %}

                        <div style="display: flex; justify-content: space-between; align-items: center; padding: 1rem; border: 1px
                        solid var(--kolor-ramka); border-radius: 8px;">

                            <div style="font-weight: 500;">

                                {{ doc_info[1] }}
```

```

</div>

<div style="display: flex; gap: 0.5rem; align-items: center;">

    <span id="status-{{ doc_key }}" style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); font-weight:
600; margin-right: 0.5rem;"></span>

    <button onclick="triggerAuto('{{ doc_key }}')" id="btn-auto-{{ doc_key }}" class="przycisk
przycisk--drugorzeczny przycisk--maly">Wyg. Automatycznie</button>

    <a href="{{ url_for('documents.edytuj', id=zapis.id, doc_type=doc_key) }}" class="przycisk przycisk--glowny
przycisk--maly" style="background: #e67e22; border-color: #e67e22;">Tryb ręczny (TeX)</a>

</div>

</div>

{% endfor %}

</div>

</div>

</div>

</div>

</div>

<div class="prawa-kolumna" style="display: flex; flex-direction: column; gap: 2rem;">

    <!-- Audit log -->

    <div class="karta">

        <div class="karta__naglowek" style="background: var(--kolor-tlo);">

            <h2 class="karta__tytul">Ostatnie modyfikacje (Log)</h2>

        </div>

        <div class="karta__tresc py-3">

            {% if logs %}

            <ul style="list-style: none; padding: 0; margin: 0; font-size: 0.85rem;">

                {% for log in logs %}

                <li style="margin-bottom: 1rem; border-bottom: 1px dashed var(--kolor-ramka); padding-bottom: 0.5rem;">

                    <strong style="color: var(--kolor-tekst);">{{ log.action }}</strong> – <span style="font-family:
monospace;">{{ log.document_type }}</span>

                    <br>

                    <span style="color: var(--kolor-tekst-drugor);">

                        {{ log.created_at.strftime('%Y-%m-%d %H:%M') }} | Kto: {{ log.user.imie if log.user else 'System' }}

                    </span>

                    {% if log.details %}

                    <div style="font-style: italic; margin-top: 0.25rem;">{{ log.details }}</div>

                    {% endif %}

                </li>

            </ul>

            {% else %}

            <div style="text-align: center; padding: 10px 0 0 0;">

                <div style="font-size: 0.85rem; margin-bottom: 0.5rem;">Brak danych do zalogowania</div>

                <div style="font-size: 0.85rem;">Brak danych do zalogowania</div>

            </div>

            {% endif %}

        </div>

    </div>

</div>

```



```

        </li>

        {% endfor %}

    </ul>

    {% else %}

    <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">Brak historii dokumentów.</p>

    {% endif %}

</div>

</div>

</div>

```

```

</div>

```

```

{% block scripts_extra %}

```

```

<script>

```

```

async function triggerAuto(docType) {

    const btn = document.getElementById('btn-auto-' + docType);

    const statusBox = document.getElementById('status-' + docType);

    btn.disabled = true;

    statusBox.innerHTML = '<span style="color: var(--kolor-glowny);">Uruchamianie...</span>';

    try {

        let url = `/dokumenty/zapis/{{ zapis.id }}/generuj_auto/${docType}`;

        if (docType === 'ZIP') {

            url = `/dokumenty/zapis/{{ zapis.id }}/generuj_zip`;

        }

        const res = await fetch(url, {

            method: 'POST',

            headers: {'X-Requested-With': 'XMLHttpRequest'}

        });

        const data = await res.json();

        if (!res.ok) throw new Error(data.error || 'Błąd serwera');
    }
}

```

```

// SSE – jedno trwałe połączenie zamiast pollingu co 2 sekundy

watchTaskSSE(data.task_id, btn, statusBox);

} catch (err) {

    statusBox.innerHTML = `

```

```
es.close();

statusBox.innerHTML = ``;

btn.disabled = false;

};

}

</script>

{% endblock %}

{% endblock %}
```

PLIK: app_admin\templates\evaluation\egzamin.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Egzamin praktyki{% endblock %}

{% block naglowek %}Protokół egzaminacyjny{% endblock %}

{% block naglowek_akcje %}

<a href="{% url_for('zarzadzanie.szczegoly_praktyki', id=praktyka.id) %}" class="przycisk
przycisk--drugorzeczny">Powrót</a>

{% endblock %}

{% block tresc %}

<div style="max-width: 800px; margin: 0 auto;">

    <!-- Info o studencie -->

    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-akcent);">

        <div class="karta__tresc" style="display: flex; justify-content: space-between; align-items: center;">

            <div>

                <h2 style="font-size: 1.1rem; margin-bottom: 0.25rem;">{{ praktyka.student.imie }} {{ praktyka.student.nazwisko
                }}</h2>

                <p style="color: var(--kolor-tekst-drugor); font-size: 0.9rem;">

                    Zakład: {{ praktyka.zaklad.nazwa }} | Godziny: {{ praktyka.lacznie_godzin }} / 160 h

                </p>

            </div>

            <div style="text-align: right;">

                <span class="status status--{{ praktyka.status.value|lower|replace('_', '-') }}">{{
                tlumacz_status(praktyka.status.value) }}</span>

            </div>

        </div>

    </div>

</div>

<!-- Wzór -->

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Wzór obliczania oceny końcowej (Zał. 8)</h3>
```

[illegible]

```

        <option value="{ { stopien } }" {% if praktyka.ocena_sprawozdania == stopien %}selected{% endif %}>{ { stopien
    }}</option>

    {% endfor %}

</select>

</div>

<div class="pole-formularza">

    <label class="pole-formularza__etykieta">Ocena Opiekuna Uczelnianego (U)</label>

    <select name="ocena_uopz" class="filtr-select" style="width: 100%;" required>

        <option value="">Wybierz...</option>

        {% for stopien in [2.0, 3.0, 3.5, 4.0, 4.5, 5.0] %}

            <option value="{ { stopien } }" {% if praktyka.ocena_uopz == stopien %}selected{% endif %}>{ { stopien
        }}</option>

        {% endfor %}

    </select>

</div>

<div class="pole-formularza">

    <label class="pole-formularza__etykieta">Ocena Opiekuna Zakładowego (Z)</label>

    <select name="ocena_zopz" class="filtr-select" style="width: 100%;" required>

        <option value="">Wybierz...</option>

        {% for stopien in [2.0, 3.0, 3.5, 4.0, 4.5, 5.0] %}

            <option value="{ { stopien } }" {% if praktyka.ocena_zopz == stopien %}selected{% endif %}>{ { stopien
        }}</option>

        {% endfor %}

    </select>

</div>

</div>

</div>

</div>

<!-- Wyliczenie K (JS) -->

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__tresc" style="text-align: center; padding: 1.5rem;">

        <div style="font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.08em; color:

```

```

var(--kolor-tekst-drugor); margin-bottom: 0.5rem;">Ocena końcowa K</div>

    <div id="wynik-k" style="font-size: 3rem; font-weight: 800; color: var(--kolor-glowny);">—</div>

</div>

</div>

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__tresc">

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Członkowie komisji egzaminacyjnej (oddzieleni przecinkami)</label>

            <input type="text" name="czlonkowie_komisji" class="pole-formularza__input"

                placeholder="np. dr Jan Kowalski, mgr Anna Nowak"

                {% if praktyka.committee_members %}value="{{ praktyka.committee_members | join(', ') }}" {% endif %}>

        </div>

    </div>

</div>

<div style="display: flex; justify-content: center; padding-bottom: 4rem;">

    <button type="submit" class="przycisk przycisk--glowny" style="padding: 1rem 3rem; font-size: 1.1rem; box-shadow: 0
4px 14px rgba(26, 58, 92, 0.3);">

        Zatwierdź egzamin i zakończ praktykę

    </button>

</div>

</form>

</div>

{% endblock %}

{% block scripts_extra %}

<script>

function obliczK() {

    const e = parseFloat(document.querySelector('[name="ocena_egzamin"]').value) || 0;

    const s = parseFloat(document.querySelector('[name="ocena_sprawozdanie"]').value) || 0;

    const u = parseFloat(document.querySelector('[name="ocena_uopz"]').value) || 0;

    const z = parseFloat(document.querySelector('[name="ocena_zopz"]').value) || 0;

    const wynik = document.getElementById('wynik-k');

    if (e && s && u && z) {

```

```
const k = (0.4 * e + 0.1 * s + 0.2 * u + 0.3 * z).toFixed(1);

wynik.textContent = k;

wynik.style.color = k >= 3.0 ? 'var(--kolor-sukces)' : 'var(--kolor-blad)';

} else {

    wynik.textContent = '-';

    wynik.style.color = 'var(--kolor-glowny)';

}

}

document.querySelectorAll('select').forEach(s => s.addEventListener('change', obliczK));

</script>

{% endblock %}
```


PLIK: app_admin\templates\evaluation\formularz_ocen.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Ocena praktyki{% endblock %}

{% block naglowek %}Karta oceny efektów uczenia się{% endblock %}

{% block naglowek_akcje %}

<a href="{% url_for('evaluation.lista_ocen') %}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

{% endblock %}

{% block tresc %}

<div class="formularz-ocen-kontener" style="max-width: 1000px; margin: 0 auto;">

    <!-- Info o studentie -->

    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-glowny);">

        <div class="karta__tresc" style="display: flex; justify-content: space-between; align-items: center;">

            <div>

                <h2 style="font-size: 1.1rem; margin-bottom: 0.25rem;">{{ praktyka.student.imie }} {{ praktyka.student.nazwisko
                }}</h2>

                <p style="color: var(--kolor-tekst-drugor); font-size: 0.9rem;">Zakład: {{ praktyka.zaklad.nazwa }} | Ścieżka: {{
                praktyka.typ_sciezki.value }}</p>

            </div>

            <div style="text-align: right;">

                <span class="status status--oczekuje">Oczekiwanie na ocenę</span>

            </div>

        </div>

    </div>

    <form method="POST">

        <!-- 13 Efektów uczenia się -->

        <section class="karta" style="margin-bottom: 2rem;">

            <header class="karta__naglowek">

                <h3 class="karta__tytul">Ocena 13 efektów uczenia się (Załącznik 4/4a)</h3>

            </header>

            <div class="karta__tresc" style="padding: 0;">
```

```

<div class="tabela-wrapper">

  <table class="tabela">

    <thead>

      <tr>

        <th scope="col" style="width: 60px;">Kod</th>

        <th scope="col">Opis efektu</th>

        <th scope="col" style="width: 280px;">Wynik</th>

      </tr>

    </thead>

    <tbody>

      {% for efekt in efekty %}

      {% set ocena = istnieje.get(efekt.id|string) %}

      <tr>

        <td><strong>{{ efekt.kod }}</strong></td>

        <td style="font-size: 0.85rem; line-height: 1.4;">{{ efekt.opis }}</td>

        <td>

          <div style="display: flex; gap: 0.75rem; flex-wrap: wrap;">

            <label class="radio-ocena">

              <input type="radio" name="wynik_{{ efekt.id }}" value="OSIAGNIETO" {% if ocena and ocena.wynik.name
== 'OSIAGNIETO' %}>checked{% endif %} required>

              <span>Osiągnięto</span>

            </label>

            {% if praktyka.typ_sciezki.value in ['ZATRUDNIENIE', 'DZIALALNOSC'] %}

            <label class="radio-ocena">

              <input type="radio" name="wynik_{{ efekt.id }}" value="CZESCIOWO" {% if ocena and ocena.wynik.name
== 'CZESCIOWO' %}>checked{% endif %}>

              <span>Częściowo</span>

            </label>

            {% endif %}

            <label class="radio-ocena">

              <input type="radio" name="wynik_{{ efekt.id }}" value="NIE_OSIAGNIETO" {% if ocena and
ocena.wynik.name == 'NIE_OSIAGNIETO' %}>checked{% endif %}>

              <span>Nie osiągnięto</span>

            </label>

          </div>

        </td>

      </tr>

      {% endfor %}

    </tbody>

  </table>

</div>

```

```

        </div>

        {% if praktyka.typ_sciezki.value in ['ZATRUDNIENIE', 'DZIALALNOSC'] %}

        <input type="text" name="uwagi_{{ efekt.id }}" value="{{ ocena.uwagi_oceniaczajacego if ocena else '' }}"

                placeholder="Uwagi (wymagane przy częściowym)"

                style="width: 100%; margin-top: 0.5rem; padding: 0.4rem; font-size: 0.8rem; border: 1px solid
var(--kolor-ramka); border-radius: 4px;">

        {% endif %}

    </td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

</div>

</section>

<!-- Oceny ogólne i opisowa -->

<div class="uklad-dashboardu" style="display: grid; grid-template-columns: 1fr 1fr; gap: 2rem;">

    <section class="karta">

        <header class="karta__naglowek">

            <h3 class="karta__tytul">Oceny parametryczne</h3>

        </header>

        <div class="karta__tresc" style="display: flex; flex-direction: column; gap: 1.5rem;">

            <div class="pole-formularza">

                <label class="pole-formularza__etykieta">Ocena sprawozdania (S)</label>

                <select name="ocena_sprawozdania" class="filtr-select" style="width: 100%;">

                    <option value="">Wybierz...</option>

                    {% for stopien in [2.0, 3.0, 3.5, 4.0, 4.5, 5.0] %}

                        <option value="{{ stopien }}" {% if praktyka.ocena_sprawozdania == stopien %}selected{% endif %}>{{ stopien
}}</option>

                    {% endfor %}

                </select>

            </div>

            <div class="pole-formularza">

```

```

<label class="pole-formularza__etykieta">Ocena Opiekuna Uczelnianego (U)</label>

<select name="ocena_uopz" class="filtr-select" style="width: 100%;">

    <option value="">Wybierz...</option>

    {% for stopien in [2.0, 3.0, 3.5, 4.0, 4.5, 5.0] %}

        <option value="{{ stopien }}" {% if praktyka.ocena_uopz == stopien %}selected{% endif %}>{{ stopien
    }}</option>

    {% endfor %}

</select>

</div>

</div>

</section>

<section class="karta">

    <header class="karta__naglowek">

        <h3 class="karta__tytul">Opinia opisowa UOPZ</h3>

    </header>

    <div class="karta__tresc">

        <div class="pole-formularza">

            <label for="ocena_opisowa_uopz" class="pole-formularza__etykieta">Uzasadnienie oceny i uwagi</label>

            <textarea id="ocena_opisowa_uopz" name="ocena_opisowa_uopz" class="pole-formularza__input" style="min-height:
120px; resize: vertical;">{{ praktyka.ocena_opisowa_uopz or '' }}</textarea>

        </div>

    </div>

</section>

</div>

<div style="margin-top: 2rem; display: flex; justify-content: center; padding-bottom: 4rem;">

    <button type="submit" class="przycisk przycisk--glowny" style="padding: 1rem 3rem; font-size: 1.1rem; box-shadow: 0
4px 14px rgba(26, 58, 92, 0.3);">

        Zapisz arkusz oceny

    </button>

</div>

</form>

</div>

```

```
<style>

.radio-ocena {

  display: flex;

  align-items: center;

  gap: 0.4rem;

  font-size: 0.85rem;

  cursor: pointer;

  background: var(--kolor-tlo);

  padding: 0.3rem 0.6rem;

  border-radius: 4px;

  border: 1px solid var(--kolor-ramka);

  transition: all 0.2s;

}

.radio-ocena:hover {

  border-color: var(--kolor-glowny-jasny);

}

.radio-ocena input:checked + span {

  font-weight: 600;

  color: var(--kolor-glowny);

}

.radio-ocena input {

  accent-color: var(--kolor-glowny);

}

</style>

{% endblock %}
```

<h1>Evaluations</h1>

PLIK: app_admin\templates\evaluation\karta_ocen.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Karta Ocen Praktyki{% endblock %}

{% block naglowek %}Karta Ocen (Zał. 3 / Zał. 8){% endblock %}

{% block naglowek_akcje %}

<a href="{{ url_for('evaluation.lista_ocen') }}" class="przycisk przycisk--drugorzeczny">Powrót do listy</a>

{% endblock %}

{% block tresc %}

<div style="max-width: 900px; margin: 0 auto;">

    <!-- Info o studencie -->

    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-akcent);">

        <div class="karta__tresc" style="display: flex; justify-content: space-between; align-items: center;">

            <div>

                <h2 style="font-size: 1.1rem; margin-bottom: 0.25rem;">{{ zapis.student.imie }} {{ zapis.student.nazwisko }}</h2>

                <p style="color: var(--kolor-tekst-drugor); font-size: 0.9rem;">

                    Firma: {{ zapis.firma_nazwa or 'Brak nazwy firmy' }} | ZOPZ: {{ zapis.zopz_imie_nazwisko or 'Brak ZOPZ' }} |

                    Godziny: {{ zapis.lacznie_godzin }} h

                </p>

            </div>

            <div style="text-align: right;">

                <span class="status status--{{ zapis.status.value|lower|replace('_', '-') }}">{{

                    tlumacz_status(zapis.status.value) }}</span>

            </div>

        </div>

    </div>

</div>

<form method="POST">

    {{ csrf_form.hidden_tag() }}

    <!-- Karta Ocen (Zał. 3) -->

    <div class="karta" style="margin-bottom: 2rem;">
```

```

<div class="karta__naglowek" style="background: var(--kolor-tlo);">

    <h3 class="karta__tytul">Karta oceny praktyki (Zař. 3)</h3>

</div>

<div class="karta__tresc">

    <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-bottom: 1rem;">

        Wprowadź oceny parametryczne. Ocenę ZOPZ przepis z dostarczonego formularza papierowego podpisanej Karty nr 3.

    </p>

    <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem; margin-bottom: 1.5rem;">

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Ocena Sprawozdania (S)</label>

            <input type="number" step="0.5" min="2" max="5" name="ocena_sprawozdania" class="pole-formularza__input ocen_field" value="{{ zapis.ocena_sprawozdania or '' }}" placeholder="2.0 - 5.0">

        </div>

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Ocena UOPZ (U)</label>

            <input type="number" step="0.5" min="2" max="5" name="ocena_uopz" class="pole-formularza__input ocen_field" value="{{ zapis.ocena_uopz or '' }}" placeholder="2.0 - 5.0">

        </div>

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Ocena z papierowej Karty ZOPZ (Z)</label>

            <input type="number" step="0.5" min="2" max="5" name="ocena_zopz" class="pole-formularza__input ocen_field" value="{{ zapis.ocena_zopz or '' }}" placeholder="2.0 - 5.0">

        </div>

    </div>

    <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem;">

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Opinia opisowa Uczelnianego Opiekuna Praktyk Zawodowych (UOPZ)</label>

            <textarea name="ocena_opisowa_uopz" class="pole-formularza__input" rows="4" placeholder="Wpisz treść opinii...">{{ zapis.ocena_opisowa_uopz or '' }}</textarea>

        </div>

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Opinia opisowa z dostarczonej karty ZOPZ (opcjonalnie)</label>

            <textarea name="ocena_opisowa_zopz" class="pole-formularza__input" rows="4" placeholder="Przepisz jeśli obecna...">{{ zapis.ocena_opisowa_zopz or '' }}</textarea>

        </div>

    </div>

```


</div>

</div>

</div>

</div>

<!-- Komisja Egzaminacyjna (Załącz. 8) -->

<div class="karta" style="margin-bottom: 2rem;">

<div class="karta__naglowek" style="background: var(--kolor-tlo);">

<h3 class="karta__tytul">Protokół Egzaminacyjny (Załącz. 8)</h3>

</div>

<div class="karta__tresc">

<p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-bottom: 1.5rem;">

Wprowadź 3 pytania zadane studentowi oraz ocenę (od 2.0 do 5.0) za każdą odpowiedź. Ocena E jest automatyczną średnią z poniższych.

</p>

<div style="display: grid; grid-template-columns: 3fr 1fr; gap: 1rem; margin-bottom: 1rem; align-items: end;">

<div class="pole-formularza">

<label class="pole-formularza__etykieta">Pytanie 1</label>

<input type="text" name="sprawdzian_pytanie_1" class="pole-formularza__input" value="{ { zapis.sprawdzian_pytanie_1 or '' } }" placeholder="Treść pytania...">

</div>

<div class="pole-formularza">

<label class="pole-formularza__etykieta">Ocena z odp. 1</label>

<input type="number" step="0.5" min="2" max="5" name="sprawdzian_ocena_1" class="pole-formularza__input egzamin_odp" value="{ { zapis.sprawdzian_ocena_1 or '' } }" placeholder="2.0 - 5.0">

</div>

</div>

<div style="display: grid; grid-template-columns: 3fr 1fr; gap: 1rem; margin-bottom: 1rem; align-items: end;">

<div class="pole-formularza">

<label class="pole-formularza__etykieta">Pytanie 2</label>

<input type="text" name="sprawdzian_pytanie_2" class="pole-formularza__input" value="{ { zapis.sprawdzian_pytanie_2 or '' } }" placeholder="Treść pytania...">

</div>

<div class="pole-formularza">

```

<label class="pole-formularza__etykieta">Ocena z odp. 2</label>

    <input type="number" step="0.5" min="2" max="5" name="sprawdzian_ocena_2" class="pole-formularza__input
egzamin_odp" value="{{ zapis.sprawdzian_ocena_2 or '' }}" placeholder="2.0 - 5.0">

</div>

</div>

<div style="display: grid; grid-template-columns: 3fr 1fr; gap: 1rem; align-items: end;">

    <div class="pole-formularza">

        <label class="pole-formularza__etykieta">Pytanie 3</label>

            <input type="text" name="sprawdzian_pytanie_3" class="pole-formularza__input" value="{{
zapis.sprawdzian_pytanie_3 or '' }}" placeholder="Treść pytania...">

        </div>

    <div class="pole-formularza">

        <label class="pole-formularza__etykieta">Ocena z odp. 3</label>

            <input type="number" step="0.5" min="2" max="5" name="sprawdzian_ocena_3" class="pole-formularza__input
egzamin_odp" value="{{ zapis.sprawdzian_ocena_3 or '' }}" placeholder="2.0 - 5.0">

        </div>

    </div>

</div>

</div>

</div>

<!-- Podsumowanie Wyników -->

<div class="karta" style="margin-bottom: 2rem; border-top: 4px solid var(--kolor-akcent);">

    <div class="karta__tresc" style="display: flex; justify-content: space-around; align-items: center; padding:
2rem;">

        <div style="text-align: center;">

            <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); text-transform: uppercase;">Średnia Egzaminu
(E)</div>

            <div id="wynik-e" style="font-size: 2.5rem; font-weight: 700; color: var(--kolor-glowny);">--</div>

        </div>

    <div style="font-size: 1.5rem; color: var(--kolor-ramka);">+</div>

    <div style="text-align: center;">

        <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); text-transform: uppercase;">Wzór
Obliczeniowy</div>

```

```

<div style="font-family: monospace; font-size: 1.1rem; font-weight: 500; margin-top: 0.5rem; color: #444;">K =
0.4*E + 0.1*S + 0.2*U + 0.3*Z</div>

</div>

<div style="font-size: 1.5rem; color: var(--kolor-ramka);">=</div>

<div style="text-align: center;">

    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); text-transform: uppercase;">Ocena Końcowa
(K)</div>

    <div id="wynik-k" style="font-size: 2.5rem; font-weight: 700; color: var(--kolor-glowny);">—</div>

</div>

</div>

<!-- Akcje -->

<div style="display: flex; justify-content: space-between; align-items: center; padding-bottom: 3rem;">

    <span style="font-size: 0.85rem; color: var(--kolor-blad);" id="ostrzezenie-z">Obliczenie K i zatwierdzenie jest
zablokowane do momentu wpisania oceny "Z".</span>

    <div style="display: flex; gap: 1rem; flex-wrap: wrap;">

        <button type="submit" name="zapisz" value="1" class="przycisk przycisk--drugorzecny" style="box-shadow: 0 2px 8px
rgba(0,0,0,0.1);">Zapisz Wersję Roboczą</button>

        <button type="submit" name="zakoncz" value="1" id="btn-zakoncz" class="przycisk przycisk--glowny"
style="box-shadow: 0 4px 14px rgba(26, 58, 92, 0.3);" disabled>Zakończ i Zatwierdź Ocenę</button>

        {% if zapis.ocena_uopz is not none %}

<a href="{% url_for('evaluation.generuj_protokol', id=zapis.id) %}"

    class="przycisk przycisk--drugorzecny" style="box-shadow: 0 2px 8px rgba(0,0,0,0.1);">

        Pobierz Protokół (Załącznik 8)

</a>

        {% endif %}

    </div>

</div>

</div>

</form>

</div>

```

```
{% block scripts_extra %}  
  
<script>  
  
document.addEventListener('DOMContentLoaded', () => {  
  
    const p1 = document.querySelector('[name="sprawdzian_ocena_1"]');  
  
    const p2 = document.querySelector('[name="sprawdzian_ocena_2"]');  
  
    const p3 = document.querySelector('[name="sprawdzian_ocena_3"]');  
  
  
    const os = document.querySelector('[name="ocena_sprawozdania"]');  
  
    const ou = document.querySelector('[name="ocena_uopz"]');  
  
    const oz = document.querySelector('[name="ocena_zopz"]');  
  
  
  
    const we = document.getElementById('wynik-e');  
  
    const wk = document.getElementById('wynik-k');  
  
    const btnZakoncz = document.getElementById('btn-zakoncz');  
  
    const ostrzezenie = document.getElementById('ostrzezenie-z');  
  
  
  
    function obl() {  
  
        let e1 = parseFloat(p1.value) || 0;  
  
        let e2 = parseFloat(p2.value) || 0;  
  
        let e3 = parseFloat(p3.value) || 0;  
  
  
  
        let s = parseFloat(os.value) || 0;  
  
        let u = parseFloat(ou.value) || 0;  
  
        let z = parseFloat(oz.value) || 0;  
  
  
  
        let exam_avg = 0;  
  
        let numPyt = 0;  
  
        if(e1>0) numPyt++;  
  
        if(e2>0) numPyt++;  
  
        if(e3>0) numPyt++;  
  
        if(numPyt > 0) exam_avg = (e1+e2+e3)/numPyt;  
  
  
  
        we.textContent = exam_avg > 0 ? exam_avg.toFixed(2) : '-';  
  
    }  
  
    btnZakoncz.addEventListener('click', () => {  
        ostrzezenie.textContent = 'Zakończ test';  
        obl();  
    });  
  
});
```

```
if(exam_avg > 0 && s > 0 && u > 0 && z > 0) {

    let k = (0.4 * exam_avg) + (0.1 * s) + (0.2 * u) + (0.3 * z);

    wk.textContent = k.toFixed(2);

    wk.style.color = k >= 3.0 ? 'var(--kolor-sukces)' : 'var(--kolor-blad)';

    btnZakoncz.disabled = false;

    ostrzezenie.style.display = 'none';

} else {

    wk.textContent = '-';

    wk.style.color = 'var(--kolor-glowny)';

    btnZakoncz.disabled = true;

    ostrzezenie.style.display = 'inline-block';

}

}

[p1, p2, p3, os, ou, oz].forEach(el => el.addEventListener('input', obl));

obl();

});

</script>

{% endblock %}

{% endblock %}
```

PLIK: app_admin\templates\evaluation\lista_ocen.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Oceny i efekty{% endblock %}

{% block naglowek %}Studenci – oceny i efekty{% endblock %}


{% block tresc %}

<div style="max-width: 1200px; margin: 0 auto;">


{% macro tabela_studentow(items, tytul, kolor) %}

<div class="karta" style="margin-bottom: 1.5rem;">

    <div class="karta__naglowek" style="border-left: 4px solid {{ kolor }};">

        <h2 class="karta__tytul">{{ tytul }} <span style="font-size:0.85rem; font-weight:400;
color:var(--kolor-tekst-drugor);">({{ items|length }})</span></h2>

    </div>

    <div class="karta__tresc">

        {% if items %}

        <div class="tabela-wrapper">

            <table class="tabela">

                <thead>

                    <tr>

                        <th>Student</th>

                        <th>Kierunek / Specjalność</th>

                        <th>Zakład pracy</th>

                        <th>Termin</th>

                        <th>Godziny</th>

                        <th>Akcje</th>

                    </tr>

                </thead>

                <tbody>

                    {% for item in items %}

                    {% set p = item.zapis %}

                    <tr {% if item.przekroczony %}style="background: rgba(220,38,38,0.04);" {% endif %}>

                        <td>
```

```

<div style="font-weight:600;">{{ p.student.last_name }} {{ p.student.first_name }}</div>

<div style="font-size:0.8rem; color:var(--kolor-tekst-drugor);">

    Nr alb.: {{ p.student.album_number or '-' }}

    {% if p.student.plec %} · {{ 'K' if p.student.plec == 'K' else 'M' }}{% endif %}

</div>

</td>

<td style="font-size:0.85rem;">

    {{ p.student.kierunek or 'Informatyka' }}<br>

    <span style="color:var(--kolor-tekst-drugor);">{{ p.student.specjalnosc or p.specjalnosc or '-' }}</span>

</td>

<td style="font-size:0.85rem;">

    {{ (p.firma.nazwa if p.firma else p.firma_nazwa) or '-' }}

</td>

<td style="font-size:0.85rem;">

    {{ p.termin_od.strftime('%d.%m.%Y') if p.termin_od else '-' }} -

    {{ p.termin_do.strftime('%d.%m.%Y') if p.termin_do else '-' }}

    {% if item.deadline %}

        <div style="font-size:0.75rem; {% if item.przekroczony %}color:var(--kolor-blad); font-weight:600;{% elif
item.dni_do_deadline <= 2 %}color:#d97706;{% else %}color:var(--kolor-tekst-drugor);{% endif %}">

            {% if item.przekroczony %}Deadline przekroczony o {{ -item.dni_do_deadline }} dni

            {% elif item.dni_do_deadline == 0 %}Dziś ostatni dzień!

            {% else %}Deadline za {{ item.dni_do_deadline }} dni{% endif %}

        </div>

    {% endif %}

</td>

<td><strong>{{ p.total_hours_logged or 0 }} h</strong></td>

<td>

    <a href="{{ url_for('evaluation.ocen_praktyke', id=p.id) }}"

        class="przycisk przycisk--maly {% if item.przekroczony %}przycisk--niebezpieczny{% elif item.zakonczone
%}przycisk--glowny{% else %}przycisk--drugorzedy{% endif %}">

        {% if item.przekroczony %}PILNE: Ocen{% elif item.zakonczone %}Wystaw ocenę{% else %}Podgląd{% endif %}

    </a>

</td>

</tr>

{% endfor %}

```

```
</tbody>

</table>

</div>

{% else %}

<div style="text-align:center; padding:2rem; color:var(--kolor-tekst-drugor);">Brak studentów.</div>

{% endif %}

</div>

</div>

{% endmacro %}


{{ tabela_studentow(zakonczone, 'Gotowi do oceny (zakończzone praktyki)', '#dc2626') }}

{{ tabela_studentow(w_trakcie, 'W trakcie praktyki', '#3b82f6') }}


</div>

{% endblock %}
```


PLIK: app_admin\templates\evaluation\podglad_sprawozdania.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Podgląd Sprawozdania{% endblock %}

{% block naglowek %}Sprawozdanie: {{ zapis.student.imie }} {{ zapis.student.nazwisko }}{% endblock %}


{% block naglowek_akcje %}

<a href="{% url_for('zarzadzanie.szczegoly_praktyki', id=zapis.internship_id) %}" class="przycisk przycisk--drugorzeczny">Powrót do szczegółów</a>

<a href="{% url_for('evaluation.ocen_praktyke', id=zapis.id) %}" class="przycisk przycisk--glowny">Przejdź do oceniania</a>

{% endblock %}


{% block tresc %}

<div style="max-width: 900px; margin: 0 auto; padding-bottom: 3rem;">

    {% if not zapis.sprawozdanie %}

    <div class="karta" style="text-align: center; padding: 3rem 2rem;">

        <p style="color: var(--kolor-tekst-drugor);">Student jeszcze nie rozpoczął generowania / pisania sprawozdania.</p>

    </div>

    {% else %}

    <div class="karta" style="margin-bottom: 2rem;">

        <div class="karta__naglowek" style="background: var(--kolor-tlo);">

            <h2 class="karta__tytul">Sekcja I: Charakterystyka miejsca roboczego</h2>

        </div>

        <div class="karta__tresc">

            <div style="white-space: pre-wrap; font-size: 0.95rem; line-height: 1.6;">{{ zapis.sprawozdanie.charakterystyka_miejsca or 'Brak' }}</div>

        </div>

    </div>

    <div class="karta" style="margin-bottom: 2rem;">

        <div class="karta__naglowek" style="background: var(--kolor-tlo);">

            <h2 class="karta__tytul">Sekcja II: Opis i analiza prac</h2>

        </div>

    </div>

    </div>

    </div>
```

```
</div>

<div class="karta__tresc">

    <div style="white-space: pre-wrap; font-size: 0.95rem; line-height: 1.6;">{{ zapis.sprawozdanie.opis_i_analiza or
'Brak' }}</div>

</div>

</div>


<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__naglowek" style="background: var(--kolor-tlo);">

        <h2 class="karta__tytul">Sekcja III (Z Dziennika)</h2>

    </div>

    <div class="karta__tresc">

        <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">Tabela zliczająca godziny dla każdego efektu z
dziennika (w PDF).</p>

        <a href="{{ url_for('journal.podglad', id=zapis.id) }}" class="przycisk przycisk--drugorzecny przycisk--maly"
style="margin-top: 1rem;">Zobacz pełny dziennik wpisów</a>

    </div>

</div>


{% endif %}


</div>

{% endblock %}
```

PLIK: app_admin\templates\journal\dziennik.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Dziennik praktyki{% endblock %}

{% block naglowek %}Dziennik praktyki{% endblock %}

{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem; align-items: center;">

    <a href="{{ url_for('journal.lista_dziennikow') }}" class="przycisk przycisk--drugorzeczny">Powrót do listy</a>

    <button id="btn-pdf" onclick="zlecPDF()" class="przycisk przycisk--glowny">

        <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"
stroke-linecap="round" stroke-linejoin="round" style="vertical-align: -1px;"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0
1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>

        <span id="btn-pdf-label">Pobierz PDF</span>

    </button>

    <span id="pdf-status" style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); display: none;"></span>

</div>

{% endblock %}

{% block tresc %}

<div style="max-width: 1000px; margin: 0 auto;">

    <!-- Info o zapisie -->

    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-glowny);">

        <div class="karta__tresc">

            <div style="display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 1.5rem;">

                <div>

                    <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom:
0.25rem;">Student</div>

                    <div style="font-weight: 600;">{{ zapis.student.first_name }} {{ zapis.student.last_name }}</div>

                    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Album: {{ zapis.student.album_number or '-'
}}</div>

                </div>

                <div>


```

```

        <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Praktyka</div>

        <div style="font-weight: 600;">{{ zapis.praktyka.rok_uczelnianny }}</div>

        <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Semestr {{ zapis.praktyka.semestr }}</div>

    </div>

    <div>

        <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Postęp godzin</div>

        <div style="font-weight: 700; font-size: 1.2rem; color: var(--kolor-glowny);">{{ suma_godzin }} / {{ zapis.praktyka.wymiar_godzin }} h</div>

        <div class="pasek-postep" style="margin-top: 0.5rem; height: 8px;">

            <div class="pasek-postep__wypelnienie {{ 'pasek-postep__wypelnienie--zakonczone' if suma_godzin >= zapis.praktyka.wymiar_godzin else '' }}"

                style="width: {{ [(suma_godzin / zapis.praktyka.wymiar_godzin) * 100]|round|int, 100]|min }}%;"></div>

            </div>

        </div>

    </div>

</div>

</div>

</div>

<!-- Alerty przerw -->

{% if przerwy %}

<div style="margin-bottom: 2rem;">

    {% for p in przerwy %}

        <div class="komunikat komunikat--danger" style="margin-bottom: 0.5rem;">

            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round" style="flex-shrink: 0; margin-top: 2px;">

                <path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"/>

                <line x1="12" y1="9" x2="12" y2="13"/>

                <line x1="12" y1="17" x2="12.01" y2="17"/>

            </svg>

            <span>Przerwa <strong>{{ p.dni_roboczych }} dni roboczych</strong> między {{ p.od.strftime('%d.%m')} } a {{ p.do.strftime('%d.%m.%Y')} }</span>

        </div>

    {% endfor %}

</div>

{% endif %}

```

```

<!-- Wpisy -->

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Wpisy dziennika ({{ wpisy|length }})</h3>

    </div>

    <div class="karta__tresc" style="padding: 0;">

        <div class="tabela-wrapper">

            <table class="tabela">

                <thead>

                    <tr>

                        <th scope="col" style="width: 110px;">Data</th>

                        <th scope="col" style="width: 80px;">Godziny</th>

                        <th scope="col">Opis czynności</th>

                        <th scope="col" style="width: 70px;">Efekt</th>

                    </tr>

                </thead>

                <tbody>

                    {% for w in wpisy %}

                        <tr>

                            <td><strong>{{ w.entry_date.strftime('%d.%m.%Y') }}</strong></td>

                            <td style="text-align: center;">{{ w.duration_hours }} h</td>

                            <td style="font-size: 0.85rem; line-height: 1.5;">{{ w.description }}</td>

                            <td style="white-space: nowrap; text-align: center;">

                                {% if w.efekty_uczenia %}

                                    {% for e in w.efekty_uczenia %}

                                        <span style="font-size: 0.75rem; background: var(--kolor-tlo); padding: 0.2rem 0.4rem; border-radius: 4px; font-weight: 600;">{{ e.kod }}</span>

                                    {% endfor %}

                                {% else %}—{% endif %}

                            </td>

                        </tr>

                    {% else %}

                        <tr>

                            <td colspan="4" class="tabela--pusta">Student nie dodał jeszcze żadnych wpisów.</td>

                        </tr>

                    {% end %}

                </tbody>

            </table>

        </div>

    </div>

</div>

```

```

        </tr>

        {% endfor %}

    </tbody>

</table>

</div>

</div>

</div>

</div>

<script>

const ZAPIS_ID = "{{ zapis.id }}";

const URL_ZLEC = `/dzienniki/zapis/${ZAPIS_ID}/pdf`;

const URL_STATUS = (taskId) => `/dzienniki/zapis/${ZAPIS_ID}/pdf/status/${taskId}`;

const URL_POBIERZ = (taskId) => `/dzienniki/zapis/${ZAPIS_ID}/pdf/pobierz/${taskId}`;


const btn = document.getElementById('btn-pdf');

const btnLabel = document.getElementById('btn-pdf-label');

const statusEl = document.getElementById('pdf-status');


let pollInterval = null;


function zlecPDF() {

    btn.disabled = true;

    btnLabel.textContent = 'Generowanie...';

    statusEl.style.display = 'inline';

    statusEl.textContent = 'Zlecam generowanie PDF...';


    fetch(URL_ZLEC, {

        method: 'POST',

        headers: { 'Content-Type': 'application/json', 'X-CSRFToken': getCsrf() },

    })

    .then(r => r.json())

    .then(data => {

```

```

    if (data.task_id) {

        statusEl.textContent = 'Kompilacja w toku...';

        pollInterval = setInterval(() => sprawdzStatus(data.task_id), 2000);

    } else {

        blad('Błąd zlecenia: ' + (data.error || 'nieznany'));

    }

})

.catch(() => blad('Błąd połączenia z serwerem.'));

}

```

```

function sprawdzStatus(taskId) {

    fetch(URL_STATUS(taskId))

    .then(r => r.json())

    .then(data => {

        if (data.status === 'SUCCESS') {

            clearInterval(pollInterval);

            statusEl.textContent = 'Gotowe!';

            btnLabel.textContent = 'Pobierz PDF';

            btn.disabled = false;

            // Automatyczne pobranie

            window.location.href = URL_POBIERZ(taskId);

        } else if (data.status === 'FAILURE') {

            clearInterval(pollInterval);

            blad('Błąd kompilacji: ' + (data.error || 'sprawdź logi workera'));

        } else {

            const progress = data.progress || 0;

            statusEl.textContent = `Kompilacja... ${progress}%`;

        }

    })

    .catch(() => {});

}

```

```

function blad(msg) {

    clearInterval(pollInterval);

}

```

```
btn.disabled = false;

btnLabel.textContent = 'Pobierz PDF';

statusEl.textContent = msg;

statusEl.style.color = 'var(--kolor-blad)';

}


function getCsrftoken() {

    const meta = document.querySelector('meta[name="csrf-token"]');

    return meta ? meta.getAttribute('content') : '';

}

</script>

{% endblock %}
```


PLIK: app_admin\templates\journal\lista.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Dzienniki praktyk{% endblock %}

{% block naglowek %}Monitor dzienników{% endblock %}

{% block tresc %}

<div class="karta">

    <div class="karta__naglowek">

        <form class="pasek-filtrow" method="GET" action="{{ url_for('journal.lista_dziennikow') }}">

            <div class="filtr-szukaj">

                <span class="filtr-szukaj__ikona" aria-hidden="true"><svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="11" cy="11"
r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg></span>

                <input type="text" name="szukaj" value="{{ request.args.get('szukaj', '') }}" class="filtr-szukaj__input"
placeholder="Szukaj po nazwisku studenta..." aria-label="Wyszukaj dziennik">

            </div>

            <button type="submit" class="przycisk przycisk--drugorzedny">Szukaj</button>

        </form>

    </div>

    <div class="karta__tresc">

        <div class="tabela-wrapper">

            <table class="tabela">

                <thead>

                    <tr>

                        <th scope="col">Student</th>

                        <th scope="col">Praktyka</th>

                        <th scope="col">Godziny</th>

                        <th scope="col">Wpisy</th>

                        <th scope="col">Ostatni wpis</th>

                        <th scope="col">Status</th>

                        <th scope="col">Akcje</th>

                    </tr>

                </thead>

                <tbody>
```

```

{% for d in dane %}

<tr {% if d.alert %}style="background: rgba(192, 57, 43, 0.05);"{% endif %}>

    <td>

        <div style="font-weight: 600;">{{ d.zapis.student.last_name }} {{ d.zapis.student.first_name }}</div>

        <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Album: {{ d.zapis.student.album_number or
'-' }}</div>

    </td>

    <td>

        <div>{{ d.zapis.praktyka.rok_uczelnianny }}</div>

        <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ d.zapis.praktyka.semestr | capitalize
}}</div>

    </td>

    <td>

        <div style="font-weight: 600;">{{ d.zapis.total_hours_logged }} / {{ d.zapis.praktyka.wymiar_godzin }}

h</div>

        <div class="pasek-postep" style="margin-top: 4px;">

            <div class="pasek-postep__wypelnienie {% 'pasek-postep__wypelnienie--zakonczone' if
d.zapis.total_hours_logged >= d.zapis.praktyka.wymiar_godzin else '' %}"

                style="width: {{ [(d.zapis.total_hours_logged / d.zapis.praktyka.wymiar_godzin) * 100]|round|int,
100]|min }}%;"></div>

            </div>

        </td>

    <td><strong>{{ d.liczba_wpisow }}</strong></td>

    <td>

        {% if d.ostatni_wpis %}

            {{ d.ostatni_wpis.strftime('%d.%m.%Y') }}

        {% else %}

            <span style="color: var(--kolor-tekst-trzeciorzed);">Brak wpisów</span>

        {% endif %}

    </td>

    <td>

        {% if d.alert %}

            <span class="status" style="background: #fef2f2; color: var(--kolor-blad);">

                Przerwa &gt;3 dni

            </span>

        {% else %}

            <span class="status status--in-progress">OK</span>


```

```

        {% endif %}

    </td>

    <td>

        <a href="{{ url_for('journal.dziennik_zapisu', id=d.zapis.id) }}" class="przycisk przycisk--drugorzedny przycisk--maly">Podgląd</a>

    </td>

</tr>

{% else %}

<tr>

    <td colspan="7" class="tabela--pusta">Brak aktywnych dzienników.</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

```

```

{% if zapisy.pages > 1 %}

<nav class="paginacja" aria-label="Nawigacja po stronach">

    <span class="paginacja__info">Strona {{ zapisy.page }} z {{ zapisy.pages }}</span>

    <div class="paginacja__przyciski">

        {% for strona in zapisy.iter_pages() %}

            {% if strona %}

                <a href="{{ url_for('journal.lista_dziennikow', strona=strona, szukaj=request.args.get('szukaj', '')) }}"

                    class="paginacja__przycisk {{ 'paginacja__przycisk--aktywny' if strona == zapisy.page else '' }}"

                    {{ strona }}

                </a>

            {% else %}

                <span class="paginacja__separator">...</span>

            {% endif %}

        {% endfor %}

    </div>

</nav>

{% endif %}

</div>

</div>

```

{% endblock %}

PLIK: app_admin\templates\layouts\base.html

```
<!DOCTYPE html>

<html lang="pl">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>{% block tytuł %}{% endblock %} – System Praktyk ANS Elbląg</title>

    <link rel="stylesheet" href="{% url_for('static', filename='css/admin.css') %}">

    <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600&family=IBM+Plex+Mono&display=swap"
rel="stylesheet">

</head>

<body class="{% block body_class %}{% endblock %}">

    <a href="#main-content" class="skip-link">Przejdź do treści głównej</a>


    {% block body %}{% endblock %}


    {% block scripts_extra %}{% endblock %}

</body>

</html>
```

PLIK: app_admin\templates\layouts\base_panel.html

```
<!DOCTYPE html>

<html lang="pl">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0, minimum-scale=1.0">

    <meta name="description" content="Panel administracyjny – System Praktyk Zawodowych IIS ANS Elbląg">

    <title>{% block tytuł %}Panel{% endblock %} – IIS ANS Elbląg</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='css/admin.css') }}">

    <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600&family=IBM+Plex+Mono&display=swap"
rel="stylesheet">

    {% block head_extra %}{% endblock %}

</head>

<body class="panel-body">


    <a href="#main-content" class="skip-link">Przejdź do głównej treści</a>


    <nav class="sidebar" aria-label="Nawigacja główna panelu">

        <div class="sidebar__naglowek">

            <div class="branding__logo-kontener" style="max-width: 140px; margin-bottom: 10px;">

                

            </div>

            <span class="sidebar__system-label">Panel administracyjny</span>

        </div>


        <div class="sidebar__uzytkownik" aria-label="Zalogowany użytkownik">

            <div class="sidebar__uzytkownik-avatar" aria-hidden="true">

                {% if current_user.is_authenticated %}

                    {{ current_user.imie[0] }}{{ current_user.nazwisko[0] }}

                {% else %}

                    ??

                {% endif %}

            </div>

            <div class="sidebar__uzytkownik-info">
```

```

{% if current_user.is_authenticated %}

<span class="sidebar__uzytkownik-imie">{{ current_user.imie }} {{ current_user.nazwisko }}</span>

<span class="sidebar__uzytkownik-rola">

    {% if current_user.is_authenticated and current_user.rola and current_user.rola.value == 'ADMIN'
}%Administrator

    {% elif current_user.is_authenticated and current_user.rola and current_user.rola.value == 'UOPZ' %}Opiekun
uczelnianny

    {% endif %}

</span>

{% else %}

<span class="sidebar__uzytkownik-imie">Niezalogowany</span>

{% endif %}

</div>

</div>

<ul class="sidebar__menu" role="list">

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('dashboard.index') }}" class="sidebar__link {% if request.endpoint == 'dashboard.index'
}%sidebar__link--aktywny{% endif %}" {% if request.endpoint == 'dashboard.index' %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><rect x="3" y="3" width="7"
height="7"/><rect x="14" y="3" width="7" height="7"/><rect x="3" y="14" width="7" height="7"/><rect x="14" y="14"
width="7" height="7"/></svg></span> Przegląd

    </a>

</li>

<li class="sidebar__menu-sekcja" aria-hidden="true">Praktyki</li>

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.lista_praktyk') }}" class="sidebar__link {% if 'lista_praktyk' in
(request.endpoint or '') %}sidebar__link--aktywny{% endif %}" {% if 'lista_praktyk' in (request.endpoint or '')
}%}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M16 4h2a2 2 0 0 1 2 2v14a2 2
0 0 1-2 2H6a2 2 0 0 1-2-2V6a2 2 0 0 1 2-2h2"/><rect x="8" y="2" width="8" height="4" rx="1" ry="1"/><line x1="9" y1="12"
x2="15" y2="12"/><line x1="9" y1="16" x2="15" y2="16"/></svg></span> Wszystkie praktyki

    </a>

</li>

{% if current_user.is_authenticated and current_user.rola and current_user.rola.value == 'ADMIN' %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.lista_zgloszen') }}" class="sidebar__link {% if 'zgloszenia' in
(request.endpoint or '') %}sidebar__link--aktywny{% endif %}" {% if 'zgloszenia' in (request.endpoint or '')
}%}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2

```

```

0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14,2 14,8 20,8"/><line x1="16" y1="13" x2="8" y2="13"/><line x1="16"
y1="17" x2="8" y2="17"/><polyline points="10,9 9,9 8,9"/></svg></span> Zgłoszenia studentów{% if nav_oczekujace %} <span
style="display:inline-flex;align-items:center;justify-content:center;background:#ef4444;color:white;border-radius:9999px;
min-width:1.25rem;height:1.25rem;font-size:0.65rem;font-weight:700;padding:0
0.25rem;margin-left:0.35rem;">{{
nav_oczekujace }}</span>{% endif %}

</a>

</li>

{% elif current_user.is_authenticated and current_user.rola and current_user.rola.value == 'UOPZ' %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.moje_zgloszenia') }}" class="sidebar__link {% if 'moje_zgloszenia' in
(request.endpoint or '') %}sidebar__link--aktywny{% endif %}" {% if 'moje_zgloszenia' in (request.endpoint or '')
%}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M14 2H6a2 2 0 0 2 2v16a2 2
0 0 2 2h12a2 2 0 0 2 2-2V8z"/><polyline points="14,2 14,8 20,8"/><line x1="16" y1="13" x2="8" y2="13"/><line x1="16"
y1="17" x2="8" y2="17"/><polyline points="10,9 9,9 8,9"/></svg></span> Moje zgłoszenia

    </a>

</li>

{% endif %}

{% if current_user.is_authenticated and current_user.rola and current_user.rola.value in ['ADMIN', 'UOPZ'] %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('evaluation.lista_ocen') }}" class="sidebar__link {% if 'evaluation' in (request.endpoint or
'') %}sidebar__link--aktywny{% endif %}" {% if 'evaluation' in (request.endpoint or '') %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M9 11l3 3L22 4"/><path d="M21
12v7a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11"/></svg></span> Oceny i efekty

    </a>

</li>

{% endif %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('journal.lista_dziennikow') }}" class="sidebar__link {% if 'journal' in (request.endpoint or
'') %}sidebar__link--aktywny{% endif %}" {% if 'journal' in (request.endpoint or '') %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5
17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/><line x1="9" y1="7" x2="16"
y2="7"/><line x1="9" y1="11" x2="16" y2="11"/><line x1="9" y1="15" x2="13" y2="15"/></svg></span> Dzienniki

    </a>

</li>

{% if current_user.is_authenticated and current_user.rola and current_user.rola.value in ['ADMIN', 'UOPZ'] %}

<li class="sidebar__menu-sekcja" aria-hidden="true">Weryfikacja</li>

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.komisja_lista') }}" class="sidebar__link {% if 'komisja' in (request.endpoint or
'') %}sidebar__link--aktywny{% endif %}" {% if 'komisja' in (request.endpoint or '') %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M9 12l2 4-4"/><path d="M21
12c0 4.97-4.03 9-9 9s-9-4.03-9-9 4.03-9 9-9c.83 0 1.63.11 2.38.32"/></svg></span> Komisja weryfikująca{% if nav_komisja

```



```

%}
style="display:inline-flex;align-items:center;justify-content:center;background:#ef4444;color:white;border-radius:9999px;
min-width:1.25rem;height:1.25rem;font-size:0.65rem;font-weight:700;padding:0 0.25rem;margin-left:0.35rem;">{{ nav_komisja
}}</span>{% endif %}

</a>

</li>

{% endif %}

{% if current_user.is_authenticated and current_user.rola and current_user.rola.value == 'ADMIN' %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.dziekan_lista') }}" class="sidebar__link {% if 'dziekan' in (request.endpoint or
    '') %}sidebar__link--aktywny{% endif %}" {% if 'dziekan' in (request.endpoint or '') %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M12 213.09 6.26L22 9.27L5
        4.87 1.18 6.88L12 17.77L-6.18 3.25L7 14.14 2 9.27L6.91-1.01L12 2z"/></svg></span> Decyzje dziekana{% if nav_dziekan %}
        <span
        style="display:inline-flex;align-items:center;justify-content:center;background:#ef4444;color:white;border-radius:9999px;
        min-width:1.25rem;height:1.25rem;font-size:0.65rem;font-weight:700;padding:0 0.25rem;margin-left:0.35rem;">{{ nav_dziekan
        }}</span>{% endif %}

    </a>

</li>

<li class="sidebar__menu-sekcja" aria-hidden="true">Zarządzanie</li>

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="sidebar__link {% if 'uzytkownik' in
    (request.endpoint or '') %}sidebar__link--aktywny{% endif %}" {% if 'uzytkownik' in (request.endpoint or '')
    %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4
        0 0 0-4 4v2"/><circle cx="9" cy="7" r="4"/><path d="M23 21v-2a4 4 0 0 0-3-3.87"/><path d="M16 3.13a4 4 0 0 1 0
        7.75"/></svg></span> Użytkownicy

    </a>

</li>

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.lista_firm') }}" class="sidebar__link {% if 'firmy' in (request.endpoint or '')
    %}sidebar__link--aktywny{% endif %}" {% if 'firmy' in (request.endpoint or '') %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><rect x="4" y="2" width="16"
        height="20" rx="2" ry="2"/><line x1="9" y1="6" x2="9" y2="6.01"/><line x1="15" y1="6" x2="15" y2="6.01"/><line x1="9"
        y1="10" x2="9" y2="10.01"/><line x1="15" y1="10" x2="15" y2="10.01"/><line x1="9" y1="14" x2="9" y2="14.01"/><line
        x1="15" y1="14" x2="15" y2="14.01"/><path d="M9 18h6"/></svg></span> Firmy

    </a>

</li>

{% endif %}

</ul>

<div class="sidebar__stopka">

    <form method="POST" action="{{ url_for('auth.wylogowanie') }}">

```

```

        <button type="submit" class="sidebar__wylogowanie">

            <svg viewBox="0 0 24 24"><path d="M9 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h4"/><polyline points="16 17 21 12 16
7"/><line x1="21" y1="12" x2="9" y2="12"/></svg> Wyloguj się

        </button>

    </form>

</div>

</nav>

<div class="panel-main">

    <header class="panel-main__naglowek">

        <div class="panel-main__naglowek-tresc">

            <h1 class="panel-main__tytul">{% block naglowek %}{% endblock %}</h1>

            {% block naglowek_akcje %}{% endblock %}

        </div>

        {# flash messages moved to bottom of the page for minimal display #}

    </header>

    <main id="main-content" class="panel-main__tresc" tabindex="-1">

        {% block tresc %}{% endblock %}

    </main>

</div>

{% with wiadomosci = get_flashed_messages(with_categories=True) %}

{% if wiadomosci %}

    <div class="komunikaty komunikaty--fixed" role="status" aria-live="polite">

        {% for kategoria, wiadomosc in wiadomosci %}

            <div class="komunikat komunikat--{{ kategoria }} komunikat--minimal" role="alert">

                <span class="komunikat__ikona" aria-hidden="true">

                    {% if kategoria == 'danger' %}{% elif kategoria == 'success' %}{% else %}{% endif %}

                </span>

                <span class="komunikat__tekst">{{ wiadomosc }}</span>

                <button type="button" class="komunikat__zamknij" aria-label="Zamknij powiadomienie"
onclick="this.parentElement.remove()"></button>

            </div>

        {% endfor %}

    </div>

{% endif %}

</div>

```

```
{% endif %}
```

```
{% endwith %}
```

```
</body>
```

```
</html>
```

PLIK: app_admin\templates\layouts\panel.html

```
<!DOCTYPE html>

<html lang="pl">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0, minimum-scale=1.0">

    <meta name="description" content="Panel administracyjny – System Praktyk Zawodowych IIS ANS Elbląg">

    <title>{% block tytuł %}Panel{% endblock %} – IIS ANS Elbląg</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='css/admin.css') }}">

    <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600&family=IBM+Plex+Mono&display=swap"
rel="stylesheet">

    {% block head_extra %}{% endblock %}

</head>

<body class="panel-body">

    <a href="#main-content" class="skip-link">Przejdź do głównej treści</a>

    <nav class="sidebar" aria-label="Nawigacja główna panelu">

        <div class="sidebar__naglowek">

            <div class="branding__logo-kontener" style="max-width: 140px; margin-bottom: 10px;">

                

            </div>

            <span class="sidebar__system-label">Panel administracyjny</span>

        </div>

        <div class="sidebar__uzytkownik" aria-label="Zalogowany użytkownik">

            <div class="sidebar__uzytkownik-avatar" aria-hidden="true">

                {{ current_user.imie[0] }}{{ current_user.nazwisko[0] }}

            </div>

            <div class="sidebar__uzytkownik-info">

                <span class="sidebar__uzytkownik-imie">{{ current_user.imie }} {{ current_user.nazwisko }}</span>

                <span class="sidebar__uzytkownik-rola">

                    {% if current_user.rola.value == 'ADMIN' %}Administrator

                    {% elif current_user.rola.value == 'UOPZ' %}Opiekun uczelniany
```

```

        {% elif current_user.rola.value == 'ZOPZ' %}Opiekun zakładowy

        {% endif %}

    </span>

</div>

</div>

<ul class="sidebar__menu" role="list">

    <li class="sidebar__menu-pozycja">

        <a href="{% url_for('dashboard.index') %}" class="sidebar__link {% if request.endpoint == 'dashboard.index' %}sidebar__link--aktywny{% endif %}" {% if request.endpoint == 'dashboard.index' %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"></span> Przegląd

        </a>

    </li>

    <li class="sidebar__menu-sekcja" aria-hidden="true">Praktyki</li>

    <li class="sidebar__menu-pozycja">

        <a href="{% url_for('zarzadzanie.lista_praktyk') %}" class="sidebar__link {% if 'lista_praktyk' in request.endpoint %}sidebar__link--aktywny{% endif %}" {% if 'lista_praktyk' in request.endpoint %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"></span> Wszystkie praktyki

        </a>

    </li>

    {% if current_user.rola.value in ['ADMIN', 'UOPZ'] %}

    <li class="sidebar__menu-pozycja">

        <a href="{% url_for('evaluation.lista_ocen') %}" class="sidebar__link {% if 'evaluation' in request.endpoint %}sidebar__link--aktywny{% endif %}" {% if 'evaluation' in request.endpoint %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"></span> Oceny i efekty

        </a>

    </li>

    {% endif %}

    {% if current_user.rola.value == 'ADMIN' %}

    <li class="sidebar__menu-sekcja" aria-hidden="true">Zarządzanie</li>

    <li class="sidebar__menu-pozycja">

        <a href="{% url_for('zarzadzanie.lista_uzytkownikow') %}" class="sidebar__link {% if 'uzytkownik' in request.endpoint %}sidebar__link--aktywny{% endif %}" {% if 'uzytkownik' in request.endpoint %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"></span> Użytkownicy

        </a>

```

```

</li>

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.lista_zakladow') }}" class="sidebar__link {% if 'zaklad' in request.endpoint %}sidebar__link--aktywny{% endif %}" {% if 'zaklad' in request.endpoint %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"></span> Zakłady pracy

    </a>

</li>

{% endif %}

</ul>

<div class="sidebar__stopka">

    <form method="POST" action="{{ url_for('auth.wylogowanie') }}">

        <button type="submit" class="sidebar__wylogowanie">

            <span aria-hidden="true"></span> Wyloguj się

        </button>

    </form>

</div>

</nav>

<div class="panel-main">

    <header class="panel-main__naglowek">

        <div class="panel-main__naglowek-tresc">

            <h1 class="panel-main__tytul">{% block naglowek %}{% endblock %}</h1>

            {% block naglowek_akcje %}{% endblock %}

        </div>

        {% with wiadomosci = get_flashed_messages(with_categories=True) %}

            {% if wiadomosci %}

                <div class="komunikaty" role="status" aria-live="polite">

                    {% for kategoria, wiadomosc in wiadomosci %}

                        <div class="komunikat komunikat--{{ kategoria }}" role="alert">

                            <span class="komunikat__ikona" aria-hidden="true">

                                {% if kategoria == 'danger' %}{% elif kategoria == 'success' %}{% else %}{% endif %}

                            </span>

                            <span>{{ wiadomosc }}</span>

                            <button type="button" class="komunikat__zamknij" aria-label="Zamknij powiadomienie"

```

onclick="this.parentElement.remove()">></button>

</div>

{% endfor %}

</div>

{% endif %}

{% endwith %}

</header>

<main id="main-content" class="panel-main__tresc" tabindex="-1">

{% block tresc %}{% endblock %}

</main>

</div>

</body>

</html>

PLIK: app_admin\templates\zarzadzanie\formularz_pracownika.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Nowy pracownik{% endblock %}

{% block naglowek %}Dodaj pracownika{% endblock %}

{% block naglowek_akcje %}

    <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="przycisk przycisk--drugorzeczny">← Powrót</a>

{% endblock %}

{% block tresc %}

<div class="formularz-kontener">

    <div class="karta">

        <div class="karta__naglowek">

            <h2 class="karta__tytul">Dane konta pracownika</h2>

            <span style="font-size: 0.825rem; color: var(--kolor-tekst-drugor); background: #faeeda; padding: 0.25rem 0.75rem;
border-radius: 999px; font-weight: 500;">

                Hasło tymczasowe = adres e-mail przed @

            </span>

        </div>

        <div class="karta__tresc">

            <form method="POST" novalidate class="formularz-panel" aria-label="Formularz pracownika">

                {{ form.hidden_tag() }}

                <div class="formularz-panel__siatka">

                    <div class="pole-formularza {% if form.imie.errors %}pole-formularza--blad{% endif %}">

                        <label for="imie" class="pole-formularza__etykieta">Imię <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>

                        {{ form.imie(class="pole-formularza__input") }}

                        {% if form.imie.errors %}<span class="pole-formularza__blad" role="alert">{{ form.imie.errors[0] }}</span>{%
endif %}

                    </div>

                    <div class="pole-formularza {% if form.nazwisko.errors %}pole-formularza--blad{% endif %}">

                        <label for="nazwisko" class="pole-formularza__etykieta">Nazwisko <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>

                        {{ form.nazwisko(class="pole-formularza__input") }}

                        {% if form.nazwisko.errors %}<span class="pole-formularza__blad" role="alert">{{ form.nazwisko.errors[0]
}}</span>{% endif %}

                    </div>

                </div>

            </form>

        </div>

    </div>

</div>
```



```
</div>
```

```
<div class="pole-formularza {% if form.email.errors %}pole-formularza--blad{% endif %}">
```

```
    <label for="email" class="pole-formularza__etykieta">E-mail <span class="pole-formularza__wymagane"
    aria-label="wymagane">*</span></label>
```

```
    {{ form.email(class="pole-formularza__input", autocomplete="off") }}
```

```
    {% if form.email.errors %}<span class="pole-formularza__blad" role="alert">{{ form.email.errors[0] }}</span>{%
endif %}
```

```
</div>
```

```
<div class="pole-formularza {% if form.rola.errors %}pole-formularza--blad{% endif %}">
```

```
    <label for="rola" class="pole-formularza__etykieta">Rola w systemie <span class="pole-formularza__wymagane"
    aria-label="wymagane">*</span></label>
```

```
    {{ form.rola(class="filtr-select", style="width:100%") }}
```

```
    {% if form.rola.errors %}<span class="pole-formularza__blad" role="alert">{{ form.rola.errors[0] }}</span>{%
endif %}
```

```
</div>
```

```
<div style="background: #f4f6f9; border-radius: var(--promien); padding: 0.875rem 1rem; font-size: 0.875rem; color:
var(--kolor-tekst-drugor);">
```

Pracownik zaloguje się e-mailem i hasłem tymczasowym (adres z e-mail, wszystko przed znakiem @), a następnie zostanie poproszony o ustawienie własnego hasła.

```
</div>
```

```
<div style="display: flex; gap: 0.75rem; flex-wrap: wrap; margin-top: 0.5rem; justify-content: flex-end;">
```

```
    <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>
```

```
    <button type="submit" class="przycisk przycisk--glowny">Utwórz konto pracownika</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\formularz_praktyki.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Nowa praktyka{% endblock %}

{% block naglowek %}Nowa praktyka{% endblock %}


{% block naglowek_akcje %}

    <a href="{{ url_for('zarzadzanie.lista_praktyk') }}" class="przycisk przycisk--drugorzedy">Anuluj</a>

{% endblock %}


{% block tresc %}

<div style="max-width: 480px; margin: 0 auto;">

    <form method="POST" action="{{ url_for('zarzadzanie.nowa_praktyka') }}" novalidate>

        {{ form.hidden_tag() }}


        <div class="karta">

            <div class="karta__naglowek">

                <h2 class="karta__tytul">Dane praktyki</h2>

            </div>

            <div class="karta__tresc" style="display: flex; flex-direction: column; gap: 1.25rem;">


                <div class="pole-formularza">

                    {{ form.rok_uczelniany.label(class="pole-formularza__etykieta") }}

                    <input type="text" name="rok_uczelniany" class="pole-formularza__input" style="width:100%;" placeholder="np.
2024/2025" value="{{ form.rok_uczelniany.data or ' ' }}">

                    {% for e in form.rok_uczelniany.errors %}

                        <span class="pole-formularza__blad">{{ e }}</span>

                    {% endfor %}

                </div>


                <div class="pole-formularza">

                    {{ form.semestr.label(class="pole-formularza__etykieta") }}

                    {{ form.semestr(class="filtr-select", style="width:100%") }}

                    {% for e in form.semestr.errors %}

                        <span class="pole-formularza__blad">{{ e }}</span>

                    </div>

            </div>

        </div>

    </form>

</div>

</div>
```

```
{% endfor %}
```

```
</div>
```

```
<div class="pole-formularza">
```

```
  {{ form.wymiar_godzin.label(class="pole-formularza__etykieta") }}
```

```
  {{ form.wymiar_godzin(class="pole-formularza__input", type="number", min="1", placeholder="np. 160") }}
```

```
  {% for e in form.wymiar_godzin.errors %}
```

```
    <span class="pole-formularza__blad">{{ e }}</span>
```

```
  {% endfor %}
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div style="display: flex; gap: 0.5rem; margin-top: 1.25rem; justify-content: flex-end;">
```

```
  <a href="{{ url_for('zarzadzanie.lista_praktyk') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>
```

```
  <button type="submit" class="przycisk przycisk--glowny">Utwórz praktykę</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\formularz_studenta.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}{% 'Edytuj' if uzytkownik else 'Nowy' %} student{% endblock %}

{% block naglowek %}{% 'Edytuj studenta' if uzytkownik else 'Dodaj studenta' %}{% endblock %}

{% block naglowek_akcje %}

    <a href="{% url_for('zarzadzanie.lista_uzytkownikow') %}" class="przycisk przycisk--drugorzeczny">← Powrót</a>

{% endblock %}

{% block tresc %}

<div style="max-width: 640px; margin: 0 auto;">

    <div class="karta">

        <div class="karta__naglowek">

            <h2 class="karta__tytul">Dane konta studenckiego</h2>

            {% if not uzytkownik %}

                <span style="font-size: 0.825rem; color: var(--kolor-tekst-drugor); background: #faeeda; padding: 0.25rem 0.75rem;
border-radius: 999px; font-weight: 500;">

                    Hasło tymczasowe = nr albumu

                </span>

            {% endif %}

        </div>

        <div class="karta__tresc">

            <form method="POST" novalidate class="formularz-panel" aria-label="Formularz studenta">

                {% form.hidden_tag() %}

                <div class="formularz-panel__siatka" style="margin-bottom: 1.5rem;">

                    <div class="pole-formularza {% if form.imie.errors %}pole-formularza--blad{% endif %}">

                        <label for="imie" class="pole-formularza__etykieta">Imię <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>

                        <input type="text" id="imie" name="imie" class="pole-formularza__input"

                            value="{% form.imie.data or '' %}" required aria-required="true"

                            {% if form.imie.errors %}aria-invalid="true" aria-describedby="imie-blad"{% endif %}>

                        {% if form.imie.errors %}<span id="imie-blad" class="pole-formularza__blad" role="alert">{% form.imie.errors[0]
%}</span>{% endif %}

                    </div>

                    <div class="pole-formularza {% if form.nazwisko.errors %}pole-formularza--blad{% endif %}">

                        <label for="nazwisko" class="pole-formularza__etykieta">Nazwisko <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>
```

```

<input type="text" id="nazwisko" name="nazwisko" class="pole-formularza__input"

value="{ { form.nazwisko.data or ' ' } }" required aria-required="true"

{% if form.nazwisko.errors %}aria-invalid="true" aria-describedby="nazwisko-blad"{% endif %}>

    {% if form.nazwisko.errors %}<span id="nazwisko-blad" class="pole-formularza__blad" role="alert">{{
form.nazwisko.errors[0] }}</span>{% endif %}

</div>

</div>

<div class="formularz-panel__siatka" style="margin-bottom: 1.5rem;">

    <div class="pole-formularza {% if form.email.errors %}pole-formularza--blad{% endif %}">

        <label for="email" class="pole-formularza__etykieta">E-mail uczelniany <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>

        <input type="email" id="email" name="email" class="pole-formularza__input"

value="{ { form.email.data or ' ' } }" required aria-required="true" autocomplete="off"

{% if form.email.errors %}aria-invalid="true" aria-describedby="email-blad"{% endif %}>

            {% if form.email.errors %}<span id="email-blad" class="pole-formularza__blad" role="alert">{{
form.email.errors[0] }}</span>{% endif %}

        </div>

        <div class="pole-formularza {% if form.numer_albumu.errors %}pole-formularza--blad{% endif %}">

            <label for="numer_albumu" class="pole-formularza__etykieta">Nr albumu <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>

            <input type="text" id="numer_albumu" name="numer_albumu" class="pole-formularza__input"

value="{ { form.numer_albumu.data or ' ' } }" required aria-required="true"

placeholder="np. 12345" maxlength="20"

{% if form.numer_albumu.errors %}aria-invalid="true" aria-describedby="album-blad"{% endif %}>

                {% if form.numer_albumu.errors %}<span id="album-blad" class="pole-formularza__blad" role="alert">{{
form.numer_albumu.errors[0] }}</span>{% endif %}

                {% if not uzytkownik %}<span style="font-size: 0.775rem; color: var(--kolor-tekst-trzeciorzed);">Będzie użyty
jako hasło tymczasowe</span>{% endif %}

            </div>

        <div class="pole-formularza">

            <label for="plec" class="pole-formularza__etykieta">Płeć</label>

            <select id="plec" name="plec" class="filtr-select" style="width:100%;">

                <option value="" {% if not form.plec.data %}selected{% endif %}>--- Wybierz ---</option>

                <option value="M" {% if form.plec.data == 'M' %}selected{% endif %}>Mężczyzna</option>

                <option value="K" {% if form.plec.data == 'K' %}selected{% endif %}>Kobieta</option>

            </select>

```

</div>

</div>

<div style="display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 1rem; margin-bottom: 1.5rem;">

<div class="pole-formularza">

<label for="kierunek" class="pole-formularza__etykieta">Kierunek studiów</label>

<input type="text" id="kierunek" name="kierunek" class="pole-formularza__input"

value="{{ form.kierunek.data or '' }}" placeholder="np. Informatyka">

</div>

<div class="pole-formularza">

<label for="specjalnosc" class="pole-formularza__etykieta">Specjalność</label>

<input type="text" id="specjalnosc" name="specjalnosc" class="pole-formularza__input"

value="{{ form.specjalnosc.data or '' }}" placeholder="np. PBDiOU">

</div>

<div class="pole-formularza">

<label for="tryb_studiow" class="pole-formularza__etykieta">Tryb studiów</label>

<select id="tryb_studiow" name="tryb_studiow" class="filtr-select" style="width:100%;">

<option value="">--- Wybierz ---</option>

<option value="stacjonarne" {% if form.tryb_studiow.data == 'stacjonarne' %}selected{% endif %}>Stacjonarne</option>

<option value="niestacjonarne" {% if form.tryb_studiow.data == 'niestacjonarne' %}selected{% endif %}>Niestacjonarne</option>

</select>

</div>

</div>

<div class="pole-formularza {% if form.uopz_id.errors %}pole-formularza--blad{% endif %}" style="margin-bottom: 1.5rem;">

<label for="uopz_id" class="pole-formularza__etykieta">Opiekun uczelniany (UOPZ) *</label>

<select id="uopz_id" name="uopz_id" class="filtr-select" style="width:100%;" required>

<option value="">— wybierz opiekuna —</option>

{% for val, label in form.uopz_id.choices %}

<option value="{{ val }}" {% if form.uopz_id.data == val %}selected{% endif %}>{{ label }}</option>

{% endfor %}

</select>

```
        {% if form.uopz_id.errors %}<span class="pole-formularza__blad" role="alert">{{ form.uopz_id.errors[0]
    }}</span>{% endif %}

    </div>

    {% if not uzytkownik %}

        <div style="background: #f4f6f9; border-radius: var(--promien); padding: 0.875rem 1rem; font-size: 0.875rem; color:
    var(--kolor-tekst-drugor); margin-bottom: 1.5rem;">

            Student zaloguje się e-mailem i hasłem tymczasowym (nr albumu), a następnie zostanie poproszony o ustawienie
            własnego hasła.

        </div>

        {% endif %}

        <div style="display: flex; gap: 0.75rem; flex-wrap: wrap; margin-top: 0.5rem; justify-content: flex-end;">

            <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

            <button type="submit" class="przycisk przycisk--glowny">{{ 'Zapisz zmiany' if uzytkownik else 'Utwórz konto'
    }}</button>

        </div>

    </form>

</div>

</div>

</div>

{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\formularz_uzytkownika.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}{% 'Edytuj użytkownika' if uzytkownik else 'Nowy użytkownik' %}{% endblock %}

{% block naglowek %}{% 'Edytuj użytkownika' if uzytkownik else 'Nowy użytkownik' %}{% endblock %}

{% block naglowek_akcje %}

<a href="{% url_for('zarzadzanie.lista_uzytkownikow') %}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

{% endblock %}

{% block tresc %}

<div style="max-width: 480px; margin: 0 auto;">

    <div class="karta">

        <div class="karta__tresc">

            <form method="POST" class="formularz-panel" novalidate>

                {% form.hidden_tag() %}

            <div class="formularz-panel__siatka">

                <div class="pole-formularza {% if form.imie.errors %}pole-formularza--blad{% endif %}">

                    <label for="imie" class="pole-formularza__etykieta">Imię</label>

                    {% form.imie(class="pole-formularza__input", placeholder="np. Jan") %}

                    {% for error in form.imie.errors %}<span class="pole-formularza__blad">{% error %}</span>{% endfor %}

                </div>

                <div class="pole-formularza {% if form.nazwisko.errors %}pole-formularza--blad{% endif %}">

                    <label for="nazwisko" class="pole-formularza__etykieta">Nazwisko</label>

                    {% form.nazwisko(class="pole-formularza__input", placeholder="np. Kowalski") %}

                    {% for error in form.nazwisko.errors %}<span class="pole-formularza__blad">{% error %}</span>{% endfor %}

                </div>

                <div class="pole-formularza {% if form.email.errors %}pole-formularza--blad{% endif %}">

                    <label for="email" class="pole-formularza__etykieta">Adres e-mail</label>

                    {% form.email(class="pole-formularza__input", placeholder="jan.kowalski@ans.elblag.edu.pl") %}

                </div>

            </div>

        </div>

    </div>

</div>
```



```

    {% for error in form.email.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

</div>

<div class="formularz-panel__siatka">

    <div class="pole-formularza {% if form.rola.errors %}pole-formularza--blad{% endif %}">

        <label for="rola" class="pole-formularza__etykieta">Rola w systemie</label>

        {{ form.rola(class="filtr-select", style="width: 100%;") }}

        {% for error in form.rola.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

    </div>

    <div id="kontener-albumu" class="pole-formularza {% if form.numer_albumu.errors %}pole-formularza--blad{% endif %}" style="display: {% if form.rola.data == 'STUDENT' %}block{% else %}none{% endif %};">

        <label for="numer_albumu" class="pole-formularza__etykieta">Numer albumu</label>

        {{ form.numer_albumu(class="pole-formularza__input", placeholder="np. 12345") }}

        {% for error in form.numer_albumu.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

    </div>

</div>

<div class="formularz-panel__sekcja">

    <h3 class="formularz-panel__sekcja-tytul">Bezpieczeństwo</h3>

    <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-bottom: 1rem;">

        {% if uzytkownik %}Pozostaw puste, aby nie zmieniać hasła.{% else %}Hasło musi mieć minimum 8 znaków.{% endif %}

    </p>

    <div class="formularz-panel__siatka">

        <div class="pole-formularza {% if form.haslo.errors %}pole-formularza--blad{% endif %}">

            <label for="haslo" class="pole-formularza__etykieta">Nowe hasło</label>

            {{ form.haslo(class="pole-formularza__input") }}

            {% for error in form.haslo.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

        </div>

        <div class="pole-formularza {% if form.haslo_potwierdzenie.errors %}pole-formularza--blad{% endif %}">

            <label for="haslo_potwierdzenie" class="pole-formularza__etykieta">Powtórz hasło</label>

            {{ form.haslo_potwierdzenie(class="pole-formularza__input") }}

```

```

        {% for error in form.haslo_potwierdzenie.errors %}<span class="pole-formularza__blad">{{ error }}</span>{%
endfor %}

        </div>

    </div>

</div>

    <div class="formularz-panel__akcje" style="margin-top: 1rem; padding-top: 1.5rem; border-top: 1px solid
var(--kolor-ramka); display:flex; gap:0.5rem; justify-content:flex-end;">

        <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="przycisk przycisk--drugorzedy">Anuluj</a>

        <button type="submit" class="przycisk przycisk--glowny">{{ 'Zapisz zmiany' if uzytkownik else 'Utwórz
użytkownika' }}</button>

    </div>

</form>

</div>

</div>

</div>

{% endblock %}

{% block scripts_extra %}

<script>

    document.getElementById('rola').addEventListener('change', function() {

        const kontener = document.getElementById('kontener-albumu');

        if (this.value === 'STUDENT') {

            kontener.style.display = 'block';

        } else {

            kontener.style.display = 'none';

            document.getElementById('numer_albumu').value = '';

        }

    });

</script>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\formularz_zakladu.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}{% 'Edytuj zakład' if zaklad else 'Nowy zakład pracy' %}{% endblock %}

{% block naglowek %}{% 'Edytuj zakład' if zaklad else 'Nowy zakład pracy' %}{% endblock %}

{% block naglowek_akcje %}

<a href="{% url_for('zarzadzanie.lista_zakladow') %}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

{% endblock %}

{% block tresc %}

<div class="karta" style="max-width: 800px; margin: 0 auto;">

    <div class="karta__tresc">

        <form method="POST" class="formularz-panel" novalidate>

            {% form.hidden_tag() %}

            <div class="pole-formularza {% if form.nazwa.errors %}pole-formularza--blad{% endif %}">

                <label for="nazwa" class="pole-formularza__etykieta">Pełna nazwa zakładu</label>

                {% form.nazwa(class="pole-formularza__input", placeholder="np. Software House Sp. z o.o.") %}

                {% for error in form.nazwa.errors %}<span class="pole-formularza__blad">{% error %}</span>{% endfor %}

            </div>

            <div class="pole-formularza {% if form.adres.errors %}pole-formularza--blad{% endif %}">

                <label for="adres" class="pole-formularza__etykieta">Adres (ulica i numer)</label>

                {% form.adres(class="pole-formularza__input", placeholder="np. ul. Grunwaldzka 137") %}

                {% for error in form.adres.errors %}<span class="pole-formularza__blad">{% error %}</span>{% endfor %}

            </div>

            <div class="formularz-panel__siatka">

                <div class="pole-formularza {% if form.kod_pocztowy.errors %}pole-formularza--blad{% endif %}">

                    <label for="kod_pocztowy" class="pole-formularza__etykieta">Kod pocztowy</label>

                    {% form.kod_pocztowy(class="pole-formularza__input", placeholder="np. 82-300") %}

                    {% for error in form.kod_pocztowy.errors %}<span class="pole-formularza__blad">{% error %}</span>{% endfor %}

                </div>

            </div>

        </form>

    </div>

</div>
```

```

<div class="pole-formularza {% if form.miasto.errors %}pole-formularza--blad{% endif %}">

  <label for="miasto" class="pole-formularza__etykieta">Miasto</label>

  {{ form.miasto(class="pole-formularza__input", placeholder="np. Elbląg") }}

  {% for error in form.miasto.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

</div>

</div>

<div class="formularz-panel__siatka">

  <div class="pole-formularza {% if form.nip_krs.errors %}pole-formularza--blad{% endif %}">

    <label for="nip_krs" class="pole-formularza__etykieta">NIP / KRS</label>

    {{ form.nip_krs(class="pole-formularza__input", placeholder="NIP lub numer rejestrowy") }}

    {% for error in form.nip_krs.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

  </div>

  <div class="pole-formularza {% if form.przedstawiciel_prawny.errors %}pole-formularza--blad{% endif %}">

    <label for="przedstawiciel_prawny" class="pole-formularza__etykieta">Przedstawiciel prawny</label>

    {{ form.przedstawiciel_prawny(class="pole-formularza__input", placeholder="Imię i nazwisko (np. Prezes Zarządu)") }}

    {% for error in form.przedstawiciel_prawny.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

  </div>

</div>

<div class="formularz-panel__akcje" style="margin-top: 1.5rem; padding-top: 1.5rem; border-top: 1px solid var(--kolor-ramka);">

  <button type="submit" class="przycisk przycisk--glowny">

    {{ 'Zapisz zmiany' if zaklad else 'Dodaj zakład pracy' }}

  </button>

</div>

</form>

</div>

</div>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\import_csv.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Import studentów{% endblock %}

{% block naglowek %}Import studentów z CSV{% endblock %}

{% block naglowek_akcje %}

    <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="przycisk przycisk--drugorzeczny">← Powrót</a>

{% endblock %}

{% block tresc %}

<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.25rem; align-items: start;">

    <div class="karta">

        <div class="karta__naglowek"><h2 class="karta__tytul">Wgraj plik CSV</h2></div>

        <div class="karta__tresc">

            <form method="POST" enctype="multipart/form-data" novalidate class="formularz-panel">

                {{ form.hidden_tag() }}

                <div class="pole-formularza">

                    <label for="plik" class="pole-formularza__etykieta">

                        Plik CSV z USOS <span class="pole-formularza__wymagane" aria-label="wymagane">*</span>

                    </label>

                    <input type="file" id="plik" name="plik" accept=".csv"

                        class="pole-formularza__input" required>

                    {% if form.plik.errors %}

                        <span class="pole-formularza__blad" role="alert">{{ form.plik.errors[0] }}</span>

                    {% endif %}

                </div>

                <div class="pole-formularza">

                    <label for="uopz_id" class="pole-formularza__etykieta">

                        Przypisz opiekuna (UOPZ) <span class="pole-formularza__wymagane" aria-label="wymagane">*</span>

                    </label>

                    <select id="uopz_id" name="uopz_id" class="filtr-select" style="width:100%;" required>

                        <option value="">– wybierz –</option>

                        {% for val, label in form.uopz_id.choices %}
```

```
<option value="{ { val } }">{ { label } }</option>

{ % endfor % }

</select>

</div>
```

```
<button type="submit" class="przycisk przycisk--glowny">Importuj</button>

</form>

</div>

</div>
```

```
<div class="karta">

  <div class="karta__naglowek"><h2 class="karta__tytul">Format pliku CSV</h2></div>

  <div class="karta__tresc">

    <p style="font-size: 0.875rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">

      Pierwsza linia musi zawierać nagłówki. Obsługiwane nazwy kolumn:

    </p>

    <code style="display: block; background: var(--kolor-tlo); padding: 0.75rem; border-radius: var(--promien);
font-size: 0.8rem; line-height: 1.8;">

      imie,nazwisko,email,numer_albumu,plec,kierunek,specjalnosc,tryb_studiow<br>

      Jan,Kowalski,j.kowalski@student.ans.elblag.pl,12345,M,Informatyka,PBDiOU,stacjonarne<br>

      Anna,Nowak,a.nowak@student.ans.elblag.pl,12346,K,Informatyka,SIA,niestacjonarne

    </code>

    <p style="font-size: 0.8rem; color: var(--kolor-tekst-trzeciorzed); margin-top: 0.75rem;">

      Kolumny <strong>plec, kierunek, specjalnosc, tryb_studiow</strong> są opcjonalne.<br>

      Hasło tymczasowe każdego studenta = jego nr albumu.<br>

      Kodowanie: UTF-8 lub UTF-8 z BOM (eksport z Excela).

    </p>

  </div>

</div>

</div>

{ % if wyniki % }

<div class="karta" style="margin-top: 1.25rem;">

  <div class="karta__naglowek">
```

```

<h2 class="karta__tytul">Wyniki importu</h2>

</div>

<div class="karta__tresc">

  <div style="display: flex; gap: 1.5rem; margin-bottom: 1rem;">

    <div style="text-align: center;">

      <p style="font-size: 2rem; font-weight: 700; color: var(--kolor-sukces);">{{ wyniki.utworzono }}</p>

      <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">Kont utworzonych</p>

    </div>

    <div style="text-align: center;">

      <p style="font-size: 2rem; font-weight: 700; color: var(--kolor-blad);">{{ wyniki.pominieto }}</p>

      <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">Pominiętych</p>

    </div>

  </div>

  {% if wyniki.bledy %}

  <details>

    <summary style="cursor: pointer; font-size: 0.875rem; font-weight: 500; color: var(--kolor-blad);">

      Pokaż błędy ({{ wyniki.bledy|length }})

    </summary>

    <ul style="margin-top: 0.5rem; font-size: 0.825rem; color: var(--kolor-tekst-drugor); list-style: none; padding: 0;">

      {% for blad in wyniki.bledy %}

      <li style="padding: 0.25rem 0; border-bottom: 1px solid var(--kolor-ramka);">{{ blad }}</li>

      {% endfor %}

    </ul>

  </details>

  {% endif %}

</div>

</div>

{% endif %}

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\index.html

<h1>Management</h1>


```

        <span class="status status--zakonczona">Aktywna</span>

    {% else %}

        <span class="status status--szkic">Nieaktywna</span>

    {% endif %}

</td>

<td style="white-space: nowrap;">

    <div style="display: flex; gap: 0.375rem;">

        <form method="POST"

            action="{% url_for('zarzadzanie.przelacz_aktywnosc_praktyki', id=p.id) %}"

            style="display: inline;">

            {{ csrf_form.hidden_tag() }}

            <button type="submit"

                class="przycisk {% if p.status.value == 'ACTIVE' %}przycisk--ostrzezenie{% else
                %}przycisk--drugorzedny{% endif %} przycisk--maly"

                onclick="return confirm('Zmienić status praktyki?')">

                {{ 'Dezaktywuj' if p.status.value == 'ACTIVE' else 'Aktywuj' }}

            </button>

        </form>

        <form method="POST"

            action="{% url_for('zarzadzanie.usun_praktyke', id=p.id) %}"

            style="display: inline;">

            {{ csrf_form.hidden_tag() }}

            <button type="submit"

                class="przycisk przycisk--niebezpieczny przycisk--maly"

                onclick="return confirm('Czy na pewno chcesz usunąć tę praktykę? Usuną się też wszystkie
                zapisy.')">

                Usuń

            </button>

        </form>

    </div>

</td>

</tr>

```

```

        {% else %}

        <tr>

            <td colspan="6" class="tabela--pusta">Brak praktyk. Utwórz pierwszą.</td>

        </tr>

        {% endfor %}

    </tbody>

</table>

</div>

{% if praktyki.pages > 1 %}

<nav class="paginacja" aria-label="Paginacja">

    <span class="paginacja__info">

        Strona {{ praktyki.page }} z {{ praktyki.pages }}

    </span>

    <div class="paginacja__przyciski">

        {% for nr in praktyki.iter_pages(left_edge=1, right_edge=1, left_current=2, right_current=2) %}

            {% if nr %}

                <a href="{{ url_for('zarzadzanie.lista_praktyk', strona=nr) }}"

                    class="paginacja__przycisk {% if nr == praktyki.page %}paginacja__przycisk--aktywny{% endif %}"

                    {% if nr == praktyki.page %}aria-current="page"{% endif %}>{{ nr }}</a>

            {% else %}

                <span class="paginacja__przycisk" aria-hidden="true">...</span>

            {% endif %}

        {% endfor %}

    </div>

</nav>

{% endif %}

</div>

<style>.sr-only { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px; overflow: hidden; clip:
rect(0,0,0,0); white-space: nowrap; border: 0; }</style>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\szczegoly_praktyki.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Szczegóły praktyki{% endblock %}

{% block naglowek %}Szczegóły praktyki{% endblock %}


{% block naglowek_akcje %}

<div class="akcje-panelu">

    <a href="{% url_for('zarzadzanie.lista_praktyk') %}" class="przycisk przycisk--drugorzeczny">Powrót do listy</a>

    {% if current_user.rola.value in ['ADMIN', 'UOPZ'] %}

        <a href="{% url_for('evaluation.ocen_praktyke', id=praktyka.id) %}" class="przycisk przycisk--glowny">Oceń / Egzaminuj</a>

        <a href="{% url_for('evaluation.podglad_sprawozdania', id=praktyka.id) %}" class="przycisk przycisk--drugorzeczny">Zobacz Sprawozdanie</a>

        <a href="{% url_for('journal.podglad', id=praktyka.id) %}" class="przycisk przycisk--drugorzeczny">Zobacz Dziennik</a>

        <a href="{% url_for('documents.panel', id=praktyka.id) %}" class="przycisk przycisk--drugorzeczny" style="border-color: #8e44ad; color: #8e44ad;"> Dokumenty (PDF)</a>

    {% endif %}

</div>

{% endblock %}


{% block tresc %}

<div class="uklad-szczegolow" style="display: grid; grid-template-columns: 2fr 1fr; gap: 2rem;">

    <div class="lewa-kolumna" style="display: flex; flex-direction: column; gap: 2rem;">

        <!-- Dane podstawowe -->

        <section class="karta">

            <header class="karta__naglowek">

                <h2 class="karta__tytul">Informacje o praktyce</h2>

                <span class="status status--{% praktyka.status.value|lower|replace('_', '-') %}">{% tlumacz_status(praktyka.status.value) %}</span>

            </header>

            <div class="karta__tresc">

                <dl style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem;">

                    <div>
```

```

        <dt style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Student</dt>

        <dd style="font-weight: 600;">{{ praktyka.student.imie }} {{ praktyka.student.nazwisko }}</dd>

        <dd style="font-size: 0.9rem;">Album: {{ praktyka.student.numer_albumu or '-' }}</dd>

    </div>

    <div>

        <dt style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Zakład pracy</dt>

        <dd style="font-weight: 600;">{{ praktyka.zaklad.nazwa }}</dd>

        <dd style="font-size: 0.9rem;">{{ praktyka.zaklad.miasto }}</dd>

    </div>

    <div>

        <dt style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Termin</dt>

        <dd>{{ praktyka.data_roz poczenia }} – {{ praktyka.data_zakonczenia or '...' }}</dd>

    </div>

    <div>

        <dt style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Ścieżka</dt>

        <dd>{{ praktyka.typ_sciezki.value }}</dd>

    </div>

</dl>

</div>

</section>

<!-- Postęp godzin -->

<section class="karta">

    <header class="karta__naglowek">

        <h2 class="karta__tytul">Realizacja godzinowa</h2>

    </header>

    <div class="karta__tresc" style="padding: 2rem;">

        <div style="display: flex; justify-content: space-between; margin-bottom: 1rem; align-items: flex-end;">

            <span style="font-size: 2rem; font-weight: 700; color: var(--kolor-glowny);">{{ praktyka.lacznie_godzin }}
            <small style="font-size: 1rem; color: var(--kolor-tekst-drugor);" / 160 h</small></span>

            <span style="font-weight: 600; color: var(--kolor-sukces);">{{ (praktyka.lacznie_godzin / 1.6)|round|int
            }}%</span>

        </div>

        <div class="pasek-postep" style="height: 12px;">

            <div class="pasek-postep__wypelnienie {{ 'pasek-postep__wypelnienie--zakonczone' if praktyka.lacznie_godzin >=
            160 else '' }}"

```

```

        style="width: {% if praktyka.lacznie_godzin >= 160 %}100{% else %}{ { praktyka.lacznie_godzin / 1.6 } }{%
endif %}%;"></div>

</div>

</div>

</section>

</div>

<div class="prawa-kolumna" style="display: flex; flex-direction: column; gap: 2rem;">

<!-- Opiekunowie -->

<section class="karta">

    <header class="karta__naglowek">

        <h2 class="karta__tytul">Opiekunowie</h2>

    </header>

    <div class="karta__tresc">

        <div style="margin-bottom: 1.5rem;">

            <div style="font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.05em; color:
var(--kolor-tekst-drugor); margin-bottom: 0.5rem;">Uczelniany (UOPZ)</div>

            <div style="font-weight: 600;">{{ praktyka.uopz.imie }} {{ praktyka.uopz.nazwisko }}</div>

        </div>

        <div>

            <div style="font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.05em; color:
var(--kolor-tekst-drugor); margin-bottom: 0.5rem;">Zakładowy (ZOPZ)</div>

            <div style="font-weight: 600;">{{ praktyka.zopz.imie }} {{ praktyka.zopz.nazwisko }}</div>

        </div>

    </div>

</section>

<!-- Oceny (jeśli są) -->

{% if praktyka.ocena_sprawozdania or praktyka.ocena_uopz %}

<section class="karta">

    <header class="karta__naglowek">

        <h2 class="karta__tytul">Wyniki oceny</h2>

    </header>

    <div class="karta__tresc">

```

```
<dl style="display: flex; flex-direction: column; gap: 1rem;">

  <div style="display: flex; justify-content: space-between;">

    <dt>Sprawozdanie (S):</dt>

    <dd style="font-weight: 700; color: var(--kolor-glowny);">{{ praktyka.ocena_sprawozdania or '-' }}</dd>

  </div>

  <div style="display: flex; justify-content: space-between;">

    <dt>Ocena UOPZ (U):</dt>

    <dd style="font-weight: 700; color: var(--kolor-glowny);">{{ praktyka.ocena_uopz or '-' }}</dd>

  </div>

  <div style="display: flex; justify-content: space-between;">

    <dt>Ocena ZOPZ (Z):</dt>

    <dd style="font-weight: 700; color: var(--kolor-glowny);">{{ praktyka.ocena_zopz or '-' }}</dd>

  </div>

</dl>

</div>

</section>

{% endif %}
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\uzytownicy.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Użytkownicy{% endblock %}

{% block naglowek %}Zarządzanie użytkownikami{% endblock %}

{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem; align-items: center;">

  <a href="{{ url_for('zarzadzanie.nowy_student') }}" class="przycisk przycisk--glowny">

    + Dodaj studenta

  </a>

  <a href="{{ url_for('zarzadzanie.nowy_pracownik') }}" class="przycisk przycisk--glowny">

    + Dodaj pracownika

  </a>

  <a href="{{ url_for('zarzadzanie.import_csv') }}" class="przycisk przycisk--drugorzedny">

    Import z CSV

  </a>

</div>

{% endblock %}

{% block tresc %}

<div class="karta">

  <div class="karta__naglowek">

    <form method="GET" class="pasek-filtrow" role="search" aria-label="Filtruj użytkowników">

      <div class="filtr-szukaj">

        <span class="filtr-szukaj__ikona" aria-hidden="true">

          <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round">

            <circle cx="11" cy="11" r="8"></circle>

            <path d="m21 21-4.3-4.3"></path>

          </svg>

        </span>

        <input

          type="search"

          name="szukaj"

        />

      </div>

    </form>

  </div>

  <div class="karta__tresc">

    <table>

      <thead>

        <tr>

          <th>Imię i nazwisko</th>

          <th>Data urodzenia</th>

          <th>Adres e-mail</th>

          <th>Telefon</th>

          <th>Stan</th>

          <th>Data ostatniej zmiany</th>

          <th>Data dodania</th>

        </tr>

      </thead>

      <tbody>

        <tr>

          <td>Jan</td>

          <td>Kowalski</td>

          <td>jan.kowalski@firma.pl</td>

          <td>123 456 789</td>

          <td>Aktywny</td>

          <td>2023-10-27 10:30</td>

          <td>2023-10-27 10:30</td>

        </tr>

        <tr>

          <td>Anna</td>

          <td>Nowak</td>

          <td>anna.nowak@firma.pl</td>

          <td>987 654 321</td>

          <td>Aktywny</td>

          <td>2023-10-27 10:30</td>

          <td>2023-10-27 10:30</td>

        </tr>

        <tr>

          <td>Piotr</td>

          <td>Wiśniewski</td>

          <td>piotr.wisniewski@firma.pl</td>

          <td>555 111 222</td>

          <td>Zrezygnowany</td>

          <td>2023-10-27 10:30</td>

          <td>2023-10-27 10:30</td>

        </tr>

        <tr>

          <td>Katarzyna</td>

          <td>Zielinski</td>

          <td>katarzyna.zielinski@firma.pl</td>

          <td>444 333 222</td>

          <td>Aktywny</td>

          <td>2023-10-27 10:30</td>

          <td>2023-10-27 10:30</td>

        </tr>

      </tbody>

    </table>

  </div>

</div>

{% endblock %}
```



```

        id="szukaj"

        class="filtr-szukaj__input"

        value="{{ request.args.get('szukaj', '') }}"

        placeholder="Szukaj po imieniu, nazwisku, e-mailu..."

        aria-label="Szukaj użytkowników"

    >

</div>

<label for="rola" class="sr-only">Filtruj po roli</label>

<select name="rola" id="rola" class="filtr-select" onchange="this.form.submit()">

    <option value="">Wszystkie role</option>

    <option value="STUDENT" {% if request.args.get('rola') == 'STUDENT' %}>selected{% endif %}>Studenci</option>

        <option value="UOPZ"          {% if request.args.get('rola') == 'UOPZ'          %}>selected{% endif %}>Opiekunowie
uczelni</option>

            <option value="ADMIN"          {% if request.args.get('rola') == 'ADMIN'          %}>selected{% endif
%}>Administratorzy</option>

</select>

<button type="submit" class="przycisk przycisk--drugorzecny">Szukaj</button>

{% if request.args.get('szukaj') or request.args.get('rola') %}

    <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="przycisk przycisk--drugorzecny">

        Wyczyść filtry

    </a>

{% endif %}

</form>


<span style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">

    {{ uzytkownicy.total }} użytkowników

</span>

</div>


<div class="tabela-wrapper">

    <table class="tabela" aria-label="Lista użytkowników systemu">

        <thead>

            <tr>

                <th scope="col">Użytkownik</th>

                <th scope="col">Rola</th>

            </tr>

        </thead>

        <tbody>

            <tr>

                <td>{{ user.username }}</td>

                <td>{{ user.role }}</td>

            </tr>

        </tbody>

    </table>

</div>

```

```

<th scope="col">Nr albumu</th>

<th scope="col">Status</th>

<th scope="col">Data rejestracji</th>

<th scope="col"><span class="sr-only">Akcje</span></th>

</tr>

</thead>

<tbody>

{% for u in uzytkownicy.items %}

<tr>

<td>

<div style="display: flex; flex-direction: column; gap: 0.15rem;">

<span style="font-weight: 500;">{{ u.first_name }} {{ u.last_name }}</span>

<span style="font-size: 0.8rem; color: var(--kolor-tekst-trzeciorzed);">{{ u.email }}</span>

</div>

</td>

<td>

{% set mapa_rol = {'ADMIN': 'admin', 'UOPZ': 'uopz', 'STUDENT': 'student'} %}

{% set klasa = mapa_rol.get(u.role.value, 'student') %}

<span class="rola-odznaka rola-odznaka--{{ klasa }}">{{ u.role.value }}</span>

</td>

<td>{{ u.album_number or '-' }}</td>

<td>

{% if u.is_active %}

<span class="status status--zakonczone">Aktywny</span>

{% else %}

<span class="status status--szkic">Nieaktywny</span>

{% endif %}

</td>

<td style="white-space: nowrap; font-size: 0.85rem; color: var(--kolor-tekst-drugor);">

{{ u.created_at.strftime('%d.%m.%Y') }}

</td>

<td style="white-space: nowrap;">

<div style="display: flex; gap: 0.375rem;">

```

```

{% if u.role.name == 'STUDENT' %}

    <a href="{{ url_for('zarzadzanie.edytuj_studenta', id=u.id) }}"

        class="przycisk przycisk--drugorzeczny przycisk--maly"

        aria-label="Edytuj studenta {{ u.first_name }} {{ u.last_name }}">

        Edytuj

    </a>

    <form method="POST"

        action="{{ url_for('zarzadzanie.reset_hasla', id=u.id) }}"

        style="display: inline;">

        {{ csrf_form.hidden_tag() }}

        <button type="submit"

            class="przycisk przycisk--drugorzeczny przycisk--maly"

            aria-label="Resetuj hasło dla {{ u.first_name }} {{ u.last_name }}"

                onclick="return confirm('Zresetować hasło do nr albumu? Nastąpi wymóg zmiany hasła przy
pierwszym logowaniu.')">

            Reset hasła

        </button>

    </form>

{% endif %}

{% if u.id != current_user.id %}

    <form method="POST"

        action="{{ url_for('zarzadzanie.przelacz_aktywnosc', id=u.id) }}"

        style="display: inline;">

        {{ csrf_form.hidden_tag() }}

        <button type="submit"

            class="przycisk {% if u.is_active %}przycisk--ostrzezenie{% else %}przycisk--drugorzeczny{%
endif %} przycisk--maly"

            aria-label="{{ 'Dezaktywuj' if u.is_active else 'Aktywuj' }}" konto {{ u.first_name }} {{
u.last_name }}"

            onclick="return confirm('Czy na pewno chcesz zmienić status tego konta?')">

            {{ 'Dezaktywuj' if u.is_active else 'Aktywuj' }}

        </button>

    </form>

    <form method="POST"

```

```

        action="{{ url_for('zarzadzanie.usun_uzytkownika', id=u.id) }}"

        style="display: inline;">

        {{ csrf_form.hidden_tag() }}

        <button type="submit"

            class="przycisk przycisk--niebezpieczny przycisk--maly"

            aria-label="Usuń konto {{ u.first_name }} {{ u.last_name }}"

            onclick="return confirm('Czy na pewno chcesz trwale usunąć konto {{ u.first_name }} {{
u.last_name }}? Tej operacji nie można cofnąć.')">

            Usuń

        </button>

    </form>

    {% endif %}

</div>

</td>

</tr>

{% else %}

<tr>

    <td colspan="6" class="tabela--pusta">

        {% if request.args.get('szukaj') %}

            Nie znaleziono użytkowników pasujących do zapytania „{{ request.args.get('szukaj') }}”.

        {% else %}

            Brak użytkowników w systemie.

        {% endif %}

    </td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

{% if uzytkownicy.pages > 1 %}

<nav class="paginacja" aria-label="Paginacja listy użytkowników">

    <span class="paginacja__info">

        Wyświetlanie {{ uzytkownicy.per_page * (uzytkownicy.page - 1) + 1 }}-{{ [uzytkownicy.per_page * uzytkownicy.page,

```

```

uzytkownicy.total] | min }}

    z {{ uzytkownicy.total }}

</span>

<div class="paginacja__przyciski" role="list">

    {% if uzytkownicy.has_prev %}

        <a href="{{ url_for('zarzadzanie.lista_uzytkownikow', strona=uzytkownicy.prev_num, **request.args) }}"

            class="paginacja__przycisk" aria-label="Poprzednia strona"><</a>

    {% endif %}

    {% for nr in uzytkownicy.iter_pages(left_edge=1, right_edge=1, left_current=2, right_current=2) %}

        {% if nr %}

            <a href="{{ url_for('zarzadzanie.lista_uzytkownikow', strona=nr, **request.args) }}"

                class="paginacja__przycisk {% if nr == uzytkownicy.page %}paginacja__przycisk--aktywny{% endif %}"

                {% if nr == uzytkownicy.page %}aria-current="page"{% endif %}>

                {{ nr }}

            </a>

        {% else %}

            <span class="paginacja__przycisk" aria-hidden="true">...</span>

        {% endif %}

    {% endfor %}

    {% if uzytkownicy.has_next %}

        <a href="{{ url_for('zarzadzanie.lista_uzytkownikow', strona=uzytkownicy.next_num, **request.args) }}"

            class="paginacja__przycisk" aria-label="Następna strona">></a>

    {% endif %}

</div>

</nav>

{% endif %}

</div>

<style>

.sr-only { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px; overflow: hidden; clip: rect(0,0,0,0);
white-space: nowrap; border: 0; }

.przycisk--ostrzezenie {

    background: #f59e0b;

```

```
    color: white;

    border: 1px solid #f59e0b;
}

.przycisk--ostrzezenie:hover {

    background: #d97706;

    border-color: #d97706;
}

</style>

{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\zaklady.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Zakłady pracy{% endblock %}

{% block naglowek %}Zakłady pracy{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem; align-items: center;">

    {% if current_user.rola.value in ['ADMIN', 'UOPZ'] %}

    <a href="{{ url_for('zarzadzanie.nowy_zaklad') }}" class="przycisk przycisk--glowny">

        <span aria-hidden="true">+</span> Dodaj zakład

    </a>

    {% endif %}

</div>

{% endblock %}


{% block tresc %}

<div class="karta">

    <div class="karta__naglowek">

        <form class="pasek-filtrow" method="GET" action="{{ url_for('zarzadzanie.lista_zakladow') }}">

            <div class="filtr-szukaj">

                <span class="filtr-szukaj__ikona" aria-hidden="true"><svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="11" cy="11"
r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg></span>

                <input type="text" name="szukaj" value="{{ request.args.get('szukaj', '') }}" class="filtr-szukaj__input"
placeholder="Nazwa zakładu lub miasto..." aria-label="Wyszukaj zakład">

            </div>

            <button type="submit" class="przycisk przycisk--drugorzeczny">Szukaj</button>

        </form>

    </div>


    <div class="karta__tresc">

        <div class="tabela-wrapper">

            <table class="tabela">

                <thead>
```

```

<tr>

    <th scope="col">Nazwa zakładu</th>

    <th scope="col">Miasto</th>

    <th scope="col">NIP / KRS</th>

    <th scope="col">Akcje</th>

</tr>

</thead>

<tbody>

    {% for z in zaklady.items %}

    <tr>

        <td>

            <div style="font-weight: 600;">{{ z.nazwa }}</div>

            <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ z.adres }}</div>

        </td>

        <td>{{ z.miasto or '-' }}</td>

        <td>{{ z.nip_krs or '-' }}</td>

        <td>

            <div class="akcje-tabeli">

                <a href="{{ url_for('zarzadzanie.edytuj_zaklad', id=z.id) }}" class="przycisk przycisk--drugorzeczny przycisk--maly">Edytuj</a>

            </div>

        </td>

    </tr>

    {% else %}

    <tr>

        <td colspan="4" class="tabela--pusta">Brak zarejestrowanych zakładów pracy.</td>

    </tr>

    {% endfor %}

</tbody>

</table>

</div>

{% if zaklady.pages > 1 %}

<nav class="paginacja" aria-label="Nawigacja po stronach">

    <span class="paginacja__info">Strona {{ zaklady.page }} z {{ zaklady.pages }}</span>

```



```
<div class="paginacja__przyciski">

    {% for strona in zaklady.iter_pages() %}

        {% if strona %}

            <a href="{% url_for('zarzadzanie.lista_zakladow', strona=strona, szukaj=request.args.get('szukaj', '')) %}"
class="paginacja__przycisk {% 'paginacja__przycisk--aktywny' if strona == zaklady.page else '' %}">

                {{ strona }}

            </a>

        {% else %}

            <span class="paginacja__separator">...</span>

        {% endif %}

    {% endfor %}

</div>

</nav>

{% endif %}

</div>

</div>

{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\dziekan\decyzja.html

```
{% extends "layouts/base_panel.html" %}
```

```
{% block tytul %}Decyzja dziekana{% endblock %}
```

```
{% block naglowek %}Decyzja dziekana w sprawie wniosku{% endblock %}
```

```
{% block naglowek_akcje %}
```

```
<div style="display: flex; gap: 0.5rem;">
```

```
<a href="{{ url_for('zarzadzanie.dziekan_lista') }}" class="przycisk przycisk--drugorzeczny">← Powrót do listy</a>
```

</div>

```
{% endblock %}
```

```
{% block tresp %}
```

```
<div class="grid-2">
```

```
<!-- Lewa kolumna: Dane wniosku -->
```

<div>

```
<div class="karta" style="margin-bottom: 2rem;">
```

```
<div class="karta_naglowek">
```

Informacje o wniosku

</div>

```
<div class="karta tresc">
```

<div class="info-grid">

<div class="info-item">

<label>Student</label>

<div>{{ zapis.student.first name }} {{ zapis.student.last name }}</div>

</div>

<div class="info-item">

<label>Nr albumu</label>

<div>{{ zapis.student.album number }}</div>

</div>

```
<div class="info-item">
```

<label>Ścieżka</label>

```

<div>

    {% if zapis.track_type.value == 'EMPLOYMENT' %}

    <span class="badge badge--info">Praca zawodowa</span>

    {% elif zapis.track_type.value == 'OWN_BUSINESS' %}

    <span class="badge badge--warning">Działalność gospodarcza</span>

    {% endif %}

</div>

</div>

<div class="info-item">

    <label>Data zgłoszenia</label>

    <div>{{ zapis.enrolled_at.strftime('%d.%m.%Y %H:%M') }}</div>

</div>

</div>

{% if zapis.firma_nazwa %}

<div style="border-top: 1px solid #e5e7eb; margin-top: 1.5rem; padding-top: 1.5rem;">

    <h3 style="font-size: 1rem; margin-bottom: 1rem;">Miejsce pracy/działalności</h3>

    <div class="info-grid">

        <div class="info-item">

            <label>Nazwa</label>

            <div><strong>{{ zapis.firma_nazwa }}</strong></div>

        </div>

        <div class="info-item">

            <label>Miasto</label>

            <div>{{ zapis.firma_miasto or 'Nie podano' }}</div>

        </div>

        {% if zapis.firma_nip_krs %}

        <div class="info-item">

            <label>NIP/KRS</label>

            <div>{{ zapis.firma_nip_krs }}</div>

        </div>

        {% endif %}

    </div>

</div>

</div>

```

```
{% endif %}
```

```
{% if zapis.uzasadnienie_sciezki %}
```

```
<div style="border-top: 1px solid #e5e7eb; margin-top: 1.5rem; padding-top: 1.5rem;">
```

```
  <h3 style="font-size: 1rem; margin-bottom: 1rem;">Uzasadnienie studenta</h3>
```

```
  <div class="uzasadnienie-box">
```

```
    {{ zapis.uzasadnienie_sciezki }}
```

```
  </div>
```

```
</div>
```

```
{% endif %}
```

```
<!-- Decyzja komisji -->
```

```
{% if zapis.decyzja_komisji %}
```

```
<div style="border-top: 1px solid #e5e7eb; margin-top: 1.5rem; padding-top: 1.5rem;">
```

```
  <h3 style="font-size: 1rem; margin-bottom: 1rem;">Decyzja komisji weryfikującej</h3>
```

```
  <div class="komisja-decyzja">
```

```
    <div class="decyzja-header">
```

```
      <span class="status status--zakonczona">Zatwierdzone</span>
```

```
      <span class="decyzja-data">
```

```
        {{ zapis.decyzja_komisji_o.strftime('%d.%m.%Y %H:%M') if zapis.decyzja_komisji_o }}
```

```
      </span>
```

```
    </div>
```

```
    {% if zapis.komentarze_komisji %}
```

```
      <div class="decyzja-komentarz">
```

```
        <strong>Komentarz komisji:</strong><br>
```

```
        {{ zapis.komentarze_komisji }}
```

```
      </div>
```

```
    {% endif %}
```

```
</div>
```

```
</div>
```

```
{% endif %}
```

```
</div>
```

```
</div>
```

```
</div>
```

```

<!-- Prawa kolumna: Formularz decyzji dziekana -->

<div>

  <div class="karta">

    <div class="karta__naglowek">

      <h2>Decyzja dziekana</h2>

      <span class="karta__opis">

        Podejmij ostateczną decyzję w sprawie wniosku o zaliczenie praktyki na podstawie pracy zawodowej/działalności
        gospodarczej

      </span>

    </div>

    <div class="karta__tresc">

      <form method="POST">

        {{ form.hidden_tag() }}

        <div class="pole-formularza">

          {{ form.decyzja.label(class="pole-formularza__etykieta") }}

          {{ form.decyzja(class="pole-formularza__select") }}

          {% for error in form.decyzja.errors %}

            <span class="pole-formularza__blad">{{ error }}</span>

          {% endfor %}

        </div>

        <div class="pole-formularza">

          {{ form.komentarz.label(class="pole-formularza__etykieta") }}

          {{ form.komentarz(class="pole-formularza__textarea", rows="8", placeholder="Uzasadnienie decyzji dziekana
          (opcjonalne)...") }}

          {% for error in form.komentarz.errors %}

            <span class="pole-formularza__blad">{{ error }}</span>

          {% endfor %}

          <div class="pole-formularza__pomoc">

            Komentarz będzie widoczny dla studenta i w dokumentacji

          </div>

        </div>

      </div>

    </div>

```

```

<div class="akcje-formularza">

    {{ form.submit(class="przycisk przycisk--glowny") }}

    <a href="{{ url_for('zarzadzanie.dziekan_lista') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

</div>

</form>

</div>

</div>

<!-- Generowanie dokumentów -->

<div class="karta" style="margin-top: 2rem;">

    <div class="karta__naglowek">

        <h2>Dokumenty praktyki</h2>

        <span class="karta__opis">Generuj dokumenty związane z wnioskiem studenta</span>

    </div>

    <div class="karta__tresc">

        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem;">

            {% if zapis.track_type.value == 'EMPLOYMENT' %}

            <!-- Ścieżka B - Praca zawodowa -->

            <div class="document-card">

                <div class="document-title">ZAL_7 - Wniosek o zaliczenie</div>

                <div class="document-desc">Wniosek o zaliczenie praktyki na podstawie pracy zawodowej</div>

                <a href="{{ url_for('documents.generuj_auto', id=zapis.id, doc_type='ZAL_7') }}"

                    class="przycisk przycisk--maly przycisk--glowny" target="_blank">

                    Generuj PDF

                </a>

            </div>

            <div class="document-card">

                <div class="document-title">ZAL_4a - Efekty uczenia</div>

                <div class="document-desc">Zestawienie efektów uczenia się dla pracy zawodowej</div>

                <a href="{{ url_for('documents.generuj_auto', id=zapis.id, doc_type='ZAL_4a') }}"

                    class="przycisk przycisk--maly przycisk--drugorzeczny" target="_blank">

                    Generuj PDF

                </a>

            </div>

        </div>

    </div>

</div>

```

```

{% elif zapis.track_type.value == 'OWN_BUSINESS' %}

<!-- Ścieżka C - Działalność gospodarcza -->

<div class="document-card">

    <div class="document-title">ZAL_4b - Wniosek działalność</div>

    <div class="document-desc">Wniosek o zaliczenie na podstawie działalności gospodarczej</div>

    <a href="{% url_for('documents.generuj_auto', id=zapis.id, doc_type='ZAL_4b') %}"

        class="przycisk przycisk--mały przycisk--główny" target="_blank">

        Generuj PDF

    </a>

</div>

<div class="document-card">

    <div class="document-title">ZAL_8 - Protokół egzaminacyjny</div>

    <div class="document-desc">Protokół z egzaminu praktycznego</div>

    <a href="{% url_for('documents.generuj_auto', id=zapis.id, doc_type='ZAL_8') %}"

        class="przycisk przycisk--mały przycisk--drugorzędny" target="_blank">

        Generuj PDF

    </a>

</div>

{% endif %}

</div>

</div>

</div>

<!-- Informacje dodatkowe -->

<div class="karta" style="margin-top: 2rem;">

    <div class="karta__naglowek">

        <h2>Dokumenty załączone przez studenta</h2>

    </div>

    <div class="karta__tresc">

        <div style="text-align: center; color: var(--kolor-tekst-drugor);">

            <p>Liczba załączonych dokumentów: <strong>{% zapis.uploaded_documents | length %}</strong></p>

            <p style="font-size: 0.9rem;">

                Szczegóły dokumentów dostępne w panelu komisji weryfikującej

            </p>

        </div>

    </div>

</div>

```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<style>
```

```
.grid-2 {
```

```
  display: grid;
```

```
  grid-template-columns: 1fr 1fr;
```

```
  gap: 2rem;
```

```
}
```

```
.info-grid {
```

```
  display: grid;
```

```
  grid-template-columns: 1fr 1fr;
```

```
  gap: 1rem;
```

```
}
```

```
.info-item {
```

```
  display: flex;
```

```
  flex-direction: column;
```

```
  gap: 0.25rem;
```

```
}
```

```
.info-item label {
```

```
  font-weight: 500;
```

```
  font-size: 0.85rem;
```

```
  color: var(--kolor-tekst-dlugor);
```

```
}
```

```
.uzasadnienie-box {
```



```
background: #f8fafc;

border: 1px solid #e2e8f0;

border-left: 4px solid var(--kolor-glowny);

border-radius: 0.5rem;

padding: 1rem;

line-height: 1.6;

}
```

```
.komisja-decyzja {

background: #f0fdf4;

border: 1px solid #bbf7d0;

border-radius: 0.5rem;

padding: 1rem;

}
```

```
.decyzja-header {

display: flex;

justify-content: space-between;

align-items: center;

margin-bottom: 0.5rem;

}
```

```
.decyzja-data {

font-size: 0.85rem;

color: var(--kolor-tekst-drugor);

}
```

```
.decyzja-komentarz {

margin-top: 0.75rem;

padding: 0.75rem;

background: white;

border-radius: 0.375rem;

font-size: 0.9rem;

line-height: 1.5;
```

```
}
```

```
.document-card {  
  
  padding: 1rem;  
  
  border: 1px solid #e5e7eb;  
  
  border-radius: 0.5rem;  
  
  background: #f9fafb;  
  
  display: flex;  
  
  flex-direction: column;  
  
  gap: 0.5rem;  
  
}
```

```
.document-title {  
  
  font-weight: 600;  
  
  font-size: 0.9rem;  
  
  color: var(--kolor-glowny);  
  
}
```

```
.document-desc {  
  
  font-size: 0.8rem;  
  
  color: var(--kolor-tekst-drugor);  
  
  flex-grow: 1;  
  
}
```

```
@media (max-width: 768px) {  
  
  .grid-2 {  
  
    grid-template-columns: 1fr;  
  
    gap: 1rem;  
  
  }  
  
  .info-grid {  
  
    grid-template-columns: 1fr;  
  
  }  
  
}
```

</style>

{% endblock %}

PLIK: app_admin\templates\zarzadzanie\dziekan\lista.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Decyzje dziekana{% endblock %}

{% block naglowek %}Wnioski oczekujące na decyzję dziekana{% endblock %}

{% block tresc %}

<div style="max-width: 1400px; margin: 0 auto;">

    {% if wnioski.items %}

    <div class="karta">

        <div class="karta__tresc" style="padding: 0;">

            <div style="overflow-x: auto;">

                <table class="tabela-pelna">

                    <thead>

                        <tr>

                            <th>Student</th>

                            <th>Ścieżka</th>

                            <th>Firma/Działalność</th>

                            <th>Decyzja komisji</th>

                            <th>Data komisji</th>

                            <th>Akcje</th>

                        </tr>

                    </thead>

                    <tbody>

                        {% for wniosek in wnioski.items %}

                        <tr>

                            <td>

                                <div>

                                    <strong>{{ wniosek.student.last_name }}, {{ wniosek.student.first_name }}</strong><br>

                                    <small style="color: var(--kolor-tekst-drugor);">{{ wniosek.student.album_number }}</small>

                                </div>

                            </td>

                            <td>
```

```

<span class="status

    {%- if wniosek.track_type.value == 'EMPLOYMENT' %} status--w-trakcie

    {%- elif wniosek.track_type.value == 'OWN_BUSINESS' %} status--szkic

    {%- endif %}">

    {% if wniosek.track_type.value == 'EMPLOYMENT' %}Praca zawodowa

    {% elif wniosek.track_type.value == 'OWN_BUSINESS' %}Działalność gospodarcza

    {% endif %}

</span>

</td>

<td>

    <div>

        {% if wniosek.firma_nazwa %}

            <strong>{{ wniosek.firma_nazwa }}</strong><br>

            <small style="color: var(--kolor-tekst-drugor);">{{ wniosek.firma_miasto or '' }}</small>

        {% else %}

            <span style="color: var(--kolor-tekst-drugor);">Brak danych</span>

        {% endif %}

    </div>

</td>

<td>

    <span class="status status--zakonczone">Zatwierdzone przez komisję</span>

    {% if wniosek.komentarze_komisji %}

        <div style="margin-top: 0.5rem; font-size: 0.8rem; color: var(--kolor-tekst-drugor);">

            {{ wniosek.komentarze_komisji[:50] }}{% if wniosek.komentarze_komisji|length > 50 %}...{% endif %}

        </div>

    {% endif %}

</td>

<td>

    <time>

        {% if wniosek.decyzja_komisji_o %}

            {{ wniosek.decyzja_komisji_o.strftime('%d.%m.%Y') }}

        {% endif %}

    </time>

</td>

```

```

        <td>

        <div style="display: flex; gap: 0.5rem;">

            <a href="{{ url_for('zarzadzanie.dziekan_decyzja', id=wniosek.id) }}"

                class="przycisk przycisk--maly przycisk--glowny">

                    Podjmij decyzję

                </a>

            </div>

        </td>

    </tr>

    {% endfor %}

</tbody>

</table>

</div>

</div>

</div>

<!-- Paginacja -->

{% if wnioski.pages > 1 %}

<div class="paginacja">

    {% if wnioski.has_prev %}

        <a href="{{ url_for('zarzadzanie.dziekan_lista', page=wnioski.prev_num) }}" class="paginacja__link">←
Poprzednia</a>

    {% endif %}

    {% for page_num in wnioski.iter_pages() %}

        {% if page_num %}

            {% if page_num != wnioski.page %}

                <a href="{{ url_for('zarzadzanie.dziekan_lista', page=page_num) }}" class="paginacja__link">{{ page_num }}</a>

            {% else %}

                <span class="paginacja__link paginacja__link--aktywny">{{ page_num }}</span>

            {% endif %}

        {% else %}

            <span class="paginacja__kropki">...</span>

        {% endif %}

    {% endfor %}


```

```
{% if wnioski.has_next %}

    <a href="{{ url_for('zarzadzanie.dziekan_lista', page=wnioski.next_num) }}" class="paginacja__link">Następna →</a>

{% endif %}

</div>

{% endif %}


{% else %}

<div class="karta">

    <div class="karta__tresc" style="text-align: center; padding: 3rem 2rem;">

        <svg width="64" height="64" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.5" style="color:
var(--kolor-tekst-drugor); margin-bottom: 1rem;">

            <path d="M12 213.09 6.26L22 9.27l-5 4.87 1.18 6.88L12 17.77l-6.18 3.25L7 14.14 2 9.27l6.91-1.01L12 22"/>

        </svg>

        <h3 style="margin-bottom: 0.5rem;">Brak wniosków oczekujących na decyzję</h3>

        <p style="color: var(--kolor-tekst-drugor); margin-bottom: 0;">

            Wszystkie wnioski zostały już przetworzone lub nie ma nowych zgłoszeń zatwierdzonych przez komisję.

        </p>

    </div>

</div>

{% endif %}


</div>

{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\enrollments\list.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Zgłoszenia studentów{% endblock %}

{% block naglowek %}Zarządzanie zgłoszeniami praktyk{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem;">

    <a href="{{ url_for('dashboard.index') }}" class="przycisk przycisk--drugorzeczny">← Powrót</a>

</div>

{% endblock %}


{% block tresc %}

<div style="max-width: 1400px; margin: 0 auto;">


    <!-- Filtry statusu -->

    <div class="karta" style="margin-bottom: 2rem;">

        <div class="karta__tresc">

            <div style="display: flex; gap: 1rem; align-items: center;">

                <span style="font-weight: 600;">Filtruj według statusu:</span>

                <a href="{{ url_for('zarzadzanie.lista_zgloszen') }}"

                    class="przycisk przycisk--{% if not request.args.get('status') %}glowny{% else %}drugorzeczny{% endif %} przycisk--maly">

                    Wszystkie

                </a>

                <a href="{{ url_for('zarzadzanie.lista_zgloszen', status='PENDING') }}"

                    class="przycisk przycisk--{% if request.args.get('status') == 'PENDING' %}glowny{% else %}drugorzeczny{% endif %} przycisk--maly">

                    Oczekujące

                </a>

                <a href="{{ url_for('zarzadzanie.lista_zgloszen', status='IN_PROGRESS') }}"

                    class="przycisk przycisk--{% if request.args.get('status') == 'IN_PROGRESS' %}glowny{% else %}drugorzeczny{% endif %} przycisk--maly">

                    Zatwierdzone

                </a>

            </div>

        </div>

    </div>

</div>

</div>

</div>
```



```

<a href="{{ url_for('zarzadzanie.lista_zgloszen', status='AWAITING_APPROVAL') }}"

        class="przycisk przycisk--{% if request.args.get('status') == 'AWAITING_APPROVAL' %}glowny{% else
%}drugorzeczny{% endif %} przycisk--maly">

        Nowe zgłoszenia

</a>

</div>

</div>

</div>

<!-- Lista zgłoszeń -->

<div class="karta">

    <div class="karta__naglowek">

        <h2 class="karta__tytul">Lista zgłoszeń ({{ zgloszenia.total }})</h2>

    </div>

    <div class="karta__tresc">

        <div class="tabela-wrapper">

            <table class="tabela">

                <thead>

                    <tr>

                        <th style="width: 25%;">Student</th>

                        <th style="width: 20%;">Praktyka</th>

                        <th style="width: 20%;">Firma</th>

                        <th style="width: 15%;">Status</th>

                        <th style="width: 12%;">Data zgłoszenia</th>

                        <th style="width: 8%; text-align: center;">Akcje</th>

                    </tr>

                </thead>

                <tbody>

                    {% for z in zgloszenia.items %}

                        <tr>

                            <td>

                                <div style="font-weight: 600;">{{ z.student.first_name }} {{ z.student.last_name }}</div>

                                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ z.student.email }}</div>

                                {% if z.student.album_number %}

                                    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Album: {{ z.student.album_number

```


Szczegóły


```
{% if not z.uopz and current_user.role.value == 'ADMIN' and z.track_type.value == 'STANDARD' %}
```

```
<a href="{{ url_for('zarzadzanie.przypisz_uopz', id=z.id) }}"
```

```
class="przycisk przycisk--maly"
```

```
style="min-width: 80px;">
```

Przypisz UOPZ


```
{% endif %}
```

</div>

</td>

</tr>

```
{% else %}
```

<tr>

<td colspan="6" class="tabela--pusta">

Brak zgłoszeń{% if request.args.get('status') %} o wybranym statusie{% endif %}.

</td>

</tr>

```
{% endfor %}
```

</tbody>

</table>

</div>

<!-- Paginacja -->

```
{% if zgloszenia.pages > 1 %}
```

```
<div style="display: flex; justify-content: center; margin-top: 2rem; gap: 0.5rem;">
```

```
{% if zgloszenia.has_prev %}
```

```
<a href="{{ url_for('zarzadzanie.lista_zgloszen', page=zgloszenia.prev_num, **request.args) }}"
```

```
class="przycisk przycisk--drugorzecny przycisk--maly">← Poprzednia</a>
```

```
{% endif %}
```

```
{% for page_num in zgloszenia.iter_pages() %}
```

```
{% if page_num %}
```

```

{% if page_num != zgloszenia.page %}

<a href="{{ url_for('zarzadzanie.lista_zgloszen', page=page_num, **request.args) }}"

    class="przycisk przycisk--drugorzeczny przycisk--maly">{{ page_num }}</a>

{% else %}

<span class="przycisk przycisk--glowny przycisk--maly">{{ page_num }}</span>

{% endif %}

{% else %}

    <span>...</span>

{% endif %}

{% endfor %}


{% if zgloszenia.has_next %}

<a href="{{ url_for('zarzadzanie.lista_zgloszen', page=zgloszenia.next_num, **request.args) }}"

    class="przycisk przycisk--drugorzeczny przycisk--maly">Następna →</a>

{% endif %}

</div>

{% endif %}


</div>

</div>

</div>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\enrollments\moje_lista.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Moje zgłoszenia - UOPZ{% endblock %}

{% block naglowek %}Zgłoszenia moich studentów{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem;">

    <a href="{{ url_for('dashboard.index') }}" class="przycisk przycisk--drugorzeczny">← Dashboard</a>

</div>

{% endblock %}


{% block tresc %}

<div style="max-width: 1400px; margin: 0 auto;">


    <!-- Info dla UOPZ -->

    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-info);">

        <div class="karta__tresc">

            <div>

                <div style="font-weight: 600; margin-bottom: 0.5rem;">Panel opiekuna uczelnianego (UOPZ)</div>

                <div style="color: var(--kolor-tekst-drugor);">

                    Tutaj znajdziesz wszystkie zgłoszenia studentów, których masz pod opieką.

                    Możesz przeglądać szczegółowe dane wypełnione przez studentów i podejmować decyzje o zatwierdzeniu lub
                    odrzuceniu zgłoszenia.

                </div>

            </div>

        </div>

    </div>


    <!-- Filtry statusu -->

    <div class="karta" style="margin-bottom: 2rem;">

        <div class="karta__tresc">

            <div style="display: flex; gap: 1rem; align-items: center; flex-wrap: wrap;">

                <span style="font-weight: 600;">Filtruj według statusu:</span>

            </div>

        </div>

    </div>

</div>
```

[illegible]

Dziś ostatni dzień na ocenę

{% else %}

Pozostało {{ item.dni_do_deadline }} dni do deadline ({{ item.deadline.strftime('%d.%m.%Y') }})

{% endif %}

</div>

</div>

<div>

<a href="{{ url_for('evaluation.ocen_praktyke', id=item.zapis.id) }}"

class="przycisk przycisk--glowny przycisk--maly">

Wystaw ocenę

</div>

</div>

</div>

{% endfor %}

</div>

</div>

{% endif %}

<!-- Lista zgłoszeń -->

<div class="karta">

<div class="karta__naglowek">

<h2 class="karta__tytul">

Moje zgłoszenia

{% if zgloszenia.total > 0 %}

{{ zgloszenia.total }}

{% endif %}

</h2>

</div>

<div class="karta__tresc">

{% if zgloszenia.items %}

<div class="tabela-wrapper">

<table class="tabela">

<thead>

```

<tr>

    <th style="width: 25%;">Student</th>

    <th style="width: 20%;">Praktyka</th>

    <th style="width: 20%;">Firma</th>

    <th style="width: 15%;">Status</th>

    <th style="width: 12%;">Zgłoszono</th>

    <th style="width: 8%; text-align: center;">Akcje</th>

</tr>

</thead>

<tbody>

    {% for z in zgloszenia.items %}

    <tr>

        <td>

            <div style="font-weight: 600;">{{ z.student.first_name }} {{ z.student.last_name }}</div>

            <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ z.student.email }}</div>

            {% if z.student.album_number %}

                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Album: {{ z.student.album_number
}}</div>

            {% endif %}

        </td>

        <td>

            <div style="font-weight: 600;">{{ z.praktyka.rok_uczelnianny }}</div>

            <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Semestr {{ z.praktyka.semestr }}</div>

            {% if z.termin_od and z.termin_do %}

                <div style="font-size: 0.75rem; color: var(--kolor-tekst-drugor); margin-top: 0.25rem;">

                    {{ z.termin_od.strftime('%d.%m') }} - {{ z.termin_do.strftime('%d.%m.%Y') }}

                </div>

            {% endif %}

        </td>

        <td>

            <div style="font-weight: 600;">{{ z.firma_nazwa or 'Nie podano' }}</div>

            {% if z.firma_miasto %}

                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ z.firma_miasto }}</div>

            {% endif %}

            {% if z.track_type.value != 'STANDARD' %}

```



```

<div style="font-size: 0.75rem; color: var(--kolor-info); margin-top: 0.25rem;">

    {% if z.track_type.value == 'EMPLOYMENT' %}Na podstawie zatrudnienia

    {% elif z.track_type.value == 'OWN_BUSINESS' %}Własna działalność

    {% endif %}

</div>

{% endif %}

</td>

<td>

    <span class="status status--{{ z.status.value|lower|replace('_', '-') }}">

        {{ tlumacz_status(z.status.value) }}

    </span>

    {% if z.komentarze_admina or z.komentarze_uopz %}

    <div style="font-size: 0.7rem; color: var(--kolor-info); margin-top: 0.25rem;">

        Są komentarze

    </div>

    {% endif %}

</td>

<td style="font-size: 0.85rem;">

    {{ z.enrolled_at.strftime('%d.%m.%Y') if z.enrolled_at else '-' }}

    <div style="font-size: 0.7rem; color: var(--kolor-tekst-drugor);">

        {{ z.enrolled_at.strftime('%H:%M') if z.enrolled_at else '' }}

    </div>

</td>

<td style="text-align: center;">

    <div style="display: flex; flex-direction: column; gap: 0.5rem; align-items: center;">

        <a href="{{ url_for('zarzadzanie.szczegoly_zgloszenia', id=z.id) }}"

            class="przycisk przycisk--{% if z.status.value == 'AWAITING_APPROVAL' %}glowny{% else
%}drugorzedny{% endif %} przycisk--maly"

            style="min-width: 80px;">

            {% if z.status.value == 'AWAITING_APPROVAL' %}Oceń{% else %}Szczegóły{% endif %}

        </a>

        {% if z.status.value == 'IN_PROGRESS' %}

        <a href="{{ url_for('evaluation.ocen_praktyke', id=z.id) }}"

            class="przycisk przycisk--maly"

```

```

        style="min-width: 80px; ">

        Ocena UOPZ

    </a>

    {% endif %}

</div>

</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

<!-- Paginacja -->

{% if zgloszenia.pages > 1 %}

<div style="display: flex; justify-content: center; margin-top: 2rem; gap: 0.5rem;">

    {% if zgloszenia.has_prev %}

    <a href="{{ url_for('zarzadzanie.moje_zgloszenia', page=zgloszenia.prev_num, **request.args) }}"

        class="przycisk przycisk--drugorzeczny przycisk--maly">← Poprzednia</a>

    {% endif %}

    {% for page_num in zgloszenia.iter_pages() %}

        {% if page_num %}

            {% if page_num != zgloszenia.page %}

                <a href="{{ url_for('zarzadzanie.moje_zgloszenia', page=page_num, **request.args) }}"

                    class="przycisk przycisk--drugorzeczny przycisk--maly">{{ page_num }}</a>

            {% else %}

                <span class="przycisk przycisk--glowny przycisk--maly">{{ page_num }}</span>

            {% endif %}

        {% else %}

            <span>...</span>

        {% endif %}

    {% endfor %}

    {% if zgloszenia.has_next %}

```

```

<a href="{{ url_for('zarzadzanie.moje_zgloszenia', page=zgloszenia.next_num, **request.args) }}"

    class="przycisk przycisk--drugorzeczny przycisk--maly">Następna →</a>

{% endif %}

</div>

{% endif %}

{% else %}

<!-- Pusty stan -->

<div style="text-align: center; padding: 4rem 2rem;">

    <div style="font-size: 3rem; margin-bottom: 1rem; opacity: 0.3;"></div>

    <div style="font-size: 1.2rem; font-weight: 600; margin-bottom: 1rem; color: var(--kolor-tekst-drugor);">

        Brak zgłoszeń

    </div>

    <div style="color: var(--kolor-tekst-drugor);">

        {% if request.args.get('status') %}

            Nie masz zgłoszeń o wybranym statusie.

        {% else %}

            Nie zostały Ci jeszcze przypisani studenci do opieki.

        {% endif %}

    </div>

    {% if request.args.get('status') %}

        <div style="margin-top: 2rem;">

            <a href="{{ url_for('zarzadzanie.moje_zgloszenia') }}" class="przycisk przycisk--drugorzeczny">

                Pokaż wszystkie zgłoszenia

            </a>

        </div>

        {% endif %}

    </div>

    {% endif %}

</div>

</div>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\enrollments\przypisz_uopz.html

```
{% extends "layouts/base.html" %}

{% block tytul %}Przypisz UOPZ{% endblock %}

{% block tresc %}

<h1>Przypisz opiekuna uczelnianego</h1>

<div class="karta">

    <form method="post">

        {{ form.hidden_tag() }}

        <div class="pole-form">

            <label>{{ form.uopz_id.label }}</label>

            {{ form.uopz_id(class_='select') }}

        </div>

        <div style="margin-top:1rem;">

            <button type="submit" class="przycisk przycisk--glowny">Zapisz</button>

            <a href="{{ url_for('zarzadzanie.lista_zgloszen') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

        </div>

    </form>

</div>

{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\enrollments\szczegoly.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Szczegóły zgłoszenia – {{ zapis.student.first_name }} {{ zapis.student.last_name }}{% endblock %}

{% block naglowek %}Szczegóły zgłoszenia{% endblock %}

{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem; align-items: center;">

    {% if current_user.role.value == 'ADMIN' %}

        <a href="{{ url_for('zarzadzanie.lista_zgloszen') }}" class="przycisk przycisk--drugorzeczny">← Powrót do listy</a>

    {% else %}

        <a href="{{ url_for('zarzadzanie.moje_zgloszenia') }}" class="przycisk przycisk--drugorzeczny">← Moje zgłoszenia</a>

    {% endif %}

</div>

{% endblock %}

{% block tresc %}

<div style="max-width: 1200px; margin: 0 auto;">

    {# — Karta identyfikacyjna zgłoszenia — #}

    {% set status_val = zapis.status.value %}

    {% if status_val == 'PENDING' %}

        {% set border_color = 'var(--kolor-akcent)' %}

        {% set status_cls = 'status--in-progress' %}

        {% set status_label = 'Oczekuje na zatwierdzenie' %}

    {% elif status_val == 'AWAITING_APPROVAL' %}

        {% set border_color = '#3b82f6' %}

        {% set status_cls = 'status--planned' %}

        {% set status_label = 'Oczekuje na akceptację UOPZ' %}

    {% elif status_val == 'IN_PROGRESS' %}

        {% set border_color = 'var(--kolor-sukces)' %}

        {% set status_cls = 'status--completed' %}

        {% set status_label = 'Zatwierdzona – w realizacji' %}

    {% elif status_val == 'COMPLETED' %}
```

```

{% set border_color = 'var(--kolor-glowny)' %}

{% set status_cls = 'status--completed' %}

{% set status_label = 'Zakończona' %}

{% else %}

{% set border_color = 'var(--kolor-ramka)' %}

{% set status_cls = 'status--draft' %}

{% set status_label = status_val %}

{% endif %}

<div style="background: var(--kolor-biale); border-radius: var(--promien-duzy); border-top: 4px solid {{ border_color
}}; box-shadow: var(--cien-karta); margin-bottom: 2rem; overflow: hidden;">

    <div style="padding: 1.75rem 2rem; display: grid; grid-template-columns: auto 1fr auto; gap: 2rem; align-items:
center;">

        {# Awatar inicjałowy #}

        <div style="width: 56px; height: 56px; border-radius: 50%; background: var(--kolor-glowny); color: #fff; display:
flex; align-items: center; justify-content: center; font-size: 1.25rem; font-weight: 700; flex-shrink: 0; letter-spacing:
0.5px;">

            {{ zapis.student.first_name[0] }}{{ zapis.student.last_name[0] }}

        </div>

        {# Dane studenta i praktyki #}

        <div style="display: grid; grid-template-columns: repeat(4, 1fr); gap: 1.5rem; min-width: 0;">

            <div>

                <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Student</div>

                <div style="font-weight: 700; font-size: 1rem; color: var(--kolor-tekst);">{{ zapis.student.first_name }} {{
zapis.student.last_name }}</div>

                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); margin-top: 0.15rem;">{{ zapis.student.email
}}</div>

                {% if zapis.student.album_number %}

                    <div style="font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed);">nr alb. {{ zapis.student.album_number
}}</div>

                {% endif %}

            </div>

            <div>

                <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Rok akademicki</div>

                <div style="font-weight: 600; color: var(--kolor-tekst);">{{ zapis.praktyka.rok_uczelnianny }}</div>

```

```

        <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Semestr {{ zapis.praktyka.semestr }}</div>

</div>

<div>

        <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Ścieżka</div>

        <div style="font-weight: 600; color: var(--kolor-tekst);">

                {% if zapis.track_type.value == 'STANDARD' %}Standardowa

                {% elif zapis.track_type.value == 'EMPLOYMENT' %}Na podst. zatrudnienia

                {% elif zapis.track_type.value == 'OWN_BUSINESS' %}Własna działalność

                {% else %}{{ zapis.track_type.value }}{% endif %}

        </div>

</div>

<div>

        <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Data zgłoszenia</div>

        <div style="font-weight: 600; color: var(--kolor-tekst);">{{ zapis.enrolled_at.strftime('%d.%m.%Y') if
zapis.enrolled_at else '-' }}</div>

        <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ zapis.enrolled_at.strftime('%H:%M') if
zapis.enrolled_at else '' }}</div>

</div>

</div>

{# Odznaka statusu #}

<div style="text-align: right; flex-shrink: 0;">

        <span class="status {{ status_cls }}" style="font-size: 0.75rem; padding: 0.4rem 1rem;">{{ status_label }}</span>

        {% if zapis.uopz %}

                <div style="margin-top: 0.75rem; font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed); text-align: right;">

                        UOPZ: <strong style="color: var(--kolor-tekst-drugor);">{{ zapis.uopz.first_name }} {{ zapis.uopz.last_name
}}</strong>

                </div>

        {% endif %}

</div>

</div>

</div>

```

```
{# — Główna siatka treści — #}
```

```
<div style="display: grid; grid-template-columns: 1fr 360px; gap: 1.5rem; align-items: start;">
```

```
{# == LEWA KOLUMNA == #}
```

```
<div style="display: flex; flex-direction: column; gap: 1.5rem;">
```

```
{# Dane podstawowe #}
```

```
<div class="karta">
```

```
<div class="karta__naglowek">
```

```
<h3 class="karta__tytul">Dane podstawowe</h3>
```

```
</div>
```

```
<div class="karta__tresc">
```

```
<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.25rem;">
```

```
<div class="pole-info">
```

```
<div class="pole-info__etykieta">Ścieżka praktyki</div>
```

```
<div class="pole-info__wartosc">
```

```
{% if zapis.track_type.value == 'STANDARD' %}Standardowa
```

```
{% elif zapis.track_type.value == 'EMPLOYMENT' %}Na podstawie zatrudnienia
```

```
{% elif zapis.track_type.value == 'OWN_BUSINESS' %}Własna działalność gospodarcza
```

```
{% else %}{{ zapis.track_type.value }}{% endif %}
```

```
</div>
```

```
</div>
```

```
<div class="pole-info">
```

```
<div class="pole-info__etykieta">Specjalność</div>
```

```
<div class="pole-info__wartosc">{{ zapis.specjalnosc or '-' }}</div>
```

```
</div>
```

```
{% if zapis.track_type.value == 'STANDARD' %}
```

```
<div class="pole-info">
```

```
<div class="pole-info__etykieta">Termin praktyki</div>
```

```
<div class="pole-info__wartosc">
```

```
{{ zapis.termin_od.strftime('%d.%m.%Y') if zapis.termin_od else 'Brak' }}
```

```
<span style="color: var(--kolor-tekst-trzeciorzed); margin: 0 0.25rem;"></span>
```

```
{{ zapis.termin_do.strftime('%d.%m.%Y') if zapis.termin_do else 'Brak' }}
```

```
</div>
```



```

</div>

<div class="pole-info">

  <div class="pole-info__etykieta">Ubezpieczenie NW</div>

  <div class="pole-info__wartosc">

    {% if zapis.ubezpieczenie_nw %}

      <span style="color: var(--kolor-sukces); display: inline-flex; align-items: center; gap: 0.35rem;">

        <span style="width: 8px; height: 8px; border-radius: 50%; background: currentColor; display: inline-block;"></span>

        Potwierdzone

      </span>

    {% else %}

      <span style="color: var(--kolor-blad); display: inline-flex; align-items: center; gap: 0.35rem;">

        <span style="width: 8px; height: 8px; border-radius: 50%; background: currentColor; display: inline-block;"></span>

        Nie potwierdzone

      </span>

    {% endif %}

  </div>

</div>

{% endif %}

</div>

</div>

```

```

{% if zapis.track_type.value == 'STANDARD' %}

```

```

{# Dane zakładu pracy #}

```

```

<div class="karta">

  <div class="karta__naglowek">

    <h3 class="karta__tytul">Zakład pracy</h3>

  </div>

  <div class="karta__tresc">

    <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.25rem;">

      <div class="pole-info">

        <div class="pole-info__etykieta">Nazwa firmy</div>

```

```

        <div class="pole-info__wartosc" style="font-weight: 600;">{{ zapis.firma_nazwa or '-' }}</div>

    </div>

    <div class="pole-info">

        <div class="pole-info__etykieta">NIP / KRS</div>

        <div class="pole-info__wartosc">{{ zapis.firma_nip_krs or '-' }}</div>

    </div>

    <div class="pole-info">

        <div class="pole-info__etykieta">Adres</div>

        <div class="pole-info__wartosc">{{ zapis.firma_adres or '-' }}</div>

    </div>

    <div class="pole-info">

        <div class="pole-info__etykieta">Miasto</div>

        <div class="pole-info__wartosc">{{ zapis.firma_miasto or '-' }}</div>

    </div>

    <div class="pole-info" style="grid-column: 1 / -1;">

        <div class="pole-info__etykieta">Przedstawiciel firmy</div>

        <div class="pole-info__wartosc">

            {{ zapis.firma_upowazniony_osoba or '-' }}

            {% if zapis.firma_upowazniony_stanowisko %}

                <span style="color: var(--kolor-tekst-drugor); font-weight: 400;"> · {{
zapis.firma_upowazniony_stanowisko }}</span>

                {% endif %}

            </div>

        </div>

    </div>

</div>

{# Opiekun zakładowy (ZOPZ) #}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Opiekun zakładowy (ZOPZ)</h3>

    </div>

    <div class="karta__tresc">

        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.25rem;">

```

```

<div class="pole-info">

    <div class="pole-info__etykieta">Imię i nazwisko</div>

    <div class="pole-info__wartosc" style="font-weight: 600;">{{ zapis.zopz_imie_nazwisko or '-' }}</div>

</div>

<div class="pole-info">

    <div class="pole-info__etykieta">Stanowisko</div>

    <div class="pole-info__wartosc">{{ zapis.zopz_stanowisko or '-' }}</div>

</div>

<div class="pole-info">

    <div class="pole-info__etykieta">Telefon</div>

    <div class="pole-info__wartosc">{{ zapis.zopz_telefon or '-' }}</div>

</div>

<div class="pole-info">

    <div class="pole-info__etykieta">E-mail</div>

    <div class="pole-info__wartosc">{{ zapis.zopz_email or '-' }}</div>

</div>

</div>

</div>

</div>

```

```
{# Harmonogram #}
```

```
<div class="karta">
```

```
    <div class="karta__naglowek" style="display: flex; justify-content: space-between; align-items: center;">
```

```
        <h3 class="karta__tytul">Harmonogram praktyki</h3>
```

```
        {% set suma_dni = namespace(wartosc=0) %}
```

```
        {% for e in efekty %}
```

```
            {% set h = harmonogram_dict.get(e.id) %}
```

```
            {% if h and h.liczba_dni > 0 %}{% set suma_dni.wartosc = suma_dni.wartosc + h.liczba_dni %}{% endif %}
```

```
        {% endfor %}
```

```

                                <span class="status" {% if suma_dni.wartosc > 120 %}status--draft"
style="background:#fee2e2;color:var(--kolor-blad){% elif suma_dni.wartosc < 100 %}status--in-progress{% else
%}status--completed{% endif %}">

```

```
                                {{ suma_dni.wartosc }} / 120 dni
```

```
                                </span>
```

```
    </div>
```

```
</div>
```

```
<div class="karta__tresc" style="padding: 0;">
```

```
<div class="tabela-wrapper">
```

```
<table class="tabela">
```

```
<thead>
```

```
<tr>
```

```
<th style="width: 4%; text-align: center;">Nr</th>
```

```
<th style="width: 38%;">Efekt uczenia się</th>
```

```
<th style="width: 25%;">Dział / Jednostka</th>
```

```
<th style="width: 25%;">Przykładowe zadania</th>
```

```
<th style="width: 8%; text-align: center;">Dni</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
{% for e in efekty %}
```

```
{% set h = harmonogram_dict.get(e.id) %}
```

```
{% set brak = not h or h.liczba_dni == 0 %}
```

```
<tr {% if brak %}style="opacity: 0.55;"{% endif %}>
```

```
<td style="text-align: center; font-weight: 600; color: var(--kolor-tekst-trzeciorzed); font-size: 0.8rem;">{{ '%02d'|format(e.id) }}</td>
```

```
<td style="font-size: 0.82rem; line-height: 1.45;">{{ e.description }}</td>
```

```
<td style="font-size: 0.85rem;">
```

```
{% if h and h.nazwa_dzialu %}{{ h.nazwa_dzialu }}
```

```
{% else %}<span style="color: var(--kolor-tekst-trzeciorzed); font-style: italic;">Nie przypisano</span>{% endif %}
```

```
</td>
```

```
<td style="font-size: 0.85rem;">
```

```
{% if h and h.przykladowe_prace %}{{ h.przykladowe_prace }}
```

```
{% else %}<span style="color: var(--kolor-tekst-trzeciorzed); font-style: italic;">Nie podano</span>{% endif %}
```

```
</td>
```

```
<td style="text-align: center; font-weight: 700; font-size: 0.95rem;
```

```
color: {% if brak %}var(--kolor-tekst-trzeciorzed){% else %}var(--kolor-glowny){% endif %};">
```

```
{{ h.liczba_dni if h else 0 }}
```

```
</td>
```

```
</tr>
```

```
{% endfor %}
```

```

</tbody>

<tfoot>

    <tr style="background: var(--kolor-tlo);">

        <td colspan="4" style="text-align: right; font-weight: 600; padding-right: 1rem; font-size: 0.85rem;
color: var(--kolor-tekst-drugor);">Suma dni:</td>

        <td style="text-align: center; font-weight: 700; font-size: 1.05rem;

            color: {% if suma_dni.wartosc > 120 %}var(--kolor-blad){% elif suma_dni.wartosc < 100 %}#92400e{%
else %}var(--kolor-sukces){% endif %};">

            {{ suma_dni.wartosc }}

        </td>

    </tr>

</tfoot>

</table>

</div>

{% if suma_dni.wartosc > 120 %}

    <div style="margin: 1rem 1.5rem; padding: 0.75rem 1rem; background: #fee2e2; border-radius: var(--promien);
color: var(--kolor-blad); font-size: 0.85rem; display: flex; gap: 0.5rem; align-items: flex-start;">

        <strong>Przekroczono limit 120 dni.</strong> Harmonogram wymaga korekty przed zatwierdzeniem.

    </div>

    {% elif suma_dni.wartosc < 100 %}

        <div style="margin: 1rem 1.5rem; padding: 0.75rem 1rem; background: #fef3c7; border-radius: var(--promien);
color: #92400e; font-size: 0.85rem; display: flex; gap: 0.5rem; align-items: flex-start;">

            <strong>Niewystarczająca liczba dni (min. 100).</strong> Harmonogram wymaga uzupełnienia.

        </div>

    {% endif %}

</div>

</div>

{% else %}

{# — Ścieżka B/C — #}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Dane pracodawcy / działalności</h3>

    </div>

    <div class="karta__tresc">

        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.25rem;">

```

```

<div class="pole-info">

    <div class="pole-info__etykieta">Nazwa firmy / działalności</div>

    <div class="pole-info__wartosc" style="font-weight: 600;">{{ zapis.firma_nazwa or '-' }}</div>

</div>

<div class="pole-info">

    <div class="pole-info__etykieta">Stanowisko / zakres obowiązków</div>

    <div class="pole-info__wartosc">{{ zapis.zopz_stanowisko or '-' }}</div>

</div>

<div class="pole-info" style="grid-column: 1 / -1;">

    <div class="pole-info__etykieta">Adres</div>

    <div class="pole-info__wartosc">{{ zapis.firma_adres or '-' }}{% if zapis.firma_miasto %}, {{
zapis.firma_miasto }}{% endif %}</div>

</div>

</div>

</div>

{% if zapis.uzasadnienie_sciezki %}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Uzasadnienie wniosku</h3>

    </div>

    <div class="karta__tresc">

        <p style="white-space: pre-line; font-size: 0.9rem; line-height: 1.7; color: var(--kolor-tekst);">{{
zapis.uzasadnienie_sciezki }}</p>

    </div>

</div>

{% endif %}

{% if zapis.decyzja_komisji or zapis.decyzja_dziekana %}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Decyzje</h3>

    </div>

    <div class="karta__tresc" style="display: flex; flex-direction: column; gap: 1rem;">

```

```

{% if zapis.decyzja_komisji %}

    <div style="padding: 1rem; border: 1px solid var(--kolor-ramka); border-radius: var(--promien); background:
var(--kolor-tlo);">

        <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.5rem;">

            <span style="font-size: 0.75rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em;
color: var(--kolor-tekst-trzeciorzed);">Komisja ds. praktyk</span>

            <span class="status {% if zapis.decyzja_komisji == 'APPROVED' %}status--completed{% elif
zapis.decyzja_komisji == 'REJECTED' %}status--draft" style="background:#fee2e2;color:var(--kolor-blad){% else
%}status--in-progress{% endif %}">

                {% if zapis.decyzja_komisji == 'APPROVED' %}Zatwierdzono

                {% elif zapis.decyzja_komisji == 'PARTIALLY_APPROVED' %}Częściowo zatwierdzono

                {% else %}Odrzucono{% endif %}

            </span>

        </div>

        {% if zapis.komentarze_komisji %}<p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); line-height:
1.5; margin: 0;">{{ zapis.komentarze_komisji }}</p>{% endif %}

    </div>

{% endif %}

{% if zapis.decyzja_dziekana %}

    <div style="padding: 1rem; border: 1px solid var(--kolor-ramka); border-radius: var(--promien); background:
var(--kolor-tlo);">

        <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.5rem;">

            <span style="font-size: 0.75rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em;
color: var(--kolor-tekst-trzeciorzed);">Dyrektor IIS</span>

            <span class="status {% if zapis.decyzja_dziekana == 'APPROVED' %}status--completed{% else %}status--draft"
style="background:#fee2e2;color:var(--kolor-blad){% endif %}">

                {{ 'Zatwierdzono' if zapis.decyzja_dziekana == 'APPROVED' else 'Odrzucono' }}

            </span>

        </div>

        {% if zapis.komentarze_dziekana %}<p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);
line-height: 1.5; margin: 0;">{{ zapis.komentarze_dziekana }}</p>{% endif %}

    </div>

{% endif %}

</div>

</div>

{% endif %}

{% endif %}

{# Załączniki #}

```

```

{% if uploaded_docs %}

<div class="karta">

  <div class="karta__naglowek">

    <h3 class="karta__tytul">Załączniki studenta</h3>

    <span style="font-size: 0.75rem; font-weight: 400; color: var(--kolor-tekst-trzeciorzed);">{{
uploaded_docs|length }} plik{% if uploaded_docs|length != 1 %}i{% endif %}</span>

  </div>

  <div class="karta__tresc" style="display: flex; flex-direction: column; gap: 0.5rem;">

    {% for dok in uploaded_docs %}

      <div style="display: flex; justify-content: space-between; align-items: center; padding: 0.75rem 1rem; border:
1px solid var(--kolor-ramka); border-radius: var(--promien); background: var(--kolor-tlo); transition: border-color
0.15s;">

        <div style="min-width: 0;">

          <div style="font-weight: 600; font-size: 0.875rem; white-space: nowrap; overflow: hidden; text-overflow:
ellipsis;">{{ dok.original_filename }}</div>

          <div style="font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed); margin-top: 0.15rem;">{{
dok.document_type }} · {{ dok.uploaded_at.strftime('%d.%m.%Y, %H:%M') }}</div>

        </div>

        <a href="{{ url_for('uploads.download_document', document_id=dok.id) }}"

          class="przycisk przycisk--maly przycisk--drugorzeczny" style="flex-shrink: 0; margin-left:
1rem;">Pobierz</a>

      </div>

    {% endfor %}

  </div>

</div>

{% endif %}

</div>

{# == PRAWA KOLUMNA == #}

<div style="display: flex; flex-direction: column; gap: 1.5rem; position: sticky; top: 1.5rem;">

  {# Komentarze #}

  {% if zapis.komentarze_admina or zapis.komentarze_uopz %}

    <div class="karta">

      <div class="karta__naglowek">

        <h3 class="karta__tytul">Komentarze</h3>

```



```

</div>

<div class="karta__tresc" style="display: flex; flex-direction: column; gap: 0.75rem;">

    {% if zapis.komentarze_admina %}

        <div style="padding: 0.875rem; border-left: 3px solid var(--kolor-glowny); background: var(--kolor-tlo);
border-radius: 0 var(--promien) var(--promien) 0;">

            <div style="font-size: 0.7rem; font-weight: 700; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-glowny); margin-bottom: 0.4rem;">Administrator</div>

            <div style="font-size: 0.875rem; color: var(--kolor-tekst); line-height: 1.55; white-space: pre-wrap;">{{
zapis.komentarze_admina }}</div>

        </div>

    {% endif %}

    {% if zapis.komentarze_uopz %}

        <div style="padding: 0.875rem; border-left: 3px solid var(--kolor-akcent); background: var(--kolor-tlo);
border-radius: 0 var(--promien) var(--promien) 0;">

            <div style="font-size: 0.7rem; font-weight: 700; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-akcent); margin-bottom: 0.4rem;">UOPZ</div>

            <div style="font-size: 0.875rem; color: var(--kolor-tekst); line-height: 1.55; white-space: pre-wrap;">{{
zapis.komentarze_uopz }}</div>

        </div>

    {% endif %}

</div>

{% endif %}

{# Panel decyzji #}

{% if zapis.status.value in ['PENDING', 'AWAITING_APPROVAL'] %}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Decyzja</h3>

    </div>

    <div class="karta__tresc">

        <form method="POST">

            {{ form.hidden_tag() }}

            <div style="margin-bottom: 1.25rem;">

                <label class="pole-formularza__etykieta" style="display: block; margin-bottom: 0.4rem;">Komentarz do
studenta <span style="font-weight: 400; color: var(--kolor-tekst-trzeciorzed);">(opcjonalny)</span></label>

                {{ form.komentarz(class="pole-formularza__textarea", rows="4", placeholder="Uwagi widoczne dla studenta w

```

```
panelu praktyk..." ) }}
```

```
</div>
```

```
<div style="border-top: 1px solid var(--kolor-ramka); padding-top: 1.25rem; display: flex; flex-direction: column; gap: 0.5rem;">
```

```
{ { form.zatwierdz(class="przycisk przycisk--glowny", style="width: 100%; justify-content: center;") } }
```

```
{ { form.odrzuc(class="przycisk przycisk--drugorzeczny", style="width: 100%; justify-content: center;") } }
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
{% else %}
```

```
{# Stan po zatwierdzeniu #}
```

```
<div class="karta">
```

```
<div class="karta__tresc" style="text-align: center; padding: 2rem 1.5rem;">
```

```
<div style="width: 48px; height: 48px; border-radius: 50%; background: #d1fae5; display: flex; align-items: center; justify-content: center; margin: 0 auto 1rem; font-size: 1.35rem; color: var(--kolor-sukces);"></div>
```

```
<div style="font-weight: 700; color: var(--kolor-sukces); font-size: 1rem; margin-bottom: 0.4rem;">Zgłoszenie zatwierdzone</div>
```

```
<div style="color: var(--kolor-tekst-drugor); font-size: 0.875rem; line-height: 1.5;">Praktyka jest w trakcie realizacji przez studenta.</div>
```

```
</div>
```

```
</div>
```

```
{% endif %}
```

```
{# Blok przypisania UOPZ #}
```

```
{% if not zapis.uopz and current_user.role.value == 'ADMIN' and zapis.track_type.value == 'STANDARD' %}
```

```
<div style="padding: 1rem; border: 1px dashed var(--kolor-ramka); border-radius: var(--promien); text-align: center;">
```

```
<div style="font-size: 0.8rem; color: var(--kolor-tekst-trzeciorzed); margin-bottom: 0.75rem;">Brak przypisanego opiekuna uczelnianego</div>
```

```
<a href="{ { url_for('zarzadzanie.przypisz_uopz', id=zapis.id) } }" class="przycisk przycisk--maly" style="width: 100%;">Przypisz UOPZ</a>
```

```
</div>
```

```
{% endif %}
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<style>
```

```
.pole-info__etykieta {  
  
  font-size: 0.7rem;  
  
  font-weight: 600;  
  
  text-transform: uppercase;  
  
  letter-spacing: 0.06em;  
  
  color: var(--kolor-tekst-trzeciorzed);  
  
  margin-bottom: 0.3rem;  
  
}
```

```
.pole-info__wartosc {  
  
  font-size: 0.9rem;  
  
  color: var(--kolor-tekst);  
  
  line-height: 1.45;  
  
}
```

```
</style>
```

```
{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\firmy\formularz.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}{% if tryb == 'dodaj' %}Dodaj firmę{% else %}Edytuj firmę{% endif %}{% endblock %}

{% block naglowek %}{% if tryb == 'dodaj' %}Dodawanie nowej firmy{% else %}Edycja firmy{% endif %}{% endblock %}

{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem;">

    <a href="{{ url_for('zarzadzanie.lista_firm') }}" class="przycisk przycisk--drugorzeczny"><- Powrót do listy</a>

</div>

{% endblock %}

{% block tresc %}

<div style="max-width: 800px; margin: 0 auto;">

    <div class="karta">

        <div class="karta__naglowek">

            <h2 class="karta__tytul">

                {% if tryb == 'dodaj' %}

                Dane nowej firmy

                {% else %}

                Edycja danych firmy: {{ firma.nazwa }}

                {% endif %}

            </h2>

        </div>

        <div class="karta__tresc">

            <form method="POST" novalidate>

                {{ form.hidden_tag() }}

                <div style="display: grid; grid-template-columns: 1fr; gap: 1.5rem; margin-bottom: 2rem;">

                    <!-- Nazwa firmy -->

                    <div class="pole-formularza {% if form.nazwa.errors %}pole-formularza--blad{% endif %}">

                        {{ form.nazwa.label(class="pole-formularza__etykieta") }}
```

```

    {{ form.nazwa(class="pole-formularza__input") }}

    {% for e in form.nazwa.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

</div>


<!-- Adres -->

<div class="pole-formularza {% if form.adres.errors %}pole-formularza--blad{% endif %}">

    {{ form.adres.label(class="pole-formularza__etykieta") }}

    {{ form.adres(class="pole-formularza__input") }}

    {% for e in form.adres.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

    <small style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">

        Pełny adres, np. "ul. Kolejowa 15/3"

    </small>

</div>


<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem;">

    <!-- Miasto -->

    <div class="pole-formularza {% if form.miasto.errors %}pole-formularza--blad{% endif %}">

        {{ form.miasto.label(class="pole-formularza__etykieta") }}

        {{ form.miasto(class="pole-formularza__input") }}

        {% for e in form.miasto.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

    </div>


    <!-- NIP/KRS -->

    <div class="pole-formularza {% if form.nip_krs.errors %}pole-formularza--blad{% endif %}">

        {{ form.nip_krs.label(class="pole-formularza__etykieta") }}

        {{ form.nip_krs(class="pole-formularza__input") }}

        {% for e in form.nip_krs.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

        <small style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">

            NIP lub numer KRS

        </small>

    </div>

</div>

</div>

```

```
<div style="margin-bottom: 1.5rem; padding: 1rem; background: #f0f9ff; border-radius: 0.5rem; border-left: 4px solid #0ea5e9;">

  <div style="font-weight: 500; color: #0c4a6e; margin-bottom: 0.5rem; display: flex; align-items: center; gap: 0.5rem;">

    <svg viewBox="0 0 24 24" style="width: 1rem; height: 1rem; fill: currentColor;">

      <circle cx="12" cy="12" r="10"/>

      <line x1="12" y1="16" x2="12" y2="12"/>

      <line x1="12" y1="8" x2="12.01" y2="8"/>

    </svg>

    Informacja o firmach w systemie

  </div>

  <div style="color: #0c4a6e; font-size: 0.9rem; line-height: 1.4;">

    Wszystkie firmy dodawane do systemu mają <strong>stałą umowę z uczelnią</strong>. Studenci którzy wybierają firmy z tej bazy nie muszą generować Załącznika 3 (Porozumienia). Studenci mogą też podać własne firmy podczas zapisów na praktyki.

  </div>

</div>

<div style="display: flex; justify-content: space-between; align-items: center;">

  <a href="{ url_for('zarzadzanie.lista_firm') }}" class="przycisk przycisk--drugorzeczny">

    Anuluj

  </a>

  <button type="submit" class="przycisk przycisk--glowny">

    {% if tryb == 'dodaj' %}

      Dodaj firmę

    {% else %}

      Zapisz zmiany

    {% endif %}

  </button>

</div>

</form>

</div>

</div>
```

</div>

{% endblock %}

PLIK: app_admin\templates\zarzadzanie\firmy\lista.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Zarządzanie firmami{% endblock %}

{% block naglowek %}Firmy w systemie{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem;">

    <a href="{{ url_for('zarzadzanie.dodaj_firme') }}" class="przycisk przycisk--glowny">+ Dodaj firmę</a>

    <a href="{{ url_for('dashboard.index') }}" class="przycisk przycisk--drugorzeczny">← Dashboard</a>

</div>

{% endblock %}


{% block tresc %}

<div style="max-width: 1400px; margin: 0 auto;">

    <!-- Info o systemie -->

    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-info);">

        <div class="karta__tresc">

            <div>

                <div style="font-weight: 600; margin-bottom: 0.5rem;">System zarządzania firmami</div>

                <div style="color: var(--kolor-tekst-drugor); font-size: 0.9rem;">

                    Firma z flagą <strong>"Stała umowa"</strong> nie wymagają generowania Załącznika 3 (Porozumienia).

                    Studenci mogą wybierać firmy z bazy podczas rejestracji na praktykę.

                </div>

            </div>

        </div>

    </div>

</div>


<!-- Lista firm -->

<div class="karta">

    <div class="karta__naglowek">

        <form method="GET" class="pasek-filtrow" role="search" aria-label="Filtruj firmy">

            <div class="filtr-szukaj">
```



```

<span class="filtr-szukaj__ikona" aria-hidden="true">

    <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round">

        <circle cx="11" cy="11" r="8"></circle>

        <path d="m21 21-4.3-4.3"></path>

    </svg>

</span>

<input

    type="search"

    name="szukaj"

    id="szukaj"

    class="filtr-szukaj__input"

    value="{{ request.args.get('szukaj', '') }}"

    placeholder="Szukaj po nazwie, adresie, NIP/KRS..."

    aria-label="Szukaj firmy"

    >

</div>

<label for="status" class="sr-only">Filtruj po statusie</label>

<select name="status" id="status" class="filtr-select" onchange="this.form.submit()">

    <option value="wszystkie" {% if request.args.get('status', 'wszystkie') == 'wszystkie' %}>selected{% endif
%}>Wszystkie</option>

    <option value="aktywne" {% if request.args.get('status') == 'aktywne' %}>selected{% endif %}>Tylko
aktywne</option>

    <option value="nieaktywne" {% if request.args.get('status') == 'nieaktywne' %}>selected{% endif %}>Tylko
nieaktywne</option>

</select>

<button type="submit" class="przycisk przycisk--drugorzecny">Szukaj</button>

{% if request.args.get('szukaj') or request.args.get('status', 'aktywne') != 'aktywne' %}

    <a href="{{ url_for('zarzadzanie.lista_firm') }}" class="przycisk przycisk--drugorzecny">

        Wyczyść filtry

    </a>

{% endif %}

</form>

<span style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">

    {{ firmy.total }} firm

```

```

</span>

</div>

<div class="karta__tresc">

  {% if firmy.items %}

  <div class="tabela-wrapper">

    <table class="tabela">

      <thead>

        <tr>

          <th style="width: 30%;">Nazwa firmy</th>

          <th style="width: 25%;">Adres</th>

          <th style="width: 15%;">NIP/KRS</th>

          <th style="width: 10%; text-align: center;">Status</th>

          <th style="width: 20%; text-align: center;">Akcje</th>

        </tr>

      </thead>

      <tbody>

        {% for firma in firmy.items %}

        <tr>

          <td>

            <div style="font-weight: 600;">{{ firma.nazwa }}</div>

          </td>

          <td>

            <div style="font-size: 0.9rem; color: var(--kolor-tekst-dugor);">

              {% if firma.adres %}

              {{ firma.adres }}

              {% if firma.miasto %}<br>{{ firma.miasto }}{% endif %}

              {% else %}

              Brak adresu

              {% endif %}

            </div>

          </td>

          <td style="font-size: 0.9rem;">

            {{ firma.nip_krs or '-' }}

          </td>

        </tr>

      </tbody>

    </table>

  </div>

  {% endif %}

</div>

```

```

<td style="text-align: center;">

    {% if firma.is_active %}

        <span class="status status--zakonczone">Aktywna</span>

    {% else %}

        <span class="status status--szkie">Nieaktywna</span>

    {% endif %}

</td>

<td style="white-space: nowrap;">

    <a href="{{ url_for('zarzadzanie.edytuj_firme', id=firma.id) }}"

        class="przycisk przycisk--maly przycisk--drugorzeczny">

        Edytuj

    </a>

    <form method="POST" action="{{ url_for('zarzadzanie.przelacz_aktynosc_firmy', id=firma.id) }}"

        style="display: inline-block; margin-left: 0.25rem;">

        {{ csrf_form.hidden_tag() }}

        {% set text = 'dezaktywować' if firma.is_active else 'aktywować' %}

        <button type="submit"

            class="przycisk {% if firma.is_active %}przycisk--ostrzezenie{% else %}przycisk--drugorzeczny{%
endif %} przycisk--maly"

            onclick="return confirm('Czy na pewno chcesz {{ text }} firmę?')">

            {{ 'Dezaktywuj' if firma.is_active else 'Aktywuj' }}

        </button>

    </form>

    <form method="POST" action="{{ url_for('zarzadzanie.usun_firme', id=firma.id) }}"

        style="display: inline-block; margin-left: 0.25rem;">

        {{ csrf_form.hidden_tag() }}

        <button type="submit"

            class="przycisk przycisk--niebezpieczny przycisk--maly"

            onclick="return confirm('Czy na pewno chcesz trwale usunąć firmę {{ firma.nazwa }}? Tej
operacji nie można cofnąć.')">

            Usuń

        </button>

    </form>

</td>

</tr>

```

```

        {% endfor %}

    </tbody>

</table>

</div>

<!-- Paginacja -->

{% if firmy.pages > 1 %}

<div style="display: flex; justify-content: center; margin-top: 2rem; gap: 0.5rem;">

    {% if firmy.has_prev %}

    <a href="{{ url_for('zarzadzanie.lista_firm', page=firmy.prev_num, **request.args) }}"

        class="przycisk przycisk--drugorzeczny przycisk--maly">← Poprzednia</a>

    {% endif %}

    {% for page_num in firmy.iter_pages() %}

        {% if page_num %}

            {% if page_num != firmy.page %}

                <a href="{{ url_for('zarzadzanie.lista_firm', page=page_num, **request.args) }}"

                    class="przycisk przycisk--drugorzeczny przycisk--maly">{{ page_num }}</a>

            {% else %}

                <span class="przycisk przycisk--glowny przycisk--maly">{{ page_num }}</span>

            {% endif %}

        {% else %}

            <span>...</span>

        {% endif %}

    {% endfor %}

    {% if firmy.has_next %}

    <a href="{{ url_for('zarzadzanie.lista_firm', page=firmy.next_num, **request.args) }}"

        class="przycisk przycisk--drugorzeczny przycisk--maly">Następna →</a>

    {% endif %}

</div>

{% endif %}

{% else %}

```

```

<!-- Pusty stan -->

<div style="text-align: center; padding: 4rem 2rem;">

    <div style="font-size: 3rem; margin-bottom: 1rem; opacity: 0.3;">

        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"
stroke-linejoin="round" style="width: 3rem; height: 3rem;">

            <rect x="4" y="2" width="16" height="20" rx="2" ry="2"/>

            <line x1="9" y1="6" x2="9" y2="6.01"/>

            <line x1="15" y1="6" x2="15" y2="6.01"/>

            <line x1="9" y1="10" x2="9" y2="10.01"/>

            <line x1="15" y1="10" x2="15" y2="10.01"/>

            <line x1="9" y1="14" x2="9" y2="14.01"/>

            <line x1="15" y1="14" x2="15" y2="14.01"/>

            <path d="M9 18h6"/>

        </svg>

    </div>

    <div style="font-size: 1.2rem; font-weight: 600; margin-bottom: 1rem; color: var(--kolor-tekst-drugor);">

        Brak firm w systemie

    </div>

    <div style="color: var(--kolor-tekst-drugor); margin-bottom: 2rem;">

        Dodaj pierwszą firmę do systemu praktyk.

    </div>

    <a href="{{ url_for('zarzadzanie.dodaj_firme') }}" class="przycisk przycisk--glowny">

        + Dodaj firmę

    </a>

</div>

{% endif %}

</div>

</div>

</div>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\komisja\lista.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Komisja weryfikująca{% endblock %}

{% block naglowek %}Wnioski do weryfikacji przez komisję{% endblock %}

{% block tresc %}

<div style="max-width: 1400px; margin: 0 auto;">

    {% if wnioski.items %}

    <div class="karta">

        <div class="karta__tresc">

            <div class="tabela-wrapper">

                <table class="tabela">

                    <thead>

                        <tr>

                            <th style="width: 25%;">Student</th>

                            <th style="width: 15%;">Ścieżka</th>

                            <th style="width: 25%;">Firma/Działalność</th>

                            <th style="width: 12%;">Data zgłoszenia</th>

                            <th style="width: 10%;">Dokumenty</th>

                            <th style="width: 13%; text-align: center;">Akcje</th>

                        </tr>

                    </thead>

                    <tbody>

                        {% for wniosek in wnioski.items %}

                        <tr>

                            <td>

                                <div style="font-weight: 600;">{{ wniosek.student.first_name }} {{ wniosek.student.last_name }}</div>

                                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ wniosek.student.email or 'Brak email' }}</div>

                                {% if wniosek.student.album_number %}

                                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Album: {{ wniosek.student.album_number }}</div>

                                {% endif %}

                            </td>

                        </tr>

                        {% endfor %}

                    </tbody>

                </table>

            </div>

        </div>

    </div>

{% endif %}
```

```

</td>

<td>

  <span class="status

    {%- if wniosek.track_type.value == 'EMPLOYMENT' %} status--w-trakcie

    {%- elif wniosek.track_type.value == 'OWN_BUSINESS' %} status--szkic

    {%- endif %}">

    {% if wniosek.track_type.value == 'EMPLOYMENT' %}Praca zawodowa

    {% elif wniosek.track_type.value == 'OWN_BUSINESS' %}Działalność gospodarcza

    {% endif %}

  </span>

</td>

<td>

  <div style="font-weight: 600;">{{ wniosek.firma_nazwa or 'Nie podano' }}</div>

  {% if wniosek.firma_miasto %}

  <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ wniosek.firma_miasto }}</div>

  {% endif %}

  {% if wniosek.track_type.value != 'STANDARD' %}

  <div style="font-size: 0.75rem; color: var(--kolor-info); margin-top: 0.25rem;">

    {% if wniosek.track_type.value == 'EMPLOYMENT' %}Na podstawie zatrudnienia

    {% elif wniosek.track_type.value == 'OWN_BUSINESS' %}Własna działalność

    {% endif %}

  </div>

  {% endif %}

</td>

<td style="font-size: 0.85rem;">

  {{ wniosek.enrolled_at.strftime('%d.%m.%Y') if wniosek.enrolled_at else '-' }}

  <div style="font-size: 0.7rem; color: var(--kolor-tekst-drugor);">

    {{ wniosek.enrolled_at.strftime('%H:%M') if wniosek.enrolled_at else '' }}

  </div>

</td>

<td style="text-align: center;">

  <span class="badge badge--neutral" style="font-size: 0.8rem;">

    {{ wniosek.uploaded_documents | length }} plików

  </span>

```

```

        </td>

        <td style="text-align: center;">

            <a href="{{ url_for('zarzadzanie.komisja_weryfikuj', id=wniosek.id) }}"

                class="przycisk przycisk--maly przycisk--glowny">

                    Weryfikuj

            </a>

        </td>

    </tr>

    {% endfor %}

</tbody>

</table>

</div>

</div>

</div>

<!-- Paginacja -->

{% if wnioski.pages > 1 %}

<div class="paginacja">

    {% if wnioski.has_prev %}

        <a href="{{ url_for('zarzadzanie.komisja_lista', page=wnioski.prev_num) }}" class="paginacja__link">←
Poprzednia</a>

    {% endif %}

    {% for page_num in wnioski.iter_pages() %}

        {% if page_num %}

            {% if page_num != wnioski.page %}

                <a href="{{ url_for('zarzadzanie.komisja_lista', page=page_num) }}" class="paginacja__link">{{ page_num }}</a>

            {% else %}

                <span class="paginacja__link paginacja__link--aktywny">{{ page_num }}</span>

            {% endif %}

        {% else %}

            <span class="paginacja__kropki">...</span>

        {% endif %}

    {% endfor %}

```



```

{% if wnioski.has_next %}

    <a href="{% url_for('zarzadzanie.komisja_lista', page=wnioski.next_num) %}" class="paginacja__link">Następna →</a>

{% endif %}

</div>

{% endif %}

{% else %}

<div class="karta">

    <div class="karta__tresc" style="text-align: center; padding: 3rem 2rem;">

        <svg width="64" height="64" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.5" style="color:
var(--kolor-tekst-drugor); margin-bottom: 1rem;">

            <path d="M9 12l2 4-4"/>

            <path d="M21 12c0 4.97-4.03 9-9 9s-9-4.03-9-9 4.03-9 9-9c.83 0 1.63.11 2.38.32"/>

        </svg>

        <h3 style="margin-bottom: 0.5rem;">Brak wniosków do weryfikacji</h3>

        <p style="color: var(--kolor-tekst-drugor); margin-bottom: 0;">

            Wszystkie wnioski zostały już przetworzone lub nie ma nowych zgłoszeń.

        </p>

    </div>

</div>

{% endif %}

</div>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\komisja\weryfikuj.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Weryfikacja wniosku – {{ zapis.student.first_name }} {{ zapis.student.last_name }}{% endblock %}

{% block naglowek %}Weryfikacja wniosku komisji{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem; align-items: center;">

    <a href="{{ url_for('zarzadzanie.komisja_lista') }}" class="przycisk przycisk--drugorzeczny"><- Powrót do listy</a>

</div>

{% endblock %}


{% block tresc %}

<div style="max-width: 1200px; margin: 0 auto;">

    {# — Karta identyfikacyjna — #}

    {% set status_val = zapis.status.value %}

    {% if status_val == 'COMMISSION_REVIEW' %}

        {% set border_color = '#3b82f6' %}

        {% set status_cls = 'status--planned' %}

        {% set status_label = 'Do rozpatrzenia przez komisję' %}

    {% elif status_val == 'AWAITING_APPROVAL' %}

        {% set border_color = 'var(--kolor-akcent)' %}

        {% set status_cls = 'status--in-progress' %}

        {% set status_label = 'Zwrócone do uzupełnienia' %}

    {% elif status_val == 'DEAN_APPROVAL' %}

        {% set border_color = 'var(--kolor-glowny)' %}

        {% set status_cls = 'status--planned' %}

        {% set status_label = 'Przekazane do dziekana' %}

    {% elif status_val == 'IN_PROGRESS' %}

        {% set border_color = 'var(--kolor-sukces)' %}

        {% set status_cls = 'status--completed' %}

        {% set status_label = 'Zatwierdzona – w realizacji' %}

    {% else %}
```

```

{% set border_color = 'var(--kolor-ramka)' %}

{% set status_cls = 'status--draft' %}

{% set status_label = status_val %}

{% endif %}

<div style="background: var(--kolor-biale); border-radius: var(--promien-duzy); border-top: 4px solid {{ border_color
}}; box-shadow: var(--cien-karta); margin-bottom: 2rem; overflow: hidden;">

    <div style="padding: 1.75rem 2rem; display: grid; grid-template-columns: auto 1fr auto; gap: 2rem; align-items:
center;">

        <div style="width: 56px; height: 56px; border-radius: 50%; background: var(--kolor-glowny); color: #fff; display:
flex; align-items: center; justify-content: center; font-size: 1.25rem; font-weight: 700; flex-shrink: 0; letter-spacing:
0.5px;">

            {{ zapis.student.first_name[0] }}{{ zapis.student.last_name[0] }}

        </div>

        <div style="display: grid; grid-template-columns: repeat(4, 1fr); gap: 1.5rem; min-width: 0;">

            <div>

                <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Student</div>

                <div style="font-weight: 700; font-size: 1rem; color: var(--kolor-tekst);">{{ zapis.student.first_name }} {{
zapis.student.last_name }}</div>

                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); margin-top: 0.15rem;">{{ zapis.student.email
}}</div>

                {% if zapis.student.album_number %}

                    <div style="font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed);">nr alb. {{ zapis.student.album_number
}}</div>

                {% endif %}

            </div>

            <div>

                <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Rok akademicki</div>

                <div style="font-weight: 600; color: var(--kolor-tekst);">{{ zapis.praktyka.rok_uczelnianny }}</div>

                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Semestr {{ zapis.praktyka.semestr }}</div>

            </div>

            <div>

                <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Ścieżka</div>

                <div style="font-weight: 600; color: var(--kolor-tekst);">

```

```

        {% if zapis.track_type.value == 'EMPLOYMENT' %}Na podst. zatrudnienia

        {% elif zapis.track_type.value == 'OWN_BUSINESS' %}Własna działalność

        {% else %}{{ zapis.track_type.value }}{% endif %}

    </div>

</div>

<div>

    <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Data zgłoszenia</div>

    <div style="font-weight: 600; color: var(--kolor-tekst);">{{ zapis.enrolled_at.strftime('%d.%m.%Y') if
zapis.enrolled_at else '-' }}</div>

    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ zapis.enrolled_at.strftime('%H:%M') if
zapis.enrolled_at else '' }}</div>

</div>

</div>

<div style="text-align: right; flex-shrink: 0;">

    <span class="status {{ status_cls }}" style="font-size: 0.75rem; padding: 0.4rem 1rem;">{{ status_label }}</span>

</div>

</div>

</div>

{# — Główna siatka treści — #}

<div style="display: grid; grid-template-columns: 1fr 360px; gap: 1.5rem; align-items: start;">

    {# == LEWA KOLUMNA == #}

    <div style="display: flex; flex-direction: column; gap: 1.5rem;">

        {# Dane pracodawcy / działalności #}

        <div class="karta">

            <div class="karta__naglowek">

                <h3 class="karta__tytul">Dane pracodawcy / działalności</h3>

            </div>

            <div class="karta__tresc">

                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.25rem;">

                    <div class="pole-info">

```

```

        <div class="pole-info__etykieta">Nazwa firmy / działalności</div>

        <div class="pole-info__wartosc" style="font-weight: 600;">{{ zapis.firma_nazwa or '-' }}</div>

    </div>

    <div class="pole-info">

        <div class="pole-info__etykieta">Stanowisko / zakres obowiązków</div>

        <div class="pole-info__wartosc">{{ zapis.zopz_stanowisko or '-' }}</div>

    </div>

    <div class="pole-info" style="grid-column: 1 / -1;">

        <div class="pole-info__etykieta">Adres</div>

        <div class="pole-info__wartosc">{{ zapis.firma_adres or '-' }}{% if zapis.firma_miasto %}, {{
zapis.firma_miasto }}{% endif %}</div>

    </div>

</div>

</div>

</div>

</div>

{# Uzasadnienie studenta #}

{% if zapis.uzasadnienie_sciezki %}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Uzasadnienie wniosku</h3>

    </div>

    <div class="karta__tresc">

        <div style="padding: 1rem; background: var(--kolor-tlo); border-left: 3px solid var(--kolor-glowny);
border-radius: 0 var(--promien) var(--promien) 0;">

            <p style="white-space: pre-line; font-size: 0.9rem; line-height: 1.7; color: var(--kolor-tekst); margin:
0;">{{ zapis.uzasadnienie_sciezki }}</p>

        </div>

    </div>

</div>

</div>

{% endif %}

{# Dokumenty do wygenerowania #}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Dokumenty praktyki</h3>

```

```

        <span style="font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed);">Generuj powiązane dokumenty</span>

    </div>

    <div class="karta__tresc">

        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 0.75rem;">

            {% if zapis.track_type.value == 'EMPLOYMENT' %}

                <div style="padding: 1rem; border: 1px solid var(--kolor-ramka); border-radius: var(--promien); background:
var(--kolor-tlo); display: flex; flex-direction: column; gap: 0.5rem;">

                    <div style="font-weight: 600; font-size: 0.875rem; color: var(--kolor-glowny);">ZAL_7 – Wniosek o
zaliczenie</div>

                    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); flex-grow: 1;">Wniosek o zaliczenie
praktyki na podstawie pracy zawodowej</div>

                    <a href="{% url_for('documents.generuj_auto', id=zapis.id, doc_type='ZAL_7') %}"

                        class="przycisk przycisk--maly przycisk--glowny" target="_blank" style="width: 100%; justify-content:
center;">Generuj PDF</a>

                </div>

                <div style="padding: 1rem; border: 1px solid var(--kolor-ramka); border-radius: var(--promien); background:
var(--kolor-tlo); display: flex; flex-direction: column; gap: 0.5rem;">

                    <div style="font-weight: 600; font-size: 0.875rem; color: var(--kolor-glowny);">ZAL_4a – Efekty
uczenia</div>

                    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); flex-grow: 1;">Zestawienie efektów uczenia
się dla pracy zawodowej</div>

                    <a href="{% url_for('documents.generuj_auto', id=zapis.id, doc_type='ZAL_4a') %}"

                        class="przycisk przycisk--maly przycisk--drugorzeczny" target="_blank" style="width: 100%;
justify-content: center;">Generuj PDF</a>

                </div>

            {% elif zapis.track_type.value == 'OWN_BUSINESS' %}

                <div style="padding: 1rem; border: 1px solid var(--kolor-ramka); border-radius: var(--promien); background:
var(--kolor-tlo); display: flex; flex-direction: column; gap: 0.5rem;">

                    <div style="font-weight: 600; font-size: 0.875rem; color: var(--kolor-glowny);">ZAL_4b – Wniosek
działalność</div>

                    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); flex-grow: 1;">Wniosek o zaliczenie na
podstawie działalności gospodarczej</div>

                    <a href="{% url_for('documents.generuj_auto', id=zapis.id, doc_type='ZAL_4b') %}"

                        class="przycisk przycisk--maly przycisk--glowny" target="_blank" style="width: 100%; justify-content:
center;">Generuj PDF</a>

                </div>

                <div style="padding: 1rem; border: 1px solid var(--kolor-ramka); border-radius: var(--promien); background:
var(--kolor-tlo); display: flex; flex-direction: column; gap: 0.5rem;">

                    <div style="font-weight: 600; font-size: 0.875rem; color: var(--kolor-glowny);">ZAL_8 – Protokół
egzaminacyjny</div>

                    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); flex-grow: 1;">Protokół z egzaminu
praktycznego</div>

```

```

        <a href="{ url_for('documents.generuj_auto', id=zapis.id, doc_type='ZAL_8') }}"
            class="przycisk przycisk--maly przycisk--drugorzeczny" target="_blank" style="width: 100%;
justify-content: center;">Generuj PDF</a>

    </div>

    {% endif %}

</div>

</div>

</div>

{# Załączniki studenta #}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Załączone dokumenty</h3>

        <span style="font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed);">{{ dokumenty|length }} plik{% if
dokumenty|length != 1 %}i{% endif %}</span>

    </div>

    <div class="karta__tresc">

        {% if dokumenty %}

            <div style="display: flex; flex-direction: column; gap: 0.5rem;">

                {% for dok in dokumenty %}

                    <div style="display: flex; justify-content: space-between; align-items: center; padding: 0.75rem 1rem;
border: 1px solid var(--kolor-ramka); border-radius: var(--promien); background: var(--kolor-tlo);">

                        <div style="display: flex; align-items: center; gap: 0.75rem; min-width: 0;">

                            <div style="color: var(--kolor-glowny); flex-shrink: 0;">

                                <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">

                                    <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/>

                                    <polyline points="14,2 14,8 20,8"/>

                                </svg>

                            </div>

                            <div style="min-width: 0;">

                                <div style="font-weight: 600; font-size: 0.875rem; white-space: nowrap; overflow: hidden;
text-overflow: ellipsis;">{{ dok.original_filename }}</div>

                                <div style="font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed); margin-top: 0.1rem;">

                                    {{ dok.document_type }} · {{ (dok.file_size / 1024)|round(1) }} KB · {{
dok.uploaded_at.strftime('%d.%m.%Y %H:%M') }}

                                </div>

                            </div>

                        </div>

                    </div>

                {% endfor %}

            </div>

        {% endif %}

    </div>

</div>

```

```

        </div>

        <a href="{ url_for('uploads.download_document', document_id=dok.id) }}"

                class="przycisk przycisk--maly przycisk--drugorzeczny" style="flex-shrink: 0; margin-left:
1rem;">Pobierz</a>

    </div>

    {% endfor %}

</div>

{% else %}

<div style="text-align: center; padding: 2rem; color: var(--kolor-tekst-trzeciorzed); font-size: 0.9rem;">

    Brak załączonych dokumentów

</div>

{% endif %}

</div>

</div>

</div>

{# == PRAWA KOLUMNA == #}

<div style="display: flex; flex-direction: column; gap: 1.5rem; position: sticky; top: 1.5rem;">

    {# Historia decyzji #}

    {% if zapis.decyzja_komisji or zapis.decyzja_dziekana %}

    <div class="karta">

        <div class="karta__naglowek">

            <h3 class="karta__tytul">Historia decyzji</h3>

        </div>

        <div class="karta__tresc" style="display: flex; flex-direction: column; gap: 0.75rem;">

            {% if zapis.decyzja_komisji %}

                <div style="padding: 0.875rem; border-left: 3px solid var(--kolor-glowny); background: var(--kolor-tlo);
border-radius: 0 var(--promien) var(--promien) 0;">

                    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.4rem;">

                        <span style="font-size: 0.7rem; font-weight: 700; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-glowny);">Komisja</span>

                        {% if zapis.decyzja_komisji_o %}

                            <span style="font-size: 0.72rem; color: var(--kolor-tekst-trzeciorzed);">{{
zapis.decyzja_komisji_o.strftime('%d.%m.%Y')}}</span>

```



```

        {% endif %}

    </div>

        <span class="status {% if zapis. decyzja_komisji == 'APPROVED' %}status--completed{% elif
zapis. decyzja_komisji == 'PARTIALLY_APPROVED' %}status--in-progress{% else %}status--draft{% endif %}"
style="margin-bottom: 0.5rem; display: inline-block;">

            {% if zapis. decyzja_komisji == 'APPROVED' %}Zatwierdzone

            {% elif zapis. decyzja_komisji == 'PARTIALLY_APPROVED' %}Częściowo zatwierdzone

            {% else %}Odrzucone{% endif %}

        </span>

        {% if zapis.komentarze_komisji %}

            <div style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); line-height: 1.5; margin-top: 0.4rem;">{{
zapis.komentarze_komisji }}</div>

            {% endif %}

        </div>

        {% endif %}

        {% if zapis. decyzja_dziekana %}

            <div style="padding: 0.875rem; border-left: 3px solid var(--kolor-akcent); background: var(--kolor-tlo);
border-radius: 0 var(--promien) var(--promien) 0;">

                <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.4rem;">

                    <span style="font-size: 0.7rem; font-weight: 700; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-akcent);">Dyrektor IIS</span>

                    {% if zapis. decyzja_dziekana_o %}

                        <span style="font-size: 0.72rem; color: var(--kolor-tekst-trzeciorzed);">{{
zapis. decyzja_dziekana_o.strftime('%d.%m.%Y') }}</span>

                        {% endif %}

                    </div>

                    <span class="status {% if zapis. decyzja_dziekana == 'APPROVED' %}status--completed{% else %}status--draft{%
endif %}" style="margin-bottom: 0.5rem; display: inline-block;">

                        {{ 'Zatwierdzone' if zapis. decyzja_dziekana == 'APPROVED' else 'Odrzucone' }}

                    </span>

                    {% if zapis.komentarze_dziekana %}

                        <div style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); line-height: 1.5; margin-top: 0.4rem;">{{
zapis.komentarze_dziekana }}</div>

                        {% endif %}

                    </div>

                    {% endif %}

                </div>

```

```

</div>

{% endif %}

{# Komentarz zwrotu (jeśli komisja wcześniej zwróciła) #}

{% if zapis.komentarze_uopz %}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Poprzedni komentarz komisji</h3>

    </div>

    <div class="karta__tresc">

        <div style="padding: 0.875rem; border-left: 3px solid var(--kolor-akcent); background: var(--kolor-tlo);
border-radius: 0 var(--promien) var(--promien) 0;">

            <div style="font-size: 0.875rem; color: var(--kolor-tekst); line-height: 1.55; white-space: pre-wrap;">{{
zapis.komentarze_uopz }}</div>

        </div>

    </div>

</div>

{% endif %}

{# Formularz decyzji komisji #}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Decyzja komisji</h3>

    </div>

    <div class="karta__tresc">

        <form method="POST">

            {{ form.hidden_tag() }}

            <div style="margin-bottom: 1.25rem;">

                {{ form.decyzja.label(class="pole-formularza__etykieta", style="display: block; margin-bottom: 0.4rem;") }}

                {{ form.decyzja(class="pole-formularza__select") }}

                {% for error in form.decyzja.errors %}

                    <span class="pole-formularza__blad">{{ error }}</span>

                {% endfor %}

            </div>

        </form>

    </div>

</div>

```

```

<div style="margin-bottom: 1.25rem;">

    <label class="pole-formularza__etykieta" style="display: block; margin-bottom: 0.4rem;">Komentarz <span
style="font-weight: 400; color: var(--kolor-tekst-trzeciorzed);">(opcjonalny)</span></label>

    {{ form.komentarz(class="pole-formularza__textarea", rows="5", placeholder="Uzasadnienie decyzji widoczne
dla studenta i dziekana...") }}

    {% for error in form.komentarz.errors %}

        <span class="pole-formularza__blad">{{ error }}</span>

    {% endfor %}

</div>

```

{{ form.submit(class="przycisk przycisk--glowny", style="width: 100%; justify-content: center;") }}

[Anuluj]({{ url_for('zarzadzanie.komisja_lista') }})

</form>

</div>

</div>

```
}

.pole-info__wartosc {

    font-size: 0.9rem;

    color: var(--kolor-tekst);

    line-height: 1.45;

}

</style>

{% endblock %}
```

PLIK: app_student\config.py

```
import os

from datetime import timedelta


class Config:

    # Osobne nazwy ciasteczek dla admina i studenta

    SESSION_COOKIE_NAME = 'student_session'


    # Podstawowe zabezpieczenie sesji i formularzy WTF

    SECRET_KEY = os.environ.get('SECRET_KEY', 'student-tajny-klucz-tylko-na-dev-xd')


    # Konfiguracja połączenia z bazą danych – sterownik psycopg2 (C, wydajny)

    SQLALCHEMY_DATABASE_URI = os.environ.get(

        'DATABASE_URL',

        'postgresql+psycopg2://ans_admin:secure_password_123@localhost:5432/ans_praktyki'

    )

    SQLALCHEMY_TRACK_MODIFICATIONS = False


    # Session

    PERMANENT_SESSION_LIFETIME = timedelta(hours=8)


    # External services

    TEX_SERVICE_URL = os.environ.get('TEX_SERVICE_URL', 'http://tex-service:5002')


    # Security

    WTF_CSRF_ENABLED = True

    WTF_CSRF_TIME_LIMIT = None


class DevelopmentConfig(Config):

    DEBUG = True

    ENV = 'development'
```

```
class ProductionConfig(Config):

    DEBUG = False

    ENV = 'production'

    SESSION_COOKIE_SECURE = True


config_dict = {

    'development': DevelopmentConfig,

    'production': ProductionConfig,

    'default': DevelopmentConfig

}
```

PLIK: app_student\Dockerfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \

    PYTHONUNBUFFERED=1 \

    PYTHONPATH=/app

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \

    curl \

    && rm -rf /var/lib/apt/lists/*

COPY app_student/requirements.txt requirements.txt

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Bezpieczeństwo: dedykowany nieuprzywilejowany użytkownik (nie root)

RUN addgroup --system appgroup \

    && adduser --system --ingroup appgroup --no-create-home appuser \

    && chown -R appuser:appgroup /app

USER appuser

EXPOSE 5001

CMD ["gunicorn", "--bind", "0.0.0.0:5001", "--workers", "2", "app_student:create_app()"]
```

PLIK: app_student__init__.py

```
from flask import Flask

from jinja2 import select_autoescape

from core.extensions import db, login_manager

from core.error_handlers import register_error_handlers

from app_student.config import config_dict


def create_app():

    from pathlib import Path as _Path

    from flask import Blueprint as _Blueprint

    app = Flask(__name__)

    # — Core static assets (CSS design system + templates) —
    core_bp = _Blueprint(
        'core_static', __name__,
        static_folder=str(_Path(__file__).parent.parent / 'core' / 'static'),
        static_url_path='/core/static',
        template_folder=str(_Path(__file__).parent.parent / 'core' / 'templates'),
    )
    app.register_blueprint(core_bp)

    app.jinja_options = app.jinja_options.copy()
    app.jinja_options.update(dict(
        autoescape=select_autoescape(['html', 'xml'])
    ))

    env = __import__('os').environ.get('FLASK_ENV', 'development')
    app.config.from_object(config_dict.get(env, config_dict['default']))

    db.init_app(app)

    login_manager.init_app(app)

    register_error_handlers(app)

    login_manager.login_view = 'auth.logowanie'

    login_manager.login_message = 'Zaloguj się, aby uzyskać dostęp.'
```



```

with app.app_context():

    from core.autoryzacja import stworz_blueprint_auth

    from core.pliki import stworz_blueprint_pliki

    from core.modele import RolaUzytkownika

    from app_student.routes.pulpit      import dashboard_bp

    from app_student.routes.praktyki    import praktyki_bp

    from app_student.routes.dziennik    import dziennik_bp

    from app_student.routes.sprawozdania import sprawozdania_bp

    from app_student.routes.dokumenty   import documents_bp


    auth_bp = stworz_blueprint_auth(

        dozwolone_role=[RolaUzytkownika.STUDENT],

        template_logowania='auth/logowanie.html',

        template_zmiany_hasla='auth/zmien_haslo.html',

    )

    uploads_bp = stworz_blueprint_pliki()


    app.register_blueprint(auth_bp)

    app.register_blueprint(dashboard_bp, url_prefix='/panel')

    app.register_blueprint(praktyki_bp, url_prefix='/praktyki')

    app.register_blueprint(dziennik_bp, url_prefix='/dziennik')

    app.register_blueprint(sprawozdania_bp, url_prefix='/sprawozdanie')

    app.register_blueprint(documents_bp, url_prefix='/dokumenty')

    app.register_blueprint(uploads_bp, url_prefix='/uploads')


# Kontekst globalny: informacje o aktywnym zapisie studenta

@app.context_processor

def inject_aktywny_zapis():

    from flask_login import current_user

    from core.modele import ZapisPraktyki, StatusZapisu

    info = {'ma_aktywny': False, 'sciezka': None, 'status': None}

    try:

        if current_user.is_authenticated:

```

```

z = db.session.query(ZapisPraktyki).filter(

    ZapisPraktyki.student_id == current_user.id,

    ZapisPraktyki.status.in_(

        StatusZapisu.IN_PROGRESS, StatusZapisu.COMPLETED,

        StatusZapisu.COMMISSION_REVIEW, StatusZapisu.DEAN_APPROVAL,

        StatusZapisu.AWAITING_APPROVAL,

    ])

).first()

if z:

    info = {

        'ma_aktywny': True,

        'sciezka': z.track_type.value if z.track_type else 'STANDARD',

        'status': z.status.value,

    }

except Exception:

    pass

wymaga_uwagi = False

try:

    if current_user.is_authenticated:

        from sqlalchemy import exists

        from core.modele import ZapisPraktyki, StatusZapisu, ZdarzenieProces, TypZdarzenia

        ma_komentarz = exists().where(

            (ZdarzenieProces.zapis_id == ZapisPraktyki.id) &

            (ZdarzenieProces.typ == TypZdarzenia.UOPZ_KOMENTARZ) &

            ZdarzenieProces.komentarz.isnot(None),

        )

        z = db.session.query(ZapisPraktyki).filter(

            ZapisPraktyki.student_id == current_user.id,

            ZapisPraktyki.status == StatusZapisu.AWAITING_APPROVAL,

            ma_komentarz,

        ).first()

        if z:

            wymaga_uwagi = True

except Exception:

```

```

pass

return {'aktywny_zapis_info': info, 'nav_wymaga_uwagi': wymaga_uwagi}

# Dodaj funkcje pomocnicze do szablonów

@app.template_global()

def tlumacz_status(status_value):

    translations = {

        'PENDING': 'Szkie',

        'AWAITING_APPROVAL': 'Oczekuje na zatwierdzenie',

        'COMMISSION_REVIEW': 'Weryfikacja komisji',

        'DEAN_APPROVAL': 'Oczekuje na dziekana',

        'IN_PROGRESS': 'W trakcie',

        'COMPLETED': 'Zakończona',

        'REJECTED': 'Odrzucony'

    }

    return translations.get(status_value, status_value)

@app.before_request

def sprawdz_studenta():

    from flask import request, redirect, url_for, abort

    from flask_login import current_user

    from core.modele import RolaUzytkownika

    if not current_user.is_authenticated:

        return

    if getattr(current_user.role, 'value', current_user.role) != 'STUDENT':

        abort(403)

    if (getattr(current_user, 'wymagana_zmiana_hasla', False)

        and request.endpoint not in ('auth.zmien_haslo', 'auth.wylogowanie', 'static')

        and not (request.endpoint and request.endpoint.endswith('.static'))):

        return redirect(url_for('auth.zmien_haslo'))

```

```
return app
```

```
@login_manager.user_loader
```

```
def wczytaj_uzytkownika(user_id):
```

```
    from core.modele import Uzytkownik
```

```
    return db.session.get(Uzytkownik, user_id)
```

PLIK: app_student\routes\dokumenty.py

```
"""

app_student/routes/dokumenty.py

"""


import io

import logging

import os

from datetime import date, datetime


import httpx

from flask import Blueprint, abort, send_file, jsonify, request, current_app, flash, redirect, url_for, make_response,
render_template

from flask_login import login_required, current_user

from core.extensions import db

from core.modele import ZapisPraktyki, StatusZapisu, HarmonogramPraktyki


logger = logging.getLogger(__name__)

documents_bp = Blueprint('documents', __name__)


def _dok(nazwa, opis=None, **kw):

    """Pomocnik budujący wpis dokumentu."""

    d = {'nazwa': nazwa, 'opis': opis, 'dostepny': True}

    d.update(kw)

    return d


def _dok_dyn(nazwa, zapis_id, typ, opis=None, dostepny=True, powod=None):

    return {'nazwa': nazwa, 'opis': opis, 'zapis_id': str(zapis_id),

            'typ': typ, 'dynamiczny': True, 'dostepny': dostepny, 'powod': powod}


def _dok_staly(nazwa, klucz, opis=None):

    return {'nazwa': nazwa, 'opis': opis, 'klucz_staly': klucz, 'staly': True, 'dostepny': True}
```

```

@documents_bp.route('/moje')

@login_required

def moje_dokumenty():

    zapisy = db.session.query(ZapisPraktyki)\

        .filter_by(student_id=current_user.id)\

        .filter(ZapisPraktyki.status.in_(

            StatusZapisu.IN_PROGRESS,

            StatusZapisu.COMPLETED,

            StatusZapisu.COMMISSION_REVIEW,

            StatusZapisu.DEAN_APPROVAL,

            StatusZapisu.AWAITING_APPROVAL,

        )))\

        .order_by(ZapisPraktyki.enrolled_at.desc())\

        .all()

    dokumenty_list = []

    for zapis in zapisy:

        sciezka = zapis.track_type.value if zapis.track_type else 'STANDARD'

        w_trakcie = zapis.status in [StatusZapisu.IN_PROGRESS, StatusZapisu.COMMISSION_REVIEW,
StatusZapisu.DEAN_APPROVAL]

        zakonczona = zapis.status == StatusZapisu.COMPLETED

        # Ocena wystawiona przez UOPZ (po egzaminie komisji)

        oceniona = zakonczona and zapis.ocena_uopz is not None

        # Dziekan zatwierdził (dla ścieżki B/C)

        dziekan_zatwierdził = zapis.decyzja_dziekana == 'APPROVED'

        harmonogram_count = db.session.query(HarmonogramPraktyki)\

            .filter_by(zapis_id=zapis.id).count()

        firma_bez_umowy = not zapis.firma or not zapis.firma.has_standing_agreement

        firma_custom = not zapis.firma_id # własna firma, nie z bazy

    docs = []

    if sciezka == 'STANDARD':

        # Faza 1-2: Przed praktyką / start

        if firma_custom:

```

```

docs.append(_dok_dyn('Załącz. 9 - Oświadczenie instytucji',
    zapis.id, 'ZAL_9', 'Do wypełnienia przez zakład pracy'))

if firma_bez_umowy:

    docs.append(_dok_dyn('Załącz. 1 - Porozumienie uczelnia ↔ zakład',
        zapis.id, 'ZAL_1', 'Dla firm bez stałej umowy z ANS'))

docs.append(_dok_dyn('Załącz. 2 - Program praktyki',
    zapis.id, 'ZAL_2', 'Z danymi studenta i firmy'))

docs.append(_dok_dyn('Załącz. 2a - Indywidualny Program Praktyk',
    zapis.id, 'ZAL_2A',
    'Harmonogram efektów – student + UOPZ + ZOPZ',
    dostepny=harmonogram_count > 0,
    powod=None if harmonogram_count > 0 else 'Wymaga wypełnionego harmonogramu (krok 2)'))

docs.append(_dok_dyn('Załącz. 3 - Karta praktyki / Skierowanie',
    zapis.id, 'ZAL_3', 'Z danymi studenta, firmy i ZOPZ'))

# Faza 5: W trakcie

docs.append(_dok_dyn('Załącz. 6 - Dziennik praktyki',
    zapis.id, 'ZAL_6', 'Generowany z wpisów dziennika',
    dostepny=w_trakcie or zakonczona,
    powod=None if (w_trakcie or zakonczona) else 'Dostępny po zatwierdzeniu praktyki'))

# — Faza 5: Dziennik (w trakcie) —————

# Faza 6: Pakiet końcowy (separator)

docs.append({'separator': True, 'nazwa': 'Pakiet końcowy – złożenie do 7 dni po zakończeniu'})

docs.append(_dok_dyn('Załącz. 7 - Sprawozdanie końcowe',
    zapis.id, 'ZAL_7', 'Podpisuje student',
    dostepny=zakonczona,
    powod=None if zakonczona else 'Dostępny po zakończeniu praktyki'))

docs.append(_dok_dyn('Załącz. 4 - Potwierdzenie efektów uczenia się',
    zapis.id, 'ZAL_4', 'Podpisuje ZOPZ + UOPZ',
    dostepny=zakonczona,
    powod=None if zakonczona else 'Dostępny po zakończeniu praktyki'))

# Faza 7: Po ocenie komisji (załącz.8 i załącz.5)

docs.append({'separator': True, 'nazwa': 'Po egzaminie komisji'})

```

```
docs.append(_dok_dyn('Załącz. 8 - Protokół egzaminu komisji',
    zapis.id, 'ZAL_8', 'Generowany po wystawieniu oceny',
    dostepny=oceniona,
    powod=None if oceniona else 'Dostępny po wystawieniu oceny przez UOPZ'))
docs.append(_dok_staly('Załącz. 5 - Ankieta oceny praktyki',
    'ankieta', 'Formularz anonimowej ankiety'))
```

```
elif sciezka in ['EMPLOYMENT', 'OWN_BUSINESS']:
```

```
# Faza 1: Wniosek (dostępny od razu)
```

```
docs.append(_dok_dyn('Załącz. 4b - Wniosek o zaliczenie',
    zapis.id, 'ZAL_4B', 'Praca etatowa / własna działalność'))
```

```
# Faza 3: Sprawozdanie (po decyzji komisji)
```

```
docs.append(_dok_dyn('Załącz. 7a - Sprawozdanie z pracy/działalności',
    zapis.id, 'ZAL_7A', 'Zatwierdza przełożony/UOPZ',
    dostepny=w_trakcie or zakonczona,
    powod=None if (w_trakcie or zakonczona) else 'Dostępny po decyzji komisji'))
```

```
# Faza 4-5: Po decyzji dziekana
```

```
docs.append(_dok_dyn('Załącz. 4a - Potwierdzenie efektów (komisja)',
    zapis.id, 'ZAL_4A', '13 efektów: uzyskał / częściowo / nie',
    dostepny=dziekan_zatwierdził or zakonczona,
    powod=None if (dziekan_zatwierdził or zakonczona) else 'Dostępny po decyzji dziekana'))
```

```
docs.append({'separator': True, 'nazwa': 'Po egzaminie komisji'})
```

```
docs.append(_dok_staly('Załącz. 5 - Ankieta', 'ankieta'))
```

```
docs.append(_dok_dyn('Załącz. 8 - Protokół egzaminu komisji',
    zapis.id, 'ZAL_8', 'Wpis do USOS',
    dostepny=dziekan_zatwierdził or zakonczona,
    powod=None if (dziekan_zatwierdził or zakonczona) else 'Dostępny po decyzji dziekana'))
```

```
dokumenty_list.append({'zapis': zapis, 'sciezka': sciezka, 'docs': docs})
```

```
return render_template('dokumenty/moje_dokumenty.html', dokumenty_list=dokumenty_list)
```



```

STALE_SZABLONY = {

    'ankieta': ('zal5_ankieta.tex.j2', 'zal5_ankieta.pdf'),

}

@documents_bp.route('/staly/<doc_key>')

@login_required

def pobierz_staly(doc_key):

    if doc_key not in STALE_SZABLONY:

        abort(404)

    template_name, filename = STALE_SZABLONY[doc_key]

    try:

        response = httpx.post(

            f"{get_tex_service_url()}/generuj",

            json={'template': template_name, 'context': {}, 'filename': filename},

            timeout=30

        )

        if response.status_code == 200:

            pdf_response = make_response(response.content)

            pdf_response.headers['Content-Type'] = 'application/pdf'

            pdf_response.headers['Content-Disposition'] = f'attachment; filename="{filename}"'

            return pdf_response

        else:

            flash('Błąd generowania dokumentu.', 'error')

    except Exception as e:

        flash(f'Błąd połączenia z serwisem PDF: {str(e)}', 'error')

    return redirect(url_for('documents.moje_dokumenty'))


@documents_bp.route('/dynamiczny/<uuid:zapis_id>/<typ>')

@login_required

def pobierz_dynamiczny(zapis_id, typ):

    """Generuje dynamiczny dokument dla danego zapisu."""

```

```

if typ not in DOC_CONFIG:

    abort(404)

zapis = db.session.get(ZapisPraktyki, zapis_id)

if not zapis or zapis.student_id != current_user.id:

    abort(404)

template_name, filename = DOC_CONFIG[typ]

# Buduj kontekst na podstawie zapisu

ctx = _build_context(zapis, typ)

try:

    response = httpx.post(

        f"{get_tex_service_url()}/generuj",

        json={'template': template_name, 'context': ctx, 'filename': filename},

        timeout=60

    )

    if response.status_code == 200:

        import unicodedata

        safe = unicodedata.normalize('NFKD', zapis.student.last_name).encode('ascii', 'ignore').decode('ascii') or
'student'

        pdf_name = template_name.replace('.tex.j2', '')

        pdf_response = make_response(response.content)

        pdf_response.headers['Content-Type'] = 'application/pdf'

        pdf_response.headers['Content-Disposition'] = f'attachment; filename="{pdf_name}_{safe}.pdf"'

        return pdf_response

    else:

        flash(f'Błąd generowania dokumentu: {response.text[:200]}', 'error')

except Exception as e:

    flash(f'Błąd połączenia z serwisem PDF: {str(e)}', 'error')

return redirect(url_for('documents.moje_dokumenty'))

def _build_context(zapis, typ):

```

```
"""Bduje kontekst dla szablonu LaTeX na podstawie zapisu."""
```

```
student = zapis.student
```

```
uopz = zapis.uopz
```

```
firma_nazwa = (zapis.firma.nazwa if zapis.firma else None) or g(zapis, 'firma_nazwa')
```

```
firma_adres = (zapis.firma.adres if zapis.firma else None) or g(zapis, 'firma_adres')
```

```
firma_miasto = (zapis.firma.miasto if zapis.firma else None) or g(zapis, 'firma_miasto')
```

```
firma_nip = (zapis.firma.nip_krs if zapis.firma else None) or g(zapis, 'firma_nip_krs')
```

```
ctx = {
```

```
    'student': {
```

```
        'first_name': g(student, 'first_name'),
```

```
        'last_name': g(student, 'last_name'),
```

```
        'album_number': g(student, 'album_number'),
```

```
        # aliasy zgodne z zalacznik_*.tex.j2
```

```
        'imie': g(student, 'first_name'),
```

```
        'nazwisko': g(student, 'last_name'),
```

```
        'nr_albumu': g(student, 'album_number'),
```

```
        'plec': g(student, 'plec', ''),
```

```
        'kierunek': g(student, 'kierunek', 'Informatyka'),
```

```
        'specjalnosc': g(student, 'specjalnosc', ''),
```

```
        'tryb_studiow': g(student, 'tryb_studiow', ''),
```

```
    },
```

```
    'praktyka': {
```

```
        'rok_uczelnianny': g(zapis.praktyka, 'rok_uczelnianny') if zapis.praktyka else '',
```

```
        'semestr': g(zapis.praktyka, 'semestr') if zapis.praktyka else '',
```

```
        'wymiar_godzin': g(zapis.praktyka, 'wymiar_godzin', 120) if zapis.praktyka else 120,
```

```
    },
```

```
    'firma': {
```

```
        'nazwa': firma_nazwa,
```

```
        'adres': firma_adres,
```

```
        'miasto': firma_miasto,
```

```
        'nip_krs': firma_nip,
```

```
    },
```

```
    'firma_upowazniony': g(zapis, 'firma_upowazniony_osoba', ''),
```

```

'firma_upowazniony_stanowisko': g(zapis, 'firma_upowazniony_stanowisko', ''),

'terminy': {

    'od': zapis.termin_od.strftime('%d.%m.%Y') if zapis.termin_od else '',

    'do': zapis.termin_do.strftime('%d.%m.%Y') if zapis.termin_do else '',

},

'zopz': {

    'imie_nazwisko': g(zapis, 'zopz_imie_nazwisko', ''),

    'stanowisko': g(zapis, 'zopz_stanowisko', ''),

    'telefon': g(zapis, 'zopz_telefon', ''),

    'email': g(zapis, 'zopz_email', ''),

},

'uopz': {

    'imie_nazwisko': f"{uopz.first_name} {uopz.last_name}" if uopz else '',

},

'specjalnosc': g(zapis, 'specjalnosc', ''),

'uzasadnienie': g(zapis, 'uzasadnienie_sciezki', ''),

'data_wniosku': '',

# Aliasy dla szablonów używających zapis.* (stare)

'zapis': {

    'specjalnosc': g(zapis, 'specjalnosc', ''),

    'firma_nazwa': firma_nazwa,

    'firma_adres': firma_adres,

    'firma_miasto': firma_miasto,

},

}

# Dane harmonogramu dla ZAL_6

if typ in ('ZAL_2A',):

    from core.modele import HarmonogramPraktyki, EfektUczenia

    harmonogramy = db.session.query(HarmonogramPraktyki)\

        .filter_by(zapis_id=zapis.id)\

        .order_by(HarmonogramPraktyki.learning_outcome_id)\

        .all()

    ctx['harmonogram'] = [{

        'efekt_kod': h.efekt.kod if h.efekt else str(h.learning_outcome_id),

```

```

        'efekt_opis': h.efekt.opis if h.efekt else '',
        'dzial': g(h, 'nazwa_dzialu', ''),
        'prace': g(h, 'przykladowe_prace', ''),
        'dni': g(h, 'liczba_dni', 0),
    } for h in harmonogramy]

if typ in ('ZAL_6',):
    from core.modele import WpisDziennika

    wpisy = db.session.query(WpisDziennika)\
        .filter_by(zapis_id=zapis.id)\
        .order_by(WpisDziennika.data_wpisu)\
        .all()

    ctx['wpisy'] = [{
        'data': _d(w.entry_date),
        'opis': g(w, 'description'),
        'godziny': g(w, 'duration_hours', 0),
        'efekt_nr': ', '.join(f"{e.id:02d}" for e in w.efekty_uczenia) if w.efekty_uczenia else '--',
    } for w in wpisy]

return ctx

```

```

DOC_CONFIG = {

    'ZAL_1': ('zal1_porozumienie.tex.j2', 'zal1_porozumienie.pdf'),
    'ZAL_2': ('zal2_program.tex.j2', 'zal2_program.pdf'),
    'ZAL_2A': ('zal2a_program.tex.j2', 'zal2a_program.pdf'),
    'ZAL_3': ('zal3_karta.tex.j2', 'zal3_karta.pdf'),
    'ZAL_4': ('zal4_efekty.tex.j2', 'zal4_efekty.pdf'),
    'ZAL_4A': ('zal4a_komisja.tex.j2', 'zal4a_komisja.pdf'),
    'ZAL_4B': ('zal4b_wniosek.tex.j2', 'zal4b_wniosek.pdf'),
    'ZAL_6': ('zal6_dziennik.tex.j2', 'zal6_dziennik.pdf'),
    'ZAL_7': ('zal7_sprawozdanie.tex.j2', 'zal7_sprawozdanie.pdf'),
    'ZAL_7A': ('zal7a_sprawozdanie.tex.j2', 'zal7a_sprawozdanie.pdf'),
    'ZAL_8': ('zal8_protokol.tex.j2', 'zal8_protokol.pdf'),
    'ZAL_9': ('zal9_oswiadczenie.tex.j2', 'zal9_oswiadczenie.pdf'),

}

```

```

def get_tex_service_url():

    """Pobiera URL tex service z konfiguracji aplikacji."""

    return current_app.config.get('TEX_SERVICE_URL', 'http://tex-service:5002')


# -----

# BEZPIECZNE GETTERY (Pancerz zapobiegający AttributeError)

# -----


def g(obj, attr, default=''):

    """Zwraca wartość atrybutu lub default, jeśli atrybut nie istnieje w modelu."""

    try:

        val = getattr(obj, attr, default)

        return val if val is not None else default

    except AttributeError:

        return default


def _d(value) -> str:

    if value is None: return ''

    if isinstance(value, (date, datetime)): return value.isoformat()

    return str(value)


def _rok_akademicki(termin_od) -> str:

    if not isinstance(termin_od, (date, datetime)): return '-'

    y = termin_od.year

    return f"{y-1}/{y}" if termin_od.month <= 7 else f"{y}/{y+1}"


def _sciezka_label(path_type) -> str:

    mapping = {

        'STANDARD': 'standardowa',

        'EMPLOYMENT': 'zatrudnienie',

        'OWN_BUSINESS': 'własna działalność',

        'ERASMUS_PLUS': 'Erasmus+',

```

```

    }

    val = getattr(path_type, 'value', str(path_type))

    return mapping.get(val, val)

# -----

# SERIALIZACJA (Dopasowana do Twojego app_admin/models.py)

# -----

def _serialize_context(doc_type: str, zapis: ZapisPraktyki) -> dict:

    p = getattr(zapis, 'praktyka', None)

    s = getattr(zapis, 'student', None)

    firma_nazwa = (zapis.firma.nazwa if zapis.firma else None) or g(zapis, 'firma_nazwa', '')

    firma_adres = (zapis.firma.adres if zapis.firma else None) or g(zapis, 'firma_adres', '')

    firma_miasto = (zapis.firma.miasto if zapis.firma else None) or g(zapis, 'firma_miasto', '')

    uopz = zapis.uopz

    context = {

        'student': {

            'first_name': g(s, 'first_name'),

            'last_name': g(s, 'last_name'),

            'album_number': g(s, 'album_number'),

            'imie': g(s, 'first_name'),

            'nazwisko': g(s, 'last_name'),

            'nr_albumu': g(s, 'album_number'),

            'numer_albumu': g(s, 'album_number'),

            'plec': g(s, 'plec', ''),

            'kierunek': g(s, 'kierunek', 'Informatyka'),

            'tryb_studiow': g(s, 'tryb_studiow', ''),

        },

        'firma': {

            'nazwa': firma_nazwa,

            'adres': firma_adres,

            'miasto': firma_miasto,

        },

    }

```

```

        'name':    firma_nazwa,

        'address': firma_adres,

        'city':    firma_miasto,

    },

    'terminy': {

        'od': zapis.termin_od.strftime('%d.%m.%Y') if zapis.termin_od else '',

        'do': zapis.termin_do.strftime('%d.%m.%Y') if zapis.termin_do else '',

    },

    'zopz': {

        'imie_nazwisko': g(zapis, 'zopz_imie_nazwisko', ''),

        'stanowisko':    g(zapis, 'zopz_stanowisko', ''),

        'telefon':        g(zapis, 'zopz_telefon', ''),

        'email':          g(zapis, 'zopz_email', ''),

    },

    'uopz': {

        'imie_nazwisko': f"{uopz.first_name} {uopz.last_name}" if uopz else '',

    },

    'praktyka': {

        'rok_uczelnianny': g(p, 'rok_uczelnianny') if p else '',

        'semestr':        g(p, 'semestr') if p else '',

        'wymiar_godzin':  g(p, 'wymiar_godzin', 120) if p else 120,

    },

    # Termin od/do i specjalność też masz w Zapisie!

    'rok_akademicki': _rok_akademicki(getattr(zapis, 'termin_od', None)),

    'specjalnosc':    g(zapis, 'specjalnosc', 'Informatyka'),

    'lacznie_godzin': g(zapis, 'total_hours_logged', 0),

}

```

```

if doc_type == 'ZAL_6':

```

```

    context.update({

        'zapis': {

            'total_hours_logged': g(zapis, 'total_hours_logged', 0),

            'praktyka': {

                'rok_uczelnianny': g(p, 'rok_uczelnianny'),

```



```

        'semestr':          g(p, 'semestr'),

    },

},

'sciezka':          _sciezka_label(getattr(zapis, 'track_type', 'STANDARD')),

'data_roz poczenia': _d(getattr(zapis, 'termin_od', None)),

'data_zakonczenia': _d(getattr(zapis, 'termin_do', None)),

'wpisy': [

    {

        'entry_date':      _d(w.entry_date),

        'duration_hours':  g(w, 'duration_hours', 0),

        'description':      g(w, 'description'),

        'efekt': {'kod': ' ', '.join(f"{e.id:02d}" for e in w.efekty_uczenia) if w.efekty_uczenia else '--'},

    }

    for w in sorted(getattr(zapis, 'wpisy_dziennika', []), key=lambda x: getattr(x, 'entry_date', date.min))

]

})

```

```

elif doc_type == 'ZAL_7':

```

```

    spr = getattr(zapis, 'sprawozdanie', None)

    context.update({

        'charakterystyka_miejsc': g(spr, 'charakterystyka_miejsc'),

        'opis_prac':              g(spr, 'opis_i_analiza'),

        'efekty_opisy':           [''] * 13,

    })

```

```

elif doc_type == 'ZAL_4':

```

```

    context.update({

        'lacznie_godzin': g(zapis, 'total_hours_logged', 0),

        'oceny': [

            {

                'learning_outcome_id': g(o, 'learning_outcome_id'),

                'grade':                o.result.value if getattr(o, 'result', None) else 'uzyskał/a',

            }

            for o in sorted(getattr(zapis, 'oceny_efektow', []), key=lambda x: g(x, 'learning_outcome_id', 0))

        ]

    })

```

```
    ],  
    'uwagi_uopz': g(zapis, 'ocena_opisowa_uopz'),  
})
```

```
return context
```

```
# -----  
# ROUTE  
# -----
```

```
@documents_bp.route('/generuj/<doc_type>', methods=['POST'])
```

```
@login_required
```

```
def generuj(doc_type: str):
```

```
    if doc_type not in DOC_CONFIG:
```

```
        abort(403)
```

```
    force = request.args.get('force') == 'true'
```

```
    # Pobranie zapisu
```

```
    zapis = db.session.query(ZapisPraktyki).filter_by(  
        student_id=current_user.id, status=StatusZapisu.IN_PROGRESS
```

```
    ).first() or db.session.query(ZapisPraktyki).filter_by(  
        student_id=current_user.id
```

```
    ).order_by(ZapisPraktyki.id.desc()).first()
```

```
    if not zapis:
```

```
        return jsonify({'error': 'Nie znaleziono zapisu na praktykę.'}), 404
```

```
    # --- WALIDACJA ---
```

```
    warnings = []
```

```
    # Sprawdzamy zapis (bo tam jest firma_nazwa), a nie p!
```

```
    if not g(zapis, 'firma_nazwa'): warnings.append("Nazwa firmy")
```

```
    if not g(zapis, 'firma_adres'): warnings.append("Adres firmy")
```

```

if doc_type == 'ZAL_6' and not getattr(zapis, 'wpisy_dziennika', None):

    warnings.append("Brak wpisów w dzienniku")

if warnings and not force:

    return jsonify({

        'requires_confirmation': True,

        'warnings': warnings,

        'message': 'Niektóre dane są puste. Dokument będzie niekompletny.'

    }), 200

# --- GENEROWANIE ---

template_name, filename = DOC_CONFIG[doc_type]

try:

    context = _serialize_context(doc_type, zapis)

    resp = httpx.post(

        f"{get_tex_service_url()}/generuj",

        json={'template': template_name, 'context': context, 'filename': filename},

        timeout=30.0,

    )

    if resp.status_code != 200:

        err = resp.json() if 'json' in resp.headers.get('content-type', '') else {}

        return jsonify({'error': err.get('error', 'Błąd serwisu PDF')}), 500

    return send_file(io.BytesIO(resp.content), mimetype='application/pdf',

                     as_attachment=True, download_name=filename)

except Exception as e:

    logger.error(f"Błąd krytyczny: {str(e)}", exc_info=True)

    return jsonify({'error': str(e)}), 500

```

PLIK: app_student\routes\dziennik.py

```
import uuid

from datetime import date

from flask import Blueprint, render_template, redirect, url_for, flash, request

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from wtforms import StringField, SelectMultipleField, TextAreaField

from wtforms.validators import DataRequired, ValidationError

from wtforms.widgets import ListWidget, CheckboxInput

from core.modele import ZapisPraktyki, WpisDziennika, EfektUczenia, StatusZapisu

from core.extensions import db

dziennik_bp = Blueprint('dziennik', __name__)

class FormularzWpisu(FlaskForm):

    data_wpisu = StringField('Data', validators=[DataRequired()])

    liczba_godzin = StringField('Liczba godzin (1-8)', validators=[DataRequired()])

    opis = TextAreaField('Opis wykonanych prac', validators=[DataRequired()])

    efekty_ids = SelectMultipleField(

        'Efekty uczenia się',

        validators=[DataRequired(message='Wybierz co najmniej jeden efekt uczenia się.')],

        widget=ListWidget(prefix_label=False),

        option_widget=CheckboxInput(),

    )

    def validate_liczba_godzin(self, pole):

        try:

            val = int(pole.data)

        except (ValueError, TypeError):

            raise ValidationError('Podaj liczbę całkowitą.')

        if val < 1 or val > 8:
```

```

        raise ValidationError('Maksymalnie 8 godzin dziennie (regulamin ANS).')

def validate_data_wpisu(self, pole):

    try:

        date.fromisoformat(pole.data)

    except ValueError:

        raise ValidationError('Nieprawidłowy format daty.')

def _aktywny_zapis():

    return db.session.query(ZapisPraktyki).filter(

        ZapisPraktyki.student_id == current_user.id,

        ZapisPraktyki.status.in_(

            StatusZapisu.IN_PROGRESS,

            StatusZapisu.COMMISSION_REVIEW,

            StatusZapisu.DEAN_APPROVAL,

            StatusZapisu.COMPLETED

        )),

    ).first()

@dziennik_bp.route('/')

@login_required

def index():

    zapis = _aktywny_zapis()

    # Jeśli student nie ma aktywnego zapisu, szukamy jakiegokolwiek innego (np. zakończonego)

    if not zapis:

        jakikolwiek = db.session.query(ZapisPraktyki).filter_by(student_id=current_user.id).first()

        # Zwracamy puste wpisy, żeby szablon się nie wysypał

        return render_template('dziennik/index.html', zapis=None, wpisy=[], jakikolwiek=jakikolwiek,
csrf_form=FlaskForm())

    # Jeśli jest aktywny zapis, normalnie ładujemy wpisy

    wpisy = db.session.query(WpisDziennika).filter_by(zapis_id=zapis.id).order_by(WpisDziennika.data_wpisu.desc()).all()

```

```
csrf_form = FlaskForm()
```

```
return render_template('dziennik/index.html', zapis=zapis, wpisy=wpisy, jakikolwiek=zapis, csrf_form=csrf_form)
```

```
@dziennik_bp.route('/nowy', methods=['GET', 'POST'])
```

```
@login_required
```

```
def nowy_wpis():
```

```
    zapis = _aktywny_zapis()
```

```
    if not zapis:
```

```
        flash('Nie masz aktywnej praktyki. Skontaktuj się z opiekunem.', 'danger')
```

```
        return redirect(url_for('dziennik.index'))
```

```
efekty = db.session.query(EfektUczenia).order_by(EfektUczenia.id).all()
```

```
form = FormularzWpisu()
```

```
form.efekty_ids.choices = [
```

```
    (str(e.id), f'{e.kod}: {e.description[:80]}...' if len(e.description) > 80 else f'{e.kod}: {e.description}')
```

```
    for e in efekty
```

```
]
```

```
if form.validate_on_submit():
```

```
    data = date.fromisoformat(form.data_wpisu.data)
```

```
    duplikat = db.session.query(WpisDziennika).filter_by(zapis_id=zapis.id, entry_date=data).first()
```

```
    if duplikat:
```

```
        flash('Wpis na ten dzień już istnieje. Możesz go edytować.', 'danger')
```

```
        return render_template('dziennik/nowy_wpis.html', form=form, zapis=zapis)
```

```
godziny = int(form.liczba_godzin.data)
```

```
wybrane_efekty = db.session.query(EfektUczenia).filter(
```

```
    EfektUczenia.id.in_([int(i) for i in form.efekty_ids.data])
```

```
).all()
```

```
wpis = WpisDziennika(
```

```
    id = uuid.uuid4(),
```

```
    enrollment_id = zapis.id,
```

```
    entry_date = data,
```

```

        duration_hours = godziny,

        description = form.opis.data.strip(),

        efekty_uczenia = wybrane_efekty,

    )

    db.session.add(wpis)

    db.session.commit()

    flash(f'Wpis z dnia {data.strftime("%d.%m.%Y")} został dodany ({godziny} h).', 'success')

    return redirect(url_for('dziennik.index'))


if request.method == 'GET':

    form.data_wpisu.data = date.today().isoformat()


return render_template('dziennik/nowy_wpis.html', form=form, zapis=zapis)


@dziennik_bp.route('/edytuj/<uuid:wpis_id>', methods=['GET', 'POST'])
@login_required
def edytuj_wpis(wpis_id):

    wpisu = db.session.get(WpisDziennika, wpis_id)

    if not wpisu:

        from flask import abort

        abort(404)

    zapis = db.session.get(ZapisPraktyki, wpisu.enrollment_id)

    if not zapis or zapis.student_id != current_user.id:

        from flask import abort

        abort(403)

    efekty = db.session.query(EfektUczenia).order_by(EfektUczenia.id).all()

    form = FormularzWpisu()

    form.efekty_ids.choices = [

        (str(e.id), f'{e.kod}: {e.description[:80]}...' if len(e.description) > 80 else f'{e.kod}: {e.description}')

        for e in efekty

    ]

```

```

if form.validate_on_submit():

    wpis.entry_date      = date.fromisoformat(form.data_wpisu.data)

    wpis.duration_hours = int(form.liczba_godzin.data)

    wpis.description     = form.opis.data.strip()

    wpis.efekty_uczenia = db.session.query(EfektUczenia).filter(
        EfektUczenia.id.in_([int(i) for i in form.efekty_ids.data])
    ).all()

    db.session.commit()

    flash('Wpis został zaktualizowany.', 'success')

    return redirect(url_for('dziennik.index'))


if request.method == 'GET':

    form.data_wpisu.data      = wpis.entry_date.isoformat()

    form.liczba_godzin.data = str(wpis.duration_hours)

    form.opis.data           = wpis.description

    form.efekty_ids.data     = [str(e.id) for e in wpis.efekty_uczenia]


return render_template('dziennik/nowy_wpis.html', form=form, zapis=zapis, edycja=True, wpis=wpis)


@dziennik_bp.route('/usun/<uuid:wpis_id>', methods=['POST'])
@login_required
def usun_wpis(wpis_id):

    wpis = db.session.get(WpisDziennika, wpis_id)

    if not wpis:

        from flask import abort

        abort(404)

    zapis = db.session.get(ZapisPraktyki, wpis.enrollment_id)

    if not zapis or zapis.student_id != current_user.id:

        from flask import abort

        abort(403)

    db.session.delete(wpis)

    db.session.commit()

```



```
flash('Wpis został usunięty.', 'success')  
  
return redirect(url_for('dziennik.index'))
```

PLIK: app_student\routes\pliki.py

```
import os

import uuid

import mimetypes

from pathlib import Path

from werkzeug.utils import secure_filename

from flask import Blueprint, request, jsonify, send_from_directory, abort, flash, redirect, url_for

from flask_login import login_required, current_user


from core.modele import ZapisPraktyki, DokumentPrzeslany, Uzytkownik, RolaUzytkownika

from core.extensions import db


uploads_bp = Blueprint('uploads', __name__)


UPLOAD_FOLDER = Path(__file__).parent.parent.parent / "uploads"

UPLOAD_FOLDER.mkdir(exist_ok=True)

MAX_FILE_SIZE = 10 * 1024 * 1024 # 10MB

ALLOWED_EXTENSIONS = {'.pdf', '.doc', '.docx', '.jpg', '.jpeg', '.png', '.zip', '.rar'}

ALLOWED_MIME_TYPES = {

    'application/pdf',

    'application/msword',

    'application/vnd.openxmlformats-officedocument.wordprocessingml.document',

    'image/jpeg',

    'image/png',

    'application/zip',

    'application/x-rar-compressed',

    'application/vnd.rar',

}


def allowed_file(filename, content_type):

    if not filename:

        return False

    ext = Path(filename).suffix.lower()
```

```
return ext in ALLOWED_EXTENSIONS and content_type in ALLOWED_MIME_TYPES
```

```
@uploads_bp.route('/enrollment/<uuid:enrollment_id>/upload', methods=['POST'])
```

```
@login_required
```

```
def upload_document(enrollment_id):
```

```
    zapis = db.session.get(ZapisPraktyki, enrollment_id)
```

```
    if not zapis or zapis.student_id != current_user.id:
```

```
        abort(404)
```

```
    if 'file' not in request.files:
```

```
        return jsonify({'error': 'Brak pliku'}), 400
```

```
    file = request.files['file']
```

```
    document_type = request.form.get('document_type', '').strip()
```

```
    if not file.filename or not document_type:
```

```
        return jsonify({'error': 'Brak pliku lub typu dokumentu'}), 400
```

```
    file.seek(0, os.SEEK_END)
```

```
    file_size = file.tell()
```

```
    file.seek(0)
```

```
    if file_size > MAX_FILE_SIZE:
```

```
        return jsonify({'error': f'Plik zbyt duży. Maksymalny rozmiar: {MAX_FILE_SIZE // (1024*1024)}MB'}), 400
```

```
    content_type = file.content_type
```

```
    if not allowed_file(file.filename, content_type):
```

```
        return jsonify({'error': 'Niedozwolony typ pliku'}), 400
```

```
    try:
```

```
        original_filename = secure_filename(file.filename)
```

```
        file_ext = Path(original_filename).suffix.lower()
```

```
        stored_filename = f"{uuid.uuid4().hex}{file_ext}"
```

```
file_path = UPLOAD_FOLDER / stored_filename
```

```
file.save(file_path)
```

```
doc = DokumentPrzeslany(
```

```
    zapis_id=enrollment_id,
```

```
    typ_dokumentu=document_type,
```

```
    oryginalna_nazwa=original_filename,
```

```
    zapisana_nazwa=stored_filename,
```

```
    sciezka_pliku=str(file_path),
```

```
    rozmiar_pliku=file_size,
```

```
    typ_mime=content_type,
```

```
    przeslane_przez_id=current_user.id,
```

```
)
```

```
db.session.add(doc)
```

```
db.session.commit()
```

```
return jsonify({'success': True, 'document_id': str(doc.id),
```

```
               'filename': original_filename, 'size': file_size})
```

```
except Exception as e:
```

```
    if 'file_path' in locals() and file_path.exists():
```

```
        file_path.unlink()
```

```
    db.session.rollback()
```

```
    return jsonify({'error': f'Błąd podczas zapisywania: {str(e)}'}), 500
```

```
@uploads_bp.route('/document/<uuid:document_id>/download')
```

```
@login_required
```

```
def download_document(document_id):
```

```
    doc = db.session.get(DokumentPrzeslany, document_id)
```

```
    if not doc or not doc.zapis:
```

```
        abort(404)
```

```
    if str(doc.zapis.student_id) != str(current_user.id):
```

```
        abort(403)
```

```

file_path = Path(doc.sciezka_pliku)

if not file_path.exists():

    abort(404)

return send_from_directory(file_path.parent, file_path.name,

                           as_attachment=True, download_name=doc.oryginalna_nazwa,

                           mimetype=doc.typ_mime)

@uploads_bp.route('/document/<uuid:document_id>/delete', methods=['POST'])
@login_required
def delete_document(document_id):

    doc = db.session.get(DokumentPrzeslany, document_id)

    if not doc or not doc.zapis:

        abort(404)

    if str(doc.zapis.student_id) != str(current_user.id):

        abort(404)

    try:

        file_path = Path(doc.sciezka_pliku)

        if file_path.exists():

            file_path.unlink()

        db.session.delete(doc)

        db.session.commit()

        flash('Dokument został usunięty.', 'success')

    except Exception as e:

        db.session.rollback()

        flash(f'Błąd podczas usuwania dokumentu: {str(e)}', 'danger')

    return redirect(request.referrer or url_for('dashboard.index'))

@uploads_bp.route('/enrollment/<uuid:enrollment_id>/documents')

```

```
@login_required
```

```
def list_documents(enrollment_id):
```

```
    zapis = db.session.get(ZapisPraktyki, enrollment_id)
```

```
    if not zapis or zapis.student_id != current_user.id:
```

```
        abort(404)
```

```
    results = (
```

```
        db.session.query(DokumentPrzeslany, Uzytkownik)
```

```
        .outerjoin(Uzytkownik, DokumentPrzeslany.przeslane_przez_id == Uzytkownik.id)
```

```
        .filter(DokumentPrzeslany.zapis_id == enrollment_id)
```

```
        .order_by(DokumentPrzeslany.przeslano_o.desc())
```

```
        .all()
```

```
    )
```

```
    payload = []
```

```
    for doc, user in results:
```

```
        uploaded_by = None
```

```
        if user is not None:
```

```
            uploaded_by = f"{user.imie} {user.nazwisko}".strip() or None
```

```
    payload.append({
```

```
        'id': str(doc.id),
```

```
        'document_type': doc.typ_dokumentu,
```

```
        'original_filename': doc.oryginalna_nazwa,
```

```
        'file_size': doc.rozmiar_pliku,
```

```
        'mime_type': doc.typ_mime,
```

```
        'uploaded_at': doc.przeslano_o.isoformat() if doc.przeslano_o else None,
```

```
        'uploaded_by': uploaded_by,
```

```
        'download_url': url_for('uploads.download_document', document_id=doc.id),
```

```
        'delete_url': url_for('uploads.delete_document', document_id=doc.id),
```

```
    })
```

```
    return jsonify(payload)
```

PLIK: app_student\routes\praktyki.py

```
import uuid

import datetime

from flask import Blueprint, render_template, redirect, url_for, flash, request, abort, make_response

from flask_wtf import FlaskForm

from wtforms import StringField, SelectField, DateField, BooleanField, TextAreaField

from wtforms.validators import DataRequired, Optional, Length, Email, ValidationError

import re

from flask_login import login_required, current_user

from core.extensions import db

import httpx

from core.modele import Praktyka, ZapisPraktyki, StatusPraktyki, StatusZapisu, SciezkaPraktyki, Uzytkownik,
RolaUzytkownika, EfektUczenia, HarmonogramPraktyki, Firma, IndywidualnyProgram, StatusDokumentu, DokumentPrzeslany


praktyki_bp = Blueprint('praktyki', __name__)


# =====
# NOWE FORMULARZE KREATORA
# =====


class FormularzSciezka(FlaskForm):

    """Krok 1: Tylko wybór ścieżki."""

    track_type = SelectField('Ścieżka praktyki', choices=[

        ('STANDARD', 'A – Standardowa praktyka'),

        ('EMPLOYMENT', 'B – Praca etatowa / staż'),

        ('OWN_BUSINESS', 'C – Własna działalność gospodarcza'),

    ], validators=[DataRequired(message='Wybierz ścieżkę.')])


class FormularzDaneFirmy(FlaskForm):

    """Krok 2A: Dane zakładu pracy + ZOPZ + terminy (tylko ścieżka A)."""

    # Terminy i dane podstawowe

    termin_od = DateField('Data rozpoczęcia', validators=[DataRequired(message='Podaj datę.')])
```

```

termin_do    = DateField('Data zakończenia', validators=[DataRequired(message='Podaj datę.')])

uopz_id      = SelectField('Opiekun uczelniany (UOPZ)', choices=[], validators=[Optional()])

ubezpieczenie_nw = BooleanField('Posiadam ubezpieczenie NW na czas trwania praktyki')


# Tryb znalezienia miejsca

firma_typ    = SelectField('Jak znalazłeś/-aś miejsce praktyki?', choices=[

    ('database', 'Uczelnia kieruje do zakładu (firma ma umowę z ANS)'),

    ('custom', 'Sam/a znalazłem/-am miejsce (wymaga Zał. 9 i Zał. 1)'),

], validators=[DataRequired()])

firma_id     = SelectField('Wybierz firmę z listy', choices=[], validators=[Optional()])


firma_nazwa      = StringField('Nazwa zakładu pracy', validators=[Optional(), Length(max=255)])

firma_adres      = StringField('Adres (ulica, nr)', validators=[Optional(), Length(max=255)])

firma_miasto     = StringField('Miasto i kod pocztowy', validators=[Optional(), Length(max=100)])

firma_nip_krs    = StringField('NIP / KRS', validators=[Optional(), Length(max=50)])

firma_upowazniony_osoba = StringField('Osoba upoważniona do podpisania porozumienia', validators=[Optional(), Length(max=255)])

firma_upowazniony_stanowisko = StringField('Stanowisko osoby upoważnionej', validators=[Optional(), Length(max=255)])


zopz_imie_nazwisko = StringField('Opiekun zakładowy (ZOPZ) – imię i nazwisko', validators=[Optional(), Length(max=255)])

zopz_stanowisko    = StringField('Stanowisko ZOPZ', validators=[Optional(), Length(max=255)])

zopz_telefon       = StringField('Telefon ZOPZ', validators=[Optional(), Length(max=50)])

zopz_email         = StringField('E-mail ZOPZ', validators=[Optional(), Email(message='Nieprawidłowy email.')])


def validate_firma_miasto(self, field):

    if not field.data:

        return

    if re.fullmatch(r'\d{2}-\d{3}', field.data.strip()):

        raise ValidationError('Podaj nazwę miasta, nie kod pocztowy. Kod pocztowy możesz dołączyć do adresu.')


def validate_zopz_imie_nazwisko(self, field):

    if not field.data:

        return

    parts = field.data.strip().split()

```



```

    if len(parts) < 2:

        raise ValidationError('Podaj imię i nazwisko (co najmniej dwa wyrazy).')

    if any(char.isdigit() for char in field.data):

        raise ValidationError('Imię i nazwisko nie może zawierać cyfr.')


def validate_firma_upowazniony_osoba(self, field):

    if not field.data:

        return

    parts = field.data.strip().split()

    if len(parts) < 2:

        raise ValidationError('Podaj imię i nazwisko osoby upoważnionej (co najmniej dwa wyrazy).')

    if any(char.isdigit() for char in field.data):

        raise ValidationError('Imię i nazwisko nie może zawierać cyfr.')


class FormularzWniosek(FlaskForm):

    """Krok 2B/C: Wniosek dla ścieżek B i C."""

    pracodawca_nazwa      = StringField('Nazwa pracodawcy / firmy', validators=[DataRequired(message='Podaj nazwę.'),
Length(max=255)])

    pracodawca_adres      = StringField('Adres', validators=[Optional(), Length(max=255)])

    pracodawca_miasto     = StringField('Miasto', validators=[Optional(), Length(max=100)])

    stanowisko            = StringField('Stanowisko / zakres działalności', validators=[DataRequired(message='Podaj
stanowisko.'), Length(max=255)])

    uzasadnienie          = TextAreaField('Uzasadnienie wniosku', validators=[DataRequired(message='Napisz uzasadnienie.'),
Length(max=2000)])


# =====

# NOWE ROUTE KREATORA

# =====


@praktyki_bp.route('/<uuid:id>/kreator/sciezka', methods=['GET', 'POST'])

@login_required

def kreator_sciezka(id):

    """Krok 1: Wybór ścieżki."""

    praktyka = db.session.get(Praktyka, id)

```

```

if not praktyka:

    flash('Praktyka niedostępna.', 'danger')

    return redirect(url_for('praktyki.lista'))

istniejacy = db.session.query(ZapisPraktyki).filter_by(

    praktyka_id=id, student_id=current_user.id

).filter(ZapisPraktyki.status == StatusZapisu.PENDING).first()

form = FormularzSciezka()

if form.validate_on_submit():

    if istniejacy:

        zapis = istniejacy

    else:

        zapis = ZapisPraktyki(id=uuid.uuid4(), praktyka_id=id,

                               student_id=current_user.id, status=StatusZapisu.PENDING)

        db.session.add(zapis)

    zapis.track_type = SciezkaPraktyki(form.track_type.data)

    db.session.commit()

    if form.track_type.data == 'STANDARD':

        return redirect(url_for('praktyki.kreator_firma', zapis_id=zapis.id))

    else:

        return redirect(url_for('praktyki.kreator_wniosek', zapis_id=zapis.id))

if istniejacy and request.method == 'GET':

    form.track_type.data = istniejacy.track_type.value if istniejacy.track_type else 'STANDARD'

return render_template('kreator/krok1_sciezka.html', form=form, praktyka=praktyka, istniejacy=istniejacy)

```

```

@praktyki_bp.route('/zgloszenie/<uuid:zapis_id>/kreator/firma', methods=['GET', 'POST'])

```

```

@login_required

```

```

def kreator_firma(zapis_id):

    """Krok 2A: Dane zakładu + ZOPZ (ścieżka A)."""

    zapis = db.session.get(ZapisPraktyki, zapis_id)

    if not zapis or zapis.student_id != current_user.id or zapis.track_type != SciezkaPraktyki.STANDARD:

        abort(404)

    form = FormularzDaneFirmy()

    firmy_list = db.session.query(Firma).filter_by(aktywna=True).order_by(Firma.nazwa).all()

    form.firma_id.choices = [('', '--- Wybierz firmę ---')] + [(str(f.id), f.nazwa) for f in firmy_list]

    uopz_list = db.session.query(Uzytkownik).filter_by(rola=RolaUzytkownika.UOPZ,
aktywny=True).order_by(Uzytkownik.nazwisko).all()

    form.uopz_id.choices = [('', '--- Wybierz ---')] + [(str(u.id), f"{u.first_name} {u.last_name}") for u in uopz_list]

    if form.validate_on_submit():

        if not form.ubezpieczenie_nw.data:

            flash('Ubezpieczenie NW jest wymagane.', 'danger')

            return render_template('kreator/krok2a_firma.html', form=form, zapis=zapis, firmy_list=firmy_list)

        zapis.termin_od = form.termin_od.data

        zapis.termin_do = form.termin_do.data

        zapis.uopz_id = form.uopz_id.data if form.uopz_id.data else None

        zapis.ubezpieczenie_nw = True

        zapis.specjalnosc = getattr(current_user, 'specjalnosc', '') or ''

    from core.modele import DaneMiejscaPraktyki

    dm = zapis.dane_miejsca or DaneMiejscaPraktyki(zapis_id=zapis.id)

    if dm not in db.session:

        db.session.add(dm)

    if form.firma_typ.data == 'database':

        if not form.firma_id.data:

            flash('Wybierz firmę z listy.', 'danger')

            return render_template('kreator/krok2a_firma.html', form=form, zapis=zapis, firmy_list=firmy_list)

        zapis.firma_id = form.firma_id.data

        dm.firma_nazwa = dm.firma_adres = dm.firma_miasto = None

```

```

dm.firma_nip_krs = dm.firma_upowazniony_osoba = dm.firma_upowazniony_stanowisko = None

else:

    if not form.firma_nazwa.data or not form.firma_adres.data or not form.firma_miasto.data:

        flash('Podaj nazwę, adres i miasto firmy.', 'danger')

        return render_template('kreator/krok2a_firma.html', form=form, zapis=zapis, firmy_list=firmy_list)

    zapis.firma_id = None

    dm.firma_nazwa = form.firma_nazwa.data

    dm.firma_adres = form.firma_adres.data

    dm.firma_miasto = form.firma_miasto.data

    dm.firma_nip_krs = form.firma_nip_krs.data

    dm.firma_upowazniony_osoba = form.firma_upowazniony_osoba.data

    dm.firma_upowazniony_stanowisko = form.firma_upowazniony_stanowisko.data

    if not form.zopz_imie_nazwisko.data or not form.zopz_email.data:

        flash('Podaj imię/nazwisko i email opiekuna zakładowego (ZOPZ).', 'danger')

        return render_template('kreator/krok2a_firma.html', form=form, zapis=zapis, firmy_list=firmy_list)

    dm.zopz_imie_nazwisko = form.zopz_imie_nazwisko.data

    dm.zopz_stanowisko = form.zopz_stanowisko.data

    dm.zopz_telefon = form.zopz_telefon.data

    dm.zopz_email = form.zopz_email.data

    db.session.commit()

    if zapis.status == StatusZapisu.AWAITING_APPROVAL:

        zapis.status = StatusZapisu.COMMISSION_REVIEW

        db.session.commit()

        flash('Dane zaktualizowane i zgłoszenie odesłane do komisji.', 'success')

        return redirect(url_for('praktyki.szczegoly_zgloszenia', id=zapis.id))

    return redirect(url_for('praktyki.zapisz_krok2', id=zapis.id))

if request.method == 'GET':

    dm = zapis.dane_miejsca

    form.termin_od.data = zapis.termin_od

    form.termin_do.data = zapis.termin_do

    form.uopz_id.data = str(zapis.uopz_id) if zapis.uopz_id else ''

```

```

form.ubezpieczenie_nw.data = zapis.ubezpieczenie_nw

form.firma_typ.data = 'database' if zapis.firma_id else 'custom'

form.firma_id.data = str(zapis.firma_id) if zapis.firma_id else ''

form.firma_nazwa.data = dm.firma_nazwa if dm else None

form.firma_adres.data = dm.firma_adres if dm else None

form.firma_miasto.data = dm.firma_miasto if dm else None

form.firma_nip_krs.data = dm.firma_nip_krs if dm else None

form.firma_upowazniony_osoba.data = dm.firma_upowazniony_osoba if dm else None

form.firma_upowazniony_stanowisko.data = dm.firma_upowazniony_stanowisko if dm else None

form.zopz_imie_nazwisko.data = dm.zopz_imie_nazwisko if dm else None

form.zopz_stanowisko.data = dm.zopz_stanowisko if dm else None

form.zopz_telefon.data = dm.zopz_telefon if dm else None

form.zopz_email.data = dm.zopz_email if dm else None

return render_template('kreator/krok2a_firma.html', form=form, zapis=zapis, firmy_list=firmy_list)

```

```

@praktyki_bp.route('/zgloszenie/<uuid:zapis_id>/kreator/wniosek', methods=['GET', 'POST'])

```

```

@login_required

```

```

def kreator_wniosek(zapis_id):

    """Krok 2B/C: Wniosek dla ścieżek B i C."""

    zapis = db.session.get(ZapisPraktyki, zapis_id)

    if not zapis or zapis.student_id != current_user.id:

        abort(404)

    if zapis.track_type == SciezkaPraktyki.STANDARD:

        return redirect(url_for('praktyki.kreator_firma', zapis_id=zapis_id))

    form = FormularzWniosek()

    if form.validate_on_submit():

        from core.modele import DaneMiejscaPraktyki, UzasadnienieSciezki

        dm = zapis.dane_miejsca or DaneMiejscaPraktyki(zapis_id=zapis.id)

        if dm not in db.session:

            db.session.add(dm)

```

```

dm.firma_nazwa = form.pracodawca_nazwa.data

dm.firma_adres = form.pracodawca_adres.data

dm.firma_miasto = form.pracodawca_miasto.data

dm.zopz_stanowisko = form.stanowisko.data


uz = zapis.uzasadnienie or UzasadnienieSieczki(zapis_id=zapis.id)

if uz not in db.session:

    db.session.add(uz)

uz.uzasadnienie = form.uzasadnienie.data


byl_awaiting = zapis.status == StatusZapisu.AWAITING_APPROVAL

zapis.status = StatusZapisu.COMMISSION_REVIEW

db.session.commit()

flash('Dane zaktualizowane i wniosek odesłany do komisji.' if byl_awaiting else 'Wniosek złożony. Oczekujesz na
decyzję komisji.', 'success')

return redirect(url_for('praktyki.szczegoly_zgloszenia', id=zapis.id))


if request.method == 'GET':

    dm = zapis.dane_miejsca

    uz = zapis.uzasadnienie

    form.pracodawca_nazwa.data = dm.firma_nazwa if dm else None

    form.pracodawca_adres.data = dm.firma_adres if dm else None

    form.pracodawca_miasto.data = dm.firma_miasto if dm else None

    form.stanowisko.data = dm.zopz_stanowisko if dm else None

    form.uzasadnienie.data = uz.uzasadnienie if uz else None


return render_template('kreator/krok2bc_wniosek.html', form=form, zapis=zapis)


@praktyki_bp.route('/')

@login_required

def lista():

    dostepne = db.session.query(Praktyka)\

```

```

        .filter_by(status=StatusPraktyki.ACTIVE)\

        .order_by(Praktyka.rok_uczelnianny.desc())\

        .all()

```

```

zapisy_data = {

    str(z.praktyka_id): {

        'id': str(z.id),

        'status': z.status.value,

        'sciezka': z.sciezka.value if z.sciezka else None,

        'wymaga_uwagi': (

            z.status == StatusZapisu.AWAITING_APPROVAL

            and bool(z.komentarze_uopz)

        ),

    }

    for z in db.session.query(ZapisPraktyki)\

        .filter_by(student_id=current_user.id).all()

}

```

```

csrf_form = FlaskForm()

return render_template('praktyki/lista.html', dostepne=dostepne, zapisy_data=zapisy_data, csrf_form=csrf_form)

```

```

@praktyki_bp.route('/zgloszenie/<uuid:id>/zakonczone', methods=['POST'])

```

```

@login_required

```

```

def zakoncz_praktyke(id):

```

```

    zapis = db.session.get(ZapisPraktyki, id)

```

```

    if not zapis or zapis.student_id != current_user.id:

```

```

        abort(404)

```

```

    if zapis.status != StatusZapisu.IN_PROGRESS:

```

```

        flash('Praktykę można zakończyć tylko gdy jest w trakcie realizacji.', 'warning')

```

```

        return redirect(url_for('praktyki.lista'))

```

```

    zapis.status = StatusZapisu.COMPLETED

```

```

    db.session.commit()

```

```

    flash('Praktyka została oznaczona jako zakończona. Dokumenty końcowe są teraz dostępne w zakładce Moje Dokumenty.',
'success')

```

```
return redirect(url_for('praktyki.lista'))
```

```
@praktyki_bp.route('/<uuid:id>/zapisz/krok1')
```

```
@login_required
```

```
def zapisz_krok1(id):
```

```
    """Stara trasa – przekierowanie do nowego kreatora."""
```

```
    return redirect(url_for('praktyki.kreator_sciezka', id=id))
```

```
@praktyki_bp.route('/zgloszenie/<uuid:id>/krok2', methods=['GET', 'POST'])
```

```
@login_required
```

```
def zapisz_krok2(id):
```

```
    zapis = db.session.get(ZapisPraktyki, id)
```

```
    if not zapis or zapis.student_id != current_user.id:
```

```
        abort(404)
```

```
    efekty = db.session.query(EfektUczenia).order_by(EfektUczenia.id).all()
```

```
    if request.method == 'POST':
```

```
        # Czyszczenie starego jeśli student wraca z jakiegoś powodu
```

```
        db.session.query(HarmonogramPraktyki).filter_by(zapis_id=zapis.id).delete()
```

```
        suma_dni = 0
```

```
        nowe_wiersze = []
```

```
        for e in efekty:
```

```
            dz = request.form.get(f'dzial_{e.id}', '')
```

```
            pr = request.form.get(f'prace_{e.id}', '')
```

```
            dni_str = request.form.get(f'dni_{e.id}', '0')
```

```
            try:
```

```
                dni = int(dni_str)
```

```
            except Exception:
```

```
                dni = 0
```



```

if dz.strip() and pr.strip():

    nowe_wiersze.append(HarmonogramPraktyki(

        id=uuid.uuid4(),

        enrollment_id=zapis.id,

        learning_outcome_id=e.id,

        nazwa_dzialu=dz,

        przykladowe_prace=pr,

        liczba_dni=dni

    ))

    suma_dni += dni

db.session.add_all(nowe_wiersze)

db.session.commit()

flash('Harmonogram zapisany.', 'success')

return redirect(url_for('praktyki.szczegoly_zgloszenia', id=zapis.id))

else:

    # GET request - pobranie istniejących danych harmonogramu

    istniejace_harmonogramy = {}

    harmonogramy = db.session.query(HarmonogramPraktyki).filter_by(zapis_id=zapis.id).all()

    for h in harmonogramy:

        istniejace_harmonogramy[str(h.learning_outcome_id)] = {

            'dzial': h.nazwa_dzialu,

            'prace': h.przykladowe_prace,

            'dni': h.liczba_dni

        }

csrf_form = FlaskForm()

return render_template('kreator/krok3_harmonogram.html',

    zapis=zapis,

    efekty=efekty,

    csrf_form=csrf_form,

```

```
istniejace_harmonogramy=istniejace_harmonogramy)
```

```
@praktyki_bp.route('/zgloszenie/<uuid:id>/szczegoly')
```

```
@login_required
```

```
def szczegoly_zgloszenia(id):
```

```
    """Szczegóły zgłoszenia studenta wraz z komentarzami UOPZ"""
```

```
    zapis = db.session.get(ZapisPraktyki, id)
```

```
    if not zapis or zapis.student_id != current_user.id:
```

```
        abort(404)
```

```
    rows = (
```

```
        db.session.query(DokumentPrzeslany)
```

```
        .filter_by(zapis_id=id)
```

```
        .order_by(DokumentPrzeslany.przeslano_o.desc())
```

```
        .all()
```

```
)
```

```
    uploaded_docs = [
```

```
        {
```

```
            'id': str(d.id),
```

```
            'original_filename': d.oryginalna_nazwa,
```

```
            'document_type': d.typ_dokumentu,
```

```
            'uploaded_at': d.przeslano_o,
```

```
        }
```

```
        for d in rows
```

```
    ]
```

```
    return render_template('praktyki/szczegoly_zgloszenia.html', zapis=zapis, uploaded_docs=uploaded_docs)
```

```
@praktyki_bp.route('/zgloszenie/<uuid:id>/resubmit', methods=['POST'])
```

```
@login_required
```

```
def resubmit_zgloszenia(id):
```

```
    """Student ponownie wysyła zgłoszenie po poprawkach."""
```

```
zapis = db.session.get(ZapisPraktyki, id)

if not zapis or zapis.student_id != current_user.id:

    abort(404)

if zapis.status != StatusZapisu.AWAITING_APPROVAL:

    flash('Zgłoszenie nie może być ponownie wysłane w tym statusie.', 'warning')

    return redirect(url_for('praktyki.szczegoly_zgloszenia', id=id))

zapis.status = StatusZapisu.COMMISSION_REVIEW

db.session.commit()

flash('Zgłoszenie zostało ponownie wysłane do weryfikacji komisji.', 'success')

return redirect(url_for('praktyki.szczegoly_zgloszenia', id=id))
```

PLIK: app_student\routes\pulpit.py

```
from flask import Blueprint, render_template

from flask_login import login_required, current_user

from core.modele import ZapisPraktyki, StatusZapisu

from core.extensions import db


dashboard_bp = Blueprint('dashboard', __name__)


@dashboard_bp.route('/')
@login_required
def index():

    zapis = db.session.query(ZapisPraktyki)\
        .filter_by(student_id=current_user.id)\
        .order_by(ZapisPraktyki.enrolled_at.desc())\
        .first()

    return render_template('dashboard/index.html', zapis=zapis)
```

PLIK: app_student\routes\sprawozdania.py

```
import uuid

from flask import Blueprint, render_template, redirect, url_for, flash, request

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from wtforms import TextAreaField

from wtforms.validators import DataRequired, Optional


from core.extensions import db

from core.modele import ZapisPraktyki, Sprawozdanie, StatusZapisu, SciezkaPraktyki


sprawozdania_bp = Blueprint('sprawozdania', __name__)


class FormularzSprawozdaniaStandard(FlaskForm):

    """Zał. 7 – ścieżka A: standardowa praktyka."""

    charakterystyka = TextAreaField('I. Charakterystyka miejsca odbywania praktyki', validators=[DataRequired()])

    opis = TextAreaField('II. Opis i analiza wykonywanych prac', validators=[DataRequired()])

    wiedza = TextAreaField('III. Wiedza i umiejętności uzyskane w trakcie praktyki', validators=[Optional()])


class FormularzSprawozdaniaPracaZawodowa(FlaskForm):

    """Zał. 7a – ścieżki B/C: praca zawodowa lub działalność gospodarcza."""

    charakterystyka = TextAreaField('I. Charakterystyka miejsca pracy / działalności', validators=[DataRequired()])

    opis = TextAreaField('II. Opis i analiza wykonywanych prac / działalności', validators=[DataRequired()])

    wiedza = TextAreaField('III. Wiedza i umiejętności uzyskane w trakcie pracy zawodowej lub działalności',
        validators=[Optional()])


@sprawozdania_bp.route('/', methods=['GET', 'POST'])
@login_required
def index():

    zapis = db.session.query(ZapisPraktyki).filter(

        ZapisPraktyki.student_id == current_user.id,
```

```

ZapisPraktyki.status.in([
    StatusZapisu.IN_PROGRESS,
    StatusZapisu.COMMISSION_REVIEW,
    StatusZapisu.DEAN_APPROVAL,
    StatusZapisu.COMPLETED,
    StatusZapisu.AWAITING_APPROVAL,
])
).first()

if not zapis:

    ma_zapis = db.session.query(ZapisPraktyki).filter_by(student_id=current_user.id).first()

    return render_template('sprawozdania/index.html', zapis=None, ma_zapis=ma_zapis)

# Utwórz sprawozdanie jeśli nie istnieje
if not zapis.sprawozdanie:

    nowe_spr = Sprawozdanie(
        id=uuid.uuid4(),
        enrollment_id=zapis.id,
        charakterystyka_miejsca='',
        opis_i_analiza=''
    )

    db.session.add(nowe_spr)

    db.session.commit()

    zapis = db.session.get(ZapisPraktyki, zapis.id)

# Wybierz formularz i szablon wg ścieżki
sciezka = zapis.track_type.value if zapis.track_type else 'STANDARD'

is_standard = sciezka == 'STANDARD'

FormClass = FormularzSprawozdaniaStandard if is_standard else FormularzSprawozdaniaPracaZawodowa

form = FormClass()

if form.validate_on_submit():

    zapis.sprawozdanie.charakterystyka_miejsca = form.charakterystyka.data

```

```
zapis.sprawozdanie.opis_i_analiza = form.opis.data

db.session.commit()

flash('Sprawozdanie zostało zapisane.', 'success')

return redirect(url_for('sprawozdania.index'))

elif request.method == 'GET':

    form.charakterystyka.data = zapis.sprawozdanie.charakterystyka_miejsca

    form.opis.data = zapis.sprawozdanie.opis_i_analiza


return render_template('sprawozdania/index.html',

                        zapis=zapis, form=form, ma_zapis=zapis,

                        is_standard=is_standard, sciezka=sciezka)
```

PLIK: app_student\static\css\student.css

```
/* =====  
  
student.css – Punkt wejścia arkusza stylów panelu studenta  
  
System Praktyk Zawodowych – ANS Elbląg  
  
Moduły ładowane w kolejności zależności:  
  
1. zmienne – tokeny CSS, reset, dostępność (WCAG 2.1 AA)  
2. komunikaty – flash messages i powiadomienia  
3. formularze – pola input, checkboxy  
4. logowanie – strona logowania i zmiana hasła  
5. układ – sidebar, panel-main, nagłówek  
6. karty – komponent .karta  
7. tabele – tabele danych  
8. przyciski – przyciski i grupy akcji  
9. statusy – odznaki statusów i ról  
10. komponenty – pasek postępu, paginacja, filtry  
11. dashboard – statystyki, strona błędu  
  
===== */  
  
@import url('/core/static/css/modul/zmienne.css');  
  
@import url('/core/static/css/modul/komunikaty.css');  
  
@import url('/core/static/css/modul/formularze.css');  
  
@import url('/core/static/css/modul/logowanie.css');  
  
@import url('/core/static/css/modul/uklad.css');  
  
@import url('/core/static/css/modul/karty.css');  
  
@import url('/core/static/css/modul/tabele.css');  
  
@import url('/core/static/css/modul/przyciski.css');  
  
@import url('/core/static/css/modul/statusy.css');  
  
@import url('/core/static/css/modul/komponenty.css');  
  
@import url('/core/static/css/modul/dashboard.css');
```


PLIK: app_student\templates\auth\logowanie.html

```
{% extends "layouts/base_auth.html" %}

{% block tytul %}Logowanie{% endblock %}


{% block body %}

<a href="#formularz-logowania" class="skip-link">Przejdź do formularza</a>


<main class="panel-logowania">


<div class="panel-logowania__branding" aria-hidden="true">

<div class="branding__tresc">

<div class="branding__logo-kontener">



</div>

<div class="branding__separator"></div>

<p class="branding__nazwa-systemu">System Praktyk Zawodowych</p>

<p class="branding__podtytul">Panel studenta</p>

<div class="branding__dekoracja">

<span></span><span></span><span></span></div>

</div>

</div>


<div class="panel-logowania__formularz">

<div class="formularz__kontener" id="formularz-logowania">

<div class="formularz__naglowek">

<h1 class="formularz__tytul">Zaloguj się</h1>

<p class="formularz__opis">Wprowadź dane swojego konta studenckiego.</p>

</div>


{% with messages = get_flashed_messages(with_categories=true) %}

{% if messages %}
```

```

<div class="komunikaty" aria-live="polite">

    {% for kategoria, wiadomosc in messages %}

    <div class="komunikat komunikat--{{ kategoria }}" role="alert">

        <span class="komunikat__ikona" aria-hidden="true">

            {% if kategoria == 'success' %}{% elif kategoria == 'danger' %}{% else %}i{% endif %}

        </span>

        {{ wiadomosc }}

    </div>

    {% endfor %}

</div>

{% endif %}

{% endwith %}

```

```

<form method="POST" action="{{ url_for('auth.logowanie') }}"

    class="formularz__forma" novalidate>

    {{ form.hidden_tag() }}

```

```

<div class="pole-formularza {% if form.email.errors %}pole-formularza--blad{% endif %}">

    {{ form.email.label(class="pole-formularza__etykieta") }}

    {{ form.email(class="pole-formularza__input",

        placeholder="jan.kowalski@student.ans-elblag.pl",

        autocomplete="email") }}

    {% for e in form.email.errors %}

        <span class="pole-formularza__blad" role="alert">{{ e }}</span>

    {% endfor %}

</div>

```

```

<div class="pole-formularza {% if form.haslo.errors %}pole-formularza--blad{% endif %}">

    {{ form.haslo.label(class="pole-formularza__etykieta") }}

    {{ form.haslo(class="pole-formularza__input",

        autocomplete="current-password") }}

    {% for e in form.haslo.errors %}

        <span class="pole-formularza__blad" role="alert">{{ e }}</span>

    {% endfor %}

```

```
</div>
```

```
<div class="pole-formularza pole-formularza--checkbox">
```

```
<label class="pole-formularza__etykieta-checkbox">
```

```
{ { form.zapamietaj(class="pole-formularza__checkbox") } }
```

```
Zapamiętaj mnie na tym urządzeniu
```

```
</label>
```

```
</div>
```

```
<button type="submit" class="przycisk-logowania">
```

```
Zaloguj się
```

```
</button>
```

```
</form>
```

```
<div class="formularz__stopka">
```

```
<p class="formularz__stopka-tekst">
```

```
Hasło tymczasowe to Twój numer albumu.<br>
```

```
Przy pierwszym logowaniu zostaniesz poproszony o jego zmianę.
```

```
</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</main>
```

```
{% endblock %}
```

PLIK: app_student\templates\dashboard\index.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Pulpit{% endblock %}

{% block naglowek %}Witaj, {{ current_user.first_name }}{% endblock %}


{% block tresc %}


{% if not zapis %}

    <div style="display: flex; justify-content: center; align-items: center; padding-top: 4rem;">

        <div class="karta" style="max-width:560px; text-align:center; padding:3rem 2rem; width: 100%;">

            <div style="color: var(--kolor-glowny); margin-bottom: 1.5rem;">

                <svg xmlns="http://www.w3.org/2000/svg" width="64" height="64" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round">

                    <path d="M14 2H6a2 2 0 0 0-2 2v16c0 1.1.9 2 2 2h12a2 2 0 0 0 2-2V8l-6-6z"/>

                    <path d="M14 3v5h5M16 13H8M16 17H8M10 9H8"/>

                </svg>

            </div>

            <h2 style="font-size:1.25rem; font-weight:600; margin-bottom:0.5rem;">Nie jesteś jeszcze zapisany na praktykę</h2>

            <p style="color:var(--kolor-tekst-drugor); margin-bottom:1.5rem;">Zapisz się do aktywnej praktyki żeby zacząć
prowadzić dziennik.</p>

            <a href="{{ url_for('praktyki.lista') }}" class="przycisk przycisk--glowny">Zobacz dostępne praktyki</a>

        </div>

    </div>

{% else %}

    <section class="siatka-statystyk" aria-label="Twój postęp">

        <div class="karta-stat"><span class="karta-stat__liczba">{{ zapis.total_hours_logged }}</span><span
class="karta-stat__etykieta">Zalogowanych godzin</span></div>

        <div class="karta-stat"><span class="karta-stat__liczba">{{ zapis.praktyka.wymiar_godzin }}</span><span
class="karta-stat__etykieta">Wymaganych godzin</span></div>

        <div class="karta-stat"><span class="karta-stat__liczba">{{ zapis.wpisy_dziennika | length }}</span><span
class="karta-stat__etykieta">Wpisów w dzienniku</span></div>

        <div class="karta-stat">{% set procent = 0 if not zapis.praktyka.wymiar_godzin else [(zapis.total_hours_logged /
zapis.praktyka.wymiar_godzin * 100)|round|int, 100]|min %}<span class="karta-stat__liczba">{{ procent }}%</span><span
class="karta-stat__etykieta">Ukończono</span></div>

    </section>

    <div class="karta" style="margin-bottom:2rem;">
```

```

<div class="karta__tresc">

    <div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:0.75rem;">

        <span style="font-weight:600;">Postęp praktyki – {{ zapis.praktyka.rok_uczelniany }} ({{ zapis.praktyka.semestr
        }})</span>

    <div style="display:flex; align-items:center; gap:0.75rem;">

        <span class="status

            {%- if zapis.status.value == 'PENDING' %} status--draft

            {%- elif zapis.status.value == 'AWAITING_APPROVAL' %} status--planned

            {%- elif zapis.status.value == 'COMMISSION_REVIEW' %} status--planned

            {%- elif zapis.status.value == 'DEAN_APPROVAL' %} status--planned

            {%- elif zapis.status.value == 'IN_PROGRESS' %} status--w-trakcie

            {%- elif zapis.status.value == 'COMPLETED' %} status--zakonczone

            {%- elif zapis.status.value == 'REJECTED' %} status--draft

            {%- endif %}">

            {% if zapis.status.value == 'PENDING' %}Szkic

            {% elif zapis.status.value == 'AWAITING_APPROVAL' %}Oczekuje na zatwierdzenie

            {% elif zapis.status.value == 'COMMISSION_REVIEW' %}Weryfikacja komisji

            {% elif zapis.status.value == 'DEAN_APPROVAL' %}Oczekuje na dziekana

            {% elif zapis.status.value == 'IN_PROGRESS' %}W trakcie

            {% elif zapis.status.value == 'COMPLETED' %}Zakończona

            {% elif zapis.status.value == 'REJECTED' %}Odrzucony

            {% endif %}

        </span>

        <span style="font-size:0.85rem; color:var(--kolor-tekst-drugor);">{{ zapis.total_hours_logged }} / {{
        zapis.praktyka.wymiar_godzin }} h</span>

    </div>

</div>

<div class="pasek-postep" style="height:12px;">

    <div class="pasek-postep__wypelnienie" style="width:{{ procent }}%;"></div>

</div>

{% if zapis.komentarze_uopz %}

    <div style="margin-top:1rem; padding:0.75rem; background:#fef3cd; border-radius:0.375rem; border-left:4px solid
    #f59e0b;">

        <div style="font-weight:500; color:#92400e; margin-bottom:0.5rem;">Uwagi UOPZ:</div>

        <div style="color:#92400e; font-size:0.9rem; white-space: pre-wrap;">{{ zapis.komentarze_uopz }}</div>

        <div style="margin-top:0.75rem;">

```

```
        <a href="{{ url_for('praktyki.szczegoly_zgloszenia', id=zapis.id) }}" class="przycisk przycisk--maly
przycisk--drugorzeczny">Zobacz szczegóły zgłoszenia</a>
```

```
    </div>
```

```
</div>
```

```
{% endif %}
```

```
</div>
```

```
</div>
```

```
<div style="display:flex; gap:1rem; flex-wrap:wrap; justify-content:center; align-items:center; margin-bottom: 2rem;">
```

```
    <a href="{{ url_for('dziennik.nowy_wpis') }}" class="przycisk przycisk--glowny">Dodaj wpis do dziennika</a>
```

```
    <a href="{{ url_for('dziennik.index') }}" class="przycisk przycisk--drugorzeczny">Przeglądaj dziennik</a>
```

```
</div>
```

```
{% endif %}
```

```
{% endblock %}
```

PLIK: app_student\templates\dokumenty\moje_dokumenty.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Moje Dokumenty{% endblock %}

{% block naglowek %}Moje Dokumenty{% endblock %}


{% block tresc %}

<div style="max-width: 900px; margin: 0 auto;">


    {% if not dokumenty_list %}

    <div class="karta">

        <div class="karta__tresc" style="text-align: center; padding: 3rem 2rem;">

            <div style="font-size: 3rem; margin-bottom: 1rem; opacity: 0.3;">

                <svg viewBox="0 0 24 24" style="width: 3rem; height: 3rem; fill: none; stroke: currentColor; stroke-width: 2;">

                    <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/>

                    <polyline points="14 2 14 8 20 8"/>

                </svg>

            </div>

            <div style="font-size: 1.1rem; font-weight: 600; margin-bottom: 0.5rem;">Brak dostępnych dokumentów</div>

            <div style="color: var(--kolor-tekst-drugor);">Dokumenty pojawią się tutaj po zatwierdzeniu praktyki przez
            opiekuna.</div>

        </div>

    </div>

    {% endif %}


    {% for wpis in dokumenty_list %}

    <div class="karta" style="margin-bottom: 1.5rem;">

        <div class="karta__naglowek">

            <div>

                <div style="font-weight: 600;">{{ wpis.zapis.praktyka.rok_uczelnianny }} · Semestr {{ wpis.zapis.praktyka.semestr
                }}</div>

                <div style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-top: 0.25rem;">

                    {% if wpis.zapis.firma %}{{ wpis.zapis.firma.nazwa }}{% elif wpis.zapis.firma_nazwa %}{{ wpis.zapis.firma_nazwa
                    }}{% endif %}

                    · Ścieżka:

                    {% if wpis.sciezka == 'STANDARD' %}Standardowa
```

```

        {% elif wpis.sciezka == 'EMPLOYMENT' %}Praca etatowa

        {% else %}Działalność gospodarcza{% endif %}

    </div>

</div>

    <span class="status {% if wpis.zapis.status.value == 'COMPLETED' %}status--zakonczone{% else %}status--aktywna{%
endif %}">

        {% if wpis.zapis.status.value == 'COMPLETED' %}Zakończona{% else %}W trakcie{% endif %}

    </span>

</div>

<div class="karta__tresc">

    <div style="display: grid; gap: 0.6rem;">

        {% for doc in wpis.docs %}

            {% if doc.separator is defined and doc.separator %}

                <div style="display: flex; align-items: center; gap: 0.75rem; margin: 0.5rem 0;">

                    <hr style="flex: 1; border: none; border-top: 1px dashed #d1d5db;">

                    <span style="font-size: 0.78rem; color: var(--kolor-tekst-drugor); white-space: nowrap; font-weight: 500;">{{
doc.nazwa }}</span>

                    <hr style="flex: 1; border: none; border-top: 1px dashed #d1d5db;">

                </div>

            {% else %}

                <div style="display: flex; justify-content: space-between; align-items: center; padding: 0.75rem 1rem;

                    background: {% if doc.dostepny %}#f8fafc{% else %}#f3f4f6{% endif %};

                    border: 1px solid {% if doc.dostepny %}#e2e8f0{% else %}#d1d5db{% endif %};

                    border-radius: 0.5rem;">

                    <div>

                        <div style="font-weight: 500; font-size: 0.9rem; {% if not doc.dostepny %}color: var(--kolor-tekst-drugor);{%
endif %}">

                            {{ doc.nazwa }}

                        </div>

                        {% if doc.opis %}

                            <div style="font-size: 0.78rem; color: var(--kolor-tekst-drugor);">{{ doc.opis }}</div>

                        {% endif %}

                        {% if not doc.dostepny and doc.powod %}

                            <div style="font-size: 0.78rem; color: #d97706; margin-top: 0.15rem;">{{ doc.powod }}</div>

                        {% endif %}

                    </div>

```



```
{% if doc.dostepny %}

    {% if doc.staly is defined and doc.staly %}

        <a href="{{ url_for('documents.pobierz_staly', doc_key=doc.klucz_staly) }}"

            class="przycisk przycisk--drugorzeczny przycisk--maly">Pobierz PDF</a>

    {% elif doc.dynamiczny is defined and doc.dynamiczny %}

        <a href="{{ url_for('documents.pobierz_dynamiczny', zapis_id=doc.zapis_id, typ=doc.typ) }}"

            class="przycisk przycisk--drugorzeczny przycisk--maly">Pobierz PDF</a>

    {% endif %}

    {% else %}

        <span style="font-size: 0.78rem; color: var(--kolor-tekst-drugor); padding: 0.3rem 0.625rem; white-space:
nowrap;">Niedostępny</span>

    {% endif %}

</div>

{% endif %}

{% endfor %}

</div>

</div>

</div>

{% endfor %}

</div>

{% endblock %}
```

PLIK: app_student\templates\dokumenty\panel.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Panel Dokumentów{% endblock %}

{% block naglowek %}Panel Dokumentów do Wydrukowania{% endblock %}

{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem;">

  <a href="{% url_for('dashboard.index') %}" class="przycisk przycisk--drugorzeczny">← Powrót do pulpitu</a>

</div>

{% endblock %}

{% block tresc %}

<div style="max-width: 1200px; margin: 0 auto;">

  <!-- Informacja o systemie -->

  <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-info);">

    <div class="karta__tresc">

      <div>

        <div style="font-weight: 600; margin-bottom: 0.5rem;">System Generowania Dokumentów</div>

        <div style="color: var(--kolor-tekst-drugor); font-size: 0.9rem;">

          Dokumenty generują się automatycznie na podstawie danych z Twojego zgłoszenia praktyki.

          <strong>Wydrukuj, podpisz i dostarcz do firmy/uczelni przed rozpoczęciem praktyki.</strong>

        </div>

      </div>

    </div>

  </div>

  <div>

    <div style="font-size: 3rem; margin-bottom: 1rem; opacity: 0.3;"></div>

    <h3 style="margin-bottom: 0.5rem;">Brak praktyk do dokumentacji</h3>

    <p style="color: var(--kolor-tekst-drugor); margin-bottom: 2rem;">

      Najpierw zapisz się na praktykę i uzupełnij harmonogram.

    </p>

  </div>

</div>

{% endblock %}
```

</p>

Zobacz dostępne praktyki

</div>

{% else %}

<!-- Lista praktyk z dokumentami -->

{% for item in dokumenty_data %}

<div class="karta" style="margin-bottom: 2rem;">

<div class="karta__naglowek" style="background: var(--kolor-tlo);">

<div style="display: flex; justify-content: space-between; align-items: center;">

<h3 class="karta__tytul">

{{ item.zapis.praktyka.rok_uczelniany }} ({{ item.zapis.praktyka.semestr }})

</h3>

{{ tлумacz_status(item.zapis.status.value) }}

</div>

</div>

<div class="karta__tresc">

<div style="margin-bottom: 1.5rem;">

<div style="font-size: 0.9rem; color: var(--kolor-tekst-drugor);">

Firma: {{ item.zapis.firma_nazwa or 'Nie podano' }} |

Okres:

{% if item.zapis.termin_od and item.zapis.termin_do %}

{{ item.zapis.termin_od.strftime('%d.%m.%Y') }} - {{ item.zapis.termin_do.strftime('%d.%m.%Y') }}

{% else %}

Nie określono

{% endif %}

</div>

</div>

```

{% if not item.harmonogram_gotowy %}

<!-- Harmonogram nie gotowy -->

    <div style="padding: 1rem; background: #fef3cd; border-radius: 0.5rem; border-left: 4px solid #f59e0b;
margin-bottom: 1.5rem;">

        <div style="font-weight: 500; color: #92400e; margin-bottom: 0.5rem;">

            Harmonogram nie został uzupełniony

        </div>

        <div style="color: #92400e; font-size: 0.9rem; margin-bottom: 1rem;">

            Przed generowaniem dokumentów musisz uzupełnić harmonogram praktyki (krok 2).

        </div>

        <a href="{{ url_for('praktyki.zapisz_krok2', id=item.zapis.id) }}" class="przycisk przycisk--maly
przycisk--glowny">

            Uzupełnij harmonogram

        </a>

    </div>

{% elif not item.mozna_generowac %}

<!-- Status nie pozwala na generowanie -->

<div style="padding: 1rem; background: #f3f4f6; border-radius: 0.5rem; margin-bottom: 1.5rem;">

    <div style="color: var(--kolor-tekst-drugor); font-size: 0.9rem;">

        Dokumenty będą dostępne po zatwierdzeniu zgłoszenia przez UOPZ.

    </div>

</div>

{% else %}

<!-- Dokumenty dostępne do generacji -->

<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 1rem;">

    <!-- Załącznik 1 - Podanie -->

    <div class="dokument-karta">

        <div class="dokument-karta__naglowek">

            <h4>Załącznik 1</h4>

            <span class="dokument-status dokument-status--dostepny">Dostępny</span>

        </div>

```

```

<div class="dokument-karta__tresc">

    <div class="dokument-tytul">Podanie o realizację praktyki studenckiej</div>

    <div class="dokument-opis">Dane studenta, firmy, planowane terminy</div>

    <a href="{{ url_for('dokumenty.generuj_dokument', enrollment_id=item.zapis.id, doc_type='ZALACZNIK_1') }}"
        class="przycisk przycisk--maly przycisk--glowny">

        Pobierz PDF

    </a>

</div>

</div>

<!-- Załącznik 2 - Skierowanie -->

<div class="dokument-karta">

    <div class="dokument-karta__naglowek">

        <h4>Załącznik 2</h4>

        <span class="dokument-status dokument-status--dostepny">Dostępny</span>

    </div>

    <div class="dokument-karta__tresc">

        <div class="dokument-tytul">Skierowanie na praktykę studencką</div>

        <div class="dokument-opis">Z automatycznym numerem pisma wychodzącego</div>

        <a href="{{ url_for('dokumenty.generuj_dokument', enrollment_id=item.zapis.id, doc_type='ZALACZNIK_2') }}"
            class="przycisk przycisk--maly przycisk--glowny">

            Pobierz PDF

        </a>

    </div>

</div>

<!-- Załącznik 3 - Porozumienie (warunkowo) -->

{% if not item.firma_ma_umowe %}

<div class="dokument-karta">

    <div class="dokument-karta__naglowek">

        <h4>Załącznik 3</h4>

        <span class="dokument-status dokument-status--dostepny">Dostępny</span>

    </div>

    <div class="dokument-karta__tresc">

```

```

<div class="dokument-tytul">Porozumienie o organizację praktyki</div>

<div class="dokument-opis">Umowa z firmą (2 egzemplarze)</div>

<a href="{{ url_for('dokumenty.generuj_dokument', enrollment_id=item.zapis.id, doc_type='ZALACZNIK_3') }}"
    class="przycisk przycisk--maly przycisk--glowny">

    Pobierz PDF

</a>

</div>

</div>

{% else %}

<div class="dokument-karta dokument-karta--nieaktywny">

    <div class="dokument-karta__naglowek">

        <h4>Załącznik 3</h4>

        <span class="dokument-status dokument-status--niewymagany">Niewymagany</span>

    </div>

    <div class="dokument-karta__tresc">

        <div class="dokument-tytul">Porozumienie o organizację praktyki</div>

        <div class="dokument-opis">Firma ma stałą umowę z uczelnią</div>

    </div>

</div>

{% endif %}

<!-- Załącznik 4 - Indywidualny Program -->

<div class="dokument-karta {% if not item.program_approved %}dokument-karta--oczekuje{% endif %}">

    <div class="dokument-karta__naglowek">

        <h4>Załącznik 4</h4>

        <span class="dokument-status dokument-status--{% if item.program_approved %}dostepny{% else %}oczekuje{%
endif %}">

            {% if item.program_approved %}Zatwierdzony{% else %}Wymaga zatwierdzenia{% endif %}

        </span>

    </div>

    <div class="dokument-karta__tresc">

        <div class="dokument-tytul">Indywidualny Program Praktyk</div>

        <div class="dokument-opis">Na podstawie Twojego harmonogramu</div>

        {% if item.program_approved %}

```

```

        <a href="{{ url_for('dokumenty.generuj_dokument', enrollment_id=item.zapis.id, doc_type='ZALACZNIK_4') }}"
            class="przycisk przycisk--maly przycisk--glowny">

            Pobierz PDF

        </a>

        {% else %}

            <form method="POST" action="{{ url_for('dokumenty.zatwierdz_program', enrollment_id=item.zapis.id) }}"
                style="margin-top: 0.5rem;">

                {{ csrf_form.hidden_tag() }}

                <button type="submit" class="przycisk przycisk--maly przycisk--drugorzedny">

                    Poproś o zatwierdzenie

                </button>

            </form>

        {% endif %}

    </div>

</div>

</div>

{% endif %}

</div>

</div>

{% endfor %}

{% endif %}

</div>

<style>

.dokument-karta {

    border: 1px solid var(--kolor-ramka);

    border-radius: var(--promien);

    overflow: hidden;

    background: white;

}

.dokument-karta--nieaktywny {

    opacity: 0.6;

```

```
background: #f9f9f9;

}

.dokument-karta--oczekuje {

  border-color: var(--kolor-ostrezenie);

}

.dokument-karta__naglowek {

  padding: 0.75rem 1rem;

  background: var(--kolor-tlo);

  display: flex;

  justify-content: space-between;

  align-items: center;

  border-bottom: 1px solid var(--kolor-ramka);

}

.dokument-karta__naglowek h4 {

  margin: 0;

  font-size: 0.9rem;

  color: var(--kolor-glowny);

}

.dokument-karta__tresc {

  padding: 1rem;

}

.dokument-tytul {

  font-weight: 600;

  margin-bottom: 0.5rem;

  font-size: 0.9rem;

}

.dokument-opis {

  font-size: 0.8rem;
```



```
    color: var(--kolor-tekst-drugor);

    margin-bottom: 1rem;

}
```

```
.dokument-status {

    padding: 0.25rem 0.5rem;

    border-radius: 3px;

    font-size: 0.7rem;

    font-weight: 500;

}
```

```
.dokument-status--dostepny {

    background: #d1fae5;

    color: #065f46;

}
```

```
.dokument-status--oczekuje {

    background: #fef3cd;

    color: #92400e;

}
```

```
.dokument-status--niewymagany {

    background: #e5e7eb;

    color: #6b7280;

}
```

```
</style>
```

```
{% endblock %}
```

PLIK: app_student\templates\dziennik\index.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Dziennik praktyki{% endblock %}

{% block naglowek %}Dziennik praktyki{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem; align-items: center;">

    {% if zapis %}

        <a href="{% url_for('dziennik.nowy_wpis') %}" class="przycisk przycisk--glowny">Nowy wpis</a>

        <a href="{% url_for('documents.moje_dokumenty') %}" class="przycisk przycisk--drugorzeczny">Pobierz PDF (Moje Dokumenty)</a>

    {% endif %}

</div>

{% endblock %}


{% block tresc %}


{% if not zapis %}

    <div style="display: flex; justify-content: center; align-items: center; padding-top: 4rem;">

        <div class="karta" style="max-width: 560px; text-align: center; padding: 3rem 2rem; width: 100%;">

            {% if jakikolwiek %}

                <p style="color: var(--kolor-tekst-drugor);">Twój zapis oczekuje na aktywację przez opiekuna.</p>

            {% else %}

                <p style="color: var(--kolor-tekst-drugor);">Najpierw zapisz się na praktykę.</p>

                <a href="{% url_for('praktyki.lista') %}" class="przycisk przycisk--glowny" style="margin-top: 1rem;">Zobacz praktyki</a>

            {% endif %}

        </div>

    </div>

{% else %}

    <div class="karta" style="margin-bottom: 1.5rem; border-left: 4px solid var(--kolor-glowny);">

        <div class="karta__tresc">

            <div style="display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 1.5rem;">
```

```

<div>

    <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Praktyka</div>

    <div style="font-weight: 600;">{{ zapis.praktyka.rok_uczelnianny }}</div>

    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Semestr {{ zapis.praktyka.semestr }}</div>

</div>

<div>

    <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Wpisy</div>

    <div style="font-weight: 600; font-size: 1.2rem;">{{ wpisy | length }}</div>

</div>

<div>

    <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Godziny</div>

    <div style="font-weight: 700; font-size: 1.2rem; color: var(--kolor-glowny);">{{ zapis.total_hours_logged }} / {{ zapis.praktyka.wymiar_godzin }} h</div>

    <div class="pasek-postep" style="margin-top: 0.5rem; margin-bottom: 1rem;">

        {% set proc_h = 0 %}

        {% if zapis and zapis.praktyka and zapis.praktyka.wymiar_godzin > 0 %}

            {% set proc_h = [(zapis.total_hours_logged / zapis.praktyka.wymiar_godzin * 100)|int, 100]|min %}

            {% endif %}

        <div class="pasek-postep__wypelnienie{% if proc_h >= 100 %} pasek-postep__wypelnienie--zakonczone{% endif %}" style="width: {{ proc_h }}%;"></div>

    </div>

    <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Dni</div>

    <div style="font-weight: 700; font-size: 1.2rem; color: var(--kolor-glowny);">{{ wpisy | length }} / 120 dni</div>

    <div class="pasek-postep" style="margin-top: 0.5rem;">

        {% set proc_d = [(wpisy | length / 120 * 100)|int, 100]|min %}

        <div class="pasek-postep__wypelnienie{% if proc_d >= 100 %} pasek-postep__wypelnienie--zakonczone{% endif %}" style="width: {{ proc_d }}%;"></div>

    </div>

</div>

</div>

</div>

```

```

<div class="karta">

  <div class="karta__naglowek"><h2 class="karta__tytul">Wpisy dziennika</h2></div>

  <div class="tabela-wrapper">

    <table class="tabela">

      <thead>

        <tr>

          <th>Data</th>

          <th>Godz.</th>

          <th>Opis czynności</th>

          <th>Efekty</th>

          <th></th>

        </tr>

      </thead>

      <tbody>

        {% for w in wpisy %}

        <tr>

          <td style="white-space: nowrap;"><strong>{{ w.entry_date.strftime('%d.%m.%Y') }}</strong></td>

          <td style="white-space: nowrap;">{{ w.duration_hours }} h</td>

          <td style="font-size: 0.85rem; line-height: 1.5;">{{ w.description }}</td>

          <td style="white-space: nowrap;">

            {% if w.efekty_uczenia %}

            {% for e in w.efekty_uczenia %}

              <span style="font-size:0.75rem; background:var(--kolor-tlo); padding:0.2rem 0.4rem; border-radius:4px; font-weight:600; display:inline-block; margin:1px;">{{ e.kod }}</span>

            {% endfor %}

            {% else %}<!--{% endif %}

          </td>

          <td style="white-space: nowrap;">

            <a href="{{ url_for('dziennik.edytuj_wpis', wpis_id=w.id) }}" class="przycisk przycisk--maly przycisk--drugorzeczny">Edytuj</a>

            <form method="POST" action="{{ url_for('dziennik.usun_wpis', wpis_id=w.id) }}" style="display:inline;">

              {{ csrf_form.hidden_tag() }}

              <button type="submit" class="przycisk przycisk--maly przycisk--niebezpieczny"

                onclick="return confirm('Usunąć ten wpis?')">Usuń</button>

            </td>

          </tr>

        {% endfor %}

      </tbody>

    </table>

  </div>

</div>

```

```
</form>
```

```
</td>
```

```
</tr>
```

```
{% else %}
```

```
<tr><td colspan="5" class="tabela--pusta">Brak wpisów. Dodaj pierwszy wpis.</td></tr>
```

```
{% endfor %}
```

```
</tbody>
```

```
</table>
```

```
</div>
```

```
</div>
```

```
{% endif %}
```

```
{% endblock %}
```

PLIK: app_student\templates\dziennik\nowy_wpis.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}{% if edycja %}Edytuj wpis{% else %}Nowy wpis{% endif %}{% endblock %}

{% block naglowek %}{% if edycja %}Edytuj wpis z {{ wpis.entry_date.strftime('%d.%m.%Y') }}{% else %}Nowy wpis do
dziennika{% endif %}{% endblock %}


{% block naglowek_akcje %}

<a href="{{ url_for('dziennik.index') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

{% endblock %}


{% block tresc %}

<div style="max-width: 640px; margin: 0 auto;">


<!-- Info o postępie -->

<div class="karta" style="margin-bottom: 1.5rem; border-left: 4px solid var(--kolor-glowny);">

    <div class="karta__tresc" style="display: flex; justify-content: space-between; align-items: center;">

        <span style="font-weight: 600;">{{ zapis.praktyka.rok_uczelniany }} – {{ zapis.praktyka.semestr }}</span>

        <span style="font-size: 0.9rem; color: var(--kolor-tekst-drugor);">

            {{ zapis.total_hours_logged }} / {{ zapis.praktyka.wymiar_godzin }} h

        </span>

    </div>

</div>

</div>


<div class="karta">

    <div class="karta__naglowek">

        <h2 class="karta__tytul">Dane wpisu</h2>

    </div>

    <div class="karta__tresc">

        <form method="POST" action="{% if edycja %}{{ url_for('dziennik.edytuj_wpis', wpis_id=wpis.id) }}{% else %}{{
url_for('dziennik.nowy_wpis') }}{% endif %}" novalidate>

            {{ form.hidden_tag() }}


            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem; margin-bottom: 1rem;">


                <div class="pole-formularza {% if form.data_wpisu.errors %}pole-formularza--blad{% endif %}">
```

```

    {{ form.data_wpisu.label(class="pole-formularza__etykieta") }}

    {{ form.data_wpisu(class="pole-formularza__input", type="date") }}

    {% for e in form.data_wpisu.errors %}

        <span class="pole-formularza__blad">{{ e }}</span>

    {% endfor %}

</div>

```

```

<div class="pole-formularza {% if form.liczba_godzin.errors %}pole-formularza--blad{% endif %}">

    {{ form.liczba_godzin.label(class="pole-formularza__etykieta") }}

    {{ form.liczba_godzin(class="pole-formularza__input", type="number", min="1", max="8", placeholder="1-8") }}

    {% for e in form.liczba_godzin.errors %}

        <span class="pole-formularza__blad">{{ e }}</span>

    {% endfor %}

</div>

```

```

</div>

```

```

    <div class="pole-formularza {% if form.efekty_ids.errors %}pole-formularza--blad{% endif %}"
style="margin-bottom: 1rem;">

```

```

    {{ form.efekty_ids.label(class="pole-formularza__etykieta") }}

```

```

    <div style="border: 1px solid var(--kolor-ramka, #d1d5db); border-radius: 6px; padding: 0.75rem; max-height:
220px; overflow-y: auto; display: flex; flex-direction: column; gap: 0.5rem;">

```

```

    {% for subfield in form.efekty_ids %}

```

```

        <label style="display: flex; align-items: flex-start; gap: 0.5rem; cursor: pointer; font-size: 0.875rem;
line-height: 1.4;">

```

```

            {{ subfield(style="margin-top: 2px; flex-shrink: 0;") }}

```

```

            <span>{{ subfield.label.text }}</span>

```

```

        </label>

```

```

    {% endfor %}

```

```

</div>

```

```

{% for e in form.efekty_ids.errors %}

```

```

    <span class="pole-formularza__blad">{{ e }}</span>

```

```

{% endfor %}

```

```

</div>

```

```

<div class="pole-formularza {% if form.opis.errors %}pole-formularza--blad{% endif %}" style="margin-bottom:

```

```

1.5rem;">

    {{ form.opis.label(class="pole-formularza__etykieta") }}

    {{ form.opis(class="pole-formularza__input", rows="5", placeholder="Opisz szczegółowo wykonane czynności...")
}}

<span style="font-size: 0.78rem; color: var(--kolor-tekst-drugor);">

    Opis powinien być szczegółowy – będzie widoczny w Dzienniku Praktyki (Załącznik 6).

</span>

{% for e in form.opis.errors %}

    <span class="pole-formularza__blad">{{ e }}</span>

{% endfor %}

</div>


<div style="display: flex; gap: 0.5rem; justify-content: center;">

    <button type="submit" class="przycisk przycisk--glowny">{% if edycja %}Zapisz zmiany{% else %}Dodaj wpis{%
endif %}</button>

    <a href="{{ url_for('dziennik.index') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

</div>


</form>

</div>

</div>

</div>

{% endblock %}

```


PLIK: app_student\templates\kreator\krok1_sciezka.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Kreator praktyki – Krok 1{% endblock %}

{% block naglowek %}Rejestracja praktyki zawodowej{% endblock %}

{% block naglowek_akcje %}

    <a href="{{ url_for('praktyki.lista') }}" class="przycisk przycisk--drugorzeczny">< Wróć</a>

{% endblock %}


{% block tresc %}

<div style="max-width: 800px; margin: 0 auto;">


<!-- Pasek -->

<div style="display:flex; gap:0.5rem; margin-bottom:2rem; font-size:0.82rem;">

    <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#3b82f6; color:white;
border-radius:0.375rem; font-weight:600;">1. Ścieżka</div>

    <div style="color:#d1d5db; align-self:center;"></div>

    <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#f3f4f6; color:#9ca3af;
border-radius:0.375rem;">2. Miejsce / Wniosek</div>

    <div style="color:#d1d5db; align-self:center;"></div>

    <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#f3f4f6; color:#9ca3af;
border-radius:0.375rem;">3. Harmonogram</div>

    <div style="color:#d1d5db; align-self:center;"></div>

    <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#f3f4f6; color:#9ca3af;
border-radius:0.375rem;">4. Realizacja</div>

</div>


<div class="karta">

    <div class="karta__naglowek">

        <h2 class="karta__tytul">Krok 1 – Wybierz ścieżkę praktyki</h2>

    </div>

    <div class="karta__tresc">

        <form method="POST" id="form-sciezka">

            {{ form.hidden_tag() }}

            <input type="hidden" name="track_type" id="selected_track" value="{{ form.track_type.data or '' }}">

            <div style="display:grid; grid-template-columns:1fr 1fr 1fr; gap:1rem; margin-bottom:1.5rem;">
```

<!-- Kafelek A -->

<div class="kafelek-sciezki" data-value="STANDARD" onclick="selectSciezka(this)"

style="border:2px solid {% if form.track_type.data == 'STANDARD' %}#3b82f6{% else %}#e2e8f0{% endif %};

background:{% if form.track_type.data == 'STANDARD' %}#eff6ff{% else %}#fafafa{% endif %};

border-radius:0.75rem; padding:1.25rem; cursor:pointer; transition:all 0.15s;">

<div style="font-weight:700; font-size:0.95rem; margin-bottom:0.4rem;">[A] Standardowa praktyka</div>

<div style="font-size:0.8rem; color:var(--kolor-tekst-drugor); line-height:1.4;">

Odbędę praktykę w firmie lub instytucji na podstawie porozumienia z uczelnią.

</div>

<div style="margin-top:0.75rem; font-size:0.78rem; color:#6b7280;">

4 dokumenty · 120 dni roboczych

</div>

</div>

<!-- Kafelek B -->

<div class="kafelek-sciezki" data-value="EMPLOYMENT" onclick="selectSciezka(this)"

style="border:2px solid {% if form.track_type.data == 'EMPLOYMENT' %}#3b82f6{% else %}#e2e8f0{% endif %};

background:{% if form.track_type.data == 'EMPLOYMENT' %}#eff6ff{% else %}#fafafa{% endif %};

border-radius:0.75rem; padding:1.25rem; cursor:pointer; transition:all 0.15s;">

<div style="font-weight:700; font-size:0.95rem; margin-bottom:0.4rem;">[B] Praca etatowa / staż</div>

<div style="font-size:0.8rem; color:var(--kolor-tekst-drugor); line-height:1.4;">

Pracuję lub pracowałem/am w branży IT w ciągu ostatnich 3 lat i chcę zaliczyć to jako praktykę.

</div>

<div style="margin-top:0.75rem; font-size:0.78rem; color:#6b7280;">

Wniosek + decyzja komisji

</div>

</div>

<!-- Kafelek C -->

<div class="kafelek-sciezki" data-value="OWN_BUSINESS" onclick="selectSciezka(this)"

style="border:2px solid {% if form.track_type.data == 'OWN_BUSINESS' %}#3b82f6{% else %}#e2e8f0{% endif

%};

background:{% if form.track_type.data == 'OWN_BUSINESS' %}#eff6ff{% else %}#fafafa{% endif %};

border-radius:0.75rem; padding:1.25rem; cursor:pointer; transition:all 0.15s;">

```
<div style="font-weight:700; font-size:0.95rem; margin-bottom:0.4rem;">[C] Własna działalność</div>
```

```
<div style="font-size:0.8rem; color:var(--kolor-tekst-drugor); line-height:1.4;">
```

```
    Prowadzę firmę informatyczną (CEIDG/KRS) i chcę zaliczyć działalność jako praktykę.
```

```
</div>
```

```
<div style="margin-top:0.75rem; font-size:0.78rem; color:#6b7280;">
```

```
    Wniosek + decyzja Dyrektora IIS
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<!-- Podgląd wybranej ścieżki -->
```

```
<div id="podglad-sciezki" style="display:none; padding:1rem; border-radius:0.5rem; background:#f0fdf4; border:1px solid #a7f3d0; margin-bottom:1.5rem; font-size:0.875rem;">
```

```
</div>
```

```
{% if form.track_type.errors %}
```

```
<div style="color:#dc2626; font-size:0.85rem; margin-bottom:1rem;">Wybierz ścieżkę praktyki.</div>
```

```
{% endif %}
```

```
<div style="display:flex; justify-content:flex-end;">
```

```
    <button type="submit" class="przycisk przycisk--glowny" id="btn-dalej" {% if not form.track_type.data %}disabled style="opacity:0.5;"{% endif %}>
```

```
        Dalej →
```

```
</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
<!-- Info praktyki -->
```

```
<div style="margin-top:1rem; font-size:0.82rem; color:var(--kolor-tekst-drugor); text-align:center;">
```

```
    Praktyka: <strong>{{ praktyka.rok_uczelniany }}</strong> · Semestr {{ praktyka.semestr }} · {{ praktyka.wymiar_godzin }} h
```

```
</div>
```

```
</div>
```

```

<style>

.kafelek-sciezki:hover { border-color:#3b82f6 !important; background:#eff6ff !important; }

.kafelek-sciezki.aktywny { border-color:#2563eb !important; background:#dbeafe !important; }

</style>


<script>

const podglady = {

    STANDARD:      'Odbędziesz 120 dni roboczych w zakładzie pracy. Potrzebujesz: Porozumienie (zał.1), Program (zał.2), Indywidualny Program (zał.2a), Kartę praktyki (zał.3). Po zakończeniu: Dziennik, Sprawozdanie, Potwierdzenie efektów.',

    EMPLOYMENT:     'Złożysz wniosek o zaliczenie zatrudnienia (zał.4b) z dokumentami od pracodawcy. Komisja podejmie decyzję. Potrzebujesz: wniosek + skan umowy/zakres obowiązków.',

    OWN_BUSINESS:   'Złożysz wniosek o zaliczenie działalności (zał.4b) z wypisem z CEIDG/KRS. Dyrektor IIS podejmie decyzję.'

};


function selectSciezka(el) {

    document.querySelectorAll('.kafelek-sciezki').forEach(k => {

        k.style.borderColor = '#e2e8f0';

        k.style.background = '#fafafa';

    });

    el.style.borderColor = '#2563eb';

    el.style.background = '#dbeafe';


    const val = el.dataset.value;

    document.getElementById('selected_track').value = val;

    const box = document.getElementById('podglad-sciezki');

    box.textContent = podglady[val];

    box.style.display = 'block';


    const btn = document.getElementById('btn-dalej');

    btn.disabled = false;

    btn.style.opacity = '1';

}


// Inicjalizacja jeśli jest wybrana ścieżka

```

```
const cur = document.getElementById('selected_track').value;

if (cur) {

    const el = document.querySelector(`[data-value="${cur}"]`);

    if (el) selectSciezka(el);

}

</script>

{% endblock %}
```

PLIK: app_student\templates\kreator\krok2a_firma.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Kreator – Krok 2: Miejsce praktyki{% endblock %}

{% block naglowek %}Rejestracja praktyki zawodowej{% endblock %}


{% block tresc %}

<div style="max-width: 800px; margin: 0 auto;">


    <!-- Pasek -->

    <div style="display:flex; gap:0.5rem; margin-bottom:2rem; font-size:0.82rem;">

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#d1fae5; color:#065f46; border-radius:0.375rem;">Ścieżka A</div>

        <div style="color:#d1d5db; align-self:center;">></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#3b82f6; color:white; border-radius:0.375rem; font-weight:600;">2. Miejsce praktyki</div>

        <div style="color:#d1d5db; align-self:center;">></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#f3f4f6; color:#9ca3af; border-radius:0.375rem;">3. Harmonogram</div>

        <div style="color:#d1d5db; align-self:center;">></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#f3f4f6; color:#9ca3af; border-radius:0.375rem;">4. Realizacja</div>

    </div>


<form method="POST" novalidate>

    {{ form.hidden_tag() }}


    <!-- === SEKCJA 1: Terminy i UOPZ === -->

    <div class="karta" style="margin-bottom:1rem;">

        <div class="karta__naglowek"><h2 class="karta__tytul">Terminy i opiekun uczelniany</h2></div>

        <div class="karta__tresc">

            <div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem; margin-bottom:1rem;">

                <div class="pole-formularza {% if form.termin_od.errors %}pole-formularza--blad{% endif %}">

                    {{ form.termin_od.label(class="pole-formularza__etykieta") }}

                    {{ form.termin_od(class="pole-formularza__input", type="date") }}

                    {% for e in form.termin_od.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

                </div>

            </div>

        </div>

    </div>

</form>
```

```

<div class="pole-formularza {% if form.termin_do.errors %}pole-formularza--blad{% endif %}">

    {{ form.termin_do.label(class="pole-formularza__etykieta") }}

    {{ form.termin_do(class="pole-formularza__input", type="date") }}

    {% for e in form.termin_do.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

</div>

</div>

<div class="pole-formularza">

    {{ form.uopz_id.label(class="pole-formularza__etykieta") }}

    {{ form.uopz_id(class="filtr-select", style="width:100%;") }}

</div>

<!-- Ubezpieczenie -->

    <div style="background:#fef3cd; border:1px solid #fde68a; border-radius:0.5rem; padding:0.875rem;
margin-top:1rem;">

        <label style="display:flex; gap:0.75rem; align-items:flex-start; cursor:pointer;">

            {{ form.ubezpieczenie_nw(style="margin-top:0.15rem; width:16px; height:16px; flex-shrink:0;") }}

            <span style="font-weight:500; font-size:0.9rem;">{{ form.ubezpieczenie_nw.label.text }}</span>

        </label>

        <div style="font-size:0.78rem; color:#92400e; margin-top:0.3rem; margin-left:22px;">Wymagane przed rozpoczęciem
praktyki (art. 304 KP)</div>

    </div>

</div>

</div>

<!-- === SEKCJA 2: Tryb znalezienia miejsca === -->

<div class="karta" style="margin-bottom:1rem;">

    <div class="karta__naglowek"><h2 class="karta__tytul">Zakład pracy</h2></div>

    <div class="karta__tresc">

        <div class="pole-formularza" style="margin-bottom:1rem;">

            {{ form.firma_typ.label(class="pole-formularza__etykieta") }}

            {{ form.firma_typ(class="filtr-select", style="width:100%; id="firma_typ") }}

        </div>

        <!-- Tryb: firma z bazy -->

        <div id="blok-baza">

            <div class="pole-formularza">

```

```

    {{ form.firma_id.label(class="pole-formularza__etykieta") }}

    {{ form.firma_id(class="filtr-select", style="width:100%; ", id="firma_id_select") }}

</div>

<div id="firma-podglad" style="display:none; margin-top:0.5rem; padding:0.75rem; background:#f0fdf4; border:1px
solid #a7f3d0; border-radius:0.5rem; font-size:0.85rem;"></div>

</div>

<!-- Tryb: własna firma -->

<div id="blok-custom" style="display:none;">

    <div style="background:#fef3cd; border:1px solid #fde68a; border-radius:0.5rem; padding:0.75rem;
margin-bottom:1rem; font-size:0.82rem; color:#92400e;">

        Własna firma wymaga wygenerowania <strong>Zał. 9</strong> (oświadczenie instytucji) i <strong>Zał. 1</strong>
(porozumienie). Możesz je wygenerować po wypełnieniu poniższych danych.

    </div>

<div class="pole-formularza">

    {{ form.firma_nazwa.label(class="pole-formularza__etykieta") }}

    {{ form.firma_nazwa(class="pole-formularza__input") }}

</div>

<div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem;">

    <div class="pole-formularza">

        {{ form.firma_adres.label(class="pole-formularza__etykieta") }}

        {{ form.firma_adres(class="pole-formularza__input", placeholder="ul. Przykładowa 1") }}

    </div>

    <div class="pole-formularza {% if form.firma_miasto.errors %}pole-formularza--blad{% endif %}">

        {{ form.firma_miasto.label(class="pole-formularza__etykieta") }}

        {{ form.firma_miasto(class="pole-formularza__input", placeholder="Elbląg") }}

        {% for e in form.firma_miasto.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

    </div>

</div>

<div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem;">

    <div class="pole-formularza">

        {{ form.firma_nip_krs.label(class="pole-formularza__etykieta") }}

        {{ form.firma_nip_krs(class="pole-formularza__input") }}

    </div>

    <div class="pole-formularza {% if form.firma_upowazniony_osoba.errors %}pole-formularza--blad{% endif %}">

        {{ form.firma_upowazniony_osoba.label(class="pole-formularza__etykieta") }}

```



```

        {{ form.firma_upowazniony_osoba(class="pole-formularza__input", placeholder="Jan Kowalski") }}

        {% for e in form.firma_upowazniony_osoba.errors %}<span class="pole-formularza__blad">{{ e }}</span>{%
endfor %}

    </div>

</div>

<div class="pole-formularza">

    {{ form.firma_upowazniony_stanowisko.label(class="pole-formularza__etykieta") }}

    {{ form.firma_upowazniony_stanowisko(class="pole-formularza__input") }}

</div>

</div>

</div>

</div>

<!-- === SEKCJA 3: ZOPZ === -->

<div class="karta" style="margin-bottom:1.5rem;">

    <div class="karta__naglowek"><h2 class="karta__tytul">Opiekun zakładowy (ZOPZ)</h2></div>

    <div class="karta__tresc">

        <div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem;">

            <div class="pole-formularza {% if form.zopz_imie_nazwisko.errors %}pole-formularza--blad{% endif %}">

                {{ form.zopz_imie_nazwisko.label(class="pole-formularza__etykieta") }}

                {{ form.zopz_imie_nazwisko(class="pole-formularza__input", placeholder="Jan Kowalski") }}

                {% for e in form.zopz_imie_nazwisko.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

            </div>

            <div class="pole-formularza">

                {{ form.zopz_stanowisko.label(class="pole-formularza__etykieta") }}

                {{ form.zopz_stanowisko(class="pole-formularza__input") }}

            </div>

        </div>

        <div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem;">

            <div class="pole-formularza">

                {{ form.zopz_telefon.label(class="pole-formularza__etykieta") }}

                {{ form.zopz_telefon(class="pole-formularza__input") }}

            </div>

            <div class="pole-formularza">

                {{ form.zopz_email.label(class="pole-formularza__etykieta") }}

```

```

        {{ form.zopz_email(class="pole-formularza__input", type="email") }}

    </div>

</div>

</div>

</div>

<div style="display:flex; justify-content:space-between;">

    <a href="{{ url_for('praktyki.kreator_sciezka', id=zapis.internship_id) }}" class="przycisk
przycisk--drugorzeczny">← Wróć</a>

    <button type="submit" class="przycisk przycisk--glowny">Dalej – Harmonogram →</button>

</div>

</form>

<!-- Dane firm do podpowiedzi -->

<script>

const firmy = {

    {% for f in firmy_list %}

        "{{ f.id }}": { nazwa: "{{ f.nazwa }}", adres: "{{ f.adres or '' }}", miasto: "{{ f.miasto or '' }}", nip: "{{
f.nip_krs or '' }}" }{% if not loop.last %},{% endif %}

    {% endfor %}

};

const typSel = document.getElementById('firma_typ');

const blokBaza = document.getElementById('blok-baza');

const blokCustom = document.getElementById('blok-custom');

const firmaSel = document.getElementById('firma_id_select');

const podglad = document.getElementById('firma-podglad');

function toggleFirma() {

    if (typSel.value === 'database') {

        blokBaza.style.display = 'block'; blokCustom.style.display = 'none';

    } else {

        blokBaza.style.display = 'none'; blokCustom.style.display = 'block';

    }

}

}

```

```
firmaSel.addEventListener('change', function() {

    const f = firmy[this.value];

    if (f) {

        podglad.innerHTML = `${f.nazwa}</strong><br>${f.adres}, ${f.miasto}${f.nip ? ' · NIP: ' + f.nip : ''}`;

        podglad.style.display = 'block';

    } else {

        podglad.style.display = 'none';

    }

});

typSel.addEventListener('change', toggleFirma);

toggleFirma();

// Pokaż podgląd jeśli firma już wybrana

if (firmaSel.value) firmaSel.dispatchEvent(new Event('change'));

</script>

</div>

{% endblock %}
```

PLIK: app_student\templates\kreator\krok2bc_wniosek.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Kreator praktyki – Wniosek{% endblock %}

{% block naglowek %}Rejestracja praktyki{% endblock %}


{% block tresc %}

<div style="max-width: 700px; margin: 0 auto;">


    <!-- Pasek postępu -->

    <div style="display: flex; gap: 0.5rem; margin-bottom: 2rem; align-items: center;">

        <div style="flex:1; text-align:center; padding:0.5rem; background:#d1fae5; color:#065f46; border-radius:0.375rem; font-size:0.85rem;">Ścieżka</div>

        <div style="color:#d1d5db;"><</div>

        <div style="flex:1; text-align:center; padding:0.5rem; background:#3b82f6; color:white; border-radius:0.375rem; font-weight:600; font-size:0.85rem;">2. Wniosek</div>

        <div style="color:#d1d5db;"><</div>

        <div style="flex:1; text-align:center; padding:0.5rem; background:#f3f4f6; color:#9ca3af; border-radius:0.375rem; font-size:0.85rem;">3. Decyzja komisji</div>

        <div style="color:#d1d5db;"><</div>

        <div style="flex:1; text-align:center; padding:0.5rem; background:#f3f4f6; color:#9ca3af; border-radius:0.375rem; font-size:0.85rem;">4. Sprawozdanie</div>

    </div>


<div class="karta">

    <div class="karta__naglowek">

        <h2 class="karta__tytul">

            {% if zapis.track_type.value == 'EMPLOYMENT' %}

                Krok 2: Wniosek – Praca etatowa / staż

            {% else %}

                Krok 2: Wniosek – Własna działalność gospodarcza

            {% endif %}

        </h2>

    </div>

    <div class="karta__tresc">


        <!-- Info o wymaganych załącznikach -->

    </div>

</div>
```

```
<div style="background:#eff6ff; border:1px solid #bfbdbfe; border-radius:0.5rem; padding:1rem; margin-bottom:1.5rem; font-size:0.85rem;">

    <div style="font-weight:600; margin-bottom:0.4rem;">Wymagane załączniki do wniosku:</div>

    {% if zapis.track_type.value == 'EMPLOYMENT' %}

    <ul style="margin:0; padding-left:1.2rem; color:#1e40af;">

        <li>Umowa o pracę / zakres obowiązków (PDF)</li>

        <li>Zaświadczenie od pracodawcy o zatrudnieniu (PDF)</li>

    </ul>

    {% else %}

    <ul style="margin:0; padding-left:1.2rem; color:#1e40af;">

        <li>Aktualny wpis do CEIDG lub KRS (PDF)</li>

        <li>Kody PKD zgodne z profilem IT</li>

    </ul>

    {% endif %}

</div>

<!-- Upload załączników -->

    <div style="background:#f9fafb; border:1px solid #e5e7eb; border-radius:0.5rem; padding:1rem; margin-bottom:1.5rem;">

    <div style="font-weight:600; margin-bottom:0.75rem; font-size:0.9rem;">Dodaj załączniki (PDF, maks. 10 MB)</div>

    <input type="file" id="upload-file" accept=".pdf,.jpg,.jpeg,.png"

        style="font-size:0.85rem; width:100%; margin-bottom:0.5rem;">

        <select id="upload-type" style="font-size:0.85rem; padding:0.4rem 0.75rem; border:1px solid #d1d5db; border-radius:0.375rem; margin-bottom:0.5rem; width:100%;">

            {% if zapis.track_type.value == 'EMPLOYMENT' %}

            <option value="umowa_pracy">Umowa o pracę</option>

            <option value="zakres_obowiazkow">Zakres obowiązków</option>

            <option value="zaswiadczenie_zatrudnienie">Zaświadczenie o zatrudnieniu</option>

            <option value="inne">Inne</option>

            {% else %}

            <option value="ceidg">Wpis z CEIDG</option>

            <option value="krs">Odpis z KRS</option>

            <option value="inne">Inne</option>

            {% endif %}

        </select>

        <button type="button" onclick="uploadZalacznik()" class="przycisk przycisk--drugorzecny przycisk--maly">Dodaj
```

plik</button>

```
<div id="upload-status" style="font-size:0.8rem; margin-top:0.5rem;"></div>
```

```
<div id="upload-lista" style="margin-top:0.75rem;"></div>
```

```
</div>
```

```
<form method="POST" novalidate>
```

```
{ { form.hidden_tag() } }
```

```
<div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem; margin-bottom:1rem;">
```

```
<div class="pole-formularza {% if form.pracodawca_nazwa.errors %}pole-formularza--blad{% endif %}">
```

```
{ { form.pracodawca_nazwa.label(class="pole-formularza__etykieta") } }
```

```
{ { form.pracodawca_nazwa(class="pole-formularza__input") } }
```

```
{% for e in form.pracodawca_nazwa.errors %}<span class="pole-formularza__blad">{ { e } }</span>{% endfor %}
```

```
</div>
```

```
<div class="pole-formularza {% if form.stanowisko.errors %}pole-formularza--blad{% endif %}">
```

```
{ { form.stanowisko.label(class="pole-formularza__etykieta") } }
```

```
{ { form.stanowisko(class="pole-formularza__input") } }
```

```
{% for e in form.stanowisko.errors %}<span class="pole-formularza__blad">{ { e } }</span>{% endfor %}
```

```
</div>
```

```
</div>
```

```
<div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem; margin-bottom:1rem;">
```

```
<div class="pole-formularza">
```

```
{ { form.pracodawca_adres.label(class="pole-formularza__etykieta") } }
```

```
{ { form.pracodawca_adres(class="pole-formularza__input") } }
```

```
</div>
```

```
<div class="pole-formularza">
```

```
{ { form.pracodawca_miasto.label(class="pole-formularza__etykieta") } }
```

```
{ { form.pracodawca_miasto(class="pole-formularza__input") } }
```

```
</div>
```

```
</div>
```

```
<div class="pole-formularza {% if form.uzasadnienie.errors %}pole-formularza--blad{% endif %}"
style="margin-bottom:1.5rem;">
```

```
{ { form.uzasadnienie.label(class="pole-formularza__etykieta") } }
```

```

        {{ form.uzasadnienie(class="pole-formularza__input", rows=5, style="height:auto;", placeholder="Opisz zakres obowiązków/działalności i dlaczego spełniają wymagania praktyki zawodowej...") }}

        {% for e in form.uzasadnienie.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

    </div>

    <div style="display:flex; justify-content:space-between; gap:1rem;">

        <a href="{{ url_for('praktyki.kreator_sciezka', id=zapis.internship_id) }}" class="przycisk przycisk--drugorzeczny">← Wróć</a>

        <button type="submit" class="przycisk przycisk--glowny">Złóż wniosek →</button>

    </div>

</form>

</div>

</div>

</div>

</div>

<script>

const uploadUrl = "{{ url_for('uploads.upload_document', enrollment_id=zapis.id) }}";

let uploadedFiles = [];

async function uploadZalacznik() {

    const fileInput = document.getElementById('upload-file');

    const typeSelect = document.getElementById('upload-type');

    const status = document.getElementById('upload-status');

    if (!fileInput.files.length) {

        status.textContent = 'Wybierz plik.';

        status.style.color = '#dc2626';

        return;

    }

    const fd = new FormData();

    fd.append('file', fileInput.files[0]);

    fd.append('document_type', typeSelect.value);

    fd.append('csrf_token', document.querySelector('[name=csrf_token]').value);

```

```

status.textContent = 'Wysyłanie...';

status.style.color = '#6b7280';

try {

  const r = await fetch(uploadUrl, { method: 'POST', body: fd });

  const data = await r.json();

  if (r.ok) {

    status.textContent = 'Plik dodany pomyslnie.';

    status.style.color = '#059669';

    uploadedFiles.push({ name: fileInput.files[0].name, type: typeSelect.options[typeSelect.selectedIndex].text });

    renderLista();

    fileInput.value = '';

  } else {

    status.textContent = 'Blad: ' + (data.error || 'Nie udalo sie przeslac.');
    status.style.color = '#dc2626';

  }

} catch(e) {

  status.textContent = 'Blad polaczenia.';

  status.style.color = '#dc2626';

}

}

function renderLista() {

  const el = document.getElementById('upload-lista');

  el.innerHTML = uploadedFiles.map(f =>

    '<div style="font-size:0.8rem; color:#059669; margin-top:0.2rem;">+ ' + f.name + ' (' + f.type + ')</div>'

  ).join('');

}

</script>

{% endblock %}

```


PLIK: app_student\templates\kreator\krok3_harmonogram.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Kreator – Krok 3: Harmonogram{% endblock %}

{% block naglowek %}Rejestracja praktyki zawodowej{% endblock %}


{% block tresc %}

<div style="max-width: 860px; margin: 0 auto;">


    <!-- Pasek kroków -->

    <div style="display:flex; gap:0.5rem; margin-bottom:2rem; font-size:0.82rem;">

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#d1fae5; color:#065f46; border-radius:0.375rem;">Ścieżka A</div>

        <div style="color:#d1d5db; align-self:center;">></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#d1fae5; color:#065f46; border-radius:0.375rem;">2. Miejsce praktyki</div>

        <div style="color:#d1d5db; align-self:center;">></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#3b82f6; color:white; border-radius:0.375rem; font-weight:600;">3. Harmonogram</div>

        <div style="color:#d1d5db; align-self:center;">></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#f3f4f6; color:#9ca3af; border-radius:0.375rem;">4. Realizacja</div>

    </div>


<form method="POST" novalidate>

    {{ csrf_form.hidden_tag() }}


    <div class="karta" style="margin-bottom:1.5rem;">

        <div class="karta__naglowek">

            <h2 class="karta__tytul">Harmonogram praktyki – efekty uczenia się</h2>

        </div>

        <div style="padding:0.6rem 1rem; background:#fef3cd; border-bottom:1px solid #fde68a; font-size:0.8rem; color:#92400e;">

            Dla każdego efektu podaj dział zakładu pracy, przykładowe czynności i planowaną liczbę dni. Wiersze bez działu lub czynności zostaną pominięte.

        </div>

        {% for e in efekty %}
```

```

{% set h = istniejejace_harmonogramy.get(e.id|string, {}) %}

<div style="border-bottom:1px solid var(--kolor-ramka); padding:0.6rem 1rem;">

    <div style="font-size:0.78rem; color:var(--kolor-tekst-drugor); margin-bottom:0.35rem;">

        {{ e.opis }}

    </div>

    <div style="display:grid; grid-template-columns:1fr 1fr 5rem; gap:0.5rem; align-items:center;">

        <input type="text" name="dzial_{{ e.id }}" value="{{ h.get('dzial', '') }}"

            class="pole-formularza__input" style="font-size:0.8rem; padding:0.3rem 0.5rem;" placeholder="Dział
zakładu pracy">

        <input type="text" name="prace_{{ e.id }}" value="{{ h.get('prace', '') }}"

            class="pole-formularza__input" style="font-size:0.8rem; padding:0.3rem 0.5rem;" placeholder="Przykładowe
czynności">

        <input type="number" name="dni_{{ e.id }}" value="{{ h.get('dni', '') }}"

            class="pole-formularza__input" style="font-size:0.8rem; padding:0.3rem 0.5rem; text-align:center;"
min="0" max="60" placeholder="0">

    </div>

</div>

{% endfor %}

</div>

<div style="display:flex; justify-content:space-between;">

    <a href="{{ url_for('praktyki.kreator_firma', zapis_id=zapis.id) }}" class="przycisk przycisk--drugorzeczny">←
Wróć</a>

    <button type="submit" class="przycisk przycisk--glowny">Zapisz harmonogram →</button>

</div>

</form>

</div>

{% endblock %}

```

PLIK: app_student\templates\layouts\base_student.html

```
<!DOCTYPE html>

<html lang="pl">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0, minimum-scale=1.0">

    {% set _csrf_token = csrf_token() if csrf_token is defined else '' %}

    <meta name="csrf-token" content="{{ _csrf_token }}">

    <title>{% block tytuł %}Panel studenta{% endblock %} – ANS Elbląg</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='css/student.css') }}">

    <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600&family=IBM+Plex+Mono&display=swap"
rel="stylesheet">

</head>

<body class="panel-body">


    <a href="#main-content" class="skip-link">Przejdź do głównej treści</a>


    <nav class="sidebar" aria-label="Nawigacja główna panelu">

        <div class="sidebar__naglowek">

            <div class="branding__logo-kontener" style="max-width: 140px; margin-bottom: 10px;">

                

            </div>

            <span class="sidebar__system-label">Panel studenta</span>

        </div>


        <div class="sidebar__uzytkownik" aria-label="Zalogowany użytkownik">

            <div class="sidebar__uzytkownik-avatar" aria-hidden="true">

                {% if current_user.is_authenticated and current_user.first_name and current_user.last_name %}

                    {{ current_user.first_name[0] }}{{ current_user.last_name[0] }}

                {% else %}

                    –

                {% endif %}

            </div>

            <div class="sidebar__uzytkownik-info">
```

```

{% if current_user.is_authenticated %}

    <span class="sidebar__uzytkownik-imie">{{ current_user.first_name }} {{ current_user.last_name }}</span>

    <span class="sidebar__uzytkownik-rola">Nr alb. {{ current_user.album_number or '-' }}</span>

{% else %}

    <span class="sidebar__uzytkownik-imie">Gość</span>

    <span class="sidebar__uzytkownik-rola">—</span>

{% endif %}

</div>

</div>

<ul class="sidebar__menu" role="list">

    <li class="sidebar__menu-pozycja">

        <a href="{{ url_for('dashboard.index') }}" class="sidebar__link {% if request.endpoint == 'dashboard.index' %}sidebar__link--aktywny{% endif %}" {% if request.endpoint == 'dashboard.index' %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="m3 9 9-7 9 7v11a2 2 0 0 1-2 2H5a2 2 0 0 1-2 2z"/><polyline points="9,22 9,12 15,12 15,22"/></svg></span> Pulpit

        </a>

    </li>

    <li class="sidebar__menu-sekcja" aria-hidden="true">Praktyki</li>

    <li class="sidebar__menu-pozycja">

        <a href="{{ url_for('praktyki.lista') }}" class="sidebar__link {% if request.blueprint == 'praktyki' %}sidebar__link--aktywny{% endif %}" {% if request.blueprint == 'praktyki' %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><rect x="2" y="3" width="20" height="14" rx="2" ry="2"/><line x1="8" y1="21" x2="16" y2="21"/><line x1="12" y1="17" x2="12" y2="21"/></svg></span> Dostępne praktyki{% if nav_wymaga_uwagi %} <span style="display:inline-flex;align-items:center;justify-content:center;background:#ef4444;color:white;border-radius:9999px;width:1.1rem;height:1.1rem;font-size:0.65rem;font-weight:700;margin-left:0.35rem;">!</span>{% endif %}

        </a>

    </li>

    {% if aktywny_zapis_info.ma_aktywny and aktywny_zapis_info.sciezka == 'STANDARD' %}

    <li class="sidebar__menu-pozycja">

        <a href="{{ url_for('dziennik.index') }}" class="sidebar__link {% if request.blueprint == 'dziennik' %}sidebar__link--aktywny{% endif %}" {% if request.blueprint == 'dziennik' %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 21.3.09 6.26L22 9.27l-5 4.87 1.18 6.88L12 17.77l-6.18 3.25L7 14.14 2 9.27l6.91-1.01L12 2z"/></svg></span> Dziennik praktyki

        </a>

    </li>

    {% endif %}

```

```

{% if aktywny_zapis_info.ma_aktywny %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('sprawozdania.index') }}" class="sidebar__link {% if request.blueprint == 'sprawozdania' %}sidebar__link--aktywny{% endif %}" {% if request.blueprint == 'sprawozdania' %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="18" y1="20" x2="18" y2="10"/><line x1="12" y1="20" x2="12" y2="4"/><line x1="6" y1="20" x2="6" y2="14"/></svg></span> Sprawozdanie

    </a>

</li>

{% endif %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('documents.moje_dokumenty') }}" class="sidebar__link {% if request.endpoint == 'documents.moje_dokumenty' %}sidebar__link--aktywny{% endif %}" {% if request.endpoint == 'documents.moje_dokumenty' %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/><line x1="16" y1="13" x2="8" y2="13"/><line x1="16" y1="17" x2="8" y2="17"/><polyline points="10 9 9 9 8 9"/></svg></span> Moje Dokumenty

    </a>

</li>

</ul>

<div class="sidebar__stopka">

    <form method="POST" action="{{ url_for('auth.wylogowanie') }}">

        {{ csrf_token_field() if csrf_token_field is defined else '' }}

        <input type="hidden" name="csrf_token" value="{{ _csrf_token }}">

        <button type="submit" class="sidebar__wylogowanie">

            <svg viewBox="0 0 24 24"><path d="M9 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h4"/><polyline points="16 17 21 12 16 7"/><line x1="21" y1="12" x2="9" y2="12"/></svg> Wyloguj się

        </button>

    </form>

</div>

</nav>

<div class="panel-main">

    <header class="panel-main__naglowek">

        <div class="panel-main__naglowek-tresc">

            <h1 class="panel-main__tytul">{% block naglowek %}{% endblock %}</h1>

            {% block naglowek_akcje %}{% endblock %}


```

```

    </div>

</header>

<main id="main-content" class="panel-main__tresc" tabindex="-1">

    {% block tresc %}{% endblock %}

</main>

</div>

{% with wiadomosci = get_flashed_messages(with_categories=True) %}

    {% if wiadomosci %}

        <div class="komunikaty komunikaty--fixed" role="status" aria-live="polite">

            {% for kategoria, wiadomosc in wiadomosci %}

                <div class="komunikat komunikat--{{ kategoria }} komunikat--minimal" role="alert">

                    <span class="komunikat__ikona" aria-hidden="true">

                        {% if kategoria == 'danger' %}{% elif kategoria == 'success' %}{% else %}{% endif %}

                    </span>

                    <span class="komunikat__tekst">{{ wiadomosc }}</span>

                    <button type="button" class="komunikat__zamknij" aria-label="Zamknij powiadomienie"
onclick="this.parentElement.remove()">×</button>

                </div>

            {% endfor %}

        </div>

    {% endif %}

{% endwith %}

<script src="{{ url_for('static', filename='js/pdf_generator.js') }}"></script>

</body>

</html>

```

PLIK: app_student\templates\praktyki\lista.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Dostępne praktyki{% endblock %}

{% block naglowek %}Dostępne praktyki{% endblock %}


{% block tresc %}

<div style="max-width: 700px; margin: 0 auto;">


    {% if not dostepne %}

    <div class="karta" style="text-align: center; padding: 3rem 2rem;">

        <p style="color: var(--kolor-tekst-drugor);">Brak aktywnych praktyk. Skontaktuj się z opiekunem uczelni.</p>

    </div>


    {% else %}

    <div style="display: flex; flex-direction: column; gap: 1rem;">

        {% for p in dostepne %}

        <div class="karta">

            <div class="karta__tresc" style="display: flex; align-items: center; justify-content: space-between; gap: 1rem;">

                <div>

                    <div style="font-weight: 600; font-size: 1.05rem;">{{ p.rok_uczelniany }}</div>

                    <div style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-top: 0.25rem;">Semestr {{ p.semestr }} · {{ p.wymiar_godzin }} h</div>

                </div>

                {% if p.id|string in zapisy_data %}

                {% set zapis_info = zapisy_data[p.id|string] %}

                <div style="display: flex; align-items: center; gap: 0.75rem;">

                    {% if zapis_info.status == 'PENDING' %}

                    <span class="status status--draft">Nieukończzone</span>

                    {% if zapis_info.track_type == 'STANDARD' %}

                    <a href="{{ url_for('praktyki.kreator_firma', zapis_id=zapis_info.id) }}" class="przycisk przycisk--maly przycisk--glowny">Kontynuuj zgłoszenie</a>

                    {% elif zapis_info.track_type in ('EMPLOYMENT', 'OWN_BUSINESS') %}

                    <a href="{{ url_for('praktyki.kreator_wniosek', zapis_id=zapis_info.id) }}" class="przycisk przycisk--maly przycisk--glowny">Kontynuuj zgłoszenie</a>

                    
```

```

{% else %}

        <a href="{ { url_for('praktyki.kreator_sciezka', id=p.id) }}" class="przycisk przycisk--maly
przycisk--glowny">Kontynuuj zgłoszenie</a>

{% endif %}

{% elif zapis_info.wymaga_uwagi %}

        <span style="display:inline-flex;align-items:center;gap:0.3rem;background:#fef2f2;color:#dc2626;border:1px
solid #fecaca;border-radius:0.375rem;padding:0.25rem 0.6rem;font-size:0.8rem;font-weight:600;">

        <span style="font-size:1rem;line-height:1;">!</span> Wymaga uzupełnienia

</span>

        <a href="{ { url_for('praktyki.szczegoly_zgloszenia', id=zapis_info.id) }}" class="przycisk przycisk--maly
przycisk--drugorzeczny">Szczegóły</a>

{% else %}

        <span class="status status--{% if zapis_info.status == 'AWAITING_APPROVAL' %}planned{% elif zapis_info.status
== 'COMMISSION_REVIEW' %}planned{% elif zapis_info.status == 'DEAN_APPROVAL' %}planned{% elif zapis_info.status ==
'IN_PROGRESS' %}w-trakcie{% elif zapis_info.status == 'COMPLETED' %}zakonczone{% elif zapis_info.status == 'REJECTED'
%}draft{% endif %}">

        {{ tlumacz_status(zapis_info.status) }}

</span>

        <a href="{ { url_for('praktyki.szczegoly_zgloszenia', id=zapis_info.id) }}" class="przycisk przycisk--maly
przycisk--drugorzeczny">Szczegóły</a>

{% if zapis_info.status == 'IN_PROGRESS' %}

        <form method="POST" action="{ { url_for('praktyki.zakoncz_praktyke', id=zapis_info.id) }}"
style="display:inline;">

        {{ csrf_form.hidden_tag() }}

        <button type="submit" class="przycisk przycisk--maly przycisk--glowny"

                onclick="return confirm('Czy na pewno chcesz oznaczyć praktykę jako zakończoną? Po zakończeniu
zostaną udostępnione dokumenty końcowe.')">

                Zakończ praktykę

        </button>

</form>

{% endif %}

{% endif %}

</div>

{% else %}

        <a href="{ { url_for('praktyki.kreator_sciezka', id=p.id) }}" class="przycisk przycisk--glowny"
style="text-decoration:none;">Zapisz się</a>

{% endif %}

</div>

</div>

```



```
{% endfor %}
```

```
</div>
```

```
{% endif %}
```

```
</div>
```

```
{% endblock %}
```

PLIK: app_student\templates\praktyki\szczegoly_zgloszenia.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Szczegóły zgłoszenia{% endblock %}

{% block naglowek %}Szczegóły zgłoszenia praktyki{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem;">

    <a href="{{ url_for('dashboard.index') }}" class="przycisk przycisk--drugorzeczny">← Powrót do pulpitu</a>

</div>

{% endblock %}


{% block tresc %}

<div style="max-width: 800px; margin: 0 auto;">


    <!-- Baner akcji – wymaga uzupełnienia -->

    {% if zapis.status.value == 'AWAITING_APPROVAL' and zapis.komentarze_uopz %}

        <div style="background:#ffffbeb; border:1px solid #f59e0b; border-left:4px solid #d97706; border-radius:0.5rem; padding:1rem 1.25rem; margin-bottom:1.5rem; display:flex; align-items:center; justify-content:space-between; gap:1rem; flex-wrap:wrap;">

            <div>

                <div style="font-weight:600; color:#92400e; margin-bottom:0.25rem;">Zgłoszenie wymaga uzupełnienia</div>

                <div style="font-size:0.85rem; color:#92400e;">{{ zapis.komentarze_uopz }}</div>

            </div>

            <div style="display:flex; gap:0.5rem; flex-shrink:0; flex-wrap:wrap;">

                {% if zapis.track_type.value == 'STANDARD' %}

                    <a href="{{ url_for('praktyki.kreator_firma', zapis_id=zapis.id) }}" class="przycisk przycisk--glowny przycisk--maly">Edytuj i odeślij →</a>

                {% else %}

                    <a href="{{ url_for('praktyki.kreator_wniosek', zapis_id=zapis.id) }}" class="przycisk przycisk--glowny przycisk--maly">Edytuj i odeślij →</a>

                {% endif %}

            </div>

        </div>

        <form method="POST" action="{{ url_for('praktyki.resubmit_zgloszenia', id=zapis.id) }}" style="margin:0;">

            <input type="hidden" name="csrf_token" id="resubmit-csrf">

            <button type="submit" class="przycisk przycisk--drugorzeczny przycisk--maly">Odeślij bez zmian</button>

        </form>

    </div>

</div>
```

</div>

```
                                <script>document.getElementById('resubmit-csrf').value
document.querySelector('meta[name="csrf-token"]')?.getAttribute('content')||'';</script>
```

=

{% endif %}

<!-- Status zgłoszenia -->

<div class="karta" style="margin-bottom: 2rem;">

<div class="karta__naglowek">

<h2>Status zgłoszenia</h2>

</div>

<div class="karta__tresc">

<div style="display: flex; justify-content: space-between; align-items: center;">

<div>

<div style="font-size: 0.9rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.5rem;">

Praktyka: {{ zapis.praktyka.rok_uczelniany }} ({{ zapis.praktyka.semestr }})

</div>

<div style="font-size: 0.9rem; color: var(--kolor-tekst-drugor);">

Wysłano: {{ zapis.enrolled_at.strftime('%d.%m.%Y %H:%M') }}

</div>

</div>

<span class="status

{%- if zapis.status.value == 'PENDING' %} status--draft

{%- elif zapis.status.value == 'AWAITING_APPROVAL' %} status--planned

{%- elif zapis.status.value == 'COMMISSION_REVIEW' %} status--planned

{%- elif zapis.status.value == 'DEAN_APPROVAL' %} status--planned

{%- elif zapis.status.value == 'IN_PROGRESS' %} status--w-trakcie

{%- elif zapis.status.value == 'COMPLETED' %} status--zakonczone

{%- elif zapis.status.value == 'REJECTED' %} status--draft

{%- endif %}">

{% if zapis.status.value == 'PENDING' %}Szkic

{% elif zapis.status.value == 'AWAITING_APPROVAL' %}Oczekuje na zatwierdzenie

{% elif zapis.status.value == 'COMMISSION_REVIEW' %}Weryfikacja komisji

{% elif zapis.status.value == 'DEAN_APPROVAL' %}Oczekuje na dziekana

{% elif zapis.status.value == 'IN_PROGRESS' %}Zatwierdzone - w trakcie

{% elif zapis.status.value == 'COMPLETED' %}Zakończone

```

        {% elif zapis.status.value == 'REJECTED' %}Odrzucony

        {% endif %}

    </span>

</div>

</div>

</div>

<!-- Uploadowane załączniki -->

{% if uploaded_docs %}

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__naglowek">

        <h2>Twoje załączniki ({{ uploaded_docs|length }})</h2>

    </div>

    <div class="karta__tresc">

        <div style="display:flex; flex-direction:column; gap:0.5rem;">

            {% for dok in uploaded_docs %}

                <div style="display:flex; justify-content:space-between; align-items:center; padding:0.625rem 0.875rem; background:#f8fafc; border:1px solid #e2e8f0; border-radius:0.375rem;">

                    <div>

                        <div style="font-weight:500; font-size:0.875rem;">{{ dok.original_filename }}</div>

                        <div style="font-size:0.75rem; color:var(--kolor-tekst-drugor);">{{ dok.document_type }}{% if dok.uploaded_at %} · {{ dok.uploaded_at.strftime('%d.%m.%Y') if dok.uploaded_at is not string else dok.uploaded_at[:10] }}{% endif %}</div>

                    </div>

                    <span style="font-size:0.75rem; color:#059669;">Przesłany</span>

                </div>

            {% endfor %}

        </div>

    </div>

</div>

{% endif %}

<!-- Komentarze i uwagi -->

{% if zapis.komentarze_uopz or zapis.komentarze_admina %}

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__naglowek">

```

```
<h2>Uwagi i komentarze</h2>

</div>

<div class="karta__tresc">

    {% if zapis.komentarze_uopz %}

        <div style="padding: 1rem; background: #fef3cd; border-radius: 0.5rem; border-left: 4px solid #f59e0b;
margin-bottom: 1rem;">

            <div style="font-weight: 500; color: #92400e; margin-bottom: 0.5rem;">

                Uwagi Opiekuna Uczelnianego (UOPZ)

            </div>

            <div style="color: #92400e; white-space: pre-wrap; line-height: 1.5;">{{ zapis.komentarze_uopz }}</div>

        </div>

    {% endif %}

    {% if zapis.komentarze_admina %}

        <div style="padding: 1rem; background: #e0f2fe; border-radius: 0.5rem; border-left: 4px solid #0288d1;">

            <div style="font-weight: 500; color: #01579b; margin-bottom: 0.5rem;">

                Komentarz Administratora

            </div>

            <div style="color: #01579b; white-space: pre-wrap; line-height: 1.5;">{{ zapis.komentarze_admina }}</div>

        </div>

    {% endif %}

</div>

</div>

{% endif %}

<!-- Dane zgłoszenia -->

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__naglowek">

        <h2>Dane zgłoszenia</h2>

    </div>

    <div class="karta__tresc">
```

```
<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem; margin-bottom: 2rem;">
```

```
<div>
```

```
<label style="font-weight: 500; margin-bottom: 0.5rem; display: block;">Ścieżka praktyki</label>
```

```
<div style="color: var(--kolor-tekst-drugor);">
```

```
{% if zapis.track_type.value == 'STANDARD' %}Standardowa
```

```
{% elif zapis.track_type.value == 'EMPLOYMENT' %}Praca etatowa
```

```
{% elif zapis.track_type.value == 'OWN_BUSINESS' %}Własna działalność gospodarcza
```

```
{% endif %}
```

```
</div>
```

```
</div>
```

```
<div>
```

```
<label style="font-weight: 500; margin-bottom: 0.5rem; display: block;">Okres praktyki</label>
```

```
<div style="color: var(--kolor-tekst-drugor);">
```

```
{% if zapis.termin_od and zapis.termin_do %}
```

```
{{ zapis.termin_od.strftime('%d.%m.%Y') }} - {{ zapis.termin_do.strftime('%d.%m.%Y') }}
```

```
{% else %}
```

```
Nie określono
```

```
{% endif %}
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% if zapis.firma_nazwa %}
```

```
<div style="border-top: 1px solid #e5e7eb; padding-top: 1.5rem; margin-top: 1.5rem;">
```

```
<h3 style="font-size: 1rem; font-weight: 600; margin-bottom: 1rem;">Zakład pracy</h3>
```

```
<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem;">
```

```
<div>
```

```
<label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">Nazwa</label>
```

```
<div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.firma_nazwa }}</div>
```

```
</div>
```

```
<div>
```

```
<label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">NIP/KRS</label>
```

```
<div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.firma_nip_krs or 'Nie podano'
```

```
}}</div>
```

```
</div>
```

```

<div>

    <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">Adres</label>

    <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.firma_adres }}</div>

</div>

<div>

    <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">Miasto</label>

    <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.firma_miasto }}</div>

</div>

</div>

</div>

{% endif %}

{% if zapis.zopz_imie_nazwisko %}

<div style="border-top: 1px solid #e5e7eb; padding-top: 1.5rem; margin-top: 1.5rem;">

    <h3 style="font-size: 1rem; font-weight: 600; margin-bottom: 1rem;">Opiekun Zakładowy (ZOPZ)</h3>

    <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem;">

        <div>

            <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">Imię i nazwisko</label>

            <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.zopz_imie_nazwisko }}</div>

        </div>

        <div>

            <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">Stanowisko</label>

            <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.zopz_stanowisko }}</div>

        </div>

        <div>

            <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">Telefon</label>

            <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.zopz_telefon }}</div>

        </div>

        <div>

            <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">E-mail</label>

            <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.zopz_email }}</div>

        </div>

    </div>

</div>

```

```

</div>

{% endif %}

{% if zapis.uopz %}

<div style="border-top: 1px solid #e5e7eb; padding-top: 1.5rem; margin-top: 1.5rem;">

  <h3 style="font-size: 1rem; font-weight: 600; margin-bottom: 1rem;">Opiekun Uczelni (UOPZ)</h3>

  <div style="color: var(--kolor-tekst-drugor);">

    {{ zapis.uopz.first_name }} {{ zapis.uopz.last_name }}

    {% if zapis.uopz.email %}

      <br><a href="mailto:{{ zapis.uopz.email }}" style="color: var(--kolor-glowny);">{{ zapis.uopz.email }}</a>

    {% endif %}

  </div>

</div>

{% endif %}

</div>

</div>

{% if zapis.status.value in ['IN_PROGRESS', 'COMPLETED'] %}

<div class="karta">

  <div class="karta__tresc" style="display: flex; align-items: center; justify-content: space-between;">

    <div>

      <div style="font-weight: 600; margin-bottom: 0.25rem;">Dokumenty praktyki gotowe do pobrania</div>

      <div style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">Załącz. 1 (Porozumienie), Załącz. 2 (Program), Załącz. 2a (Indywidualny Program), Załącz. 3 (Karta ze Skierowaniem) – dostępne w zakładce Moje Dokumenty.</div>

    </div>

    <a href="{{ url_for('documents.moje_dokumenty') }}" class="przycisk przycisk--glowny">

      Przejdź do Moich Dokumentów →

    </a>

  </div>

</div>

{% endif %}

</div>

{% endblock %}

```


PLIK: app_student\templates\sprawozdania\index.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Sprawozdanie z praktyki{% endblock %}

{% block naglowek %}Sprawozdanie z przebiegu praktyki{% endblock %}


{% block tresc %}

<div style="max-width: 900px; margin: 0 auto; padding-bottom: 3rem;">


    {% if not ma_zapis %}

    <div class="karta" style="text-align: center; padding: 3rem 2rem;">

        <p style="color: var(--kolor-tekst-drugor);">Najpierw zapisz się na praktykę.</p>

        <a href="{{ url_for('praktyki.lista') }}" class="przycisk przycisk--glowny" style="margin-top: 1rem;">Dostępne
praktyki</a>

    </div>

    {% elif not zapis %}

    <div class="karta" style="text-align: center; padding: 3rem 2rem;">

        <p style="color: var(--kolor-tekst-drugor);">Twój zapis oczekuje na akceptację przez UOPZ, dlatego projekt
sprawozdania jest jeszcze niedostępny.</p>

    </div>

    {% else %}


    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-glowny);">

        <div class="karta__tresc" style="display: flex; justify-content: space-between; align-items: center;">

            <div>

                <div style="font-weight: 600; font-size: 1.1rem;">

                    {% if is_standard %}Załącznik 7 – Sprawozdanie z praktyki{% else %}Załącznik 7a – Sprawozdanie z pracy
zawodowej{% endif %}

                </div>

                <div style="font-size: 0.9rem; color: var(--kolor-tekst-drugor);">{{ zapis.praktyka.rok_ucelniiany }} – Semestr
{{ zapis.praktyka.semestr }}</div>

            </div>

            <a href="{{ url_for('documents.moje_dokumenty') }}" class="przycisk przycisk--drugorzeczny">Pobierz PDF (Moje
Dokumenty)</a>

        </div>

    </div>

</div>
```

```
<form method="POST" action="{ { url_for('sprawozdania.index') } }" novalidate>
```

```
{ { form.hidden_tag() } }
```

```
<div class="karta" style="margin-bottom: 2rem;">
```

```
<div class="karta__naglowek">
```

```
<h2 class="karta__tytul">{ { form.charakterystyka.label.text } }</h2>
```

```
</div>
```

```
<div class="karta__tresc">
```

```
<p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-bottom: 1rem;">
```

```
{ % if is_standard % }
```

Opisz zwięźle zakład pracy (branża, wielkość, używane technologie) oraz konkretny dział/komórkę organizacyjną, w której realizowana była praktyka.

```
{ % else % }
```

Opisz zwięźle miejsce pracy / działalność (branża, wielkość firmy, technologie IT, zakres działalności).

```
{ % endif % }
```

```
</p>
```

```
<div class="pole-formularza { % if form.charakterystyka.errors % }pole-formularza--blad{ % endif % }">
```

```
{ { form.charakterystyka(class="pole-formularza__input", rows="8", placeholder="Tutaj wpisz sekcję I...") } }
```

```
{ % for e in form.charakterystyka.errors % }<span class="pole-formularza__blad">{ { e } }</span>{ % endfor % }
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="karta" style="margin-bottom: 2rem;">
```

```
<div class="karta__naglowek">
```

```
<h2 class="karta__tytul">{ { form.opis.label.text } }</h2>
```

```
</div>
```

```
<div class="karta__tresc">
```

```
<p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-bottom: 1rem;">
```

```
{ % if is_standard % }
```

Przedstaw szczegółowy opis i analizę wykonanych prac w ramach zrealizowanego harmonogramu.

```
{ % else % }
```

Opisz syntetycznie jakie zadania wykonywałeś/aś w trakcie pracy zawodowej lub działalności gospodarczej (najważniejsze z punktu widzenia efektów uczenia się).

```
{ % endif % }
```

```

</p>

<div class="pole-formularza {% if form.opis.errors %}pole-formularza--blad{% endif %}">

    {{ form.opis(class="pole-formularza__input", rows="12", placeholder="Tutaj wpisz sekcję II...") }}

    {% for e in form.opis.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

</div>

</div>

</div>

{% if is_standard %}

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__naglowek">

        <h2 class="karta__tytul">Sekcja III: Wykonane zadania (Zestawienie efektów)</h2>

    </div>

    <div class="karta__tresc py-3">

        <div style="background: var(--kolor-tlo); border-radius: 8px; padding: 1.5rem; border: 1px dashed
var(--kolor-ramka);">

            <p style="font-size: 0.9rem; color: var(--kolor-tekst);">

                <strong>Uwaga:</strong> Sekcja III zawierająca szczegółowe zestawienie godzinowych nakładów pracy dla każdego
zdefiniowanego efektu uczenia się <strong style="color: var(--kolor-glowny);">jest generowana w 100%
automatycznie</strong> na podstawie Twojego Dziennika Praktyk.

            </p>

            <ul style="margin-top: 1rem; padding-left: 1.5rem; font-size: 0.85rem; color: var(--kolor-tekst-drugor);">

                <li>Nie musisz nic tu wpisywać. Wystarczy regularnie uzupełniać dziennik.</li>

                <li>System sumuje godziny i przypisuje je do właściwych komórek tabeli z załącznika.</li>

            </ul>

        </div>

    </div>

</div>

{% endif %}

<div style="display: flex; justify-content: flex-end; gap: 1rem;">

    <button type="submit" class="przycisk przycisk--glowny" style="padding: 1rem 3rem; font-size: 1.1rem; box-shadow: 0
4px 14px rgba(26, 58, 92, 0.3);">

        Zapisz robocze sprawozdanie

    </button>

</div>

```

</form>

{% endif %}

</div>

{% endblock %}

PLIK: core\autoryzacja.py

"""

core/autoryzacja.py

Kanoniczny moduł autoryzacji – jeden dla całej platformy SOP.

Rejestracja w aplikacji:

```
from core.autoryzacja import stworz_blueprint_auth

app.register_blueprint(stworz_blueprint_auth(

    dozwolone_role=[RolaUzytkownika.STUDENT],

    template_logowania='auth/logowanie.html',

    template_zmiany_hasla='auth/zmien_haslo.html',

))
```

Jeśli dozwolone_role=None – brak filtrowania po roli (admin akceptuje ADMIN i UOPZ, student akceptuje tylko STUDENT).

"""

```
from functools import wraps

from flask import Blueprint, render_template, redirect, url_for, flash, request, abort, current_app

from flask_login import login_user, logout_user, login_required, current_user

from flask_wtf import FlaskForm

from wtforms import StringField, PasswordField, BooleanField

from wtforms.validators import DataRequired, Email, Length, EqualTo

from werkzeug.security import check_password_hash, generate_password_hash

from sqlalchemy.exc import SQLAlchemyError, OperationalError

from core.modele import Uzytkownik, RolaUzytkownika

from core.extensions import db
```

— Formularze —————

```
class FormularzLogowania(FlaskForm):

    email = StringField('Adres e-mail', validators=[

        DataRequired('Podaj adres e-mail.'),
```

```

        Email('Podaj poprawny adres e-mail.'),

        Length(max=255),

    ])

    haslo = PasswordField('Hasło', validators=[

        DataRequired('Podaj hasło.'),

        Length(min=1, max=128),

    ])

    zapamietaj = BooleanField('Zapamiętaj mnie')

```

```

class FormularzZmianyHasla(FlaskForm):

    nowe_haslo = PasswordField('Nowe hasło', validators=[

        DataRequired(), Length(min=8, max=128),

    ])

    potwierdzenie = PasswordField('Powtórz hasło', validators=[

        DataRequired(),

        EqualTo('nowe_haslo', message='Hasła muszą być identyczne.'),

    ])

```

— Dekorator RBAC – używany przez oba podmodele —————

```

def wymaga_rol(*dozwolone_role):

    """Sprawdza czy zalogowany użytkownik ma jedną z podanych ról."""

    def dekorator(func):

        @wraps(func)

        @login_required

        def wrapper(*args, **kwargs):

            if current_user.role not in dozwolone_role:

                abort(403)

            return func(*args, **kwargs)

        return wrapper

    return dekorator

```

```
def stworz_blueprint_auth(

    dozwolone_role: list | None = None,

    template_logowania: str = 'auth/logowanie.html',

    template_zmiany_hasla: str = 'auth/zmien_haslo.html',

) -> Blueprint:

    """

    Tworzy blueprint 'auth' skonfigurowany dla danego kontekstu.

    dozwolone_role - lista RolaUzytkownika dopuszczonych do logowania;

                    None = akceptuj wszystkie aktywne konta.

    """

    auth_bp = Blueprint('auth', __name__)

    @auth_bp.route('/', methods=['GET'])

    @auth_bp.route('/logowanie', methods=['GET', 'POST'])

    def logowanie():

        if current_user.is_authenticated:

            return redirect(url_for('dashboard.index'))

        form = FormularzLogowania()

        if form.validate_on_submit():

            email = form.email.data.lower().strip()

            try:

                uzytkownik = db.session.query(Uzytkownik).filter_by(email=email).first()

            except (OperationalError, SQLAlchemyError):

                current_app.logger.exception('Błąd połączenia z bazą podczas logowania')

                flash('Błąd połączenia z bazą danych. Spróbuj ponownie później.', 'danger')

                return render_template(template_logowania, form=form)

            if uzytkownik and uzytkownik.is_active and \
```

```

        check_password_hash(uzyskownik.password_hash, form.haslo.data):

        # Weryfikacja roli

        if dozwolone_role and uzyskownik.role not in dozwolone_role:

            flash('Twoje konto nie ma dostępu do tego panelu.', 'danger')

            return render_template(template_logowania, form=form)

        login_user(uzyskownik, remember=form.zapamietaj.data)

        next_page = request.args.get('next')

        return redirect(

            next_page if next_page and next_page.startswith('/') else url_for('dashboard.index')

        )

        flash('Nieprawidłowy adres e-mail lub hasło.', 'danger')

        return render_template(template_logowania, form=form)

@auth_bp.route('/wylogowanie', methods=['GET', 'POST'])
@login_required
def wylogowanie():

    logout_user()

    flash('Wylogowano pomyślnie.', 'success')

    return redirect(url_for('auth.logowanie'))

@auth_bp.route('/zmien-haslo', methods=['GET', 'POST'])
@login_required
def zmien_haslo():

    form = FormularzZmianyHasla()

    if form.validate_on_submit():

        current_user.password_hash = generate_password_hash(form.nowe_haslo.data)

        current_user.wymagana_zmiana_hasla = False

        db.session.commit()

        flash('Hasło zostało zmienione. Możesz korzystać z systemu.', 'success')

        return redirect(url_for('dashboard.index'))

```



```
return render_template(template_zmiany_hasla, form=form)
```

```
return auth_bp
```

PLIK: core\error_handlers.py

```
# core/error_handlers.py

"""

Centralizowana obsługa błędów HTTP dla całej aplikacji.

Rejestruje handlersy 403, 404, 500 z wspólnymi szablonami.

"""


from flask import render_template


def register_error_handlers(app):

    """

    Rejestruje wszystkie error handlers dla aplikacji.

    Użycie:

        from core.error_handlers import register_error_handlers

        app = Flask(__name__)

        register_error_handlers(app)

    """


@app.errorhandler(403)

def blad_403(e):

    return render_template('errors/blad.html',

                           kod='403',

                           tytul='Brak dostępu',

                           opis='Nie masz uprawnień do wyświetlenia tej strony.',

                           tekst_przycisku='Wróć do pulpitu'), 403


@app.errorhandler(404)

def blad_404(e):

    return render_template('errors/blad.html',

                           kod='404',

                           tytul='Nie znaleziono strony',

                           opis='Strona, której szukasz nie istnieje lub została przeniesiona.',
```

```
tekst_przycisku='Wróć do pulpitu'), 404
```

```
@app.errorhandler(500)
```

```
def blad_500(e):
```

```
    app.logger.exception('Unhandled Exception:')
```

```
    return render_template('errors/blad.html',
```

```
        kod='500',
```

```
        tytul='Błąd serwera',
```

```
        opis='Wystąpił nieoczekiwany błąd. Administratorzy zostali powiadomieni.',
```

```
        tekst_przycisku='Wróć do pulpitu'), 500
```

PLIK: `core\extensions.py`

```
from flask_sqlalchemy import SQLAlchemy
```

```
from flask_login import LoginManager
```

```
db = SQLAlchemy()
```

```
login_manager = LoginManager()
```

PLIK: core\pliki.py

"""

core/pliki.py

Kanoniczny moduł obsługi przesyłania plików – jeden dla całej platformy SOP.

Rejestracja w aplikacji:

```
from core.pliki import stworz_blueprint_pliki, SprawdzaczDostepuPliku

app.register_blueprint(stworz_blueprint_pliki(

    sprawdzacz=SprawdzaczDostepuPliku.tylko_student,

    url_prefix='/uploads',

))
```

Centralny punkt wszystkich zabezpieczeń: walidacja MIME, rozszerzeń,
rozmiaru, secure_filename oraz ochrona przed Path Traversal.

"""

import os

import uuid

from pathlib import Path

from werkzeug.utils import secure_filename

from flask import Blueprint, request, jsonify, send_from_directory, abort, flash, redirect, url_for

from flask_login import login_required, current_user

from sqlalchemy import text

from core.modele import ZapisPraktyki, RolaUzytkownika, UploadedDocument

from core.extensions import db

— Polityka bezpieczeństwa plików – JEDEN punkt konfiguracji —————

UPLOAD_FOLDER = Path(__file__).parent.parent / "uploads"

UPLOAD_FOLDER.mkdir(exist_ok=True)

MAX_FILE_SIZE = 10 * 1024 * 1024 # 10 MB

```
# Dozwolone rozszerzenia – wyłącznie te
```

```
ALLOWED_EXTENSIONS = frozenset({  
  
    '.pdf', '.doc', '.docx',  
  
    '.jpg', '.jpeg', '.png',  
  
    '.zip', '.rar',  
  
})
```

```
# Dozwolone typy MIME – muszą odpowiadać rozszerzeniu (double-check)
```

```
ALLOWED_MIME_TYPES = frozenset({  
  
    'application/pdf',  
  
    'application/msword',  
  
    'application/vnd.openxmlformats-officedocument.wordprocessingml.document',  
  
    'image/jpeg',  
  
    'image/png',  
  
    'application/zip',  
  
    'application/x-rar-compressed',  
  
    'application/vnd.rar',  
  
})
```

```
def _dozwolony_plik(filename: str, content_type: str) -> bool:
```

```
    """Walidacja rozszerzenia + MIME. Obie warstwy muszą być zgodne."""
```

```
    if not filename:
```

```
        return False
```

```
    ext = Path(filename).suffix.lower()
```

```
    return ext in ALLOWED_EXTENSIONS and content_type in ALLOWED_MIME_TYPES
```

```
def _sprawdz_dostep_do_zapisu(zapis: ZapisPraktyki) -> bool:
```

```
    """Domyślna kontrola dostępu: student widzi tylko swoje, admin/uopz wszystko."""
```

```
    if current_user.role == RolaUzytkownika.STUDENT:
```

```
        return zapis.student_id == current_user.id
```

```
    if current_user.role == RolaUzytkownika.UOPZ:
```

```
        return zapis.uopz_id == current_user.id
```

```
return current_user.role == RolaUzytkownika.ADMIN
```

```
# — Fabryka blueprintu —————
```

```
def stworz_blueprint_pliki(
    sprawdzacz_dostepu=None,
) -> Blueprint:
    """
    Tworzy blueprint 'uploads' ze scentralizowaną logiką bezpieczeństwa.

    sprawdzacz_dostepu – callable(zapis) → bool; None = domyślna logika RBAC.
    """
    uploads_bp = Blueprint('uploads', __name__)

    _sprawdz = sprawdzacz_dostepu or _sprawdz_dostep_do_zapisu

    @uploads_bp.route('/enrollment/<uuid:enrollment_id>/upload', methods=['POST'])
    @login_required
    def upload_document(enrollment_id):
        zapis = db.session.get(ZapisPraktyki, enrollment_id)

        if not zapis or not _sprawdz(zapis):
            abort(403)

        if 'file' not in request.files:
            return jsonify({'error': 'Brak pliku'}), 400

        file = request.files['file']

        document_type = request.form.get('document_type', '').strip()

        if not file.filename or not document_type:
            return jsonify({'error': 'Brak pliku lub typu dokumentu'}), 400

# — Walidacja rozmiaru —————

file.seek(0, os.SEEK_END)
```

```

file_size = file.tell()

file.seek(0)

if file_size > MAX_FILE_SIZE:

    return jsonify({'error': f'Plik zbyt duży (max {MAX_FILE_SIZE // (1024*1024)} MB)'}), 400

# — Walidacja MIME + rozszerzenia —————

content_type = file.content_type

if not _dozwolony_plik(file.filename, content_type):

    return jsonify({'error': 'Niedozwolony typ pliku'}), 400

try:

    # — secure_filename: ochrona przed Path Traversal —————

    original_filename = secure_filename(file.filename)

    if not original_filename:

        return jsonify({'error': 'Nieprawidłowa nazwa pliku'}), 400

    file_ext = Path(original_filename).suffix.lower()

    stored_filename = f"{uuid.uuid4().hex}{file_ext}"

    # — Zapis poza katalogiem webowym —————

    file_path = UPLOAD_FOLDER / stored_filename

    file.save(file_path)

    doc = UploadedDocument(

        enrollment_id = enrollment_id,

        document_type = document_type,

        original_filename = original_filename,

        stored_filename = stored_filename,

        file_path = str(file_path),

        file_size = file_size,

        mime_type = content_type,

        uploaded_by_id = current_user.id,

    )

```



```
db.session.add(doc)
```

```
db.session.commit()
```

```
return jsonify({
```

```
    'success':      True,
```

```
    'document_id': str(doc.id),
```

```
    'filename':     original_filename,
```

```
    'size':         file_size,
```

```
})
```

```
except Exception as e:
```

```
    if 'file_path' in locals() and Path(file_path).exists():
```

```
        Path(file_path).unlink()
```

```
    db.session.rollback()
```

```
    return jsonify({'error': f'Błąd podczas zapisywania: {str(e)}'}), 500
```

```
@uploads_bp.route('/document/<uuid:document_id>/download')
```

```
@login_required
```

```
def download_document(document_id):
```

```
    doc = db.session.get(UploadedDocument, document_id)
```

```
    if not doc:
```

```
        abort(404)
```

```
    zapis = doc.enrollment
```

```
    if not _sprawdz(zapis):
```

```
        abort(403)
```

```
    file_path = Path(doc.file_path)
```

```
    if not file_path.exists():
```

```
        abort(404)
```

```
# — Bezpieczne wysyłanie: send_from_directory zapobiega Path Traversal —
```

```
return send_from_directory(
```

```
    file_path.parent, file_path.name,
```

```
    as_attachment=True,
```

```
        download_name=doc.original_filename,

        mimetype=doc.mime_type,

    )
```

```
@uploads_bp.route('/document/<uuid:document_id>/delete', methods=['POST'])
```

```
@login_required
```

```
def delete_document(document_id):
```

```
    doc = db.session.get(UploadedDocument, document_id)
```

```
    if not doc:
```

```
        abort(404)
```

```
    if not _sprawdz(doc.enrollment):
```

```
        abort(403)
```

```
    try:
```

```
        file_path = Path(doc.file_path)
```

```
        if file_path.exists():
```

```
            file_path.unlink()
```

```
        db.session.delete(doc)
```

```
        db.session.commit()
```

```
        flash('Dokument został usunięty.', 'success')
```

```
    except Exception as e:
```

```
        db.session.rollback()
```

```
        flash(f'Błąd podczas usuwania: {str(e)}', 'danger')
```

```
    return redirect(request.referrer or url_for('dashboard.index'))
```

```
@uploads_bp.route('/enrollment/<uuid:enrollment_id>/documents')
```

```
@login_required
```

```
def list_documents(enrollment_id):
```

```
    zapis = db.session.get(ZapisPraktyki, enrollment_id)
```

```
    if not zapis or not _sprawdz(zapis):
```

```
        abort(403)
```

```
    docs = db.session.query(UploadedDocument)\
```

```
.filter_by(zapis_id=enrollment_id)\n\n.order_by(UploadedDocument.przeslano_o.desc())\n\n.all()
```

```
return jsonify([{\n\n    'id':                str(d.id),\n\n    'document_type':     d.document_type,\n\n    'original_filename': d.original_filename,\n\n    'file_size':         d.file_size,\n\n    'mime_type':         d.mime_type,\n\n    'uploaded_at':       d.uploaded_at.isoformat(),\n\n    'uploaded_by':       (f"{d.uploaded_by.first_name} {d.uploaded_by.last_name}"\n\n                          if d.uploaded_by else None),\n\n    'download_url':      url_for('uploads.download_document', document_id=d.id),\n\n    'delete_url':        url_for('uploads.delete_document',    document_id=d.id),\n\n    } for d in docs])
```

```
return uploads_bp
```

PLIK: `core\__init__.py`

PLIK: core\modele\dokumenty.py

```
"""core/modele/dokumenty.py
```

```
Modele domenowe: Przesłane dokumenty, Log audytu.
```

```
"""
```

```
import uuid
```

```
import enum
```

```
from sqlalchemy.dialects.postgresql import UUID
```

```
from core.extensions import db
```

```
class StatusDokumentu(enum.Enum):
```

```
    SZKIC          = 'DRAFT'
```

```
    OCZEKUJACY     = 'AWAITING_APPROVAL'
```

```
    ZATWIERDZONY   = 'APPROVED'
```

```
    ODRZUCONY      = 'REJECTED'
```

```
    # Compat
```

```
    DRAFT          = 'DRAFT'
```

```
    AWAITING_APPROVAL = 'AWAITING_APPROVAL'
```

```
    APPROVED       = 'APPROVED'
```

```
    REJECTED       = 'REJECTED'
```

```
class LogAudytuDokumentow(db.Model):
```

```
    """Ślad audytu dla operacji na dokumentach PDF."""
```

```
    __tablename__ = 'logi_audytu_dokumentow'
```

```
    id          = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    uzytkownik_id = db.Column('uzytkownik_id', UUID(as_uuid=True), db.ForeignKey('uzytkownicy.id', ondelete='SET NULL'),  
                             nullable=True)
```

```
    zapis_id      = db.Column('zapis_id',          UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id', ondelete='SET  
NULL'), nullable=True)
```

```
    typ_dokumentu = db.Column('typ_dokumentu', db.String(50), nullable=False)
```

```
akcja          = db.Column('akcja',          db.String(50), nullable=False)

szczegoly      = db.Column('szczegoly',      db.Text, nullable=True)

wykonano_o     = db.Column('wykonano_o',     db.DateTime, server_default=db.func.now())
```

```
uzytkownik = db.relationship('Uzytkownik')
```

```
zapis        = db.relationship('ZapisPraktyki')
```

```
# Compat
```

```
@property
```

```
def user_id(self):
```

```
    return self.uzytkownik_id
```

```
@property
```

```
def enrollment_id(self):
```

```
    return self.zapis_id
```

```
@property
```

```
def document_type(self):
```

```
    return self.typ_dokumentu
```

```
@property
```

```
def action(self):
```

```
    return self.akcja
```

```
@property
```

```
def details(self):
```

```
    return self.szczegoly
```

```
@property
```

```
def created_at(self):
```

```
    return self.wykonano_o
```

```
# Alias dla istniejącego kodu
```

```
DocumentAuditLog = LogAudytuDokumentow
```

```
class DokumentPrzeslany(db.Model):
```

```
    """Plik przesłany przez studenta lub pracownika (umowy, zaświadczenia itp.)."""
```

```
    __tablename__ = 'dokumenty_przeslane'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    zapis_id = db.Column('zapis_id', UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id',  
ondelete='CASCADE'), nullable=False)
```

```
    typ_dokumentu = db.Column('typ_dokumentu', db.String(50), nullable=False)
```

```
    oryginalna_nazwa = db.Column('oryginalna_nazwa', db.String(255), nullable=False)
```

```
    zapisana_nazwa = db.Column('zapisana_nazwa', db.String(255), nullable=False)
```

```
    sciezka_pliku = db.Column('sciezka_pliku', db.String(500), nullable=False)
```

```
    rozmiar_pliku = db.Column('rozmiar_pliku', db.Integer, nullable=False)
```

```
    typ_mime = db.Column('typ_mime', db.String(100), nullable=False)
```

```
    przeslano_o = db.Column('przeslano_o', db.DateTime, server_default=db.func.now())
```

```
    przeslane_przez_id = db.Column('przeslane_przez_id', UUID(as_uuid=True), db.ForeignKey('uzytkownicy.id',  
ondelete='SET NULL'), nullable=True)
```

```
    zapis = db.relationship('ZapisPraktyki', backref=db.backref('uploaded_documents', passive_deletes=True))
```

```
    przeslane_przez = db.relationship('Uzytkownik')
```

```
    # Compat
```

```
    @property
```

```
    def enrollment_id(self):
```

```
        return self.zapis_id
```

```
    @property
```

```
    def document_type(self):
```

```
        return self.typ_dokumentu
```

```
    @property
```

```
    def original_filename(self):
```

```
        return self.oryginalna_nazwa
```

```
@property

def stored_filename(self):

    return self.zapisana_nazwa
```

```
@property

def file_path(self):

    return self.sciezka_pliku
```

```
@property

def file_size(self):

    return self.rozmiar_pliku
```

```
@property

def mime_type(self):

    return self.typ_mime
```

```
@property

def uploaded_at(self):

    return self.przeslano_o
```

```
@property

def uploaded_by_id(self):

    return self.przeslane_przez_id
```

```
# Alias dla istniejącego kodu
```

```
UploadedDocument = DokumentPrzeslany
```


PLIK: core\modele\dziennik.py

```
"""core/modele/dziennik.py
```

```
Modele domenowe: Dziennik praktyki, Efekty uczenia, Oceny.
```

```
"""
```

```
import uuid
```

```
import enum
```

```
from sqlalchemy.dialects.postgresql import UUID
```

```
from core.extensions import db
```

```
class WynikOceny(enum.Enum):
```

```
    OSIAGNIETO      = 'ACHIEVED'
```

```
    CZESCIOWO      = 'PARTIALLY_ACHIEVED'
```

```
    NIE_OSIAGNIETO = 'NOT_ACHIEVED'
```

```
class EfektUczenia(db.Model):
```

```
    """Efekty uczenia się – słownik (tabela referencyjna)."""
```

```
    __tablename__ = 'efekty_uczenia'
```

```
    id = db.Column(db.Integer, primary_key=True)
```

```
    opis = db.Column('opis', db.Text, nullable=False)
```

```
    # Compat
```

```
    @property
```

```
    def description(self):
```

```
        return self.opis
```

```
    @property
```

```
    def kod(self) -> str:
```

```
        return str(self.id).zfill(2)
```

```
# Tabela asocjacyjna: wpis ↔ efekty
```

```
wpisy_efekty = db.Table(

    'wpisy_efekty',

    db.Column(

        'wpis_id',

        UUID(as_uuid=True),

        db.ForeignKey('wpisy_dziennika.id', ondelete='CASCADE'),

        primary_key=True,

    ),

    db.Column(

        'efekt_id',

        db.Integer,

        db.ForeignKey('efekty_uczenia.id'),

        primary_key=True,

    ),

)
```

```
class WpisDziennika(db.Model):
```

```
    """Pojedynczy wpis w dzienniku praktyki studenta."""
```

```
    __tablename__ = 'wpisy_dziennika'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    zapis_id = db.Column('zapis_id', UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id', ondelete='CASCADE'),
nullable=False)
```

```
    data_wpisu = db.Column('data_wpisu', db.Date, nullable=False)
```

```
    liczba_godzin = db.Column('liczba_godzin', db.Integer, nullable=False)
```

```
    opis = db.Column('opis', db.Text, nullable=False)
```

```
    efekty_uczenia = db.relationship('EfektUczenia', secondary=wpisy_efekty, lazy='subquery')
```

```
# Compat
```

```
@property
```

```
def enrollment_id(self):
```

```
    return self.zapis_id
```

```
@property
```

```
def entry_date(self):
```

```
    return self.data_wpisu
```

```
@property
```

```
def duration_hours(self):
```

```
    return self.liczba_godzin
```

```
@property
```

```
def description(self):
```

```
    return self.opis
```

```
class OcenaPraktyki(db.Model):
```

```
    """Ocena osiągnięcia efektu uczenia dla jednego zapisu."""
```

```
    __tablename__ = 'oceny_praktyk'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    zapis_id = db.Column('zapis_id', UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id', ondelete='CASCADE'),  
nullable=False)
```

```
    efekt_id = db.Column('efekt_id', db.Integer, db.ForeignKey('efekty_uczenia.id'), nullable=False)
```

```
    wynik = db.Column(  
    'wynik',  
    db.Enum(WynikOceny, name='wynik_oceny', values_callable=lambda e: [x.value for x in e]),  
    nullable=False,  
    )
```

```
    uwagi = db.Column('uwagi', db.Text, nullable=True)
```

```
    efekt = db.relationship('EfektUczenia', lazy='select')
```

```
# Compat
```

```
@property
```

```
def enrollment_id(self):
```

```
return self.zapis_id
```

```
@property
```

```
def learning_outcome_id(self):
```

```
    return self.efekt_id
```

```
@property
```

```
def result(self):
```

```
    return self.wynik
```

```
@result.setter
```

```
def result(self, v):
```

```
    self.wynik = v
```

```
@property
```

```
def evaluator_notes(self):
```

```
    return self.uwagi
```

```
@evaluator_notes.setter
```

```
def evaluator_notes(self, v):
```

```
    self.uwagi = v
```

PLIK: core\modele\firmy.py

```
"""core/modele/firmy.py
```

Modele domenowe: Zakłady pracy (firmy współpracujące z uczelnią).

```
"""
```

```
import uuid
```

```
from sqlalchemy.dialects.postgresql import UUID
```

```
from core.extensions import db
```

```
class Firma(db.Model):
```

```
    """Zakład pracy – partner praktyk o stałej umowie z ANS."""
```

```
    __tablename__ = 'firmy'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    nazwa = db.Column(db.String(255), nullable=False)
```

```
    adres = db.Column(db.String(255), nullable=True)
```

```
    miasto = db.Column(db.String(100), nullable=True)
```

```
    nip_krs = db.Column(db.String(50), nullable=True)
```

```
    ma_stala_umowe = db.Column(db.Boolean, default=True)
```

```
    aktywna = db.Column(db.Boolean, default=True)
```

```
    utworzono = db.Column(db.DateTime, server_default=db.func.now())
```

```
# Compat
```

```
@property
```

```
def stala_umowa(self):
```

```
    return self.ma_stala_umowe
```

```
@property
```

```
def has_standing_agreement(self):
```

```
    return self.ma_stala_umowe
```

```
@property
```

```
def is_active(self):
```

```
return self.aktywna
```

```
def __repr__(self) -> str:
```

```
    return f'<Firma {self.nazwa}>'
```

PLIK: core\modele\praktyki.py

```
"""core/modele/praktyki.py
```

```
Modele domenowe: Praktyki, Zapisy i powiązane encje satelitarne.
```

```
"""
```

```
import uuid
```

```
import enum
```

```
from sqlalchemy.dialects.postgresql import UUID
```

```
from sqlalchemy.ext.hybrid import hybrid_property
```

```
from core.extensions import db
```

```
# — Enumy dziedziny praktyk —————
```

```
class StatusPraktyki(enum.Enum):
```

```
    AKTYWNA = 'ACTIVE'
```

```
    NIEAKTYWNA = 'INACTIVE'
```

```
class StatusZapisu(enum.Enum):
```

```
    OCZEKUJACY = 'PENDING'
```

```
    OCZEKUJE_NA_AKCEPT = 'AWAITING_APPROVAL'
```

```
    WERYFIKACJA_KOMISJI = 'COMMISSION_REVIEW'
```

```
    AKCEPTACJA_DZIEKANA = 'DEAN_APPROVAL'
```

```
    W_REALIZACJI = 'IN_PROGRESS'
```

```
    ZAKONCZONA = 'COMPLETED'
```

```
    ODRZUCONA = 'REJECTED'
```

```
class SciezkaPraktyki(enum.Enum):
```

```
    STANDARDOWA = 'STANDARD'
```

```
    ZATRUDNIENIE = 'EMPLOYMENT'
```

```
WLASNA_DZIALALNOSC = 'OWN_BUSINESS'
```

```
class TypZdarzenia(enum.Enum):
```

```
    ADMIN_KOMENTARZ      = 'ADMIN_KOMENTARZ'
```

```
    UOPZ_KOMENTARZ       = 'UOPZ_KOMENTARZ'
```

```
    POWIADOMIENIE_STUDENTA = 'POWIADOMIENIE_STUDENTA'
```

```
    KOMISJA_DECYZJA       = 'KOMISJA_DECYZJA'
```

```
    DZIEKAN_DECYZJA       = 'DZIEKAN_DECYZJA'
```

```
# Aliasy dla istniejącego kodu (nie usuwać do pełnej migracji kontrolerów)
```

```
StatusPraktyki.ACTIVE     = StatusPraktyki.AKTYWNA
```

```
StatusPraktyki.INACTIVE  = StatusPraktyki.NIEAKTYWNA
```

```
StatusZapisu.PENDING     = StatusZapisu.OCZEKUJACY
```

```
StatusZapisu.AWAITING_APPROVAL = StatusZapisu.OCZEKUJE_NA_AKCEPT
```

```
StatusZapisu.COMMISSION_REVIEW = StatusZapisu.WERYFIKACJA_KOMISJI
```

```
StatusZapisu.DEAN_APPROVAL   = StatusZapisu.AKCEPTACJA_DZIEKANA
```

```
StatusZapisu.IN_PROGRESS    = StatusZapisu.W_REALIZACJI
```

```
StatusZapisu.COMPLETED     = StatusZapisu.ZAKONCZONA
```

```
StatusZapisu.REJECTED       = StatusZapisu.ODRZUCONA
```

```
SciezkaPraktyki.STANDARD    = SciezkaPraktyki.STANDARDOWA
```

```
SciezkaPraktyki.EMPLOYMENT  = SciezkaPraktyki.ZATRUDNIENIE
```

```
SciezkaPraktyki.OWN_BUSINESS = SciezkaPraktyki.WLASNA_DZIALALNOSC
```

```
# — Modele —————
```

```
class Praktyka(db.Model):
```

```
    """Edycja praktyk – rok akademicki i semestr."""
```

```
    __tablename__ = 'praktyki'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    rok_uczelniany = db.Column(db.String(9), nullable=False)
```



```

semestr          = db.Column(db.String(10), nullable=False)

wymiar_godzin    = db.Column(db.Integer, nullable=False, default=160)

status           = db.Column(

    db.Enum(StatusPraktyki, name='status_praktyki', values_callable=lambda e: [x.value for x in e]),

    nullable=False, default=StatusPraktyki.NIEAKTYWNA,

)

utworzono = db.Column(db.DateTime, server_default=db.func.now())

zapisy = db.relationship(

    'ZapisPraktyki', backref='praktyka',

    lazy='select', cascade='all, delete-orphan', passive_deletes=True,

)

```

```

class ZapisPraktyki(db.Model):

```

```

    """Rdzeń zgłoszenia studenta do edycji praktyki."""

```

```

    __tablename__ = 'zapisy_praktyk'

```

```

id              = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

praktyka_id     = db.Column(UUID(as_uuid=True), db.ForeignKey('praktyki.id',      ondelete='CASCADE'), nullable=False)

student_id      = db.Column(UUID(as_uuid=True), db.ForeignKey('uzytkownicy.id', ondelete='CASCADE'), nullable=False)

uopz_id         = db.Column(UUID(as_uuid=True), db.ForeignKey('uzytkownicy.id', ondelete='SET NULL'), nullable=True)

firma_id        = db.Column(UUID(as_uuid=True), db.ForeignKey('firmy.id',        ondelete='SET NULL'), nullable=True)

```

```

status         = db.Column(

    db.Enum(StatusZapisu, name='status_zapisu', values_callable=lambda e: [x.value for x in e]),

    nullable=False, default=StatusZapisu.OCZEKUJACY,

)

```

```

sciezka = db.Column(

    db.Enum(SciezkaPraktyki, name='sciezka_praktyki', values_callable=lambda e: [x.value for x in e]),

    nullable=False, default=SciezkaPraktyki.STANDARDOWA,

)

```

```

termin_od       = db.Column(db.Date,          nullable=True)

```

```

termin_do          = db.Column(db.Date,          nullable=True)

specjalnosc        = db.Column(db.String(255), nullable=True)

ubezpieczenie_nw   = db.Column(db.Boolean,       default=False)


lacznie_godzin     = db.Column(db.Integer,  default=0)

zapisano_o         = db.Column(db.DateTime, server_default=db.func.now())


# — Relacje do encji satelitarnych —————

student           = db.relationship('Uzytkownik',          foreign_keys=[student_id], lazy='select')

uopz              = db.relationship('Uzytkownik',          foreign_keys=[uopz_id],    lazy='select')

firma             = db.relationship('Firma',                foreign_keys=[firma_id],  lazy='select')

dane_miejscas     = db.relationship('DaneMiejscasPraktyki', back_populates='zapis',    uselist=False, cascade='all,
delete-orphan')

uzasadnienie      = db.relationship('UzasadnienieSciezki', back_populates='zapis',    uselist=False, cascade='all,
delete-orphan')

sprawdzian        = db.relationship('Sprawdzian',          back_populates='zapis',    uselist=False, cascade='all,
delete-orphan')

oceny_koncowe     = db.relationship('OcenyKoncowe',         back_populates='zapis',    uselist=False, cascade='all,
delete-orphan')

zdarzenia         = db.relationship('ZdarzenieProces',     back_populates='zapis',    lazy='select', cascade='all,
delete-orphan', order_by='ZdarzenieProces.wykonano_o')

wpisy_dziennika   = db.relationship('WpisDziennika',       backref='zapis',          lazy='select', cascade='all,
delete-orphan')

oceny             = db.relationship('OcenaPraktyki',       backref='zapis',          lazy='select', cascade='all,
delete-orphan')

harmonogram       = db.relationship('HarmonogramPraktyki', backref='zapis',          lazy='select', cascade='all,
delete-orphan')

sprawozdanie      = db.relationship('Sprawozdanie',       backref='zapis',          uselist=False, lazy='select',
cascade='all, delete-orphan')


# — Compat – stare nazwy atrybutów —————

@property

def internship_id(self):

    return self.praktyka_id


@internship_id.setter

def internship_id(self, v):

    self.praktyka_id = v

```

```
@property
```

```
def track_type(self):  
    return self.sciezka
```

```
@track_type.setter
```

```
def track_type(self, v):  
    self.sciezka = v
```

```
@hybrid_property
```

```
def enrolled_at(self):  
    return self.zapisano_o
```

```
@enrolled_at.expression
```

```
def enrolled_at(cls):  
    return cls.zapisano_o
```

```
@property
```

```
def total_hours_logged(self):  
    return self.lacznie_godzin
```

```
# — Skróty do odczytu danych z encji satelitarnych (compat dla g() i szablonów) —
```

```
# dane_miejsca
```

```
@property
```

```
def firma_nazwa(self): return self.dane_miejsca.firma_nazwa if self.dane_miejsca else None
```

```
@property
```

```
def firma_adres(self): return self.dane_miejsca.firma_adres if self.dane_miejsca else None
```

```
@property
```

```
def firma_miasto(self): return self.dane_miejsca.firma_miasto if self.dane_miejsca else None
```

```
@property
```

```
def firma_nip_krs(self): return self.dane_miejsca.firma_nip_krs if self.dane_miejsca else None
```

```
@property
```

```
def firma_upowazniony_osoba(self): return self.dane_miejsca.firma_upowazniony_osoba if self.dane_miejsca else None
```

```
@property
```

```
def firma_upowazniony_stanowisko(self): return self.dane_miejsca.firma_upowazniony_stanowisko if self.dane_miejsca else None
```

```
@property
```

```
def zopz_imie_nazwisko(self): return self.dane_miejsca.zopz_imie_nazwisko if self.dane_miejsca else None
```

```
@property
```

```
def zopz_stanowisko(self): return self.dane_miejsca.zopz_stanowisko if self.dane_miejsca else None
```

```
@property
```

```
def zopz_telefon(self): return self.dane_miejsca.zopz_telefon if self.dane_miejsca else None
```

```
@property
```

```
def zopz_email(self): return self.dane_miejsca.zopz_email if self.dane_miejsca else None
```

```
# uzasadnienie ścieżki
```

```
@property
```

```
def uzasadnienie_sciezki(self): return self.uzasadnienie.uzasadnienie if self.uzasadnienie else None
```

```
@property
```

```
def zalaczniki_sciezki(self): return self.uzasadnienie.zalaczniki if self.uzasadnienie else None
```

```
# oceny końcowe
```

```
@property
```

```
def ocena_sprawozdania(self): return self.oceny_koncowe.ocena_sprawozdania if self.oceny_koncowe else None
```

```
@property
```

```
def ocena_uopz(self): return self.oceny_koncowe.ocena_uopz if self.oceny_koncowe else None
```

```
@property
```

```
def ocena_zopz(self): return self.oceny_koncowe.ocena_zopz if self.oceny_koncowe else None
```

```
@property
```

```
def ocena_opisowa_uopz(self): return self.oceny_koncowe.ocena_opisowa_uopz if self.oceny_koncowe else None
```

```
@property
```

```
def ocena_opisowa_zopz(self): return self.oceny_koncowe.ocena_opisowa_zopz if self.oceny_koncowe else None
```

```
# sprawdzian
```

```
@property
```

```
def sprawdzian_pytanie_1(self): return self.sprawdzian.pytanie_1 if self.sprawdzian else None
```

```
@property
```

```
def sprawdzian_ocena_1(self): return self.sprawdzian.ocena_1 if self.sprawdzian else None
```

```
@property
```

```
def sprawdzian_pytanie_2(self): return self.sprawdzian.pytanie_2 if self.sprawdzian else None
```

```
@property
```

```
def sprawdzian_ocena_2(self): return self.sprawdzian.ocena_2 if self.sprawdzian else None
```

```
@property
```

```
def sprawdzian_pytanie_3(self): return self.sprawdzian.pytanie_3 if self.sprawdzian else None
```

```
@property
```

```
def sprawdzian_ocena_3(self): return self.sprawdzian.ocena_3 if self.sprawdzian else None
```

```
# — Skróty do danych procesowych (odczytywane z event log) —————
```

```
def _ostatnie_zdarzenie(self, typ: TypZdarzenia):
```

```
    return next(
        (z for z in reversed(self.zdarzenia) if z.typ == typ),
        None,
    )
```

```
@property
```

```
def decyzja_komisji(self):
```

```
    z = self._ostatnie_zdarzenie(TypZdarzenia.KOMISJA_DECYZJA)
    return z.decyzja if z else None
```

```
@property
```

```
def komentarze_komisji(self):
```

```
    z = self._ostatnie_zdarzenie(TypZdarzenia.KOMISJA_DECYZJA)
    return z.komentarz if z else None
```

```
@property
```

```
def decyzja_komisji_o(self):
```

```
    z = self._ostatnie_zdarzenie(TypZdarzenia.KOMISJA_DECYZJA)
    return z.wykonano_o if z else None
```

```
@property
```

```
def decyzja_dziekana(self):
```

```
    z = self._ostatnie_zdarzenie(TypZdarzenia.DZIEKAN_DECYZJA)
```

```
return z. decyzja if z else None
```

```
@property
```

```
def komentarze_dziekana(self):
```

```
    z = self._ostatnie_zdarzenie(TypZdarzenia.DZIEKAN_DECYZJA)
```

```
    return z.komentarz if z else None
```

```
@property
```

```
def decyzja_dziekana_o(self):
```

```
    z = self._ostatnie_zdarzenie(TypZdarzenia.DZIEKAN_DECYZJA)
```

```
    return z.wykonano_o if z else None
```

```
@property
```

```
def komentarze_admina(self):
```

```
    z = self._ostatnie_zdarzenie(TypZdarzenia.ADMIN_KOMENTARZ)
```

```
    return z.komentarz if z else None
```

```
@property
```

```
def komentarze_uopz(self):
```

```
    z = self._ostatnie_zdarzenie(TypZdarzenia.UOPZ_KOMENTARZ)
```

```
    return z.komentarz if z else None
```

```
@property
```

```
def powiadomiono_studenta_o(self):
```

```
    z = self._ostatnie_zdarzenie(TypZdarzenia.POWIADOMIENIE_STUDENTA)
```

```
    return z.wykonano_o if z else None
```

```
# — Oceny (przez encję OcenyKoncowe i Sprawdzian) —————
```

```
@hybrid_property
```

```
def ocena_e(self):
```

```
    """Średnia ze sprawdzianu (instancja)."""
```

```
    if self.sprawdzian is None:
```

```
        return None
```

```

oceny = [

    float(v) for v in (

        self.sprawdzian.ocena_1,

        self.sprawdzian.ocena_2,

        self.sprawdzian.ocena_3,

    ) if v is not None

]

return round(sum(oceny) / len(oceny), 2) if oceny else None

```

@hybrid_property

def ocena_k(self):

"""Ocena końcowa: 0.4E + 0.1S + 0.2U + 0.3Z."""

if self.oceny_koncowe is None:

return None

e = self.ocena_e

s = float(self.oceny_koncowe.ocena_sprawozdania) if self.oceny_koncowe.ocena_sprawozdania is not None else None

u = float(self.oceny_koncowe.ocena_uopz) if self.oceny_koncowe.ocena_uopz is not None else None

z = float(self.oceny_koncowe.ocena_zopz) if self.oceny_koncowe.ocena_zopz is not None else None

if None in (e, s, u, z):

return None

return round(0.4 * e + 0.1 * s + 0.2 * u + 0.3 * z, 2)

— Encje satelitarne zapisu —————

class DaneMiejscaPraktyki(db.Model):

"""Snapshot danych zakładu i ZOPZ kopiowany z formularza do dokumentów TeX."""

__tablename__ = 'dane_miejsca_praktyki'

id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

zapis_id = db.Column(UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id', ondelete='CASCADE'), nullable=False, unique=True)

firma_nazwa = db.Column(db.String(255), nullable=True)

firma_adres = db.Column(db.String(255), nullable=True)

```
firma_miasto = db.Column(db.String(255), nullable=True)

firma_nip_krs = db.Column(db.String(50), nullable=True)

firma_upowazniony_osoba = db.Column(db.String(255), nullable=True)

firma_upowazniony_stanowisko = db.Column(db.String(255), nullable=True)
```

```
zopz_imie_nazwisko = db.Column(db.String(255), nullable=True)

zopz_stanowisko = db.Column(db.String(255), nullable=True)

zopz_telefon = db.Column(db.String(50), nullable=True)

zopz_email = db.Column(db.String(255), nullable=True)
```

```
zapis = db.relationship('ZapisPraktyki', back_populates='dane_miejsca')
```

```
class UzasadnienieSciezki(db.Model):
```

```
    """Uzasadnienie wyboru ścieżki B lub C (opcjonalne)."""
```

```
    __tablename__ = 'uzasadnienia_sciezki'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    zapis_id = db.Column(UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id', ondelete='CASCADE'), nullable=False,
unique=True)
```

```
    uzasadnienie = db.Column(db.Text, nullable=True)
```

```
    zalaczniki = db.Column(db.Text, nullable=True)
```

```
    zapis = db.relationship('ZapisPraktyki', back_populates='uzasadnienie')
```

```
class Sprawdzian(db.Model):
```

```
    """Trzy pytania sprawdzające z ocenami (wystawiane przez UOPZ)."""
```

```
    __tablename__ = 'sprawdziany'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    zapis_id = db.Column(UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id', ondelete='CASCADE'), nullable=False,
unique=True)
```

```
    pytanie_1 = db.Column(db.Text, nullable=True)
```



```

ocena_1 = db.Column(db.Numeric(3, 1), nullable=True)

pytanie_2 = db.Column(db.Text, nullable=True)

ocena_2 = db.Column(db.Numeric(3, 1), nullable=True)

pytanie_3 = db.Column(db.Text, nullable=True)

ocena_3 = db.Column(db.Numeric(3, 1), nullable=True)


zapis = db.relationship('ZapisPraktyki', back_populates='sprawdzian')

```

```

class OcenyKoncowe(db.Model):

```

```

    """Oceny składowe finalne praktyki."""

```

```

    __tablename__ = 'oceny_koncowe'

```

```

    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

```

```

    zapis_id = db.Column(UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id', ondelete='CASCADE'), nullable=False,
unique=True)

```

```

    ocena_sprawozdania = db.Column(db.Numeric(3, 1), nullable=True)

```

```

    ocena_uopz = db.Column(db.Numeric(3, 1), nullable=True)

```

```

    ocena_zopz = db.Column(db.Numeric(3, 1), nullable=True)

```

```

    ocena_opisowa_uopz = db.Column(db.Text, nullable=True)

```

```

    ocena_opisowa_zopz = db.Column(db.Text, nullable=True)

```

```

    zapis = db.relationship('ZapisPraktyki', back_populates='oceny_koncowe')

```

```

class ZdarzenieProces(db.Model):

```

```

    """Event log przepływu pracy: komentarze, decyzje komisji i dziekana."""

```

```

    __tablename__ = 'zdarzenia_procesowe'

```

```

    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

```

```

    zapis_id = db.Column(UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id', ondelete='CASCADE'), nullable=False)

```

```

    typ = db.Column(

```

```

        db.Enum(TypZdarzenia, name='typ_zdarzenia', values_callable=lambda e: [x.value for x in e]),

```

```

        nullable=False,

```

```

)

decyzja          = db.Column(db.String(20), nullable=True)  # 'APPROVED' / 'REJECTED'

komentarz        = db.Column(db.Text, nullable=True)

    wykonane_przez_id = db.Column(UUID(as_uuid=True), db.ForeignKey('uzytkownicy.id', ondelete='SET NULL'),
nullable=True)

wykonano_o       = db.Column(db.DateTime, server_default=db.func.now())

zapis            = db.relationship('ZapisPraktyki', back_populates='zdarzenia')

wykonane_przez   = db.relationship('Uzytkownik', foreign_keys=[wykonane_przez_id], lazy='select')

```

— Pozostałe modele —————

```

class HarmonogramPraktyki(db.Model):

    """Harmonogram realizacji efektów uczenia dla jednego zapisu."""

    __tablename__ = 'harmonogram_praktyk'

    id              = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

    zapis_id        = db.Column(UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id', ondelete='CASCADE'),
nullable=False)

    efekt_id        = db.Column(db.Integer, db.ForeignKey('efekty_uczenia.id'), nullable=False)

    nazwa_dzialu    = db.Column(db.String(255), nullable=False)

    przykladowe_prace = db.Column(db.Text, nullable=False)

    liczba_dni      = db.Column(db.Integer, nullable=False, default=0)

    efekt = db.relationship('EfektUczenia', lazy='select')

# Compat

@property
def enrollment_id(self):

    return self.zapis_id

@enrollment_id.setter
def enrollment_id(self, v):

    self.zapis_id = v

```

```
@property
```

```
def learning_outcome_id(self):
```

```
    return self.efekt_id
```

```
@learning_outcome_id.setter
```

```
def learning_outcome_id(self, v):
```

```
    self.efekt_id = v
```

```
class Sprawozdanie(db.Model):
```

```
    """Sprawozdanie studenta z odbytej praktyki."""
```

```
    __tablename__ = 'sprawozdania'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    zapis_id = db.Column(UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id', ondelete='CASCADE'),  
nullable=False, unique=True)
```

```
    charakterystyka_miejsca = db.Column(db.Text, nullable=True)
```

```
    opis_i_analiza = db.Column(db.Text, nullable=True)
```

```
    zaktualizowano = db.Column(db.DateTime, server_default=db.func.now(), onupdate=db.func.now())
```

```
@property
```

```
def enrollment_id(self):
```

```
    return self.zapis_id
```

```
class IndywidualnyProgram(db.Model):
```

```
    """Indywidualny program praktyki (opcjonalny)."""
```

```
    __tablename__ = 'programy_indywidualne'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    zapis_id = db.Column(UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id', ondelete='CASCADE'),  
nullable=False, unique=True)
```

```
    status = db.Column(db.String(30), nullable=False, default='DRAFT')
```

```
    zatwierdzony_przez_uopz = db.Column(db.Boolean, default=False)
```

```
zatwierdzono_o      = db.Column(db.DateTime,    nullable=True)

komentarz_uopz      = db.Column(db.Text,        nullable=True)

utworzono            = db.Column(db.DateTime,    server_default=db.func.now())
```

```
zapis = db.relationship('ZapisPraktyki', backref=db.backref('indywidualny_program', passive_deletes=True))
```

```
@property
```

```
def enrollment_id(self):

    return self.zapis_id
```

```
class NumerPisma(db.Model):
```

```
    """Kolejny numer pisma administracyjnego dla dokumentu."""
```

```
    __tablename__ = 'numery_pism'
```

```
id          = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

zapis_id    = db.Column(UUID(as_uuid=True), db.ForeignKey('zapisy_praktyk.id', ondelete='CASCADE'), nullable=False)

typ_dokumentu = db.Column(db.String(50), nullable=False)

numer       = db.Column(db.String(100), nullable=False)

wygenerowano = db.Column(db.DateTime,    server_default=db.func.now())
```

```
zapis = db.relationship('ZapisPraktyki')
```

```
@property
```

```
def enrollment_id(self):

    return self.zapis_id
```

```
@property
```

```
def document_type(self):

    return self.typ_dokumentu
```

PLIK: core\modele\uzytkownicy.py

```
"""core/modele/uzytkownicy.py
```

```
Modele domenowe: Uzytkownicy systemu.
```

```
Joined Table Inheritance (JTI): wspólna tabela uzytkownicy,  
rozszerzona przez Studenta, Administratora i OpikunaUczelnianego.
```

```
"""
```

```
import uuid
```

```
import enum
```

```
from flask_login import UserMixin
```

```
from sqlalchemy.dialects.postgresql import UUID
```

```
from core.extensions import db
```

```
# — Enumy dziedziny użytkowników —————
```

```
class RolaUzytkownika(enum.Enum):
```

```
    STUDENT      = 'STUDENT'
```

```
    UOPZ         = 'UOPZ'
```

```
    ADMIN        = 'ADMIN'
```

```
# — Klasa bazowa —————
```

```
class Uzytkownik(UserMixin, db.Model):
```

```
    """Wspólna tabela wszystkich kont w systemie.
```

```
    Discriminator: kolumna `rola` – SQLAlchemy automatycznie  
    zwraca właściwą podklasę przy zapytaniu przez klasę bazową.
```

```
    """
```

```
    __tablename__ = 'uzytkownicy'
```

```
    __mapper_args__ = {
```

```
    'polymorphic_on':      'rola',

    'polymorphic_identity': None,

}
```

```
id                = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

email             = db.Column(db.String(255), unique=True, nullable=False)

hash_hasla        = db.Column('hash_hasla', db.String(255), nullable=False)

imie              = db.Column('imie', db.String(100), nullable=False)

nazwisko          = db.Column('nazwisko', db.String(100), nullable=False)

rola              = db.Column(

    'rola',

    db.Enum(RolaUzytkownika, name='rola_uzytkownika', values_callable=lambda e: [x.value for x in e]),

    nullable=False,

)

aktywny           = db.Column('aktywny', db.Boolean, default=True)

wymagana_zmiana_hasla = db.Column(db.Boolean, default=True)

utworzono         = db.Column('utworzono', db.DateTime, server_default=db.func.now())
```

```
# — Compat z istniejącym kodem (Flask-Login, szablony) —————
```

```
@property

def password_hash(self):

    return self.hash_hasla
```

```
@password_hash.setter

def password_hash(self, v):

    self.hash_hasla = v
```

```
@property

def first_name(self):

    return self.imie
```

```
@first_name.setter

def first_name(self, v):
```

```
self.imie = v
```

```
@property
```

```
def last_name(self):  
    return self.nazwisko
```

```
@last_name.setter
```

```
def last_name(self, v):  
    self.nazwisko = v
```

```
@property
```

```
def role(self):  
    return self.rola
```

```
@role.setter
```

```
def role(self, v):  
    self.rola = v
```

```
@property
```

```
def is_active(self):  
    return self.aktywny
```

```
@is_active.setter
```

```
def is_active(self, v):  
    self.aktywny = v
```

```
@property
```

```
def created_at(self):  
    return self.utworzono
```

```
def get_id(self) -> str:  
    return str(self.id)
```

```
def __repr__(self) -> str:
```

```
return f'<Uzytkownik {self.email} ({self.rola})>'
```

```
# — Podklasy per rola — JTI —————
```

```
class Student(Uzytkownik):
```

```
    """Dane specyficzne dla studenta – tabela studenci."""
```

```
    __tablename__ = 'studenci'
```

```
    __mapper_args__ = {'polymorphic_identity': RolaUzytkownika.STUDENT}
```

```
    id = db.Column(UUID(as_uuid=True), db.ForeignKey('uzytkownicy.id', ondelete='CASCADE'), primary_key=True)
```

```
    numer_albumu = db.Column('numer_albumu', db.String(20), nullable=True)
```

```
    plec = db.Column('plec', db.String(1), nullable=True) # 'M' lub 'K'
```

```
    kierunek = db.Column('kierunek', db.String(100), nullable=True)
```

```
    specjalnosc = db.Column('specjalnosc', db.String(100), nullable=True)
```

```
    tryb_studiow = db.Column('tryb_studiow', db.String(20), nullable=True) # 'stacjonarne'/'niestacjonarne'
```

```
    # Compat
```

```
    @property
```

```
    def album_number(self):
```

```
        return self.numer_albumu
```

```
    @album_number.setter
```

```
    def album_number(self, v):
```

```
        self.numer_albumu = v
```

```
    def __repr__(self) -> str:
```

```
        return f'<Student {self.email} nr={self.numer_albumu}>'
```

```
class Administrator(Uzytkownik):
```

```
    """Konto administratora systemu."""
```

```
    __tablename__ = 'administratorzy'
```

```
    __mapper_args__ = {'polymorphic_identity': RolaUzytkownika.ADMIN}
```



```
id = db.Column(UUID(as_uuid=True), db.ForeignKey('uzytkownicy.id', ondelete='CASCADE'), primary_key=True)
```

```
def __repr__(self) -> str:
```

```
    return f'<Administrator {self.email}>'
```

```
class OpikunUczelniany(Uzytkownik):
```

```
    """Uczelniany Opiekun Praktyk Zawodowych (UOPZ)."""
```

```
    __tablename__ = 'opiekunowie_uczelniani'
```

```
    __mapper_args__ = {'polymorphic_identity': RolaUzytkownika.UOPZ}
```

```
id = db.Column(UUID(as_uuid=True), db.ForeignKey('uzytkownicy.id', ondelete='CASCADE'), primary_key=True)
```

```
def __repr__(self) -> str:
```

```
    return f'<OpikunUczelniany {self.email}>'
```

PLIK: core\modele__init__.py

```
"""core/modele/__init__.py
```

Eksportuje wszystkie modele domenowe.

Flask-Migrate (Alembic) musi wiedzieć każdą klasę db.Model,

zanim wygeneruje skrypt migracji – stąd gwiazdkowe importy.

```
"""
```

```
from core.modele.uzytownicy import (
```

```
    RolaUzytkownika,
```

```
    Uzytkownik,
```

```
    Student,
```

```
    Administrator,
```

```
    OpikunUczelni,
```

```
)
```

```
from core.modele.firmy import Firma
```

```
from core.modele.praktyki import (
```

```
    StatusPraktyki,
```

```
    StatusZapisu,
```

```
    SciezkaPraktyki,
```

```
    TypZdarzenia,
```

```
    Praktyka,
```

```
    ZapisPraktyki,
```

```
    DaneMiejscPraktyki,
```

```
    UzasadnienieSciezki,
```

```
    Sprawdzian,
```

```
    OcenyKoncowe,
```

```
    ZdarzenieProces,
```

```
    HarmonogramPraktyki,
```

```
    Sprawozdanie,
```

```
    IndywidualnyProgram,
```

```
    NumerPisma,
```

)

```
from core.modele.dziennik import (
```

```
    WynikOceny,  
    EfektUczenia,  
    wpisy_efekty,  
    WpisDziennika,  
    OcenaPraktyki,
```

)

```
from core.modele.dokumenty import (
```

```
    StatusDokumentu,  
    LogAudytuDokumentow,  
    DocumentAuditLog,          # alias  
    DokumentPrzeslany,  
    UploadedDocument,          # alias
```

)

```
__all__ = [
```

```
    # uzytkownicy  
  
    'RolaUzytkownika', 'Uzytkownik', 'Student', 'Administrator', 'OpikunUczelnianny',  
  
    # firmy  
  
    'Firma',  
  
    # praktyki  
  
    'StatusPraktyki', 'StatusZapisu', 'SciezkaPraktyki', 'TypZdarzenia',  
  
    'Praktyka', 'ZapisPraktyki',  
  
    'DaneMiejscPraktyki', 'UzasadnienieSciezki', 'Sprawdzian', 'OcenyKoncowe', 'ZdarzenieProces',  
  
    'HarmonogramPraktyki', 'Sprawozdanie', 'IndywidualnyProgram', 'NumerPisma',  
  
    # dziennik  
  
    'WynikOceny', 'EfektUczenia', 'wpisy_efekty', 'WpisDziennika', 'OcenaPraktyki',  
  
    # dokumenty  
  
    'StatusDokumentu', 'LogAudytuDokumentow', 'DocumentAuditLog',  
  
    'DokumentPrzeslany', 'UploadedDocument',
```

]

PLIK: core\repozytoria\praktyki.py

```
"""core/repozytoria/praktyki.py

Repozytoria edycji praktyk i zapisów studentów.

"""

from __future__ import annotations

from typing import Optional

import uuid

from core.extensions import db

from core.modele.praktyki import (
    Praktyka,
    StatusPraktyki,
    ZapisPraktyki,
    StatusZapisu,
)

class RepozytoriumPraktyk:

    """Dostęp do edycji praktyk (rok akademicki / semestr)."""

    def znajdz_po_id(self, praktyka_id: uuid.UUID) -> Optional[Praktyka]:

        return db.session.get(Praktyka, praktyka_id)

    def wszystkie(self) -> list[Praktyka]:

        return db.session.query(Praktyka).order_by(Praktyka.rok_ucelniany.desc(), Praktyka.semestr).all()

    def aktywne(self) -> list[Praktyka]:

        return (
            db.session.query(Praktyka)
                .filter_by(status=StatusPraktyki.AKTYWNA)
                .order_by(Praktyka.rok_ucelniany.desc())
                .all()
        )
```

)

```
def aktywna_edycja(self) -> Optional[Praktyka]:  
  
    """Zwraca pierwszą aktywną edycję praktyk lub None."""  
  
    return (  
  
        db.session.query(Praktyka)  
  
        .filter_by(status=StatusPraktyki.AKTYWNA)  
  
        .order_by(Praktyka.rok_uczelniany.desc())  
  
        .first()  
  
    )
```

```
def zapisz(self, praktyka: Praktyka) -> Praktyka:  
  
    db.session.add(praktyka)  
  
    db.session.flush()  
  
    return praktyka
```

```
def usun(self, praktyka: Praktyka) -> None:  
  
    db.session.delete(praktyka)
```

```
class RepozytoriumZapisow:
```

```
    """Dostęp do zapisów studentów na edycje praktyk."""
```

```
def znajdz_po_id(self, zapis_id: uuid.UUID) -> Optional[ZapisPraktyki]:  
  
    return db.session.get(ZapisPraktyki, zapis_id)
```

```
def wszystkie(self) -> list[ZapisPraktyki]:  
  
    return db.session.query(ZapisPraktyki).order_by(ZapisPraktyki.zapisano_o.desc()).all()
```

```
def dla_studenta(self, student_id: uuid.UUID) -> list[ZapisPraktyki]:  
  
    return (  
  
        db.session.query(ZapisPraktyki)  
  
        .filter_by(student_id=student_id)  
  
        .order_by(ZapisPraktyki.zapisano_o.desc())
```

```
.all()

)
```

```
def dla_praktyki(self, praktyka_id: uuid.UUID) -> list[ZapisPraktyki]:
```

```
    return (

        db.session.query(ZapisPraktyki)

        .filter_by(praktyka_id=praktyka_id)

        .order_by(ZapisPraktyki.zapisano_o.desc())

        .all()

    )
```

```
def dla_opiekuna(self, uopz_id: uuid.UUID) -> list[ZapisPraktyki]:
```

```
    return (

        db.session.query(ZapisPraktyki)

        .filter_by(uopz_id=uopz_id)

        .order_by(ZapisPraktyki.zapisano_o.desc())

        .all()

    )
```

```
def po_statusie(self, status: StatusZapisu) -> list[ZapisPraktyki]:
```

```
    return (

        db.session.query(ZapisPraktyki)

        .filter_by(status=status)

        .order_by(ZapisPraktyki.zapisano_o.desc())

        .all()

    )
```

```
def student_ma_aktywny_zapis(self, student_id: uuid.UUID, praktyka_id: uuid.UUID) -> bool:
```

```
    """Sprawdza czy student ma już zapis do danej edycji (inny niż ODRZUCONY)."""

    q = (

        db.session.query(ZapisPraktyki)

        .filter_by(student_id=student_id, praktyka_id=praktyka_id)

        .filter(ZapisPraktyki.status != StatusZapisu.ODRZUCONA)

    )
```

```
return db.session.query(q.exists()).scalar()
```

```
def zapisz(self, zapis: ZapisPraktyki) -> ZapisPraktyki:
```

```
    db.session.add(zapis)
```

```
    db.session.flush()
```

```
    return zapis
```

```
def usun(self, zapis: ZapisPraktyki) -> None:
```

```
    db.session.delete(zapis)
```

PLIK: core\repozytoria\uzytkownicy.py

```
"""core/repozytoria/uzytkownicy.py
```

Repozytorium użytkowników – jedyne miejsce zapytań ORM

dotyczących tabeli `uzytkownicy` i jej podtabel JTII.

```
"""
```

```
from __future__ import annotations
```

```
from typing import Optional
```

```
import uuid
```

```
from core.extensions import db
```

```
from core.modele.uzytkownicy import RolaUzytkownika, Uzytkownik, Student, Administrator, OpikunUczelni
```

```
class RepozytoriumUzytkownikow:
```

```
    """Dostęp do danych użytkowników systemu."""
```

```
    # — Wyszukiwanie —————
```

```
    def znajdz_po_id(self, uzytkownik_id: uuid.UUID) -> Optional[Uzytkownik]:
```

```
        return db.session.get(Uzytkownik, uzytkownik_id)
```

```
    def znajdz_po_emailu(self, email: str) -> Optional[Uzytkownik]:
```

```
        return db.session.query(Uzytkownik).filter_by(email=email).first()
```

```
    def wszyscy(self) -> list[Uzytkownik]:
```

```
        return db.session.query(Uzytkownik).order_by(Uzytkownik.nazwisko, Uzytkownik.imie).all()
```

```
    def aktywni(self) -> list[Uzytkownik]:
```

```
        return db.session.query(Uzytkownik).filter_by(aktywny=True).order_by(Uzytkownik.nazwisko).all()
```

```
    # — Według roli —————
```



```
def wszyscy_studenci(self) -> list[Student]:  
  
    return db.session.query(Student).order_by(Uzytkownik.nazwisko, Uzytkownik.imie).all()
```

```
def wszyscy_administratorzy(self) -> list[Administrator]:  
  
    return db.session.query(Administrator).order_by(Uzytkownik.nazwisko).all()
```

```
def wszyscy_opiekunowie(self) -> list[OpikunUczelniacy]:  
  
    return db.session.query(OpikunUczelniacy).order_by(Uzytkownik.nazwisko).all()
```

```
def opiekunowie_aktywni(self) -> list[OpikunUczelniacy]:  
  
    return (  
  
        db.session.query(OpikunUczelniacy)  
  
        .filter(Uzytkownik.aktywny.is_(True))  
  
        .order_by(Uzytkownik.nazwisko)  
  
        .all()  
  
    )
```

— Zapis / usunięcie —————

```
def zapisz(self, uzytkownik: Uzytkownik) -> Uzytkownik:  
  
    db.session.add(uzytkownik)  
  
    db.session.flush()  
  
    return uzytkownik
```

```
def usun(self, uzytkownik: Uzytkownik) -> None:  
  
    db.session.delete(uzytkownik)
```

— Pomocnicze —————

```
def istnieje_email(self, email: str, pomini_j_id: Optional[uuid.UUID] = None) -> bool:  
  
    q = db.session.query(Uzytkownik).filter_by(email=email)  
  
    if pomini_j_id is not None:  
  
        q = q.filter(Uzytkownik.id != pomini_j_id)  
  
    return db.session.query(q.exists()).scalar()
```

PLIK: core\repozytoria__init__.py

"""core/repozytoria – warstwa dostępu do danych (Repository Pattern).

Każda klasa repozytorium jest jedynym miejscem, w którym

konstruowane są zapytania ORM do konkretnej domeny.

Kontrolery i serwisy nigdy nie wywołują db.session.query() bezpośrednio.

"""

from core.repozytoria uzytkownicy import RepozytoriumUzytkownikow

from core.repozytoria.praktyki import RepozytoriumPraktyk, RepozytoriumZapisow

__all__ = [

'RepozytoriumUzytkownikow',

'RepozytoriumPraktyk',

'RepozytoriumZapisow',

]

PLIK: core\static\css\autoryzacja.css

```
/*  
  
autoryzacja.css  
  
Jedyny arkusz stylów dla ekranów autoryzacji (logowanie, zmiana hasła).  
  
Cel: jedno podejście bez rozproszenia stylów po wielu plikach.  
  
*/  
  
:root {  
  
  --kolor-glowny:          #1a3a5c;  
  
  --kolor-glowny-ciemny:   #0f2540;  
  
  --kolor-glowny-jasny:    #2a5080;  
  
  --kolor-akcent:          #c8963e;  
  
  --kolor-akcent-jasny:    #e8b45a;  
  
  
  --kolor-tlo:             #f4f6f9;  
  
  --kolor-biale:           #ffffff;  
  
  --kolor-tekst:           #1a1a2e;  
  
  --kolor-tekst-drugor:    #4a5568;  
  
  --kolor-tekst-trzeciorzed: #718096;  
  
  
  --kolor-ramka:           #d1d9e6;  
  
  --kolor-blad:            #c0392b;  
  
  --kolor-sukces:          #1e6e3a;  
  
  --kolor-info:            #1a5276;  
  
  
  --fokus-kolor:           #c8963e;  
  
  --fokus-grubosc:         3px;  
  
  --fokus-offset:         3px;  
  
  
  --font-glowny:           'IBM Plex Sans', 'Segoe UI', Tahoma, sans-serif;  
  
  
  --promien:               8px;  
  
  --cien-karta:            0 2px 12px rgba(26, 58, 92, 0.12);  
  
}
```

```
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
```

```
html {  
  
    font-size: 100%;  
  
    -webkit-text-size-adjust: 100%;  
  
    scroll-behavior: smooth;  
  
}
```

```
body {  
  
    font-family: var(--font-glowny), serif;  
  
    font-size: 1rem;  
  
    line-height: 1.6;  
  
    color: var(--kolor-tekst);  
  
    background-color: var(--kolor-tlo);  
  
    min-height: 100vh;  
  
}
```

```
:focus-visible {  
  
    outline: var(--fokus-grubosc) solid var(--fokus-kolor);  
  
    outline-offset: var(--fokus-offset);  
  
    border-radius: calc(var(--promien) - 2px);  
  
}
```

```
:focus:not(:focus-visible) { outline: none; }
```

```
.skip-link {  
  
    position: absolute; top: -100px; left: 1rem; z-index: 9999;  
  
    padding: 0.75rem 1.5rem; background: var(--kolor-glowny); color: var(--kolor-biale);  
  
    font-weight: 600; border-radius: var(--promien); text-decoration: none; transition: top 0.2s;  
  
}
```

```
.skip-link:focus { top: 1rem; }
```

```
.strona-logowania { background: var(--kolor-tlo); }
```

```
.panel-logowania { display: grid; grid-template-columns: 1fr 1fr; min-height: 100vh; }
```

```

.panel-logowania__branding { background: var(--kolor-glowny); display: flex; align-items: center; justify-content: center; padding: 3rem 2.5rem; position: relative; overflow: hidden; }

.panel-logowania__branding::before { content: ''; position: absolute; inset: 0; background-image: repeating-linear-gradient(45deg, transparent, transparent 40px, rgba(255,255,255,0.025) 40px, rgba(255,255,255,0.025) 41px); pointer-events: none; }

.panel-logowania__branding::after { content: ''; position: absolute; top: 0; right: 0; width: 6px; height: 100%; background: linear-gradient(to bottom, var(--kolor-akcent) 0%, var(--kolor-akcent-jasny) 50%, var(--kolor-akcent) 100%); }

.branding__tresc { position: relative; z-index: 1; display: flex; flex-direction: column; align-items: flex-start; gap: 1.5rem; max-width: 320px; }

.branding__logo-kontener { background: white; padding: 10px; border-radius: var(--promien); display: flex; align-items: center; justify-content: center; box-shadow: var(--cien-karta); }

.branding__logo { max-width: 200px; height: auto; display: block; }

.branding__separator { width: 48px; height: 3px; background: var(--kolor-akcent); border-radius: 2px; }

.branding__nazwa-systemu { font-size: 1.5rem; font-weight: 600; color: var(--kolor-biale); line-height: 1.3; letter-spacing: -0.01em; }

.branding__podtytul { font-size: 0.9rem; color: rgba(255,255,255,0.65); letter-spacing: 0.08em; text-transform: uppercase; }

.branding__dekoracja { display: flex; gap: 8px; margin-top: 1rem; }

.branding__dekoracja span { display: block; width: 8px; height: 8px; border-radius: 50%; background: var(--kolor-akcent); opacity: 0.6; }

.branding__dekoracja span:first-child { opacity: 1; }

.branding__dekoracja span:last-child { opacity: 0.3; }

.panel-logowania__formularz { display: flex; align-items: center; justify-content: center; padding: 3rem 2rem; background: var(--kolor-biale); }

.formularz__kontener { width: 100%; max-width: 420px; }

.formularz__naglowek { margin-bottom: 2rem; }

.formularz__tytul { font-size: 1.75rem; font-weight: 600; color: var(--kolor-glowny); line-height: 1.2; letter-spacing: -0.02em; margin-bottom: 0.5rem; }

.formularz__opis { font-size: 0.95rem; color: var(--kolor-tekst-dlugor); }

.komunikaty { margin-bottom: 1.5rem; display: flex; flex-direction: column; gap: 0.5rem; }

.komunikat { display: flex; align-items: center; gap: 0.5rem; padding: 0.5rem 0.75rem; border-radius: 10px; font-size: 0.9rem; line-height: 1.2; border-left: 4px solid; }

.komunikat--danger { background: #fdf2f2; border-color: var(--kolor-blad); color: #8b1a1a; }

.komunikat--success { background: #f0faf4; border-color: var(--kolor-sukces); color: #14452a; }

.komunikat--info { background: #f0f6fc; border-color: var(--kolor-info); color: #0f3449; }

.komunikat__ikona { font-size: 0.85rem; font-weight: 700; flex-shrink: 0; }

```

```

.formularz__forma { display: flex; flex-direction: column; gap: 1.25rem; }

.pole-formularza { display: flex; flex-direction: column; gap: 0.375rem; }

.pole-formularza__etykieta { font-size: 0.875rem; font-weight: 500; color: var(--kolor-tekst); }

.pole-formularza__wymagane { color: var(--kolor-blad); margin-left: 0.2em; font-weight: 700; }


.pole-formularza__input { width: 100%; padding: 0.75rem 1rem; font-family: var(--font-glowny), serif; font-size: 1rem;
color: var(--kolor-tekst); background: var(--kolor-biale); border: 2px solid var(--kolor-ramka); border-radius:
var(--promien); transition: border-color 0.15s, box-shadow 0.15s; }

.pole-formularza__input:hover { border-color: var(--kolor-glowny-jasny); }

.pole-formularza__input:focus { border-color: var(--kolor-glowny); box-shadow: 0 0 0 3px rgba(26,58,92,0.15); }

.pole-formularza--blad .pole-formularza__input { border-color: var(--kolor-blad); }


.pole-formularza__input-wrapper { position: relative; display: flex; align-items: center; }

.pole-formularza__input-wrapper .pole-formularza__input { padding-right: 3rem; }

.pole-formularza__toggle-haslo { position: absolute; right: 0.75rem; background: none; border: none; cursor: pointer;
padding: 0.25rem; color: var(--kolor-tekst-drugor); border-radius: var(--promien); font-size: 1rem; line-height: 1; }

.pole-formularza__toggle-haslo:hover { color: var(--kolor-glowny); }

.pole-formularza__blad { font-size: 0.825rem; color: var(--kolor-blad); font-weight: 500; }


.pole-formularza--checkbox { flex-direction: row; }

.pole-formularza__etykieta-checkbox { display: flex; align-items: center; gap: 0.625rem; cursor: pointer; font-size:
0.9rem; color: var(--kolor-tekst-drugor); }

.pole-formularza__checkbox { width: 18px; height: 18px; flex-shrink: 0; cursor: pointer; accent-color:
var(--kolor-glowny); }


.przycisk-logowania { width: 100%; padding: 0.875rem 1.5rem; margin-top: 0.5rem; font-family: var(--font-glowny), serif;
font-size: 1rem; font-weight: 600; color: var(--kolor-biale); background: var(--kolor-glowny); border: none;
border-radius: var(--promien); cursor: pointer; letter-spacing: 0.02em; transition: background-color 0.15s, transform
0.1s; min-height: 48px; }

.przycisk-logowania:hover { background: var(--kolor-glowny-ciemny); }

.przycisk-logowania:active { transform: translateY(1px); }


.formularz__stopka { margin-top: 2rem; padding-top: 1.5rem; border-top: 1px solid var(--kolor-ramka); display: flex;
flex-direction: column; gap: 0.375rem; }

.formularz__stopka-tekst { font-size: 0.825rem; color: var(--kolor-tekst-trzeciorzed); text-align: center; }

@media (max-width: 900px) {

    .panel-logowania { grid-template-columns: 1fr; }

```

```
.panel-logowania__branding { padding: 2.5rem 2rem; min-height: auto; }

.branding__tresc { flex-direction: row; flex-wrap: wrap; align-items: center; max-width: 100%; gap: 1rem; }

.branding__separator, .branding__podtytul, .branding__dekoracja { display: none; }

.branding__nazwa-systemu { font-size: 1.1rem; }

}

@media (max-width: 480px) {

    .panel-logowania__formularz { padding: 1.5rem 1rem; }

    .formularz__kontener { max-width: 100%; }

    .formularz__tytul { font-size: 1.4rem; }

}
```

PLIK: core\static\css\modul\dashboard.css

```
/* dashboard.css – Statystyki, strona błędu */
```

```
.statystyki-grid, .siatka-statystyk { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 1.5rem; margin-bottom: 2rem; }
```

```
.stat-karta, .karta-stat { background: var(--kolor-biale); padding: 1.5rem; border-radius: var(--promien-duzy); border: 1px solid var(--kolor-ramka); box-shadow: var(--cien-karta); display: flex; flex-direction: column; align-items: center; text-align: center; gap: 0.5rem; }
```

```
.stat-karta__etykieta, .karta-stat__etykieta { font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.08em; color: var(--kolor-tekst-drugor); }
```

```
.stat-karta__wartosc, .karta-stat__liczba { font-size: 2rem; font-weight: 700; color: var(--kolor-glowny); line-height: 1; }
```

```
.karta-stat__ikona { color: var(--kolor-akcent); line-height: 1; }
```

```
.karta-stat__ikona svg { width: 28px; height: 28px; }
```

```
@media (max-width: 768px) { .statystyki-grid { grid-template-columns: 1fr 1fr; } }
```

```
.strona-bledu { display: flex; flex-direction: column; align-items: center; justify-content: center; text-align: center; padding: 4rem 2rem; min-height: 60vh; }
```

```
.strona-bledu__kod { font-size: 5rem; font-weight: 800; color: var(--kolor-ramka); line-height: 1; margin-bottom: 1rem; }
```

```
.strona-bledu__tytul { font-size: 1.5rem; font-weight: 600; color: var(--kolor-tekst); margin-bottom: 0.5rem; }
```

```
.strona-bledu__opis { color: var(--kolor-tekst-drugor); max-width: 400px; margin: 0 auto; }
```


PLIK: core\static\css\modul\formularze.css

```
/* formularze.css – Pola formularzy, inputy, checkboxy */
```

```
.formularz__forma { display: flex; flex-direction: column; gap: 1.25rem; }

.formularz-kontener { display: flex; justify-content: center; }

.formularz-kontener > .karta { width: 100%; max-width: 640px; }

.pole-formularza { display: flex; flex-direction: column; gap: 0.375rem; }

.pole-formularza__naglowek-haslo { display: flex; align-items: center; justify-content: space-between; }

.pole-formularza__etykieta { font-size: 0.875rem; font-weight: 500; color: var(--kolor-tekst); }

.pole-formularza__wymagane { color: var(--kolor-blad); margin-left: 0.2em; font-weight: 700; }

.pole-formularza__input { width: 100%; padding: 0.75rem 1rem; font-family: var(--font-glowny), serif; font-size: 1rem;
color: var(--kolor-tekst); background: var(--kolor-biale); border: 2px solid var(--kolor-ramka); border-radius:
var(--promien); transition: border-color 0.15s, box-shadow 0.15s; }

.pole-formularza__input:hover { border-color: var(--kolor-glowny-jasny); }

.pole-formularza__input:focus { border-color: var(--kolor-glowny); box-shadow: 0 0 0 3px rgba(26,58,92,0.15); outline:
var(--fokus-grubosc) solid var(--fokus-kolor); outline-offset: var(--fokus-offset); }

.pole-formularza--blad .pole-formularza__input { border-color: var(--kolor-blad); }

.pole-formularza--blad .pole-formularza__input:focus { box-shadow: 0 0 0 3px rgba(192,57,43,0.15); }

.pole-formularza__input-wrapper { position: relative; display: flex; align-items: center; }

.pole-formularza__input-wrapper .pole-formularza__input { padding-right: 3rem; }

.pole-formularza__toggle-haslo { position: absolute; right: 0.75rem; background: none; border: none; cursor: pointer;
padding: 0.25rem; color: var(--kolor-tekst-drugor); border-radius: var(--promien); font-size: 1rem; line-height: 1;
transition: color 0.15s; }

.pole-formularza__toggle-haslo:hover { color: var(--kolor-glowny); }

.pole-formularza__blad { font-size: 0.825rem; color: var(--kolor-blad); font-weight: 500; }

.pole-formularza--checkbox { flex-direction: row; }

.pole-formularza__etykieta-checkbox { display: flex; align-items: center; gap: 0.625rem; cursor: pointer; font-size:
0.9rem; color: var(--kolor-tekst-drugor); }

.pole-formularza__checkbox { width: 18px; height: 18px; flex-shrink: 0; cursor: pointer; accent-color:
var(--kolor-glowny); }
```

PLIK: core\static\css\modul\karty.css

```
/* karty.css – Komponent karty (.karta) */
```

```
.karta { background: var(--kolor-biale); border: 1px solid var(--kolor-ramka); border-radius: var(--promien-duzy);  
box-shadow: var(--cien-karta); overflow: hidden; }
```

```
.karta__naglowek { padding: 1rem 1.5rem; border-bottom: 1px solid var(--kolor-ramka); display: flex; align-items: center;  
justify-content: space-between; gap: 1rem; flex-wrap: wrap; }
```

```
.karta__tytul { font-size: 1rem; font-weight: 600; color: var(--kolor-tekst); }
```

```
.karta__tresc { padding: 1.5rem; }
```

PLIK: core\static\css\modul\komponenty.css

/* komponenty.css – Pasek postępu, paginacja, filtry */

```
.pasek-postep { height: 6px; background: var(--kolor-tlo); border-radius: 10px; overflow: hidden; }
```

```
.pasek-postep__wypelnienie { height: 100%; background: var(--kolor-glowny); border-radius: 10px; transition: width 0.4s ease; }
```

```
.pasek-postep__wypelnienie--zakonczone { background: var(--kolor-sukces); }
```

```
.paginacja { display: flex; align-items: center; justify-content: space-between; padding: 1rem 1.5rem; border-top: 1px solid var(--kolor-ramka); }
```

```
.paginacja__info { font-size: 0.8rem; color: var(--kolor-tekst-drugor); }
```

```
.paginacja__przyciski { display: flex; align-items: center; gap: 0.25rem; }
```

```
.paginacja__przycisk { display: inline-flex; align-items: center; justify-content: center; min-width: 32px; height: 32px; padding: 0 0.5rem; font-size: 0.8rem; font-weight: 500; text-decoration: none; color: var(--kolor-tekst-drugor); background: var(--kolor-biale); border: 1px solid var(--kolor-ramka); border-radius: var(--promien); transition: background 0.15s; }
```

```
.paginacja__przycisk:hover { background: var(--kolor-tlo); }
```

```
.paginacja__przycisk--aktywny { background: var(--kolor-glowny); color: var(--kolor-biale); border-color: var(--kolor-glowny); }
```

```
.paginacja__separator { padding: 0 0.25rem; color: var(--kolor-tekst-trzeciorzed); }
```

```
.pasek-filtrow { display: flex; align-items: center; gap: 0.75rem; flex-wrap: wrap; width: 100%; }
```

```
.filtr-szukaj { display: flex; align-items: center; gap: 0.5rem; flex: 1; min-width: 200px; background: var(--kolor-tlo); border: 1px solid var(--kolor-ramka); border-radius: var(--promien); padding: 0 0.75rem; }
```

```
.filtr-szukaj__ikona { color: var(--kolor-tekst-trzeciorzed); font-size: 0.85rem; flex-shrink: 0; }
```

```
.filtr-szukaj__input { flex: 1; border: none; background: transparent; padding: 0.5rem 0; font-family: var(--font-glowny); font-size: 0.85rem; color: var(--kolor-tekst); outline: none; }
```

```
.filtr-select { padding: 0.5rem 0.75rem; font-family: var(--font-glowny); font-size: 0.85rem; color: var(--kolor-tekst); background: var(--kolor-tlo); border: 1px solid var(--kolor-ramka); border-radius: var(--promien); cursor: pointer; }
```

PLIK: core\static\css\modul\komunikaty.css

/* komunikaty.css – Komunikaty flash i powiadomienia */

```
.komunikaty { margin-bottom: 1.5rem; display: flex; flex-direction: column; gap: 0.5rem; }
```

```
.komunikat { display: flex; align-items: center; gap: 0.5rem; padding: 0.5rem 0.75rem; border-radius: 10px; font-size: 0.9rem; line-height: 1.2; border-left: 4px solid; }
```

```
.komunikat--danger { background: #fdf2f2; border-color: var(--kolor-blad); color: #8b1a1a; }
```

```
.komunikat--success { background: #f0faf4; border-color: var(--kolor-sukces); color: #14452a; }
```

```
.komunikat--info { background: #f0f6fc; border-color: var(--kolor-info); color: #0f3449; }
```

```
.komunikat__ikona { font-size: 0.85rem; font-weight: 700; flex-shrink: 0; }
```

```
.komunikat__tekst { flex: 1; }
```

```
.komunikaty--fixed { position: fixed; right: 1rem; bottom: 1rem; z-index: 1200; width: calc(100% - 2rem); max-width: 360px; }
```

```
.komunikat--minimal { box-shadow: 0 4px 16px rgba(10,10,20,0.08); }
```

```
.komunikat__zamknij { margin-left: auto; background: none; border: none; cursor: pointer; font-size: 1.1rem; color: inherit; opacity: 0.5; padding: 0 0.5rem; line-height: 1; }
```

```
.komunikat__zamknij:hover { opacity: 1; }
```

PLIK: core\static\css\modul\logowanie.css

```
/* logowanie.css – Strona logowania i zmiana hasła */
```

```
.strona-logowania { background: var(--kolor-tlo); }
```

```
.panel-logowania { display: grid; grid-template-columns: 1fr 1fr; min-height: 100vh; }
```

```
.panel-logowania__branding { background: var(--kolor-glowny); display: flex; align-items: center; justify-content: center; padding: 3rem 2.5rem; position: relative; overflow: hidden; }
```

```
.panel-logowania__branding::before { content: ''; position: absolute; inset: 0; background-image: repeating-linear-gradient(45deg, transparent, transparent 40px, rgba(255,255,255,0.025) 40px, rgba(255,255,255,0.025) 41px); pointer-events: none; }
```

```
.panel-logowania__branding::after { content: ''; position: absolute; top: 0; right: 0; width: 6px; height: 100%; background: linear-gradient(to bottom, var(--kolor-akcent) 0%, var(--kolor-akcent-jasny) 50%, var(--kolor-akcent) 100%); }
```

```
.branding__tresc { position: relative; z-index: 1; display: flex; flex-direction: column; align-items: flex-start; gap: 1.5rem; max-width: 320px; }
```

```
.branding__logo-kontener { background: white; padding: 10px; border-radius: var(--promien); display: flex; align-items: center; justify-content: center; box-shadow: var(--cien-karta); }
```

```
.branding__logo { max-width: 200px; height: auto; display: block; }
```

```
.branding__separator { width: 48px; height: 3px; background: var(--kolor-akcent); border-radius: 2px; }
```

```
.branding__nazwa-systemu { font-size: 1.5rem; font-weight: 600; color: var(--kolor-biale); line-height: 1.3; letter-spacing: -0.01em; }
```

```
.branding__podtytul { font-size: 0.9rem; color: rgba(255,255,255,0.65); letter-spacing: 0.08em; text-transform: uppercase; }
```

```
.branding__dekoracja { display: flex; gap: 8px; margin-top: 1rem; }
```

```
.branding__dekoracja span { display: block; width: 8px; height: 8px; border-radius: 50%; background: var(--kolor-akcent); opacity: 0.6; }
```

```
.branding__dekoracja span:first-child { opacity: 1; }
```

```
.branding__dekoracja span:last-child { opacity: 0.3; }
```

```
.panel-logowania__formularz { display: flex; align-items: center; justify-content: center; padding: 3rem 2rem; background: var(--kolor-biale); }
```

```
.formularz__kontener { width: 100%; max-width: 420px; }
```

```
.formularz__naglowek { margin-bottom: 2rem; }
```

```
.formularz__tytul { font-size: 1.75rem; font-weight: 600; color: var(--kolor-glowny); line-height: 1.2; letter-spacing: -0.02em; margin-bottom: 0.5rem; }
```

```
.formularz__opis { font-size: 0.95rem; color: var(--kolor-tekst-drugor); }
```

```
.przycisk-logowania { width: 100%; padding: 0.875rem 1.5rem; margin-top: 0.5rem; font-family: var(--font-glowny), serif; }
```

```
font-size: 1rem; font-weight: 600; color: var(--kolor-biale); background: var(--kolor-glowny); border: none;
border-radius: var(--promien); cursor: pointer; letter-spacing: 0.02em; transition: background-color 0.15s, transform
0.1s; min-height: 48px; }
```

```
.przycisk-logowania:hover { background: var(--kolor-glowny-ciemny); }
```

```
.przycisk-logowania:active { transform: translateY(1px); }
```

```
.przycisk-logowania:focus-visible { outline: var(--fokus-grubosc) solid var(--fokus-kolor); outline-offset:
var(--fokus-offset); }
```

```
.formularz__stopka { margin-top: 2rem; padding-top: 1.5rem; border-top: 1px solid var(--kolor-ramka); display: flex;
flex-direction: column; gap: 0.375rem; }
```

```
.formularz__stopka-tekst { font-size: 0.825rem; color: var(--kolor-tekst-trzeciorzed); text-align: center; }
```

```
.formularz__link { color: var(--kolor-glowny-jasny); text-decoration: underline; text-underline-offset: 2px; font-weight:
500; }
```

```
.formularz__link:hover { color: var(--kolor-glowny); }
```

```
@media (max-width: 900px) {
```

```
  .panel-logowania { grid-template-columns: 1fr; }
```

```
  .panel-logowania__branding { padding: 2.5rem 2rem; min-height: auto; }
```

```
  .branding__tresc { flex-direction: row; flex-wrap: wrap; align-items: center; max-width: 100%; gap: 1rem; }
```

```
  .branding__separator, .branding__podtytul, .branding__dekoracja { display: none; }
```

```
  .branding__nazwa-systemu { font-size: 1.1rem; }
```

```
}
```

```
@media (max-width: 480px) {
```

```
  .panel-logowania__formularz { padding: 1.5rem 1rem; }
```

```
  .formularz__kontener { max-width: 100%; }
```

```
  .formularz__tytul { font-size: 1.4rem; }
```

```
}
```

PLIK: core\static\css\modul\przyciski.css

```
/* przyciski.css – Przyciski i grupy akcji */
```

```
.przycisk { display: inline-flex; align-items: center; gap: 0.375rem; padding: 0.5rem 1rem; font-family: var(--font-glowny); font-size: 0.85rem; font-weight: 600; text-decoration: none; border: none; border-radius: var(--promien); cursor: pointer; transition: background 0.15s, color 0.15s, box-shadow 0.15s; min-height: 36px; white-space: nowrap; }
```

```
.przycisk--glowny { background: var(--kolor-glowny); color: var(--kolor-biale); }
```

```
.przycisk--glowny:hover { background: var(--kolor-glowny-ciemny); }
```

```
.przycisk--drugorzeczny { background: var(--kolor-tlo); color: var(--kolor-tekst); border: 1px solid var(--kolor-ramka); }
```

```
.przycisk--drugorzeczny:hover { background: var(--kolor-ramka); }
```

```
.przycisk--ostrzezenie { background: #f59e0b; color: white; border: 1px solid #f59e0b; }
```

```
.przycisk--ostrzezenie:hover { background: #d97706; border-color: #d97706; }
```

```
.przycisk--niebezpieczny { background: var(--kolor-blad); color: var(--kolor-biale); }
```

```
.przycisk--niebezpieczny:hover { background: #a93226; }
```

```
.przycisk--maly { padding: 0.3rem 0.625rem; font-size: 0.75rem; min-height: 28px; }
```

```
.akcje-tabeli { display: flex; align-items: center; gap: 0.375rem; }
```

```
.akcje-panelu { display: flex; align-items: center; gap: 0.5rem; }
```

PLIK: core\static\css\modul\statusy.css

```
/* statusy.css – Odznaki statusów i ról */
```

```
.status { display: inline-block; padding: 0.25rem 0.625rem; font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.04em; border-radius: 100px; white-space: nowrap; }
```

```
.status--draft, .status--szkic { background: #e2e8f0; color: var(--kolor-tekst-drugor); }
```

```
.status--planned, .status--zaplanowana { background: #dbeafe; color: #1e40af; }
```

```
.status--in-progress, .status--w-trakcie { background: #fef3c7; color: #92400e; }
```

```
.status--pending-evaluation, .status--oczekuje-oceny { background: #fce7f3; color: #9d174d; }
```

```
.status--completed, .status--zakonczone { background: #d1fae5; color: var(--kolor-sukces); }
```

```
.rola-odznaka { display: inline-block; padding: 0.2rem 0.5rem; font-size: 0.7rem; font-weight: 600; border-radius: 100px; text-transform: uppercase; letter-spacing: 0.03em; }
```

```
.rola-odznaka--admin { background: #fef3c7; color: #92400e; }
```

```
.rola-odznaka--uopz { background: #dbeafe; color: #1e40af; }
```

```
.rola-odznaka--zopz { background: #d1fae5; color: var(--kolor-sukces); }
```

```
.rola-odznaka--student { background: #e2e8f0; color: var(--kolor-tekst-drugor); }
```


PLIK: core\static\css\modul\tabele.css

```
/* tabele.css – Tabele danych */
```

```
.tabela-wrapper { overflow-x: auto; }
```

```
.tabela { width: 100%; border-collapse: collapse; font-size: 0.875rem; }
```

```
.tabela thead { background: var(--kolor-tlo); }
```

```
.tabela th { padding: 0.75rem 1rem; text-align: left; font-weight: 600; font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.05em; color: var(--kolor-tekst-drugor); border-bottom: 2px solid var(--kolor-ramka); white-space: nowrap; }
```

```
.tabela td { padding: 0.875rem 1rem; border-bottom: 1px solid var(--kolor-ramka); vertical-align: middle; color: var(--kolor-tekst); }
```

```
.tabela tbody tr:hover { background: rgba(26,58,92,0.03); }
```

```
.tabela--pusta { text-align: center; color: var(--kolor-tekst-trzeciorzed); padding: 2rem 1rem !important; font-style: italic; }
```

```
.komorka-uzytkownik { display: flex; flex-direction: column; }
```

```
.komorka-uzytkownik__nazwa { font-weight: 600; }
```

```
.komorka-uzytkownik__info { font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed); }
```

PLIK: core\static\css\modul\uklad.css

/* uklad.css – Layout panelu: sidebar, panel-main, nagłówek */

```
.panel-body { display: flex; min-height: 100vh; background: var(--kolor-tlo); }
```

```
.sidebar {  
    width: 260px; height: 100vh; background: var(--kolor-glowny);  
  
    color: rgba(255,255,255,0.85); display: flex; flex-direction: column;  
  
    flex-shrink: 0; position: sticky; top: 0; overflow-y: auto;
```

```
}
```

```
.sidebar__naglowek { padding: 1.5rem 1.25rem 1rem; border-bottom: 1px solid rgba(255,255,255,0.1); }
```

```
.sidebar__logo { max-width: 100%; height: auto; display: block; }
```

```
.sidebar__system-label { display: block; font-size: 0.7rem; text-transform: uppercase; letter-spacing: 0.12em; color:  
var(--kolor-akcent); font-weight: 600; margin-top: 0.5rem; }
```

```
.sidebar__uzytkownik { display: flex; align-items: center; gap: 0.75rem; padding: 1.25rem; border-bottom: 1px solid  
rgba(255,255,255,0.08); }
```

```
.sidebar__uzytkownik-avatar { width: 38px; height: 38px; border-radius: 50%; background: var(--kolor-akcent); color:  
var(--kolor-glowny-ciemny); display: flex; align-items: center; justify-content: center; font-weight: 700; font-size:  
0.8rem; flex-shrink: 0; }
```

```
.sidebar__uzytkownik-info { display: flex; flex-direction: column; min-width: 0; }
```

```
.sidebar__uzytkownik-imie { font-size: 0.85rem; font-weight: 600; color: #fff; white-space: nowrap; overflow: hidden;  
text-overflow: ellipsis; }
```

```
.sidebar__uzytkownik-rola { font-size: 0.7rem; color: rgba(255,255,255,0.5); text-transform: uppercase; letter-spacing:  
0.05em; }
```

```
.sidebar__menu { list-style: none; padding: 0.75rem 0; flex: 1; }
```

```
.sidebar__menu-sekcja { font-size: 0.65rem; text-transform: uppercase; letter-spacing: 0.1em; color:  
rgba(255,255,255,0.35); padding: 1.25rem 1.25rem 0.375rem; font-weight: 600; }
```

```
.sidebar__menu-pozycja { margin: 1px 0; }
```

```
.sidebar__link { display: flex; align-items: center; gap: 0.75rem; padding: 0.625rem 1.25rem; color:  
rgba(255,255,255,0.75); text-decoration: none; font-size: 0.875rem; font-weight: 500; border-left: 3px solid transparent;  
transition: background 0.15s, color 0.15s, border-color 0.15s; }
```

```
.sidebar__link:hover { background: rgba(255,255,255,0.08); color: #fff; }
```

```
.sidebar__link--aktywny { background: rgba(255,255,255,0.12); color: #fff; border-left-color: var(--kolor-akcent);  
font-weight: 600; }
```

```
.sidebar__ikona { display: inline-flex; align-items: center; justify-content: center; width: 20px; height: 20px;  
flex-shrink: 0; }
```

```
.sidebar__ikona svg { width: 18px; height: 18px; fill: none; stroke: currentColor; stroke-width: 2; stroke-linecap:  
round; stroke-linejoin: round; }
```

```
.sidebar__stopka { padding: 1rem 1.25rem; border-top: 1px solid rgba(255,255,255,0.08); margin-top: auto; }

.sidebar__wylogowanie { display: flex; align-items: center; gap: 0.5rem; width: 100%; padding: 0.625rem 0.75rem;
background: rgba(255,255,255,0.06); border: 1px solid rgba(255,255,255,0.1); border-radius: var(--promien); color:
rgba(255,255,255,0.7); font-family: var(--font-glowny); font-size: 0.85rem; font-weight: 500; cursor: pointer;
transition: background 0.15s, color 0.15s; }

.sidebar__wylogowanie:hover { background: rgba(192,57,43,0.3); color: #fff; }

.sidebar__wylogowanie svg { width: 16px; height: 16px; fill: none; stroke: currentColor; stroke-width: 2; stroke-linecap:
round; stroke-linejoin: round; }


.panel-main { flex: 1; display: flex; flex-direction: column; min-width: 0; }

.panel-main__naglowek { background: var(--kolor-biale); border-bottom: 1px solid var(--kolor-ramka); padding: 1.25rem
2rem; }

.panel-main__naglowek-tresc { display: flex; align-items: center; justify-content: space-between; gap: 1rem; }

.panel-main__tytul { font-size: 1.35rem; font-weight: 600; color: var(--kolor-tekst); letter-spacing: -0.01em; }

.panel-main__tresc { flex: 1; padding: 2rem; }


@media (max-width: 768px) {

    .sidebar { display: none; }

    .panel-main__tresc { padding: 1rem; }

    .panel-main__naglowek { padding: 1rem; }

}
```

PLIK: core\static\css\modul\zmiennosc.css

```
/* =====  
  
zmiennosc.css – Zmienne CSS, reset, podstawy, dostępność  
  
System Praktyk Zawodowych – ANS Elbląg  
  
WCAG 2.1 AA: kontrast ≥ 4.5:1, fokus widoczny  
  
===== */  
  
:root {  
  
  --kolor-glowny:      #1a3a5c;  
  
  --kolor-glowny-ciemny: #0f2540;  
  
  --kolor-glowny-jasny:  #2a5080;  
  
  --kolor-akcent:       #c8963e;  
  
  --kolor-akcent-jasny:  #e8b45a;  
  
  
  --kolor-tlo:          #f4f6f9;  
  
  --kolor-biale:         #ffffff;  
  
  --kolor-tekst:         #1a1a2e;  
  
  --kolor-tekst-drugor:  #4a5568;  
  
  --kolor-tekst-trzeciorzed: #718096;  
  
  
  --kolor-ramka:         #d1d9e6;  
  
  --kolor-blad:          #c0392b;  
  
  --kolor-sukces:        #1e6e3a;  
  
  --kolor-info:          #1a5276;  
  
  
  --fokus-kolor:         #c8963e;  
  
  --fokus-grubosc:       3px;  
  
  --fokus-offset:        3px;  
  
  
  --font-glowny:         'IBM Plex Sans', 'Segoe UI', Tahoma, sans-serif;  
  
  --font-mono:           'IBM Plex Mono', 'Courier New', monospace;  
  
  
  --promien:             8px;  
  
  --promien-duzy:        16px;
```

```

--cien-karta:    0 2px 12px rgba(26, 58, 92, 0.12);

--cien-glowny:   0 8px 32px rgba(26, 58, 92, 0.18);

}

*, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }

html {

    font-size: 100%;

    -webkit-text-size-adjust: 100%;

    scroll-behavior: smooth;

}

@media (prefers-reduced-motion: reduce) {

    *, ::before, ::after {

        animation-duration: 0.01ms !important;

        transition-duration: 0.01ms !important;

    }

}

body {

    font-family: var(--font-glowny), serif;

    font-size: 1rem;

    line-height: 1.6;

    color: var(--kolor-tekst);

    background-color: var(--kolor-tlo);

    min-height: 100vh;

}

.skip-link {

    position: absolute; top: -100px; left: 1rem; z-index: 9999;

    padding: 0.75rem 1.5rem; background: var(--kolor-glowny); color: var(--kolor-biale);

    font-weight: 600; border-radius: var(--promien); text-decoration: none; transition: top 0.2s;

}

.skip-link:focus { top: 1rem; }

```

```
:focus-visible {
    outline: var(--fokus-grubosc) solid var(--fokus-kolor);

    outline-offset: var(--fokus-offset);

    border-radius: calc(var(--promien) - 2px);
}

:focus:not(:focus-visible) { outline: none; }

@media (prefers-contrast: more) {
    :root { --kolor-ramka: #000; --fokus-kolor: #000; --fokus-grubosc: 4px; }

    .pole-formularza__input { border-width: 2px; }
}
```

PLIK: core\templates\auth\zmien_haslo.html

```
{% extends "layouts/base_auth.html" %}

{% block tytul %}Ustaw hasło{% endblock %}


{% block body %}

<div class="panel-logowania" role="main">


    <!-- Lewa kolumna: branding -->

    <aside class="panel-logowania__branding" aria-label="Informacje o instytucji">

        <div class="branding__tresc">

            <div class="branding__logo-kontener">

                

            </div>

            <div class="branding__separator" aria-hidden="true"></div>

            <p class="branding__nazwa-systemu">Pierwsza konfiguracja</p>

            <p class="branding__podtytul">Ustaw własne hasło</p>

        </div>

    </aside>


    <!-- Prawa kolumna: formularz -->

    <main class="panel-logowania__formularz" id="main-content">

        <div class="formularz__kontener">

            <div class="formularz__naglowek">

                <h1 class="formularz__tytul">Ustaw własne hasło</h1>

                <p class="formularz__opis">

                    Twoje konto wymaga ustawienia hasła przed kontynuowaniem.

                    Hasło musi mieć minimum 8 znaków.

                </p>

            </div>

        </div>

    </main>

    {% with messages = get_flashed_messages(with_categories=true) %}

        {% if messages %}
```

```

<div class="komunikaty" aria-live="polite">

    {% for kategoria, wiadomosc in messages %}

    <div class="komunikat komunikat--{{ kategoria }}" role="alert">

        <span class="komunikat__ikona" aria-hidden="true">

            {% if kategoria == 'success' %}{% elif kategoria == 'danger' %}{% else %}i{% endif %}

        </span>

        {{ wiadomosc }}

    </div>

    {% endfor %}

</div>

{% endif %}

{% endwith %}

<form method="POST" class="formularz__forma" novalidate>

    {{ form.hidden_tag() }}

    <div class="pole-formularza {% if form.nowe_haslo.errors %}pole-formularza--blad{% endif %}">

        <label for="nowe_haslo" class="pole-formularza__etykieta">

            Nowe hasło <span class="pole-formularza__wymagane" aria-label="wymagane">*</span>

        </label>

        <div class="pole-formularza__input-wrapper">

            <input type="password" id="nowe_haslo" name="nowe_haslo"

                class="pole-formularza__input" required

                autocomplete="new-password" minlength="8"

                {% if form.nowe_haslo.errors %}aria-invalid="true" aria-describedby="haslo-blad"{% endif %}>

            <button type="button" class="pole-formularza__toggle-haslo"

                aria-label="Pokaż hasło" aria-pressed="false"

                onclick="toggleHaslo(this, 'nowe_haslo')">

                <span aria-hidden="true"></span>

            </button>

        </div>

        {% if form.nowe_haslo.errors %}

            <span id="haslo-blad" class="pole-formularza__blad" role="alert">{{ form.nowe_haslo.errors[0] }}</span>

        {% endif %}

```



```
</div>
```

```
<div class="pole-formularza {% if form.potwierdzenie.errors %}pole-formularza--blad{% endif %}">
```

```
<label for="potwierdzenie" class="pole-formularza__etykieta">
```

```
    Powtórz hasło <span class="pole-formularza__wymagane" aria-label="wymagane">*</span>
```

```
</label>
```

```
<div class="pole-formularza__input-wrapper">
```

```
<input type="password" id="potwierdzenie" name="potwierdzenie"
```

```
    class="pole-formularza__input" required autocomplete="new-password"
```

```
    {% if form.potwierdzenie.errors %}aria-invalid="true" aria-describedby="pow-blad"{% endif %}>
```

```
</div>
```

```
{% if form.potwierdzenie.errors %}
```

```
<span id="pow-blad" class="pole-formularza__blad" role="alert">{{ form.potwierdzenie.errors[0] }}</span>
```

```
{% endif %}
```

```
</div>
```

```
<button type="submit" class="przycisk-logowania">
```

```
    Ustaw hasło i przejdź do systemu
```

```
</button>
```

```
</form>
```

```
</div>
```

```
</main>
```

```
</div>
```

```
{% endblock %}
```

```
{% block scripts_extra %}
```

```
<script>
```

```
function toggleHaslo(btn, id) {
```

```
    const inp = document.getElementById(id);
```

```
    const ukryte = inp.type === 'password';
```

```
    inp.type = ukryte ? 'text' : 'password';
```

```
    btn.setAttribute('aria-pressed', ukryte ? 'true' : 'false');
```

```
    btn.setAttribute('aria-label', ukryte ? 'Ukryj hasło' : 'Pokaż hasło');  
  
}  
  
</script>  
  
{% endblock %}
```

PLIK: core\templates\errors\blad.html

```
{% extends "layouts/base_auth.html" %}
```

```
{% block tytul %}Błąd {{ kod }}{% endblock %}
```

```
{% block body %}
```

```
    {% set url_glowna = url_for('dashboard.index') %}
```

```
    {% include "errors/blad_bazowy.html" %}
```

```
{% endblock %}
```

PLIK: core\templates\errors\blad_bazowy.html

```
{# core/templates/errors/blad_bazowy.html
```

```
    Parametryczny fragment błędu – includowany przez blad.html każdej aplikacji.
```

```
    Zmienne: kod, tytuł, opis, url_glowna, tekst_przycisku #}
```

```
<div class="strona-bledu">
```

```
    <div class="strona-bledu__kod" aria-hidden="true">{{ kod }}</div>
```

```
    <h1 class="strona-bledu__tytul">{{ tytul }}</h1>
```

```
    <p class="strona-bledu__opis">{{ opis }}</p>
```

```
    <div style="margin-top: 1.5rem;">
```

```
        <a href="{{ url_glowna }}" class="przycisk przycisk--glowny">{{ tekst_przycisku }}</a>
```

```
    </div>
```

```
</div>
```

PLIK: core\templates\layouts\base_auth.html

```
<!DOCTYPE html>

<html lang="pl">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>{% block tytuł %}Autoryzacja{% endblock %} – ANS Elbląg</title>

    <link rel="stylesheet" href="{% url_for('core_static.static', filename='css/autoryzacja.css') %}">

    <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600&display=swap" rel="stylesheet">

    {% block head_extra %}{% endblock %}

</head>

<body class="strona-logowania">

    <a href="#main-content" class="skip-link">Przejdź do treści głównej</a>

    <main id="main-content" class="auth-main">

        {% block body %}{% endblock %}

    </main>

    {% block scripts_extra %}{% endblock %}

</body>

</html>
```

PLIK: core\uslugi\ocenie.py

```
"""
```

```
core/uslugi/ocenie.py
```

Usługa oceniań praktyk – przeniesiona z app_admin/services/serwis_oceniania.py.

Należy do domeny core, bo operuje na modelu domenowym ZapisPraktyki.

```
"""
```

```
from __future__ import annotations
```

```
from datetime import date, timedelta
```

```
from core.extensions import db
```

```
from core.modele import ZapisPraktyki, StatusZapisu
```

```
class SerwisOceniania:
```

```
    @staticmethod
```

```
    def get_pilne_oceny(uopz_id=None) -> list[dict]:
```

```
        """Zwraca listę praktyk z pilnymi ocenami (termin ≤ 3 dni)."""
```

```
        q = db.session.query(ZapisPraktyki).filter_by(status=StatusZapisu.COMPLETED)
```

```
        if uopz_id:
```

```
            q = q.filter_by(uopz_id=uopz_id)
```

```
        pilne = []
```

```
        for zapis in q.all():
```

```
            if zapis.termin_do:
```

```
                deadline = zapis.termin_do + timedelta(days=7)
```

```
                dni_do_konca = (deadline - date.today()).days
```

```
                if dni_do_konca <= 3:
```

```
                    pilne.append({
```

```
                        'zapis': zapis,
```

```
                        'deadline': deadline,
```

```
                        'dni_do_deadline': dni_do_konca,
```

```
                        'przekroczony': dni_do_konca < 0,
```

```
                    })
```

```
        return sorted(pilne, key=lambda x: x['dni_do_deadline'])
```

```
@staticmethod

def auto_complete_internships() -> None:

    """Automatycznie zamyka praktyki z przekroczonym terminem."""

    do_zakonczenia = db.session.query(ZapisPraktyki).filter(

        ZapisPraktyki.status == StatusZapisu.IN_PROGRESS,

        ZapisPraktyki.termin_do < date.today(),

    ).all()

    for p in do_zakonczenia:

        p.status = StatusZapisu.COMPLETED

    if do_zakonczenia:

        db.session.commit()
```

PLIK: core\uslugi\praktyki.py

```
"""core/uslugi/praktyki.py

Usługa zarządzania praktykami i zapisami studentów.

"""

from __future__ import annotations

import uuid

from datetime import datetime

from typing import Optional

from core.extensions import db

from core.modele.praktyki import (
    Praktyka,
    StatusPraktyki,
    ZapisPraktyki,
    StatusZapisu,
    SciezkaPraktyki,
    TypZdarzenia,
    ZdarzenieProces,
    Sprawozdanie,
)

from core.repozytoria.praktyki import RepozytoriumPraktyk, RepozytoriumZapisow


class UsługaPraktyk:

    """Logika biznesowa edycji praktyk i procesowania zapisów."""

    def __init__(
        self,
        repo_praktyk: Optional[RepozytoriumPraktyk] = None,
        repo_zapisow: Optional[RepozytoriumZapisow] = None,
    ) -> None:

        self._praktyki = repo_praktyk or RepozytoriumPraktyk()
```



```
self._zapisy = repo_zapisow or RepozytoriumZapisow()
```

```
# — Edycje praktyk —————
```

```
def utworz_edycje(self, rok_uczelniany: str, semestr: str, wymiar_godzin: int = 160) -> Praktyka:
```

```
    praktyka = Praktyka(
        rok_uczelniany=rok_uczelniany,
        semestr=semestr,
        wymiar_godzin=wymiar_godzin,
        status=StatusPraktyki.NIEAKTYWNA,
    )
    self._praktyki.zapisz(praktyka)
    db.session.commit()
    return praktyka
```

```
def aktywuj_edycje(self, praktyka: Praktyka) -> None:
```

```
    praktyka.status = StatusPraktyki.AKTYWNA
    db.session.commit()
```

```
def dezaktywuj_edycje(self, praktyka: Praktyka) -> None:
```

```
    praktyka.status = StatusPraktyki.NIEAKTYWNA
    db.session.commit()
```

```
# — Zapisy studentów —————
```

```
def zapisz_studenta(
```

```
    self,
    student_id: uuid.UUID,
    praktyka_id: uuid.UUID,
    sciezka: SciezkaPraktyki = SciezkaPraktyki.STANDARDOWA,
```

```
) -> ZapisPraktyki:
```

```
    if self._zapisy.student_ma_aktywny_zapis(student_id, praktyka_id):
        raise ValueError('Student ma już aktywne zgłoszenie do tej edycji praktyk.')
    zapis = ZapisPraktyki(
```

```

        student_id=student_id,

        praktyka_id=praktyka_id,

        sciezka=sciezka,

        status=StatusZapisu.OCZEKUJACY,

    )

    self._zapisy.zapisz(zapis)

    db.session.commit()

    return zapis

def zmien_status(

    self,

    zapis: ZapisPraktyki,

    nowy_status: StatusZapisu,

    komentarz: Optional[str] = None,

    wykonane_przez_id: Optional[uuid.UUID] = None,

) -> None:

    """Zmienia status zapisu i opcjonalnie dodaje zdarzenie do logu."""

    zapis.status = nowy_status

    if komentarz is not None:

        if nowy_status in (StatusZapisu.ODRZUCONA, StatusZapisu.OCZEKUJE_NA_AKCEPT):

            typ = TypZdarzenia.ADMIN_KOMENTARZ

        elif nowy_status == StatusZapisu.WERYFIKACJA_KOMISJI:

            typ = TypZdarzenia.UOPZ_KOMENTARZ

        else:

            typ = TypZdarzenia.ADMIN_KOMENTARZ

        self._dodaj_zdarzenie(zapis, typ, komentarz=komentarz, wykonane_przez_id=wykonane_przez_id)

    db.session.commit()

def przypisz_opiekuna(self, zapis: ZapisPraktyki, uopz_id: uuid.UUID) -> None:

    zapis.uopz_id = uopz_id

    db.session.commit()

```

```

def zatwierdz_przez_komisje(

    self,

    zapis: ZapisPraktyki,

    decyzja: str,

    komentarz: Optional[str] = None,

    wykonane_przez_id: Optional[uuid.UUID] = None,

) -> None:

    self._dodaj_zdarzenie(

        zapis, TypZdarzenia.KOMISJA_DECYZJA,

        decyzja=decyzja, komentarz=komentarz,

        wykonane_przez_id=wykonane_przez_id,

    )

    zapis.status = StatusZapisu.AKCEPTACJA_DZIEKANA if decyzja == 'APPROVED' else StatusZapisu.ODRZUCONA

    db.session.commit()

```

```

def zatwierdz_przez_dziekana(

    self,

    zapis: ZapisPraktyki,

    decyzja: str,

    komentarz: Optional[str] = None,

    wykonane_przez_id: Optional[uuid.UUID] = None,

) -> None:

    self._dodaj_zdarzenie(

        zapis, TypZdarzenia.DZIEKAN_DECYZJA,

        decyzja=decyzja, komentarz=komentarz,

        wykonane_przez_id=wykonane_przez_id,

    )

    zapis.status = StatusZapisu.W_REALIZACJI if decyzja == 'APPROVED' else StatusZapisu.ODRZUCONA

    db.session.commit()

```

```

def powiadom_studenta(

    self,

    zapis: ZapisPraktyki,

    komentarz: Optional[str] = None,

```

```

        wykonane_przez_id: Optional[uuid.UUID] = None,
    ) -> None:

        self._dodaj_zdarzenie(

            zapis, TypZdarzenia.POWIADOMIENIE_STUDENTA,

            komentarz=komentarz,

            wykonane_przez_id=wykonane_przez_id,

        )

        db.session.commit()


def zakoncz(self, zapis: ZapisPraktyki) -> None:

    zapis.status = StatusZapisu.ZAKONCZONA

    db.session.commit()


# — Sprawozdania —————

def pobierz_lub_utworz_sprawozdanie(self, zapis: ZapisPraktyki) -> Sprawozdanie:

    if zapis.sprawozdanie is None:

        sprawozdanie = Sprawozdanie(zapis_id=zapis.id)

        db.session.add(sprawozdanie)

        db.session.flush()

        return sprawozdanie

    return zapis.sprawozdanie


# — Helpers —————

def _dodaj_zdarzenie(

    self,

    zapis: ZapisPraktyki,

    typ: TypZdarzenia,

    decyzja: Optional[str] = None,

    komentarz: Optional[str] = None,

    wykonane_przez_id: Optional[uuid.UUID] = None,

) -> ZdarzenieProces:

    zdarzenie = ZdarzenieProces(

```

```
        zapis_id=zapis.id,  
  
        typ=typ,  
  
        decyzja=decyzja,  
  
        komentarz=komentarz,  
  
        wykonane_przez_id=wykonane_przez_id,  
  
        wykonano_o=datetime.utcnow(),  
  
    )  
  
    db.session.add(zdarzenie)  
  
    return zdarzenie
```

— Dostęp do repozytoriów —————

@property

```
def praktyki(self) -> RepozytoriumPraktyk:  
  
    return self._praktyki
```

@property

```
def zapisy(self) -> RepozytoriumZapisow:  
  
    return self._zapisy
```

PLIK: core\uslugi\uzytkownicy.py

```
"""core/uslugi/uzytkownicy.py

Usługa zarządzania kontami użytkowników.

"""

from __future__ import annotations

from typing import Optional

import uuid

from werkzeug.security import generate_password_hash, check_password_hash

from core.extensions import db

from core.modele.uzytkownicy import RolaUzytkownika, Uzytkownik, Student, Administrator, OpikunUczelniacy

from core.repozytoria.uzytkownicy import RepozytoriumUzytkownikow


class UsługaUzytkownikow:

    """Logika biznesowa kont użytkowników."""

    def __init__(self, repozytorium: Optional[RepozytoriumUzytkownikow] = None) -> None:

        self._repo = repozytorium or RepozytoriumUzytkownikow()

    # — Uwierzytelnienie —————

    def uwierzytelnij(self, email: str, haslo: str) -> Optional[Uzytkownik]:

        """Zwraca użytkownika jeśli dane są poprawne, None w przeciwnym razie."""

        uzytkownik = self._repo.znajdz_po_emailu(email)

        if uzytkownik is None or not uzytkownik.aktywny:

            return None

        if not check_password_hash(uzytkownik.hash_hasla, haslo):

            return None

        return uzytkownik
```

```

def zmien_haslo(self, uzytkownik: Uzytkownik, nowe_haslo: str) -> None:

    uzytkownik.hash_hasla = generate_password_hash(nowe_haslo)

    uzytkownik.wymagana_zmiana_hasla = False

    db.session.commit()

```

— Tworzenie kont —————

```

def utworz_studenta(

    self,

    email: str,

    haslo: str,

    imie: str,

    nazwisko: str,

    numer_albumu: Optional[str] = None,

    **dane_studenta,

) -> Student:

    if self._repo.istnieje_email(email):

        raise ValueError(f'Konto z adresem {email} już istnieje.')

    student = Student(

        email=email,

        hash_hasla=generate_password_hash(haslo),

        imie=imie,

        nazwisko=nazwisko,

        rola=RolaUzytkownika.STUDENT,

        numer_albumu=numer_albumu,

        **dane_studenta,

    )

    self._repo.zapisz(student)

    db.session.commit()

    return student


def utworz_administratora(self, email: str, haslo: str, imie: str, nazwisko: str) -> Administrator:

    if self._repo.istnieje_email(email):

        raise ValueError(f'Konto z adresem {email} już istnieje.')

```

```

admin = Administrator(

    email=email,

    hash_hasla=generate_password_hash(haslo),

    imie=imie,

    nazwisko=nazwisko,

    rola=RolaUzytkownika.ADMIN,

)

self._repo.zapisz(admin)

db.session.commit()

return admin

```

```

def utworz_opiekuna(self, email: str, haslo: str, imie: str, nazwisko: str) -> OpikunUczelniany:

```

```

    if self._repo.istnieje_email(email):

        raise ValueError(f'Konto z adresem {email} już istnieje.')

    opiekun = OpikunUczelniany(

        email=email,

        hash_hasla=generate_password_hash(haslo),

        imie=imie,

        nazwisko=nazwisko,

        rola=RolaUzytkownika.UOPZ,

    )

    self._repo.zapisz(opiekun)

    db.session.commit()

    return opiekun

```

```

# — Aktualizacja —————

```

```

def aktualizuj(self, uzytkownik: Uzytkownik, **pola) -> Uzytkownik:

    """Aktualizuje dowolne pola modelu i zapisuje."""

    if 'haslo' in pola:

        uzytkownik.hash_hasla = generate_password_hash(pola.pop('haslo'))

    for klucz, wartosc in pola.items():

        setattr(uzytkownik, klucz, wartosc)

    db.session.commit()

```



```
return uzytkownik
```

```
def dezaktywuj(self, uzytkownik: Uzytkownik) -> None:
```

```
    uzytkownik.aktywny = False
```

```
    db.session.commit()
```

```
def aktywuj(self, uzytkownik: Uzytkownik) -> None:
```

```
    uzytkownik.aktywny = True
```

```
    db.session.commit()
```

```
# — Dostęp —————
```

```
@property
```

```
def repozytorium(self) -> RepozytoriumUzytkownikow:
```

```
    return self._repo
```

PLIK: core\uslugi__init__.py

```
"""core/uslugi – warstwa usług biznesowych.
```

```
Usługi zawierają logikę domenową i orkiestrują  
repozytoria. Kontrolery wywołują wyłącznie usługi –  
nie tworzą zapytań ORM ani nie operują bezpośrednio  
na sesjach bazy danych.
```

```
"""
```

```
from core.uslugi.praktyki    import UsługaPraktyk  
  
from core.uslugi.uzytkownicy import UsługaUzytkownikow  
  
from core.uslugi.ocenianie   import SerwisOceniania
```

```
__all__ = ['UsługaPraktyk', 'UsługaUzytkownikow', 'SerwisOceniania']
```

PLIK: database\docker-compose.yml

version: '3.8'

services:

db:

image: postgres:15-alpine

container_name: ans_postgres

environment:

POSTGRES_USER: ans_admin

POSTGRES_PASSWORD: secure_password_123

POSTGRES_DB: ans_praktyki

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

- ./init.sql:/docker-entrypoint-initdb.d/init.sql

restart: unless-stopped

redis:

image: redis:7-alpine

container_name: ans_redis

ports:

- "6379:6379"

restart: unless-stopped

worker:

build:

context: ../app_admin

dockerfile: Dockerfile.worker

container_name: ans_celery_worker

environment:

DATABASE_URL: postgresql+psycopg2://ans_admin:secure_password_123@db:5432/ans_praktyki

CELERY_BROKER_URL: redis://redis:6379/0

CELERY_RESULT_BACKEND: redis://redis:6379/0

FLASK_ENV: development

SECRET_KEY: super-tajny-klucz-tylko-na-dev-xd

volumes:

- pdf_output:/app/pdf_output

depends_on:

- db
- redis

restart: unless-stopped

volumes:

postgres_data:

pdf_output:

PLIK: shared\static\css\modul\dashboard.css

```
/* =====  
  
    dashboard.css – Statystyki, strona błędu, misc  
  
    ===== */  
  
  
/* — Dashboard: statystyki ————— */  
  
.statystyki-grid,  
  
.siatka-statystyk {  
  
    display: grid;  
  
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
  
    gap: 1.5rem;  
  
    margin-bottom: 2rem;  
  
}  
  
  
  
.stat-karta,  
  
.karta-stat {  
  
    background: var(--kolor-biale);  
  
    padding: 1.5rem;  
  
    border-radius: var(--promien-duzy);  
  
    border: 1px solid var(--kolor-ramka);  
  
    box-shadow: var(--cien-karta);  
  
    display: flex;  
  
    flex-direction: column;  
  
    align-items: center;  
  
    text-align: center;  
  
    gap: 0.5rem;  
  
}  
  
  
  
.stat-karta__etykieta,  
  
.karta-stat__etykieta {  
  
    font-size: 0.75rem;  
  
    text-transform: uppercase;  
  
    letter-spacing: 0.08em;  
  
    color: var(--kolor-tekst-drugor);
```

```
}
```

```
.stat-karta__wartosc,  
.karta-stat__liczba {  
  
  font-size: 2rem;  
  
  font-weight: 700;  
  
  color: var(--kolor-glowny);  
  
  line-height: 1;  
}
```

```
.karta-stat__ikona {  
  
  color: var(--kolor-akcent);  
  
  line-height: 1;  
}
```

```
.karta-stat__ikona svg {  
  
  width: 28px;  
  
  height: 28px;  
}
```

```
/* — Responsywność dashboard ————— */
```

```
@media (max-width: 768px) {  
  
  .statystyki-grid {  
  
    grid-template-columns: 1fr 1fr;  
  
  }  
}
```

```
/* — Strona błędu ————— */
```

```
.strona-bledu {  
  
  display: flex;  
  
  flex-direction: column;  
  
  align-items: center;  
  
  justify-content: center;  
  
  text-align: center;
```

```
padding: 4rem 2rem;

min-height: 60vh;

}
```

```
.strona-bledu__kod {

font-size: 5rem;

font-weight: 800;

color: var(--kolor-ramka);

line-height: 1;

margin-bottom: 1rem;

}
```

```
.strona-bledu__tytul {

font-size: 1.5rem;

font-weight: 600;

color: var(--kolor-tekst);

margin-bottom: 0.5rem;

}
```

```
.strona-bledu__opis {

color: var(--kolor-tekst-drugor);

max-width: 400px;

margin: 0 auto;

}
```

PLIK: shared\static\css\modul\formularze.css

```
/* =====  
  
formularze.css – Pola formularzy, inputy, checkboxy  
  
===== */  
  
.formularz__forma {  
  
    display: flex;  
  
    flex-direction: column;  
  
    gap: 1.25rem;  
  
}  
  
/* Kontener formularzy – ujednolicone centrowanie dla panelu */  
  
.formularz-kontener {  
  
    display: flex;  
  
    justify-content: center;  
  
}  
  
.formularz-kontener > .karta {  
  
    width: 100%;  
  
    max-width: 640px;  
  
}  
  
/* — Pola formularza ————— */  
  
.pole-formularza {  
  
    display: flex;  
  
    flex-direction: column;  
  
    gap: 0.375rem;  
  
}  
  
.pole-formularza__naglowek-haslo {  
  
    display: flex;  
  
    align-items: center;  
  
    justify-content: space-between;  
  
}
```



```
.pole-formularza__etykieta {  
  
    font-size: 0.875rem;  
  
    font-weight: 500;  
  
    color: var(--kolor-tekst);  
  
    /* WCAG 1.3.1: etykiety są elementami <label> powiązanymi z polem */  
  
}
```

```
.pole-formularza__wymagane {  
  
    color: var(--kolor-blad);  
  
    margin-left: 0.2em;  
  
    font-weight: 700;  
  
}
```

```
.pole-formularza__input {  
  
    width: 100%;  
  
    padding: 0.75rem 1rem;  
  
    font-family: var(--font-glowny), serif;  
  
    font-size: 1rem;  
  
    color: var(--kolor-tekst);  
  
    background: var(--kolor-biale);  
  
    border: 2px solid var(--kolor-ramka);  
  
    border-radius: var(--promien);  
  
    transition: border-color 0.15s, box-shadow 0.15s;  
  
    /* WCAG 1.4.11: komponent interfejsu – kontrast granicy co najmniej 3:1 */  
  
}
```

```
.pole-formularza__input:hover {  
  
    border-color: var(--kolor-glowny-jasny);  
  
}
```

```
.pole-formularza__input:focus {  
  
    border-color: var(--kolor-glowny);  
  
    box-shadow: 0 0 0 3px rgba(26, 58, 92, 0.15);  
  
}
```

```
outline: var(--fokus-grubosc) solid var(--fokus-kolor);

outline-offset: var(--fokus-offset);

}

.pole-formularza--blad .pole-formularza__input {

  border-color: var(--kolor-blad);

}

.pole-formularza--blad .pole-formularza__input:focus {

  box-shadow: 0 0 0 3px rgba(192, 57, 43, 0.15);

}

.pole-formularza__input-wrapper {

  position: relative;

  display: flex;

  align-items: center;

}

.pole-formularza__input-wrapper .pole-formularza__input {

  padding-right: 3rem;

}

.pole-formularza__toggle-haslo {

  position: absolute;

  right: 0.75rem;

  background: none;

  border: none;

  cursor: pointer;

  padding: 0.25rem;

  color: var(--kolor-tekst-drugor);

  border-radius: var(--promien);

  font-size: 1rem;

  line-height: 1;

  transition: color 0.15s;
```

```
}
```

```
.pole-formularza__toggle-haslo:hover {  
  
  color: var(--kolor-glowny);  
  
}
```

```
.pole-formularza__blad {  
  
  font-size: 0.825rem;  
  
  color: var(--kolor-blad);  
  
  font-weight: 500;  
  
  /* role="alert" w HTML zapewnia odczyt przez czytniki ekranu */  
  
}
```

```
/* — Checkbox ————— */
```

```
.pole-formularza--checkbox {  
  
  flex-direction: row;  
  
}
```

```
.pole-formularza__etykieta-checkbox {  
  
  display: flex;  
  
  align-items: center;  
  
  gap: 0.625rem;  
  
  cursor: pointer;  
  
  font-size: 0.9rem;  
  
  color: var(--kolor-tekst-drugor);  
  
}
```

```
.pole-formularza__checkbox {  
  
  width: 18px;  
  
  height: 18px;  
  
  flex-shrink: 0;  
  
  cursor: pointer;  
  
  accent-color: var(--kolor-glowny);  
  
}
```

PLIK: shared\static\css\modul\karty.css

```
/* =====  
  
karty.css – Komponent karty (.karta)  
  
===== */  
  
.karta {  
  
    background: var(--kolor-biale);  
  
    border: 1px solid var(--kolor-ramka);  
  
    border-radius: var(--promien-duzy);  
  
    box-shadow: var(--cien-karta);  
  
    overflow: hidden;  
  
}  
  
.karta__naglowek {  
  
    padding: 1rem 1.5rem;  
  
    border-bottom: 1px solid var(--kolor-ramka);  
  
    display: flex;  
  
    align-items: center;  
  
    justify-content: space-between;  
  
    gap: 1rem;  
  
    flex-wrap: wrap;  
  
}  
  
.karta__tytul {  
  
    font-size: 1rem;  
  
    font-weight: 600;  
  
    color: var(--kolor-tekst);  
  
}  
  
.karta__tresc {  
  
    padding: 1.5rem;  
  
}
```

PLIK: shared\static\css\modul\komponenty.css

```
/* =====  
  
komponenty.css – Pasek postępu, paginacja, filtry  
  
===== */  
  
  
/* — Pasek postępu ————— */  
  
.pasek-postep {  
  
    height: 6px;  
  
    background: var(--kolor-tlo);  
  
    border-radius: 10px;  
  
    overflow: hidden;  
  
}  
  
  
.pasek-postep__wypelnienie {  
  
    height: 100%;  
  
    background: var(--kolor-glowny);  
  
    border-radius: 10px;  
  
    transition: width 0.4s ease;  
  
}  
  
  
.pasek-postep__wypelnienie--zakonczoney {  
  
    background: var(--kolor-sukces);  
  
}  
  
  
  
  
/* — Paginacja ————— */  
  
.paginacja {  
  
    display: flex;  
  
    align-items: center;  
  
    justify-content: space-between;  
  
    padding: 1rem 1.5rem;  
  
    border-top: 1px solid var(--kolor-ramka);  
  
}  
  
  
  
  
.paginacja__info {
```

```
font-size: 0.8rem;

color: var(--kolor-tekst-drugor);

}
```

```
.paginacja__przyciski {

display: flex;

align-items: center;

gap: 0.25rem;

}
```

```
.paginacja__przycisk {

display: inline-flex;

align-items: center;

justify-content: center;

min-width: 32px;

height: 32px;

padding: 0 0.5rem;

font-size: 0.8rem;

font-weight: 500;

text-decoration: none;

color: var(--kolor-tekst-drugor);

background: var(--kolor-biale);

border: 1px solid var(--kolor-ramka);

border-radius: var(--promien);

transition: background 0.15s;

}
```

```
.paginacja__przycisk:hover {

background: var(--kolor-tlo);

}
```

```
.paginacja__przycisk--aktywny {

background: var(--kolor-glowny);

color: var(--kolor-biale);

}
```

```
border-color: var(--kolor-glowny);  
}
```

```
.paginacja__separator {  
  
padding: 0 0.25rem;  
  
color: var(--kolor-tekst-trzeciorzed);  
}
```

```
/* — Filtry ————— */
```

```
.pasek-filtrow {  
  
display: flex;  
  
align-items: center;  
  
gap: 0.75rem;  
  
flex-wrap: wrap;  
  
width: 100%;  
}
```

```
.filtr-szukaj {  
  
display: flex;  
  
align-items: center;  
  
gap: 0.5rem;  
  
flex: 1;  
  
min-width: 200px;  
  
background: var(--kolor-tlo);  
  
border: 1px solid var(--kolor-ramka);  
  
border-radius: var(--promien);  
  
padding: 0 0.75rem;  
}
```

```
.filtr-szukaj__ikona {  
  
color: var(--kolor-tekst-trzeciorzed);  
  
font-size: 0.85rem;  
  
flex-shrink: 0;  
}
```

```
.filtr-szukaj__input {  
  
  flex: 1;  
  
  border: none;  
  
  background: transparent;  
  
  padding: 0.5rem 0;  
  
  font-family: var(--font-glowny);  
  
  font-size: 0.85rem;  
  
  color: var(--kolor-tekst);  
  
  outline: none;  
  
}
```

```
.filtr-select {  
  
  padding: 0.5rem 0.75rem;  
  
  font-family: var(--font-glowny);  
  
  font-size: 0.85rem;  
  
  color: var(--kolor-tekst);  
  
  background: var(--kolor-tlo);  
  
  border: 1px solid var(--kolor-ramka);  
  
  border-radius: var(--promien);  
  
  cursor: pointer;  
  
}
```


PLIK: shared\static\css\modul\komunikaty.css

```
/* =====  
  
komunikaty.css – Komunikaty flash i powiadomienia  
  
===== */  
  
.komunikaty {  
  
    margin-bottom: 1.5rem;  
  
    display: flex;  
  
    flex-direction: column;  
  
    gap: 0.5rem;  
  
}  
  
.komunikat {  
  
    display: flex;  
  
    align-items: center;  
  
    gap: 0.5rem;  
  
    padding: 0.5rem 0.75rem;  
  
    border-radius: 10px;  
  
    font-size: 0.9rem;  
  
    line-height: 1.2;  
  
    border-left: 4px solid;  
  
}  
  
.komunikat--danger {  
  
    background: #fdf2f2;  
  
    border-color: var(--kolor-blad);  
  
    color: #8b1a1a; /* kontrast 7.2:1 */  
  
}  
  
.komunikat--success {  
  
    background: #f0faf4;  
  
    border-color: var(--kolor-sukces);  
  
    color: #14452a; /* kontrast 8.1:1 */  
  
}
```

```
.komunikat--info {  
  
    background: #f0f6fc;  
  
    border-color: var(--kolor-info);  
  
    color: #0f3449; /* kontrast 8.5:1 */  
  
}
```

```
.komunikat__ikona {  
  
    font-size: 0.85rem;  
  
    font-weight: 700;  
  
    flex-shrink: 0;  
  
}
```

```
.komunikat__tekst { flex: 1; }
```

```
/* Stałe powiadomienia w prawym dolnym rogu */
```

```
.komunikaty--fixed {  
  
    position: fixed;  
  
    right: 1rem;  
  
    bottom: 1rem;  
  
    z-index: 1200;  
  
    width: calc(100% - 2rem);  
  
    max-width: 360px;  
  
}
```

```
.komunikat--minimal {  
  
    box-shadow: 0 4px 16px rgba(10,10,20,0.08);  
  
}
```

```
.komunikat__zamknij {  
  
    margin-left: auto;  
  
    background: none;  
  
    border: none;  
  
    cursor: pointer;
```

```
font-size: 1.1rem;

color: inherit;

opacity: 0.5;

padding: 0 0.5rem;

line-height: 1;

}

.komunikat__zamknij:hover { opacity: 1; }
```

PLIK: shared\static\css\modul\logowanie.css

```
/* =====  
  
    logowanie.css – Strona logowania i zmiana hasła  
  
    ===== */  
  
.strona-logowania {  
  
    background: var(--kolor-tlo);  
  
}  
  
.panel-logowania {  
  
    display: grid;  
  
    grid-template-columns: 1fr 1fr;  
  
    min-height: 100vh;  
  
}  
  
/* — Lewa kolumna: branding ————— */  
  
.panel-logowania__branding {  
  
    background: var(--kolor-glowny);  
  
    display: flex;  
  
    align-items: center;  
  
    justify-content: center;  
  
    padding: 3rem 2.5rem;  
  
    position: relative;  
  
    overflow: hidden;  
  
}  
  
/* Subtelny wzór geometryczny w tle – dekoracyjny */  
  
.panel-logowania__branding::before {  
  
    content: '';  
  
    position: absolute;  
  
    inset: 0;  
  
    background-image:  
  
        repeating-linear-gradient(  
  
            45deg,
```

```
        transparent,

        transparent 40px,

        rgba(255,255,255,0.025) 40px,

        rgba(255,255,255,0.025) 41px

    );

    pointer-events: none;
}
```

```
/* Złoty akcent w górnym rogu */

.panel-logowania__branding::after {

    content: '';

    position: absolute;

    top: 0;

    right: 0;

    width: 6px;

    height: 100%;

    background: linear-gradient(

        to bottom,

        var(--kolor-akcent) 0%,

        var(--kolor-akcent-jasny) 50%,

        var(--kolor-akcent) 100%

    );

}
```

```
.branding__tresc {

    position: relative;

    z-index: 1;

    display: flex;

    flex-direction: column;

    align-items: flex-start;

    gap: 1.5rem;

    max-width: 320px;

}
```

```
.branding__logo-kontener {  
  
  background: white;  
  
  padding: 10px;  
  
  border-radius: var(--promien);  
  
  display: flex;  
  
  align-items: center;  
  
  justify-content: center;  
  
  box-shadow: var(--cien-karta);  
  
}
```

```
.branding__logo {  
  
  max-width: 200px;  
  
  height: auto;  
  
  display: block;  
  
}
```

```
.branding__separator {  
  
  width: 48px;  
  
  height: 3px;  
  
  background: var(--kolor-akcent);  
  
  border-radius: 2px;  
  
}
```

```
.branding__nazwa-systemu {  
  
  font-size: 1.5rem;  
  
  font-weight: 600;  
  
  color: var(--kolor-biale);  
  
  line-height: 1.3;  
  
  letter-spacing: -0.01em;  
  
}
```

```
.branding__podtytul {  
  
  font-size: 0.9rem;  
  
  color: rgba(255, 255, 255, 0.65);  
  
}
```

```
letter-spacing: 0.08em;

text-transform: uppercase;

}
```

```
.branding__dekoracja {

    display: flex;

    gap: 8px;

    margin-top: 1rem;

}
```

```
.branding__dekoracja span {

    display: block;

    width: 8px;

    height: 8px;

    border-radius: 50%;

    background: var(--kolor-akcent);

    opacity: 0.6;

}
```

```
.branding__dekoracja span:first-child { opacity: 1; }

.branding__dekoracja span:last-child { opacity: 0.3; }
```

```
/* — Prawa kolumna: formularz ————— */
```

```
.panel-logowania__formularz {

    display: flex;

    align-items: center;

    justify-content: center;

    padding: 3rem 2rem;

    background: var(--kolor-biale);

}
```

```
.formularz__kontener {

    width: 100%;

    max-width: 420px;
```

```
}
```

```
.formularz__naglowek {  
  
  margin-bottom: 2rem;  
  
}
```

```
.formularz__tytul {  
  
  font-size: 1.75rem;  
  
  font-weight: 600;  
  
  color: var(--kolor-glowny);  
  
  line-height: 1.2;  
  
  letter-spacing: -0.02em;  
  
  margin-bottom: 0.5rem;  
  
}
```

```
.formularz__opis {  
  
  font-size: 0.95rem;  
  
  color: var(--kolor-tekst-drugor);  
  
}
```

```
/* — Przycisk logowania ————— */
```

```
.przycisk-logowania {  
  
  width: 100%;  
  
  padding: 0.875rem 1.5rem;  
  
  margin-top: 0.5rem;  
  
  font-family: var(--font-glowny), serif;  
  
  font-size: 1rem;  
  
  font-weight: 600;  
  
  color: var(--kolor-biale);  
  
  background: var(--kolor-glowny);  
  
  border: none;  
  
  border-radius: var(--promien);  
  
  cursor: pointer;  
  
  letter-spacing: 0.02em;
```



```
transition: background-color 0.15s, transform 0.1s;

/* WCAG 2.5.5: minimalny obszar dotyku 44x44px */

min-height: 48px;
}

.przycisk-logowania: hover { background: var(--kolor-glowny-ciemny); }

.przycisk-logowania: active { transform: translateY(1px); }

.przycisk-logowania: focus-visible {

    outline: var(--fokus-grubosc) solid var(--fokus-kolor);

    outline-offset: var(--fokus-offset);
}

/* — Stopka formularza ————— */

.formularz__stopka {

    margin-top: 2rem;

    padding-top: 1.5rem;

    border-top: 1px solid var(--kolor-ramka);

    display: flex;

    flex-direction: column;

    gap: 0.375rem;
}

.formularz__stopka-tekst {

    font-size: 0.825rem;

    color: var(--kolor-tekst-trzeciorzed);

    text-align: center;
}

.formularz__link {

    color: var(--kolor-glowny-jasny);

    text-decoration: underline;

    text-underline-offset: 2px;

    font-weight: 500;
}
```

```
.formularz__link:hover { color: var(--kolor-glowny); }
```

```
/* — Responsywność ————— */
```

```
@media (max-width: 900px) {
```

```
  .panel-logowania {  
    grid-template-columns: 1fr;  
  }
```

```
  .panel-logowania__branding {  
    padding: 2.5rem 2rem;  
    min-height: auto;  
  }
```

```
  .branding__tresc {  
    flex-direction: row;  
    flex-wrap: wrap;  
    align-items: center;  
    max-width: 100%;  
    gap: 1rem;  
  }
```

```
  .branding__separator,  
  .branding__podtytul,  
  .branding__dekoracja {  
    display: none;  
  }
```

```
  .branding__nazwa-systemu { font-size: 1.1rem; }  
}
```

```
@media (max-width: 480px) {
```

```
  .panel-logowania__formularz { padding: 1.5rem 1rem; }  
  .formularz__kontener { max-width: 100%; }
```

```
.formularz__tytul { font-size: 1.4rem; }

}

/* — Wysoki kontrast – WCAG SC 1.4.11 ————— */

@media (prefers-contrast: more) {

  :root {

    --kolor-ramka: #000000;

    --fokus-kolor: #000000;

    --fokus-grubosc: 4px;

  }

  .pole-formularza__input { border-width: 2px; }

}
```

PLIK: shared\static\css\modul\przyciski.css

```
/* =====  
  
przyciski.css – Przyciski i grupy akcji  
  
===== */  
  
.przycisk {  
  
    display: inline-flex;  
  
    align-items: center;  
  
    gap: 0.375rem;  
  
    padding: 0.5rem 1rem;  
  
    font-family: var(--font-glowny);  
  
    font-size: 0.85rem;  
  
    font-weight: 600;  
  
    text-decoration: none;  
  
    border: none;  
  
    border-radius: var(--promien);  
  
    cursor: pointer;  
  
    transition: background 0.15s, color 0.15s, box-shadow 0.15s;  
  
    min-height: 36px;  
  
    white-space: nowrap;  
  
}  
  
.przycisk--glowny {  
  
    background: var(--kolor-glowny);  
  
    color: var(--kolor-biale);  
  
}  
  
.przycisk--glowny:hover {  
  
    background: var(--kolor-glowny-ciemny);  
  
}  
  
.przycisk--drugorzeczny {  
  
    background: var(--kolor-tlo);  
  
    color: var(--kolor-tekst);
```

```
border: 1px solid var(--kolor-ramka);  
}
```

```
.przycisk--drugorzeczny:hover {  
    background: var(--kolor-ramka);  
}
```

```
.przycisk--ostrzezenie {  
    background: #f59e0b;  
    color: white;  
    border: 1px solid #f59e0b;  
}
```

```
.przycisk--ostrzezenie:hover {  
    background: #d97706;  
    border-color: #d97706;  
}
```

```
.przycisk--niebezpieczny {  
    background: var(--kolor-blad);  
    color: var(--kolor-biale);  
}
```

```
.przycisk--niebezpieczny:hover {  
    background: #a93226;  
}
```

```
.przycisk--maly {  
    padding: 0.3rem 0.625rem;  
    font-size: 0.75rem;  
    min-height: 28px;  
}
```

```
/* — Grupy przycisków ————— */
```

```
.akcje-tabeli {  
  
  display: flex;  
  
  align-items: center;  
  
  gap: 0.375rem;  
  
}
```

```
.akcje-panelu {  
  
  display: flex;  
  
  align-items: center;  
  
  gap: 0.5rem;  
  
}
```

PLIK: shared\static\css\modul\statusy.css

```
/* =====  
  
    statusy.css – Odznaki statusów i ról  
  
    ===== */  
  
/* — Statusy ————— */  
  
.status {  
  
    display: inline-block;  
  
    padding: 0.25rem 0.625rem;  
  
    font-size: 0.7rem;  
  
    font-weight: 600;  
  
    text-transform: uppercase;  
  
    letter-spacing: 0.04em;  
  
    border-radius: 100px;  
  
    white-space: nowrap;  
  
}  
  
.status--draft,  
.status--szkic {  
  
    background: #e2e8f0;  
  
    color: var(--kolor-tekst-drugor);  
  
}  
  
.status--planned,  
.status--zaplanowana {  
  
    background: #dbeafe;  
  
    color: #1e40af;  
  
}  
  
.status--in-progress,  
.status--w-trakcie {  
  
    background: #fef3c7;  
  
    color: #92400e;  
  
}
```

```
.status--pending-evaluation,  
  
.status--oczekuje-oceny {  
  
    background: #fce7f3;  
  
    color: #9d174d;  
  
}
```

```
.status--completed,  
  
.status--zakonczone {  
  
    background: #d1fae5;  
  
    color: var(--kolor-sukces);  
  
}
```

```
/* — Role ————— */
```

```
.rola-odznaka {  
  
    display: inline-block;  
  
    padding: 0.2rem 0.5rem;  
  
    font-size: 0.7rem;  
  
    font-weight: 600;  
  
    border-radius: 100px;  
  
    text-transform: uppercase;  
  
    letter-spacing: 0.03em;  
  
}
```

```
.rola-odznaka--admin {  
  
    background: #fef3c7;  
  
    color: #92400e;  
  
}
```

```
.rola-odznaka--uopz {  
  
    background: #dbeeaf;  
  
    color: #1e40af;  
  
}
```



```
.rola-odznaka--zopz {  
  
  background: #d1fae5;  
  
  color: var(--kolor-sukces);  
  
}
```

```
.rola-odznaka--student {  
  
  background: #e2e8f0;  
  
  color: var(--kolor-tekst-drugor);  
  
}
```

PLIK: shared\static\css\modul\tabele.css

```
/* =====  
  
    tabele.css – Tabele danych  
  
    ===== */  
  
.tabela-wrapper {  
    overflow-x: auto;  
}  
  
.tabela {  
    width: 100%;  
    border-collapse: collapse;  
    font-size: 0.875rem;  
}  
  
.tabela thead {  
    background: var(--kolor-tlo);  
}  
  
.tabela th {  
    padding: 0.75rem 1rem;  
    text-align: left;  
    font-weight: 600;  
    font-size: 0.75rem;  
    text-transform: uppercase;  
    letter-spacing: 0.05em;  
    color: var(--kolor-tekst-dugor);  
    border-bottom: 2px solid var(--kolor-ramka);  
    white-space: nowrap;  
}  
  
.tabela td {  
    padding: 0.875rem 1rem;  
    border-bottom: 1px solid var(--kolor-ramka);
```

```
vertical-align: middle;

color: var(--kolor-tekst);

}

.tabela tbody tr:hover {

    background: rgba(26, 58, 92, 0.03);

}

.tabela--pusta {

    text-align: center;

    color: var(--kolor-tekst-trzeciorzed);

    padding: 2rem 1rem !important;

    font-style: italic;

}

/* — Komórka użytkownika ————— */

.komorka-uzytkownik {

    display: flex;

    flex-direction: column;

}

.komorka-uzytkownik__nazwa {

    font-weight: 600;

}

.komorka-uzytkownik__info {

    font-size: 0.75rem;

    color: var(--kolor-tekst-trzeciorzed);

}
```

PLIK: shared\static\css\modul\uklad.css

```
/* =====  
  
    uklad.css – Layout panelu: sidebar, panel-main, nagłówek  
  
    ===== */  
  
/* — Layout panelu ————— */  
  
.panel-body {  
  
    display: flex;  
  
    min-height: 100vh;  
  
    background: var(--kolor-tlo);  
  
}  
  
/* — Sidebar ————— */  
  
.sidebar {  
  
    width: 260px;  
  
    height: 100vh;  
  
    background: var(--kolor-glowny);  
  
    color: rgba(255, 255, 255, 0.85);  
  
    display: flex;  
  
    flex-direction: column;  
  
    flex-shrink: 0;  
  
    position: sticky;  
  
    top: 0;  
  
    overflow-y: auto;  
  
}  
  
.sidebar__naglowek {  
  
    padding: 1.5rem 1.25rem 1rem;  
  
    border-bottom: 1px solid rgba(255, 255, 255, 0.1);  
  
}  
  
.sidebar__logo {  
  
    max-width: 100%;  
  
    height: auto;
```

```
display: block;

}
```

```
.sidebar__system-label {

display: block;

font-size: 0.7rem;

text-transform: uppercase;

letter-spacing: 0.12em;

color: var(--kolor-akcent);

font-weight: 600;

margin-top: 0.5rem;

}
```

```
/* Użytkownik */
```

```
.sidebar__uzytkownik {

display: flex;

align-items: center;

gap: 0.75rem;

padding: 1.25rem;

border-bottom: 1px solid rgba(255, 255, 255, 0.08);

}
```

```
.sidebar__uzytkownik-avatar {

width: 38px;

height: 38px;

border-radius: 50%;

background: var(--kolor-akcent);

color: var(--kolor-glowny-ciemny);

display: flex;

align-items: center;

justify-content: center;

font-weight: 700;

font-size: 0.8rem;

flex-shrink: 0;
```

```
}
```

```
.sidebar__uzytkownik-info {  
  
    display: flex;  
  
    flex-direction: column;  
  
    min-width: 0;  
  
}
```

```
.sidebar__uzytkownik-imie {  
  
    font-size: 0.85rem;  
  
    font-weight: 600;  
  
    color: #fff;  
  
    white-space: nowrap;  
  
    overflow: hidden;  
  
    text-overflow: ellipsis;  
  
}
```

```
.sidebar__uzytkownik-rola {  
  
    font-size: 0.7rem;  
  
    color: rgba(255, 255, 255, 0.5);  
  
    text-transform: uppercase;  
  
    letter-spacing: 0.05em;  
  
}
```

```
/* Menu */
```

```
.sidebar__menu {  
  
    list-style: none;  
  
    padding: 0.75rem 0;  
  
    flex: 1;  
  
}
```

```
.sidebar__menu-sekcja {  
  
    font-size: 0.65rem;  
  
    text-transform: uppercase;
```

```
letter-spacing: 0.1em;

color: rgba(255, 255, 255, 0.35);

padding: 1.25rem 1.25rem 0.375rem;

font-weight: 600;
}
```

```
.sidebar__menu-pozycja {

margin: 1px 0;
}
```

```
.sidebar__link {

display: flex;

align-items: center;

gap: 0.75rem;

padding: 0.625rem 1.25rem;

color: rgba(255, 255, 255, 0.75);

text-decoration: none;

font-size: 0.875rem;

font-weight: 500;

border-left: 3px solid transparent;

transition: background 0.15s, color 0.15s, border-color 0.15s;
}
```

```
.sidebar__link:hover {

background: rgba(255, 255, 255, 0.08);

color: #fff;
}
```

```
.sidebar__link--aktywny {

background: rgba(255, 255, 255, 0.12);

color: #fff;

border-left-color: var(--kolor-akcent);

font-weight: 600;
}
```

```
.sidebar__ikona {  
  
    display: inline-flex;  
  
    align-items: center;  
  
    justify-content: center;  
  
    width: 20px;  
  
    height: 20px;  
  
    flex-shrink: 0;  
  
    font-size: 1rem;  
  
    line-height: 1;  
  
}
```

```
.sidebar__ikona svg {  
  
    width: 18px;  
  
    height: 18px;  
  
    fill: none;  
  
    stroke: currentColor;  
  
    stroke-width: 2;  
  
    stroke-linecap: round;  
  
    stroke-linejoin: round;  
  
}
```

```
/* Stopka sidebara */
```

```
.sidebar__stopka {  
  
    padding: 1rem 1.25rem;  
  
    border-top: 1px solid rgba(255, 255, 255, 0.08);  
  
    margin-top: auto;  
  
}
```

```
.sidebar__wylogowanie {  
  
    display: flex;  
  
    align-items: center;  
  
    gap: 0.5rem;  
  
    width: 100%;  
  
}
```



```
padding: 0.625rem 0.75rem;

background: rgba(255, 255, 255, 0.06);

border: 1px solid rgba(255, 255, 255, 0.1);

border-radius: var(--promien);

color: rgba(255, 255, 255, 0.7);

font-family: var(--font-glowny);

font-size: 0.85rem;

font-weight: 500;

cursor: pointer;

transition: background 0.15s, color 0.15s;

}
```

```
.sidebar__wylogowanie:hover {

    background: rgba(192, 57, 43, 0.3);

    color: #fff;

}
```

```
.sidebar__wylogowanie svg {

    width: 16px;

    height: 16px;

    fill: none;

    stroke: currentColor;

    stroke-width: 2;

    stroke-linecap: round;

    stroke-linejoin: round;

}
```

```
/* — Panel main ————— */

.panel-main {

    flex: 1;

    display: flex;

    flex-direction: column;

    min-width: 0;

}
```

```
.panel-main__naglowek {  
  
    background: var(--kolor-biale);  
  
    border-bottom: 1px solid var(--kolor-ramka);  
  
    padding: 1.25rem 2rem;  
  
}
```

```
.panel-main__naglowek-tresc {  
  
    display: flex;  
  
    align-items: center;  
  
    justify-content: space-between;  
  
    gap: 1rem;  
  
}
```

```
.panel-main__tytul {  
  
    font-size: 1.35rem;  
  
    font-weight: 600;  
  
    color: var(--kolor-tekst);  
  
    letter-spacing: -0.01em;  
  
}
```

```
.panel-main__tresc {  
  
    flex: 1;  
  
    padding: 2rem;  
  
}
```

```
/* — Responsywność panelu ————— */
```

```
@media (max-width: 768px) {  
  
    .sidebar {  
  
        display: none;  
  
    }  
  
}
```

```
.panel-main__tresc {  
  
    padding: 1rem;  
  
}
```

```
}
```

```
.panel-main__naglowek {
```

```
padding: 1rem;
```

```
}
```

```
}
```

PLIK: shared\static\css\modul\zmiennie.css

```
/* =====

zmiennie.css – Zmienne CSS, reset, podstawy, dostępność

System Praktyk Zawodowych – ANS Elbląg

WCAG 2.1 AA: kontrast ≥ 4.5:1, fokus widoczny

===== */

/* — Zmienne CSS ————— */

:root {

  /* Paleta ANS – granat instytucjonalny jako kolor główny */

  --kolor-glowny:      #1a3a5c;   /* navy – kolor uczelni */

  --kolor-glowny-ciemny: #0f2540; /* hover */

  --kolor-glowny-jasny:  #2a5080; /* aktywny */

  --kolor-akcent:       #c8963e;   /* złoty – akcent ANS */

  --kolor-akcent-jasny:  #e8b45a;

  --kolor-tlo:          #f4f6f9;

  --kolor-biale:         #ffffff;

  --kolor-tekst:         #1a1a2e;   /* kontrast 13.5:1 na białym */

  --kolor-tekst-drugor:  #4a5568;   /* kontrast 5.9:1 na białym */

  --kolor-tekst-trzeciorzed: #718096; /* kontrast 4.6:1 */

  --kolor-ramka:         #d1d9e6;

  --kolor-blad:          #c0392b;   /* kontrast 5.2:1 na białym */

  --kolor-sukces:        #1e6e3a;   /* kontrast 6.1:1 */

  --kolor-info:          #1a5276;

  /* Fokus – WCAG 2.4.7 wymagany, 3.2.4 spójny */

  --fokus-kolor:         #c8963e;

  --fokus-grubosc:       3px;

  --fokus-offset:        3px;

  /* Typografia */

  --font-glowny:         'IBM Plex Sans', 'Segoe UI', Tahoma, sans-serif;
```

```
--font-mono:      'IBM Plex Mono', 'Courier New', monospace;

/* Odstępy */

--promien:        8px;

--promien-duzy:    16px;

--cien-karta:      0 2px 12px rgba(26, 58, 92, 0.12);

--cien-glowny:     0 8px 32px rgba(26, 58, 92, 0.18);

}

/* — Reset i podstawy ————— */

*, *::before, *::after {

  box-sizing: border-box;

  margin: 0;

  padding: 0;

}

html {

  /* WCAG 1.4.4: tekst powiększalny do 200% bez utraty treści */

  font-size: 100%;

  -webkit-text-size-adjust: 100%;

  scroll-behavior: smooth;

}

/* WCAG 2.3.3: respektuj preferencje użytkownika dot. animacji */

@media (prefers-reduced-motion: reduce) {

  *, *::before, *::after {

    animation-duration: 0.01ms !important;

    transition-duration: 0.01ms !important;

  }

}

body {

  font-family: var(--font-glowny), serif;

  font-size: 1rem;
```

```
    line-height: 1.6;

    color: var(--kolor-tekst);

    background-color: var(--kolor-tlo);

    min-height: 100vh;
}

/* — WCAG: Skip link (SC 2.4.1) ————— */

.skip-link {

    position: absolute;

    top: -100px;

    left: 1rem;

    z-index: 9999;

    padding: 0.75rem 1.5rem;

    background: var(--kolor-glowny);

    color: var(--kolor-biale);

    font-weight: 600;

    border-radius: var(--promien);

    text-decoration: none;

    transition: top 0.2s;
}

.skip-link:focus {

    top: 1rem;
}

/* — Fokus globalny (SC 2.4.7) ————— */

:focus-visible {

    outline: var(--fokus-grubosc) solid var(--fokus-kolor);

    outline-offset: var(--fokus-offset);

    border-radius: calc(var(--promien) - 2px);
}

/* Usuń fokus dla kliknięcia myszą, zachowaj dla klawiatury */

:focus:not(:focus-visible) {
```

outline: none;

}

PLIK: tex_service\app.py

```
import io

import logging

import shutil

from pathlib import Path


from flask import Flask, request, send_file, jsonify

from compiler import compile_pdf, render_tex, TexCompilationError


logging.basicConfig(level=logging.INFO)

logger = logging.getLogger(__name__)


app = Flask(__name__)


TEMPLATES_DIR = Path(__file__).parent / "templates"


def _allowed_templates() -> set[str]:

    """Zwraca nazwy plików .tex.j2 z katalogu templates."""

    return {f.name for f in TEMPLATES_DIR.glob("*.tex.j2")}


@app.route("/health")

def health():

    lualatex_ok = bool(shutil.which("lualatex"))

    templates = list(_allowed_templates())

    return jsonify({

        "status": "ok" if lualatex_ok else "degraded",

        "lualatex": lualatex_ok,

        "templates": templates,

    })
```



```

@app.route("/generuj", methods=["POST"])

def generuj():
    """
    Body JSON:

    {
        "template": "zal6_dziennik.tex.j2",
        "context": { ... },
        "filename": "dziennik.pdf"    # opcjonalne
    }

    Odpowiedź: application/pdf (bajty pliku)
    lub          application/json z {"error": "..."} przy błędzie
    """

    data = request.get_json(silent=True)

    if not data:

        return jsonify({"error": "Brak body JSON"}), 400

    template_name = data.get("template", "")
    context = data.get("context", {})
    filename = data.get("filename", "dokument.pdf")

    # Walidacja szablonu

    allowed = _allowed_templates()

    if template_name not in allowed:

        return jsonify({
            "error": f"Nieznany szablon: {template_name!r}",
            "available": sorted(allowed),
        }), 400

    if not isinstance(context, dict):

        return jsonify({"error": "context musi być obiektem JSON"}), 400

    try:

```

```

pdf_bytes = compile_pdf(template_name, context)

except TexCompilationError as e:

    logger.error("Błąd kompilacji LaTeX dla %s: %s\nLOG:\n%s", template_name, e, e.log)

    return jsonify({

        "error": str(e),

        "log": e.log[-3000:],

    }), 500

except Exception as e:

    logger.exception("Nieoczekiwany błąd dla %s", template_name)

    return jsonify({"error": str(e)}), 500


return send_file(

    io.BytesIO(pdf_bytes),

    mimetype="application/pdf",

    as_attachment=True,

    download_name=filename,

)


if __name__ == "__main__":

    app.run(host="0.0.0.0", port=5002, debug=False)

```

PLIK: tex_service\compiler.py

```
"""tex_service/compiler.py

Silnik kompilacji PDF. Przyjmuje nazwę szablonu i kontekst danych,
wyrenderowuje szablon Jinja2, wywołuje lualatex,
zwraca bajty PDF lub rzuca TexCompilationError.
"""

import os

import shutil

import subprocess

import tempfile

import logging

from pathlib import Path

from jinja2 import Environment, FileSystemLoader, StrictUndefined

from sanitizer import sanitize, sanitize_date, sanitize_int

logger = logging.getLogger(__name__)

TEMPLATES_DIR = Path(__file__).parent / "templates"

COMPILE_TIMEOUT = int(os.environ.get('LATEX_TIMEOUT', '60'))

COMPILE_PASSES = 1

class TexCompilationError(RuntimeError):

    def __init__(self, message: str, log: str = ""):

        super().__init__(message)

        self.log = log

def _build_jinja_env() -> Environment:
```

```

env = Environment(

    loader=FileSystemLoader(str(TEMPLATES_DIR)),

    block_start_string="<<%",

    block_end_string="%>>",

    variable_start_string="<<",

    variable_end_string=">>",

    comment_start_string="<<#",

    comment_end_string="#>>",

    undefined=StrictUndefined,

    keep_trailing_newline=True,

)

env.filters["s"] = sanitize

env.filters["date"] = sanitize_date

env.filters["num"] = sanitize_int

return env

```

`_JINJA_ENV: Environment | None = None`

```

def get_jinja_env() -> Environment:

    global _JINJA_ENV

    if _JINJA_ENV is None:

        _JINJA_ENV = _build_jinja_env()

    return _JINJA_ENV

```

```

def _find_lualatex() -> str:

    binary = shutil.which("lualatex")

    if binary is None:

        raise EnvironmentError(

            "Nie znaleziono lualatex w PATH. "

            "Sprawdź czy texlive jest zainstalowany w kontenerze."

        )

```

```
return binary
```

```
def _run_lualatex(lualatex: str, tex_file: Path, workdir: Path) -> str:
```

```
    cmd = [  
        lualatex,  
        "-no-shell-escape",  
        "-interaction=nonstopmode",  
        "-halt-on-error",  
        "-output-directory", str(workdir),  
        str(tex_file),  
    ]
```

```
    try:
```

```
        result = subprocess.run(  
            cmd,  
            cwd=str(workdir),  
            capture_output=True,  
            timeout=COMPILE_TIMEOUT,  
            stdin=subprocess.DEVNULL,  
        )
```

```
    except subprocess.TimeoutExpired:
```

```
        raise TexCompilationError(  
            f"lualatex przekroczył limit czasu ({COMPILE_TIMEOUT}s)."  
        )
```

```
    log_file = workdir / tex_file.with_suffix(".log").name
```

```
    log_text = log_file.read_text(encoding="utf-8", errors="replace") if log_file.exists() else ""
```

```
    if result.returncode != 0:
```

```
        raise TexCompilationError(  
            f"lualatex zakończył się błędem (kod {result.returncode}).",  
            log=log_text,  
        )
```

```
return log_text
```

```
def render_tex(template_name: str, context: dict) -> str:
```

```
    """Renderuje szablon Jinja2 → surowy tekst TeX."""
```

```
    env = get_jinja_env()
```

```
    template = env.get_template(template_name)
```

```
    return template.render(**context)
```

```
_MAX_RAW_TEX_SIZE = 100_000 # 100 KB – wystarczy dla każdego dokumentu
```

```
# Wzorce niebezpiecznych komend LaTeX.
```

```
# Każdy wpis: (wzorzec_regex, opis_dla_uzytkownika).
```

```
# WAŻNE: to jest lista blokowanych wzorców (blocklist), nie lista dozwolonych.
```

```
# Blocklist nigdy nie jest kompletna – główną ochroną jest sandboxing kontenera.
```

```
_DANGEROUS_PATTERNS: list[tuple[str, str]] = [
```

```
    # — Shell escape – bezpośrednie wykonanie kodu powłoki —————
```

```
    (r'\\write\s*18\b',          r'\write18 – shell escape'),
```

```
    (r'\\write\s*-\s*1\b',      r'\write-1 – zapis na terminal'),
```

```
    (r'\\immediate\s*\\write\s*18\b', r'\immediate\write18 – shell escape'),
```

```
    (r'\\ShellEscape\b',        r'\ShellEscape – shell escape (shellesc)'),
```

```
    # — Lua – wykonanie dowolnego kodu Lua/systemu —————
```

```
    (r'\\directlua\b', r'\directlua – wykonanie kodu Lua'),
```

```
    (r'\\latelua\b',   r'\latelua – wykonanie kodu Lua'),
```

```
    (r'\\luacode\b',   r'\luacode – środowisko Lua'),
```

```
    (r'\\luaexec\b',   r'\luaexec – wykonanie Lua'),
```

```
    (r'\\luastring\b', r'\luastring – ciąg Lua'),
```

```
    (r'\\lua[A-Za-z]+' , r'\lua* – komenda Lua'),
```

```
    # — Odczyt plików systemu —————
```

```
    (r'\\input\s*\{',        r'\input{} – dołączenie pliku'),
```

```
(r'\include\s*{',      r'\include{} – dołączenie pliku'),
(r'\verbatiminput\b',  r'\verbatiminput – odczyt pliku jako tekst'),
(r'\lstinputlisting\b', r'\lstinputlisting – odczyt pliku (listings)'),
(r'\inputminted\b',     r'\inputminted – odczyt pliku (minted)'),
(r'\subfile\b',         r'\subfile – dołączenie podpliku'),
(r'\import\b',          r'\import – dołączenie z zewnętrznego katalogu'),
(r'\openin\b',          r'\openin – otwarcie strumienia odczytu'),
(r'\read\b',            r'\read – odczyt ze strumienia'),
```

— Zapis plików systemu —————

```
(r'\openout\b',        r'\openout – otwarcie strumienia zapisu'),
(r'\newwrite\b',       r'\newwrite – rejestracja strumienia zapisu'),
(r'\newread\b',        r'\newread – rejestracja strumienia odczytu'),
(r'\immediate\s*\write', r'\immediate\write – natychmiastowy zapis'),
(r'\write\s*\\',        r'\write\register – zapis przez rejestr'),
```

— Niebezpieczne pakiety —————

```
(r'\usepackage\s*(?:\[.*?\])?\s*\{shellesc\}', r'pakiet shellesc – shell escape'),
(r'\usepackage\s*(?:\[.*?\])?\s*\{minted\}',   r'pakiet minted – wykonanie kodu powłoki'),
(r'\usepackage\s*(?:\[.*?\])?\s*\{sagetex\}',   r'pakiet sagetex – wykonanie kodu Python/Sage'),
(r'\usepackage\s*(?:\[.*?\])?\s*\{pythontex\}', r'pakiet pythontex – wykonanie kodu Python'),
```

— Obejście interpretera – zmiana znaczenia znaków —————

```
(r'\catcode\b', r'\catcode – zmiana kodu kategorii znaków'),
(r'\csname\b',  r'\csname – obfuskacja nazwy komendy'),
```

— Niskopoziomowe prymitywy pdftex/luatex —————

```
(r'\pdfshellescape\b', r'\pdfshellescape – flaga shell escape'),
(r'\pdfobj\b',         r'\pdfobj – niskopoziomowy obiekt PDF'),
(r'\special\b',        r'\special – komenda sterownika wyjścia'),
```

]

```
def _check_dangerous_commands(tex_source: str) -> None:
```

```
"""Sprawdza surowy kod TeX pod kątem niebezpiecznych komend.
```

Rzuca `TexCompilationError` przy pierwszym dopasowaniu.

Sprawdzanie jest case-insensitive i wieloliniowe.

WAŻNE: Funkcja jest drugą linią obrony – pierwsze są:

1. Sandboxing kontenera (`read_only`, `no-new-privileges`, `cap_drop`)
2. `lualatex -no-shell-escape`

Nie należy polegać wyłącznie na tej funkcji.

```
"""
```

```
if len(tex_source) > _MAX_RAW_TEX_SIZE:
```

```
    raise TexCompilationError(
        f"Kod TeX jest zbyt długi ({len(tex_source)} znaków). "
        f"Maksymalny rozmiar to {_MAX_RAW_TEX_SIZE} znaków."
    )
```

```
for pattern, description in _DANGEROUS_PATTERNS:
```

```
    if re.search(pattern, tex_source, re.IGNORECASE | re.DOTALL):
        raise TexCompilationError(
            f"Zablokowana niebezpieczna komenda LaTeX: {description}. "
            f"Ze względów bezpieczeństwa ta komenda jest niedozwolona "
            f"w trybie ręcznej kompilacji."
        )
```

```
def compile_raw_tex(tex_source: str) -> bytes:
```

```
    """Kompiluje surowy kod TeX → bajty PDF.
```

Przed kompilacją sprawdza kod pod kątem niebezpiecznych komend

(ochrona przed LaTeX Command Injection).

```
"""
```

```
    _check_dangerous_commands(tex_source)
```



```
lualatex = _find_lualatex()
```

```
with tempfile.TemporaryDirectory(prefix="tex_") as tmpdir:
```

```
    workdir = Path(tmpdir)
```

```
    tex_file = workdir / "document.tex"
```

```
    tex_file.write_text(tex_source, encoding="utf-8")
```

```
    for _ in range(COMPILE_PASSES):
```

```
        _run_lualatex(lualatex, tex_file, workdir)
```

```
    pdf_file = workdir / "document.pdf"
```

```
    if not pdf_file.exists():
```

```
        raise TexCompilationError("lualatex zakończył się sukcesem, ale brak pliku PDF.")
```

```
    return pdf_file.read_bytes()
```

```
def compile_pdf(template_name: str, context: dict) -> bytes:
```

```
    """Publiczne API: template + context → bajty PDF."""
```

```
    logger.info("Kompilacja %s", template_name)
```

```
    tex_source = render_tex(template_name, context)
```

```
    pdf_bytes = compile_raw_tex(tex_source)
```

```
    logger.info("Wygenerowano %d bajtów", len(pdf_bytes))
```

```
    return pdf_bytes
```

PLIK: tex_service\Dockfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \

    PYTHONUNBUFFERED=1 \

    DEBIAN_FRONTEND=noninteractive


WORKDIR /app


RUN apt-get update && apt-get install -y --no-install-recommends \

    texlive-luatex \

    texlive-latex-base \

    texlive-latex-recommended \

    texlive-latex-extra \

    texlive-fonts-recommended \

    texlive-lang-polish \

    lmodern \

    curl \

    && rm -rf /var/lib/apt/lists/*


COPY requirements.txt ./

RUN pip install --no-cache-dir -r requirements.txt


COPY sanitizer.py compiler.py app.py pdf_service.py ./

COPY templates/ ./templates/


# Bezpieczeństwo: dedykowany nieuprzywilejowany użytkownik (nie root).

# LuaLaTeX potrzebuje /tmp do plików pomocniczych – tmpfs montowany

# jest w docker-compose, więc chown /tmp nie jest wymagany.

RUN addgroup --system appgroup \

    && adduser --system --ingroup appgroup --no-create-home appuser \

    && chown -R appuser:appgroup /app


USER appuser
```

EXPOSE 5002

CMD ["gunicorn", "--bind", "0.0.0.0:5002", "--workers", "2", "--timeout", "120", "app:app"]

PLIK: tex_service\pdf_service.py

```
"""
```

```
tex_service/pdf_service.py
```

Klasa PDFService – buduje konteksty danych dla szablonów LaTeX.

Przeniesiona z tex_engine/pdf_service.py (tex_engine usunięty).

```
"""
```

```
import logging
```

```
from typing import Dict, Any
```

```
logger = logging.getLogger(__name__)
```

```
class PDFServiceInterface:
```

```
    """Interface dla usługi generowania PDF."""
```

```
    def build_context_dziennik(self, zapis) -> Dict[str, Any]:
```

```
        raise NotImplementedError
```

```
    def build_context_efekty(self, zapis) -> Dict[str, Any]:
```

```
        raise NotImplementedError
```

```
    def build_context_sprawozdanie(self, zapis, tresc: Dict) -> Dict[str, Any]:
```

```
        raise NotImplementedError
```

```
class PDFService(PDFServiceInterface):
```

```
    def build_context_dziennik(self, zapis) -> Dict[str, Any]:
```

```
        wpisy = getattr(zapis, 'wpisy_dziennika', [])
```

```
        return {
```

```
            'student': {
```

```
                'first_name': zapis.student.first_name,
```

```

        'last_name':      zapis.student.last_name,

        'album_number': zapis.student.album_number,

    },

    'zapis': {

        'total_hours_logged': zapis.total_hours_logged,

        'praktyka': {

            'rok_uczelnianny': zapis.praktyka.rok_uczelnianny,

            'semestr':      zapis.praktyka.semestr,

        },

    },

    'wpisy': [

        {

            'entry_date':      w.entry_date.isoformat(),

            'duration_hours':  w.duration_hours,

            'description':      w.description,

            'efekt': {'kod': f"{w.learning_outcome_id:02d}"},

        }

        for w in sorted(wpisy, key=lambda x: x.entry_date)

    ],

}

```

```

def build_context_efekty(self, zapis) -> Dict[str, Any]:

    return {

        'student': {

            'nazwisko':      zapis.student.last_name,

            'imie':          zapis.student.first_name,

            'numer_albumu':  zapis.student.album_number,

        },

        'specjalnosc':      zapis.specjalnosc or '',

        'harmonogram':      getattr(zapis, 'harmonogram', []),

        'efekty_uczenia':   [],      # uzupełniane w route

        'oceny_efektow':    [],      # uzupełniane w route

    }

```

```
def build_context_sprawozdanie(self, zapis, tresc: Dict) -> Dict[str, Any]:
```

```
    return {  
        'student': {  
            'imie':      zapis.student.first_name,  
            'nazwisko':  zapis.student.last_name,  
            'album_number': zapis.student.album_number,  
        },  
        'firma': {  
            'nazwa': zapis.firma_nazwa,  
        },  
        'tresc': tresc,  
        'zapis': zapis,  
    }
```

```
# Singleton używany przez Flask routes i Celery tasks
```

```
pdf_service = PDFService()
```

PLIK: tex_service\sanitizer.py

```
r"""tex_service/sanitizer.py
```

Sanityzacja danych wejściowych przed wstrzyknięciem do szablonów LaTeX.

Warstwa ochrony: dane z bazy/requestu -> sanitize() -> szablon LaTeX.

Nawet jeśli użytkownik wpisze '\\directlua{...}' w pole formularza,

po przejściu przez sanitize() zostanie to zamienione na bezpieczny

ciąg znaków LaTeX (np. \textbackslash{}directlua\{...\}).

```
"""
```

```
import re
```

```
from datetime import date, datetime
```

```
from pylatexenc.latexencode import unicode_to_latex
```

```
# Znaki specjalne LaTeX wymagające escapowania.
```

```
# unicode_to_latex z pylatexenc obsługuje je wszystkie, ale jawna lista
```

```
# służy jako dokumentacja i drugi poziom ochrony.
```

```
_LATEX_SPECIAL_CHARS = {
```

```
    '\\': r'\textbackslash{}',
```

```
    '{': r'\{',
```

```
    '}': r'\}',
```

```
    '$': r'\$',
```

```
    '&': r'\&',
```

```
    '#': r'\#',
```

```
    '^': r'\^{}',
```

```
    '_': r'\_',
```

```
    '~': r'\textasciitilde{}',
```

```
    '%': r'\%',
```

```
}
```

```
def sanitize(value, max_length: int = 500) -> str:

    """Oczyszcza wartość do bezpiecznego wstawienia w szablon LaTeX.

    Używa pylatexenc.unicode_to_latex, które:

    - Zamienia wszystkie znaki specjalne LaTeX (\\, {, }, $, &, #, ^, _, ~, %)
      na ich escapowane odpowiedniki
    - Zamienia znaki Unicode na komendy LaTeX (np. a → \\k{a})
```

Dzięki temu nawet jeśli wartość zawiera złośliwy kod LaTeX
(np. z pola tekstowego w formularzu), zostanie on zrenderowany
jako dosłowny tekst, nie wykonany jako komenda.

```
    """

    if value is None:

        return ''

    text = str(value).strip()

    if max_length:

        text = text[:max_length]

    # unicode_to_latex escapuje wszystkie znaki specjalne LaTeX

    return unicode_to_latex(text)
```

```
def sanitize_date(value, fmt: str = '%d.%m.%Y') -> str:

    """Formatuje datę do bezpiecznego ciągu – tylko cyfry i kropki."""

    if value is None:

        return '-'

    if isinstance(value, (date, datetime)):

        return value.strftime(fmt)

    try:

        return date.fromisoformat(str(value)).strftime(fmt)

    except ValueError:

        return sanitize(str(value))
```

```
def sanitize_int(value) -> str:
```



```
"""Zwraca wyłącznie cyfry całkowite – zero ryzyka injection."""
```

```
if value is None:
```

```
    return '0'
```

```
try:
```

```
    return str(int(value))
```

```
except (ValueError, TypeError):
```

```
    return '0'
```

PLIK: tex_service__init__.py

```
"""
```

```
tex_service – pakiet kompilacji LaTeX do PDF.
```

```
Eksportuje:
```

- compile_pdf(template_name, context) -> bytes
- render_tex(template_name, context) -> str
- compile_raw_tex(tex_source) -> bytes
- TexCompilationError
- pdf_service (instancja PDFService)

```
"""
```

```
import sys
```

```
import os
```

```
# Gwarantuj że katalog tex_service/ jest w sys.path (potrzebne dla
```

```
# lokalnych importów: 'from sanitizer import ...' w compiler.py)
```

```
_pkg_dir = os.path.dirname(__file__)
```

```
if _pkg_dir not in sys.path:
```

```
    sys.path.insert(0, _pkg_dir)
```

```
from compiler import (          # noqa: E402
```

```
    TexCompilationError,
```

```
    render_tex,
```

```
    compile_raw_tex,
```

```
    compile_pdf,
```

```
)
```

```
from pdf_service import pdf_service # noqa: E402
```

```
__all__ = [
```

```
    "TexCompilationError",
```

```
    "render_tex",
```

```
    "compile_raw_tex",
```

```
    "compile_pdf",
```

"pdf_service",

]